

In [12]: `from IPython.display import Image`

`Image('3-1.png')`

Out[12]: **시험장 환경 정보**

Python: 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]

모듈	버전
pandas	0.25.1
numpy	1.18.5
sklearn	0.21.3
scipy	1.5.2
mlxtend	0.15.0.0
statsmodels	0.11.1

강사: 멀티캠퍼스 강선구(sunku0316.kang@multicampus.com, sun9sun9@gmail.com)

```
In [13]: # 실행 환경 확인

import pandas as pd
import numpy as np
import sklearn
import scipy
import statsmodels
import mlxtend
import sys
import xgboost as xgb

import warnings
warnings.filterwarnings(action = 'ignore')

print(sys.version)
for i in [pd, np, sklearn, scipy, mlxtend, statsmodels, xgb]:
    print(i.__name__, i.__version__)
```

```
3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
pandas 0.25.1
numpy 1.18.5
sklearn 0.21.3
scipy 1.5.2
mlxtend 0.15.0.0
statsmodels 0.11.1
xgboost 0.80
```

```
In [14]: # 시각화 모듈 설정
# 참고용 차트를 출력하기 위함
import matplotlib.pyplot as plt
import seaborn as sns

import matplotlib as mpl
import matplotlib.font_manager as fm
plt.rc('font', family='Malgun Gothic')
mpl.rcParams['axes.unicode_minus'] = False
```

```
In [15]: Image('3-2.png')
```

Out[15]:

문제 개요

생활가전 A부서는 친환경 미니 세탁기를 개발 중에 있다.

프로젝트의 핵심은 세탁물을 자동 인식하고 이에 따른 최적 코스를 (수량, 수온, 세제종류, 세제량, 세탁시간, 세탁방법) 제시하여 세탁지수(target)를 높이는 것이다.

이를 위해 세탁지수에 영향을 주는 빨랫감의 종류와 오염량, 급수량, 세제종류, 투입량 등을 측정한 데이터를 확보하였다.

데이터의 변수명은 보안을 위해 cat0, cont0등으로 코드화 되어 있다.

아래 데이터를 사용하여 과제를 해결하고 세탁지수(target)를 예측하시오.

데이터 구성

학습데이터: train_prob.csv, 240,000 rows, 26 columns

테스트데이터: test_prob.csv, 60,000 rows, 25 columns

테스트정답셋: test_prob_ans.csv, 60,000 rows, 1 columns

변수명	설명	타입
Id	실험 고유 번호	정수형
cat0 ~ cat9	범주형 변수 0~9	문자형
cont0 ~ cont13	연속형 변수 0~13	실수형
target	세탁지수	실수형

전처리(Preprocessing)

train_prob.csv 데이터셋을 대상으로 다음 전처리를 진행하라.

```
In [16]: df_train = pd.read_csv('train_prob.csv', index_col='id')
display(df_train.info())
df_train.head()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 240000 entries, 267387 to 415856
Data columns (total 25 columns):
cat0      240000 non-null object
cat1      240000 non-null object
cat2      240000 non-null object
cat3      240000 non-null object
cat4      240000 non-null object
cat5      240000 non-null object
cat6      240000 non-null object
cat7      240000 non-null object
cat8      240000 non-null object
cat9      240000 non-null object
cont0     240000 non-null float64
cont1     240000 non-null float64
cont2     240000 non-null float64
cont3     240000 non-null float64
cont4     240000 non-null float64
cont5     240000 non-null float64
cont6     240000 non-null float64
cont7     240000 non-null float64
cont8     240000 non-null float64
cont9     240000 non-null float64
cont10    240000 non-null float64
cont11    240000 non-null float64
cont12    240000 non-null float64
cont13    240000 non-null float64
target    240000 non-null float64
dtypes: float64(15), object(10)
memory usage: 47.6+ MB
None
```

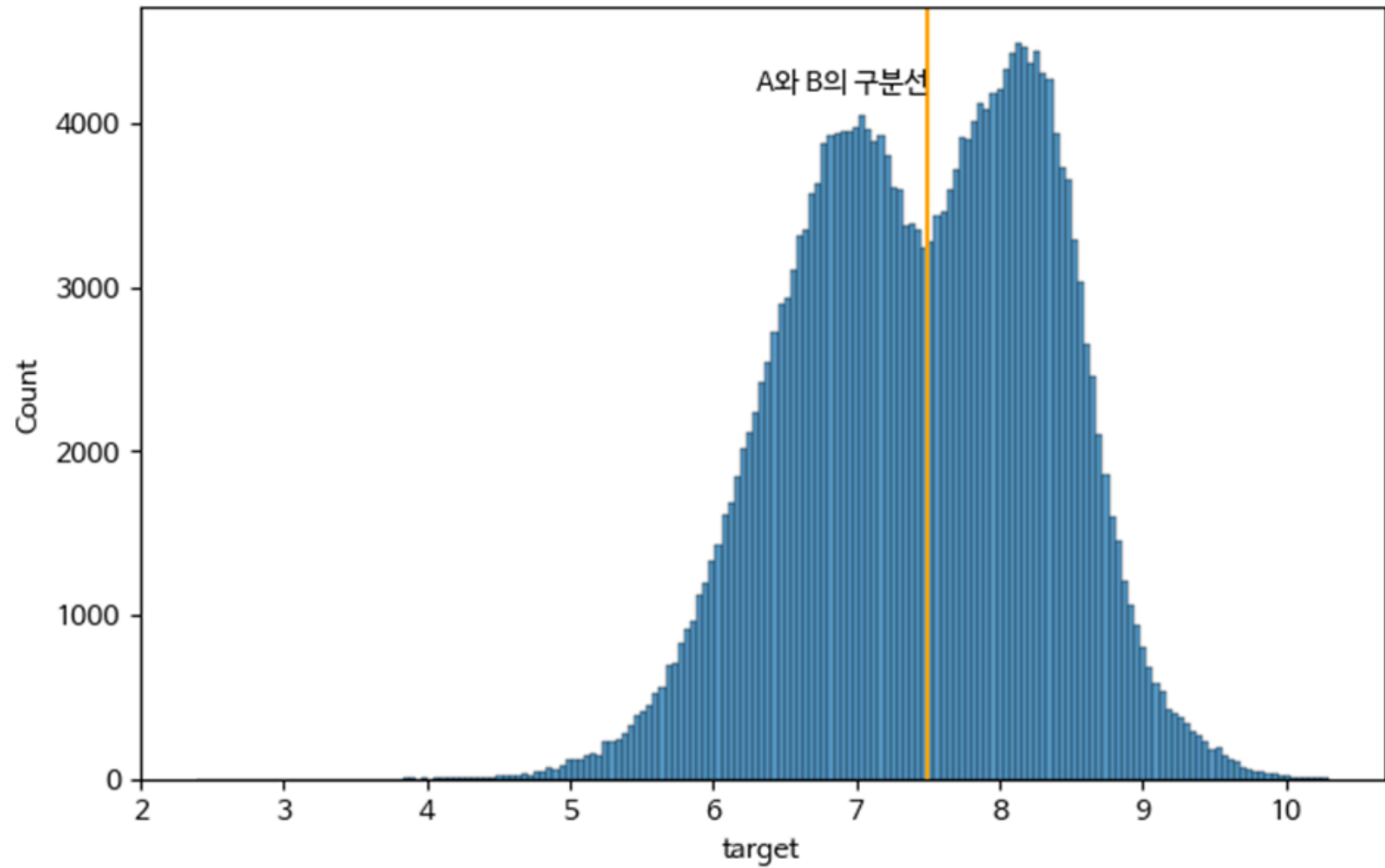
Out[16]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont5	cont6	cont7	cont8	cont9	cont10	cont11	cont12	cont13	1
id																					
267387	A	B	B	C	B	B	A	E	G	G	...	0.22443	0.69220	0.30912	0.47134	0.53290	0.97139	0.59414	0.36856	0.42951	8.
410037	A	A	A	A	B	B	A	E	C	F	...	0.43278	0.97199	0.62596	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.
139373	A	A	B	A	B	B	A	E	C	J	...	0.36238	0.41310	0.27010	0.40112	0.53052	0.35635	0.33341	0.33354	0.82542	7.
113765	A	A	A	C	B	B	A	E	C	G	...	0.32670	0.42301	0.28331	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.
179915	A	B	A	C	B	B	A	E	E	L	...	0.20121	0.37466	0.46316	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.

5 rows x 25 columns

단계 1

target은 2개의 정규 분포가 합쳐진 모양을 하고 있다.



두 분포의 구분점 7.45를 기준으로 target값이 작거나 같으면 A, 초과하면 B로 나눈다.

A 범주이면 True, B이면 False인 파생 변수 target A를 만든다.

```
In [17]: df_train['target']
```

```
Out[17]: id
267387    8.13741
410037    7.40427
139373    7.98022
113765    7.26085
179915    7.26448
...
320975    7.60537
29591     8.10343
46717     6.93285
463191    6.64854
415856    7.43852
Name: target, Length: 240000, dtype: float64
```

```
In [18]: df_train['targetA'] = df_train['target'] <= 7.45
df_train['targetA']
```

```
Out[18]: id
267387    False
410037     True
139373    False
113765     True
179915     True
...
320975    False
29591     False
46717     True
463191     True
415856     True
Name: targetA, Length: 240000, dtype: bool
```

```
In [19]: x_col = ['cat{}'.format(i) for i in range(3,10)]
df_train[x_col]
```

Out[19]:

	cat3	cat4	cat5	cat6	cat7	cat8	cat9
id							
267387	C	B	B	A	E	G	G
410037	A	B	B	A	E	C	F
139373	A	B	B	A	E	C	J
113765	C	B	B	A	E	C	G
179915	C	B	B	A	E	E	L
...	...	...	...	...	...	...	...
320975	C	B	B	A	E	C	I
29591	C	B	B	A	E	C	F
46717	C	B	D	A	E	G	F
463191	C	B	D	A	E	E	F
415856	C	B	D	A	E	G	I

240000 rows × 7 columns

```
In [20]: for i in range(3, 10) :
          col = 'cat{}'.format(i)
          print(df_train[col].value_counts(normalize = True))
```

```
C    0.611917
A    0.348475
D    0.037521
B    0.002087
Name: cat3, dtype: float64
B    0.991179
A    0.004292
C    0.002512
D    0.002017
Name: cat4, dtype: float64
B    0.496879
D    0.451021
C    0.039179
A    0.012921
Name: cat5, dtype: float64
A    0.975263
B    0.021346
C    0.002708
D    0.000492
I    0.000092
E    0.000058
H    0.000033
G    0.000008
Name: cat6, dtype: float64
E    0.891779
D    0.081492
B    0.019117
G    0.006596
F    0.000942
A    0.000042
C    0.000021
I    0.000013
Name: cat7, dtype: float64
C    0.403096
E    0.315167
G    0.141121
A    0.126408
D    0.012304
F    0.001854
B    0.000050
Name: cat8, dtype: float64
F    0.358100
```

```
I    0.166958
L    0.140558
H    0.082596
K    0.069763
A    0.044492
G    0.034667
M    0.032579
J    0.022921
O    0.020700
N    0.013879
B    0.011513
C    0.000683
D    0.000346
E    0.000246
Name: cat9, dtype: float64
```

```
In [21]: Image('3-3.png')
```

Out[21]:

단계 2

cat3 ~ 9 각각의 범주 중에서 빈도가 적은 범주를 target을 예측하는데 영향이 없는 방향으로 정리하려고 한다.

아래 가이드에 맞추어 cat3 ~ 9 에 대한 전처리를 수행한다.

- cat3는 B→C 로 치환한다.

Hint] 치환 후의 빈도 테이블

A	C	D
83,634	147,361	9,005

- cat4는 A→B, D→B 로 치환한다.

Hint] 치환 후의 빈도 테이블

B	C
239,397	603

- cat6는 D→A, E→B, G→C, H→B, I→A로 치환한다.

Hint] 치환 후의 빈도 테이블

A	B	C
234,203	5,145	652

- cat7는 A→B, C→B, F→D, I→B로 치환한다.

Hint] 치환 후의 빈도 테이블

B	D	E	G
4,606	19,784	214,027	1,583

- cat8는 B→G, F→E로 치환한다.

Hint] 치환 후의 빈도 테이블

A	C	D	E	G
30,338	96,743	2,953	76,085	33,881

- cat9는 C→H, D→B, E→L로 치환한다.

Hint] 치환 후의 빈도 테이블

A	B	F	G	H	I	J	K	L	M	N	O
10,678	2,846	85,944	8,320	19,987	40,070	5,501	16,743	33,793	7,819	3,331	4,968

범주형 변수들(cat0~9)을 **cat_cols**, 연속형 변수(cont0~13)들을 **cont_cols** 로 정의합니다.

전처리 과정을 마친 데이터프레임을 **basetable**이라고 합니다.

In [22]:

```
'''
`pandas.replace()` 함수는 DataFrame 또는 Series에서 특정 값을 다른 값으로 바꾸는 데 사용됩니다.
이 함수는 다양한 형태의 값 교체 작업을 지원하며, 단일 값, 리스트, 사전, 정규 표현식 등 여러 가지를 사용할 수 있습니다.

### 함수 시그니처
DataFrame.replace(to_replace=None, value=None, *, inplace=False, limit=None, regex=False, method='pad')

### 주요 인자
- to_replace: 바꿀 값 또는 값들의 집합입니다.
  - 단일 값: 특정 값 하나를 지정하여 변경합니다.
  - 리스트: 여러 값을 리스트로 지정하여 일괄 변경합니다.
  - 사전: {old_value: new_value} 형태의 사전을 사용하여 여러 값을 한번에 변경할 수 있습니다.
- value: 새롭게 대체할 값입니다. `to_replace`가 단일 값, 리스트, 사전일 때 각각에 대해 대응되는 새로운 값을 지정합니다.
  - 단일 값: `to_replace`에서 지정한 값을 모두 이 값으로 바꿉니다.
  - 리스트: `to_replace`가 리스트일 때, 각 값에 대응하는 새로운 값을 리스트로 지정합니다.
  - 사전: `to_replace`가 사전일 때, 각 key에 대응하는 값을 지정합니다.
- inplace: 불리언 값으로, True로 설정하면 원본 DataFrame 또는 Series를 직접 변경하고, False이면 변경된 새로운 객체를 반환합니다. 기본값은 False입니다.
- limit: 변경할 최대 횟수를 지정합니다. None이면 제한 없이 모든 값을 변경합니다.
- regex: 불리언 값 또는 사전으로, 정규 표현식을 사용할지를 지정합니다.
  True로 설정하면 `to_replace` 값을 정규 표현식으로 간주합니다.
- method: 결측치 채우기 방식으로, {'pad', 'ffill', 'bfill', 'nearest'} 중 하나를 선택할 수 있습니다.
  결측치를 앞이나 뒤 또는 가까운 값으로 채웁니다.
```

반환값

- 변경된 값을 반영한 새로운 DataFrame 또는 Series를 반환합니다.
`inplace=True`로 설정한 경우에는 None을 반환합니다.

사용 예시

1. 단일 값 변경

```
df = pd.DataFrame({'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]})
df.replace(1, 100)
```

2. 여러 값 변경

```
df.replace([1, 2], [100, 200])
```

3. 사전을 사용한 변경

```
df.replace({1: 100, 2: 200})
```

4. 정규 표현식 사용

```
df = pd.DataFrame({'A': ['foo', 'fuz', 'bar'], 'B': ['baz', 'qux', 'quux']})
df.replace(to_replace=r'^f.*', value='prefix', regex=True)
```

5. inplace 사용

```
df.replace(1, 100, inplace=True)
```

6. 제한된 횟수 변경

```
df.replace(1, 100, limit=1)
```

이 함수는 데이터 클리닝 작업에서 매우 유용하게 사용할 수 있으며, 다양한 데이터 변환 작업에 유연하게 적용할 수 있습니다.

```
print('')
```

```
In [23]: df = pd.DataFrame({'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]})
print(df.replace(1, 100))
print('-----')

print(df.replace([1, 2], [100, 200]))
print('-----')

print(df.replace({2: 100, 3: 200}))
print('-----')
```

```
df = pd.DataFrame({'A': ['foo', 'fuz', 'bar'], 'B': ['baz', 'qux', 'quux']})
print(df.replace(to_replace=r'^f.*', value='prefix', regex=True))
```

```

      A  B
0  100  5
1    2  6
2    3  7
3    4  8
-----
      A  B
0  100  5
1  200  6
2    3  7
3    4  8
-----
      A  B
0    1  5
1  100  6
2  200  7
3    4  8
-----
      A  B
0  prefix  baz
1  prefix  qux
2    bar  quux

```

In [24]:

```
'''
`pandas.all()` 함수는 Pandas 라이브러리에서 DataFrame 또는 Series의 모든 요소가 True인지 확인하는 데 사용됩니다.
이 함수는 주로 Boolean 데이터나 조건식을 평가할 때 사용됩니다.

### 기본 사용법
#### Series에서의 사용
Series에서 `all()`을 호출하면 모든 요소가 True인지 검사합니다.
# 예제 Series
s = pd.Series([True, True, False])

# 모든 요소가 True인지 검사
result = s.all()
print(result) # False

#### DataFrame에서의 사용
DataFrame에서 `all()`을 호출하면 기본적으로 각 열(column)에 대해 모든 요소가 True인지 검사합니다.
'''
```

```
# 예제 DataFrame
df = pd.DataFrame({
    'A': [True, True, True],
    'B': [True, False, True]
})

# 각 열에 대해 모든 요소가 True인지 검사
result = df.all()
print(result)
출력:
A      True
B     False
dtype: bool

### 매개 변수
`pandas.all()` 함수는 다음과 같은 매개변수를 가집니다:
- axis: {0 or 'index', 1 or 'columns'}, 기본값 0
  - 0 또는 'index': 각 열(column)에 대해 평가
  - 1 또는 'columns': 각 행(row)에 대해 평가
- bool_only: bool, 기본값 None
  - Boolean 열만 고려할지 여부를 지정합니다. 기본값은 None으로, 이는 모든 데이터 유형을 고려한다는 의미입니다.
- skipna: bool, 기본값 True
  - NA/null 값은 무시하고 계산할지 여부를 지정합니다. True일 경우 NA 값은 무시됩니다.
- level: int 또는 레이블, 기본값 None
  - 지정된 레벨에서만 동작하도록 멀티 인덱스 (hierarchical index)에 사용할 수 있습니다.

### 예제
#### 특정 축(axis)에 대해 평가
# 예제 DataFrame
df = pd.DataFrame({
    'A': [True, True, True],
    'B': [True, False, True]
})

# 각 행(row)에 대해 모든 요소가 True인지 검사
result = df.all(axis=1)
print(result)
출력:
0      True
1     False
2      True
dtype: bool
```

```
#### NA 값 무시
# 예제 Series
s = pd.Series([True, np.nan, True])
# NA 값을 무시하고 검사
result = s.all(skipna=True)
print(result) # True
# NA 값을 고려하여 검사
result = s.all(skipna=False)
print(result) # nan
```

이러한 방식으로 `pandas.all()` 함수는 Series나 DataFrame의 모든 요소가 True인지 간단하게 확인할 수 있는 유용한 도구입니다.

```
'''
print('')
```

In [25]:

```
'''
코드는 `repl_list`를 순회하면서, 각 카테고리 열(`catX`)에 대해 특정 값들을
다른 값으로 치환하고, 치환된 결과가 예상된 횟수와 일치하는지 확인하는 작업을 수행합니다.
```

1. 데이터 설명

- `repl\_list`: 튜플로 구성된 리스트. 각 튜플은 세 개의 요소로 이루어져 있습니다.
 - 첫 번째 요소(`c`): 카테고리 열 이름 (`catX` 형식의 문자열).
 - 두 번째 요소(`d`): 치환할 값들의 딕셔너리.
 - 예를 들어, `{ 'B': 'C' }`는 'B'를 'C'로 치환한다는 의미입니다.
 - 세 번째 요소(`cnt`): 치환 후 각 값들의 예상 빈도수를 나타내는 리스트.

2. 코드 동작 단계

- `repl\_list`를 순회하면서 각 튜플의 요소들을 변수 `c`, `d`, `cnt`에 할당하고 출력합니다.
- `df\_train` 데이터프레임의 `c` 열에 대해 `d` 딕셔너리에 정의된 값들을 치환합니다.
 - 예를 들어, `d`가 `{ 'B': 'C' }`인 경우 `df\_train[c]`의 'B' 값을 'C'로 치환합니다.
- 치환된 열 `s\_repl`의 고유값 개수가 `cnt`의 길이와 일치하는지 확인합니다.
 - 일치하지 않으면 오류를 출력합니다.
- `s\_repl.value\_counts().sort\_index() == cnt`은 치환된 열의 값별 빈도수가 `cnt` 리스트와 일치하는지 확인하는 조건입니다.
 - `sort\_index()`를 사용하여 값들을 정렬한 후 비교합니다.
- 위 조건이 만족되지 않으면 오류 메시지와 함께 `s\_repl`의 값별 빈도수를 출력합니다.
- 오류가 발생하면 반복을 중단합니다.
- 치환된 열 `s\_repl`를 원래 데이터프레임의 열 `df\_train[c]`에 다시 할당합니다.

전체 흐름 정리

1. `repl\_list`의 각 요소를 순회하면서 각 카테고리 열(`catX`)에 대해 치환 작업을 수행합니다.
2. 치환된 값의 빈도수가 예상 빈도수와 일치하는지 확인합니다.

3. 일치하지 않으면 오류 메시지를 출력하고 반복을 중단합니다.

4. 일치하면 치환된 값을 데이터프레임에 다시 저장합니다.

코드는 데이터프레임의 특정 열에 대해 지정된 값 치환 작업을 수행하고, 그 결과가 예상된 빈도수와 일치하는지 검증하는 기능을 합니다.

#cat3는 B→C 로 치환한다.

A, C,D

83634, 147361, 9005

```
repl_list = [
    ('cat3', {'B': 'C'}, [83634, 147361, 9005]),
    ('cat4', {'A': 'B', 'D': 'B'}, [239397, 603]),
    ('cat6', {'D': 'A', 'E': 'B', 'G': 'C', 'H': 'B', 'I': 'A'}, [234203, 5145, 652]),
    ('cat7', {'A': 'B', 'C': 'B', 'F': 'D', 'I': 'B'}, [4606, 19784, 214027, 1583]),
    ('cat8', {'B': 'G', 'F': 'E'}, [30338, 96743, 2953, 76085, 33881]),
    ('cat9', {'C': 'H', 'D': 'B', 'E': 'L'}, [10678, 2846, 85944, 8320, 19987, 40070, 5501, 16743, 33793, 7819, 3331, 4968])
]
```

```
print(type(repl_list[0]))
```

```
for c, d, cnt in repl_list:
    print(c, d, cnt)
    s_repl = df_train[c].replace(d)
    if s_repl.nunique() != len(cnt) or not (s_repl.value_counts().sort_index() == cnt).all():
        print("error", s_repl.value_counts())
        break
    df_train[c] = s_repl
```

```
<class 'tuple'>
```

```
cat3 {'B': 'C'} [83634, 147361, 9005]
```

```
cat4 {'A': 'B', 'D': 'B'} [239397, 603]
```

```
cat6 {'D': 'A', 'E': 'B', 'G': 'C', 'H': 'B', 'I': 'A'} [234203, 5145, 652]
```

```
cat7 {'A': 'B', 'C': 'B', 'F': 'D', 'I': 'B'} [4606, 19784, 214027, 1583]
```

```
cat8 {'B': 'G', 'F': 'E'} [30338, 96743, 2953, 76085, 33881]
```

```
cat9 {'C': 'H', 'D': 'B', 'E': 'L'} [10678, 2846, 85944, 8320, 19987, 40070, 5501, 16743, 33793, 7819, 3331, 4968]
```

```
In [26]: cat_cols = ['cat{}'.format(i) for i in range(10)]
print('cat_cols = ', cat_cols)
```

```
cont_cols = ['cont{}'.format(i) for i in range(14)]
print('cont_cols = ', cont_cols)
```

```
df_basetable = df_train.copy()
df_basetable

cat_cols = ['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8', 'cat9']
cont_cols = ['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13']
```

Out[26]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont6	cont7	cont8	cont9	cont10	cont11	cont12	cont13	target	ta
id																					
267387	A	B	B	C	B	B	A	E	G	G	...	0.69220	0.30912	0.47134	0.53290	0.97139	0.59414	0.36856	0.42951	8.13741	
410037	A	A	A	A	B	B	A	E	C	F	...	0.97199	0.62596	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.40427	
139373	A	A	B	A	B	B	A	E	C	J	...	0.41310	0.27010	0.40112	0.53052	0.35635	0.33341	0.33354	0.82542	7.98022	
113765	A	A	A	C	B	B	A	E	C	G	...	0.42301	0.28331	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.26085	
179915	A	B	A	C	B	B	A	E	E	L	...	0.37466	0.46316	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.26448	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
320975	A	A	A	C	B	B	A	E	C	I	...	0.40968	0.62906	0.31427	0.43718	0.40494	0.69718	0.34519	0.67524	7.60537	
29591	A	B	A	C	B	B	A	E	C	F	...	0.74914	0.82530	0.46395	0.83396	0.71724	0.69450	0.33008	0.28460	8.10343	
46717	A	B	B	C	B	D	A	E	G	F	...	0.47447	0.79777	0.87541	0.63918	0.58994	0.77751	0.86388	0.74568	6.93285	
463191	A	A	A	C	B	D	A	E	E	F	...	0.21360	0.22158	0.06595	0.61991	0.62167	0.25545	0.33329	0.28366	6.64854	
415856	A	A	A	C	B	D	A	E	G	I	...	0.83486	0.48463	0.90318	0.81729	0.57672	0.46053	0.91726	0.25874	7.43852	

240000 rows × 26 columns



던(Dunn) 지수는 군집화(clustering) 품질을 평가하는 데 사용되는 지표 중 하나입니다.

이 지수는 군집 간 분리도를 최대화하고 군집 내 응집도를 최소화하려고 합니다.

Dunn 지수는 다음과 같은 개념을 기반으로 합니다:

- 1. **군집 간 분리도 (Inter-cluster distance):** 서로 다른 군집 간의 거리입니다. 이는 군집 간의 최소 거리(minimum distance)를 의미합니다.
- 2. **군집 내 응집도 (Intra-cluster distance):** 같은 군집 내 데이터 포인트들 간의 거리입니다. 이는 군집 내의 최대 거리(maximum distance)를 의미합니다.

Dunn 지수 정의

던 지수는 다음과 같이 정의됩니다:

$$D = \frac{\min_{1 \leq i < j \leq k} \delta(C_i, C_j)}{\max_{1 \leq l \leq k} \Delta(C_l)}$$

여기서, 여기서,

- $(\delta(C_i, C_j))$: 군집 (C_i) 와 군집 (C_j) 간의 거리 (보통 최소 거리로 정의됨)
- $(\Delta(C_l))$: 군집 (C_l) 내의 거리 (보통 최대 거리로 정의됨)
- (k) : 군집의 수

계산 방법

1. 군집 간 최소 거리 ($\delta(C_i, C_j)$): 각 군집 쌍에 대해 모든 데이터 포인트 간의 거리를 계산하고, 그 중 최소 거리를 선택합니다.
2. 군집 내 최대 거리 ($\Delta(C_l)$): 각 군집에 대해 모든 데이터 포인트 간의 거리를 계산하고, 그 중 최대 거리를 선택합니다.
3. Dunn 지수 계산: 위의 공식을 사용하여 Dunn 지수를 계산합니다.

특성

- Dunn 지수가 클수록 군집화의 품질이 높다고 간주됩니다.
- 높은 Dunn 지수는 군집 간의 분리도가 크고, 군집 내 응집도가 낮음을 의미합니다.

예제

다음은 Python을 사용하여 Dunn 지수를 계산하는 간단한 예제입니다:

```
import numpy as np
from sklearn.metrics import pairwise_distances

def dunn_index(X, labels):
    # 데이터 포인트 간의 거리 행렬 계산
    distances = pairwise_distances(X)
```

```

# 군집 식별
clusters = np.unique(labels)

# 군집 내 최대 거리 계산
intra_dists = []
for cluster in clusters:
    cluster_points = X[labels == cluster]
    if len(cluster_points) > 1:
        intra_dist = np.max(pairwise_distances(cluster_points))
    else:
        intra_dist = 0
    intra_dists.append(intra_dist)
max_intra_dist = np.max(intra_dists)

# 군집 간 최소 거리 계산
inter_dists = []
for i in range(len(clusters)):
    for j in range(i + 1, len(clusters)):
        cluster_i_points = X[labels == clusters[i]]
        cluster_j_points = X[labels == clusters[j]]
        inter_dist = np.min(pairwise_distances(cluster_i_points, cluster_j_points))
        inter_dists.append(inter_dist)
min_inter_dist = np.min(inter_dists)

# Dunn 지수 계산
dunn_index = min_inter_dist / max_intra_dist
return dunn_index

# 예제 데이터
from sklearn.datasets import make_blobs

X, labels = make_blobs(n_samples=150, centers=3, random_state=42)

# Dunn 지수 계산
dunn = dunn_index(X, labels)
print(f'Dunn Index: {dunn}')

```

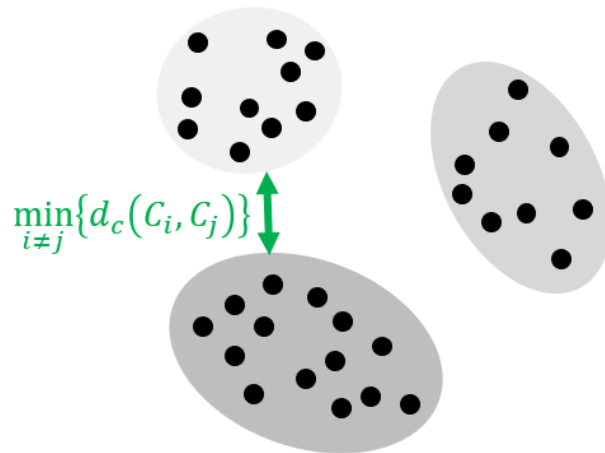
이 예제에서는 `pairwise_distances` 함수를 사용하여 데이터 포인트 간의 거리를 계산하고, Dunn 지수를 계산합니다.

이 방법을 통해 군집화의 품질을 평가할 수 있습니다.

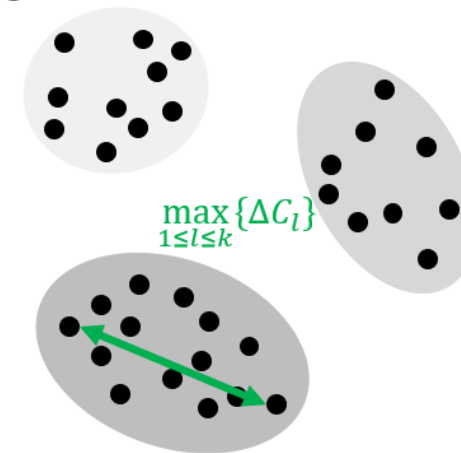
TIP!: Dunn Index 구하기

$$I(C) = \frac{\min_{i \neq j} \{d_c(C_i, C_j)\}}{\max_{1 \leq l \leq k} \{\Delta C_l\}}$$

Minimize
Smallest distance between Clusters



Maximize
Largest distance within a cluster



```
In [27]: # 예시를 들기에는 데이터셋이 커서 1%만 뽑겠습니다.
df_dunn = df_train.sample(frac=0.01, random_state=123)
len(df_dunn)
```

Out[27]: 2400

Question

cat5는 cont0~13 연속형 변수에 대한 군집화의 결과 생긴

데이터 포인트의 소속 군집이라고 가정합니다.

df_dunn에서 cat5 군집에 대한 Dunn-Index로 측정하세요.

Python 함수가이드

```
from scipy.spatial.distance import pdist, cdist, squareform
```

In [28]: `Image('4.png')`

Out[28]: 쌍별 거리(pairwise distance)는 데이터셋 내의 모든 데이터 포인트 쌍 간의 거리를 의미합니다. 이는 데이터 분석과 머신 러닝에서 자주 사용되는 개념으로, 데이터 포인트 간의 유사성 또는 차이를 정량적으로 측정하는 데 사용됩니다.

예시

만약 세 개의 데이터 포인트 (A, B, C) 가 있다면, 쌍별 거리는 다음과 같습니다:

- $d(A, B)$: 데이터 포인트 A 와 B 간의 거리
- $d(A, C)$: 데이터 포인트 A 와 C 간의 거리
- $d(B, C)$: 데이터 포인트 B 와 C 간의 거리

즉, N 개의 데이터 포인트가 있는 경우, 총 $\frac{N(N-1)}{2}$ 개의 쌍별 거리를 계산하게 됩니다.

거리 계산 방법

쌍별 거리를 계산하는 방법에는 여러 가지가 있으며, 가장 일반적인 방법은 유클리드 거리입니다. 그러나 다른 거리 메트릭도 사용될 수 있습니다:

- **유클리드 거리 (Euclidean distance)**: 두 점 사이의 직선 거리를 계산합니다.

$$d(A, B) = \sqrt{(A_1 - B_1)^2 + (A_2 - B_2)^2 + \cdots + (A_n - B_n)^2}$$

- **맨해튼 거리 (Manhattan distance)**: 각 축을 따라 이동하는 거리를 합산합니다.

$$d(A, B) = |A_1 - B_1| + |A_2 - B_2| + \cdots + |A_n - B_n|$$

- **코사인 유사도 (Cosine similarity):** 벡터의 방향을 기준으로 두 벡터 간의 코사인 값을 계산합니다.

$$d(A, B) = 1 - \frac{A \cdot B}{\|A\| \|B\|}$$

- **자카드 유사도 (Jaccard similarity):** 집합 간의 교집합을 기준으로 유사도를 측정합니다.

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

In [29]: `Image('5.png')`

Out[29]:

예제 코드 설명

`pdist`와 `squareform` 함수를 사용하는 예제 코드에서는 유클리드 거리를 기본으로 사용합니다.

python

 코드 복사

```
from scipy.spatial.distance import pdist, squareform

# df_dunn에서 연속형 특성 선택
continuous_features = df_dunn[cont_cols]

# 쌍별 거리 계산 (기본적으로 유클리드 거리)
pairwise_distances = pdist(continuous_features)

# 거리 행렬을 정방형 대칭 행렬로 변환
distance_matrix = squareform(pairwise_distances)
```

`pdist` 함수

- `pdist(continuous_features)`: `df_dunn`의 연속형 특성들에 대해 쌍별 유클리드 거리를 계산합니다.
 - 결과는 길이 $\frac{N(N-1)}{2}$ 의 1차원 배열입니다. 여기서 N은 샘플의 수입니다.

`squareform` 함수

- `squareform(pairwise_distances)`: `pdist`가 반환한 1차원 배열을 NxN 형태의 정방형 대칭

앵달도 먼완입니다.

- 이 행렬의 (i, j) 요소는 i 번째 샘플과 j 번째 샘플 간의 거리를 나타냅니다.

요약

쌍별 거리는 데이터셋 내의 모든 데이터 포인트 쌍 간의 거리를 의미하며, 다양한 거리 메트릭을 사용하여 계산할 수 있습니다. ``pdist``와 ``squareform`` 함수를 사용하면, 데이터셋의 쌍별 거리를 계산하고 이를 정방형 대칭 행렬로 변환할 수 있습니다. 이 행렬은 각 데이터 포인트 간의 유사성 또는 차이를 정량적으로 분석하는 데 유용합니다.

In [30]:

```
'''
`pdist()` 함수는 SciPy 라이브러리의 `scipy.spatial.distance` 모듈에 속해 있으며,
여러 관측값 사이의 쌍별 거리를 계산하는 데 사용됩니다.
`pdist()`는 입력으로 주어진  $n \times m$  크기의 배열에서  $n$ 개의 관측값에 대해
모든 쌍의 거리를 계산하고, 결과를 압축된 형식으로 반환합니다. 이 함수는 다양한 거리 메트릭을 지원합니다.

## pdist() 함수의 인자
1. X (array_like):
    -  $n \times m$  크기의 입력 배열입니다. 여기서  $n$ 은 관측값의 수이고,  $m$ 은 각 관측값의 차원입니다.

2. metric (str or function, optional):
    - 거리를 계산할 메트릭을 지정합니다. 기본값은 'euclidean' (유클리드 거리)입니다.
    - 문자열로 메트릭을 지정할 수 있으며, 지원되는 메트릭에는 'euclidean', 'cityblock' (맨해튼 거리),
      'cosine', 'hamming' 등 여러 가지가 있습니다.
    - 사용자 정의 함수도 사용할 수 있습니다. 이 경우 함수는 두 개의 1차원 배열을 입력으로 받아 거리를 계산하고
      하나의 스칼라 값을 반환해야 합니다.

3. Other Parameters:
    - `p`: Minkowski 거리 메트릭을 사용하는 경우, 이 인자는  $p$ -노름의 차수를 나타냅니다.
      예를 들어, `metric='minkowski'`와 `p=3`를 지정하면 3-노름을 사용하여 거리를 계산합니다.

## pdist() 함수의 반환값
- Y (ndarray):
    -  $n \times n$  거리 행렬의 상삼각 부분을 1차원 배열로 평탄화한 배열입니다. 길이는  $n(n-1)/2$ 입니다.
    - 이 배열은 `scipy.spatial.distance.squareform()` 함수를 사용하여 거리 행렬로 변환할 수 있습니다.
```

```
'''
# 아래 코드는 세 개의 2차원 데이터 포인트 간의 유클리드 거리를 계산하고, 그 결과를 거리 행렬 형태로 출력합니다.

import numpy as np
from scipy.spatial.distance import pdist, squareform

# 예제 데이터
data = np.array([[0, 1], [1, 0], [2, 2]])

# 유클리드 거리 계산
distances = pdist(data, metric='euclidean')
print('distances = ', distances)

distance_matrix = squareform(distances)
print('distance_matrix')
print('-----')
```

```
distance_matrix
distances = [1.41421356 2.23606798 2.23606798]
distance_matrix
-----
```

```
Out[30]: array([[0.          , 1.41421356, 2.23606798],
               [1.41421356, 0.          , 2.23606798],
               [2.23606798, 2.23606798, 0.          ]])
```

```
In [31]: # 샘플 데이터 (4개의 2차원 포인트)
X = np.array([[0, 1], [1, 0], [2, 0], [3, 4]])

# 유클리드 거리 계산
Y = pdist(X, metric='euclidean')

# 거리 벡터 출력
print("거리 벡터:", Y)

# 거리 행렬로 변환
distance_matrix = squareform(Y)
print("거리 행렬:\n", distance_matrix)
```

```

거리 벡터: [1.41421356 2.23606798 4.24264069 1.
거리 행렬:
[[0.          1.41421356 2.23606798 4.24264069]
 [1.41421356 0.          1.          4.47213595]
 [2.23606798 1.          0.          4.12310563]
 [4.24264069 4.47213595 4.12310563 0.          ]]

```

```

4.47213595 4.12310563]

```

```

In [32]: import numpy as np
from scipy.spatial.distance import pdist

# 샘플 데이터 (4개의 2차원 포인트)
X = np.array([[0, 1], [1, 0], [2, 0], [3, 4]])

# 사용자 정의 거리 함수 (예: 두 벡터의 원소 차이의 절대값 합)
def custom_metric(u, v):
    return np.sum(np.abs(u - v))

# 사용자 정의 거리 함수로 거리 계산
Y = pdist(X, metric=custom_metric)

# 거리 벡터 출력
print("사용자 정의 거리 벡터:", Y)

```

사용자 정의 거리 벡터: [2. 3. 6. 1. 6. 5.]

```

In [33]: '''
코드는 `df_dunn` 데이터프레임의 연속형 특성(`cont_cols`)에 대한 쌍별 거리 행렬을 계산하고, 이를 정방행렬 형태로 변환합니다.

* 쌍별 거리(pairwise distance)는 데이터 분석 및 기계 학습에서 각 데이터 포인트 간의 거리를 계산하는 방법을 의미합니다.
이를 통해 데이터의 구조와 유사성을 파악할 수 있습니다.
쌍별 거리 계산은 특히 군집화(clustering), 최근접 이웃 탐색(knn), 다차원 척도법(MDS) 등 여러 알고리즘에서 중요하게 사용됩니다.

# 1. 데이터 샘플링
`df_train` 데이터프레임에서 1% 샘플링하여 `df_dunn` 데이터프레임을 생성합니다.
- `frac=0.01`: `df_train` 데이터프레임의 1%를 무작위로 샘플링합니다.
- `random_state=123`: 무작위 샘플링의 재현성을 보장하기 위해 시드를 설정합니다.

# 2. 쌍별 거리 계산
`df_dunn`의 연속형 특성에 대해 쌍별 거리를 계산합니다.
## 내부 동작 설명
- `df_dunn[cont_cols]`: `df_dunn` 데이터프레임에서 연속형 특성만 선택합니다.
  `cont_cols`는 연속형 특성의 컬럼 이름 리스트입니다.

```

```

- `pdist(df_dunn[cont_cols])`: 선택된 연속형 특성에 대해 각 데이터 포인트 간의 쌍별 거리를 계산합니다.
  `pdist`는 압축된 거리 행렬을 반환합니다. 기본적으로 유클리드 거리를 계산하지만, 다른 거리 메트릭을 사용할 수도 있습니다.
- 예를 들어, `pdist(df_dunn[cont_cols], metric='euclidean')`는 유클리드 거리를 계산합니다.
## 거리 행렬 변환
- `squareform(pdist(...))`: `pdist`가 반환한 압축된 거리 행렬을 정방형 거리 행렬로 변환합니다.
  - `squareform` 함수는 압축된 거리 벡터를 정방형 대칭 행렬로 변환합니다. 결과는 NxN 형태의 행렬로, 여기서 N은 샘플 수입니다.

# 전체 과정 요약
1. `df_train` 데이터프레임에서 1% 샘플링하여 `df_dunn` 데이터프레임을 생성합니다.
2. `df_dunn`의 연속형 특성에 대해 쌍별 거리를 계산합니다.
3. 쌍별 거리 행렬을 정방형 대칭 행렬로 변환합니다.

결과적으로, `squareform(pdist(df_dunn[cont_cols]))`은 `df_dunn` 데이터프레임의
연속형 특성 간의 쌍별 거리 정보를 포함하는 NxN 형태의 거리 행렬을 반환합니다.
이 행렬의 (i, j) 요소는 i번째 샘플과 j번째 샘플 간의 거리를 나타냅니다.
...

from scipy.spatial.distance import pdist, cdist, squareform

# pdist는 주어진 행렬의 거리 행렬을 구합니다.
# 대칭 행렬인 거리 행렬을 효율적으로 담기 위해
# 삼각 영역의 값을 1차원 배열로 저장합니다.
# 효율화된 1차원 배열의 거리 행렬 -> 행렬 형태로,
# 행렬 형태는 -> 효율화된 1차원 배열의 거리 행렬 로 바꾸어줍니다.

squareform(pdist(df_dunn[cont_cols]))

```

```

Out[33]: array([[0.          , 1.66286356, 1.67872855, ..., 1.61131483, 1.74132879,
0.75583433],
[1.66286356, 0.          , 1.22627566, ..., 1.33120631, 0.85189025,
1.71017092],
[1.67872855, 1.22627566, 0.          , ..., 0.9789981 , 1.17031726,
1.48386611],
...,
[1.61131483, 1.33120631, 0.9789981 , ..., 0.          , 1.13333429,
1.42148633],
[1.74132879, 0.85189025, 1.17031726, ..., 1.13333429, 0.          ,
1.79540053],
[0.75583433, 1.71017092, 1.48386611, ..., 1.42148633, 1.79540053,
0.          ]])

```

```

In [34]: df_dunn.groupby('cat5')['cont0'].apply(lambda x: print(x))

```

```
id
366778    0.68308
255453    0.36268
300436    0.52435
153248    0.76304
37546     0.22057
278208    0.47434
290192    0.49814
317072    0.50376
279773    0.47409
67084     0.61741
408658    0.34193
159028    0.13903
72799     0.46590
172901    0.68643
377501    0.48276
368007    0.71123
385324    0.62845
207529    0.73967
148602    0.47553
190241    0.28887
479956    0.30776
289047    0.33306
305850    0.72816
378947    0.49299
321550    0.42562
266644    0.68860
61026     0.65493
371642    0.47764
481135    0.34094
144388    0.73686
235634    0.46695
253602    0.65600
480253    0.07623
374251    0.28666
466408    0.50555
95119     0.45647
Name: A, dtype: float64
id
487072    0.47681
490811    0.46057
255712    0.52201
```

```

319842    0.69704
220725    0.64340
...
400667    0.80848
25152     0.57363
291689    0.48443
277966    0.51043
137263    0.32016
Name: B, Length: 1185, dtype: float64
id
150375    0.46629
215453    0.64320
353226    0.34671
445765    0.59929
480011    0.24915
...
453134    0.86269
306620    0.72571
108700    0.66572
448660    0.76887
6837      0.62973
Name: C, Length: 93, dtype: float64
id
309145    0.93484
368647    0.32245
484687    0.95435
139863    0.43725
86723     0.22010
...
486182    0.99650
44652     0.72750
13831     0.70526
108894    0.20930
159345    0.99884
Name: D, Length: 1086, dtype: float64

```

Out[34]: cat5

```

A    None
B    None
C    None
D    None
Name: cont0, dtype: object

```

```
In [35]: aa = df_dunn.groupby('cat5')[cont_cols].apply(lambda x: np.max(pdist(x)))
print(aa)
type(aa)
```

```
cat5
A    1.748697
B    2.093003
C    1.957944
D    2.201251
dtype: float64
```

```
Out[35]: pandas.core.series.Series
```

```
In [36]: '''
# `max_diam` 계산
`df_dunn`의 `cat5` 그룹별로 연속형 특성의 최대 쌍별 거리를 계산하고, 그 중 최대 값을 찾습니다.
## 내부 동작 설명
- `df_dunn.groupby('cat5')[cont_cols]`: `cat5` 별로 데이터를 그룹화하고, 연속형 특성(`cont_cols`)만 선택합니다.
- `.apply(lambda x: np.max(pdist(x)))`: 각 그룹에 대해 함수(`lambda x: np.max(pdist(x))`)를 적용합니다.
  - `pdist(x)`: 그룹 별로 데이터 포인트 간의 쌍별 거리를 계산합니다.
  - `np.max(pdist(x))`: 계산된 쌍별 거리 중 최대 값을 찾습니다.
- `.max()`: 각 그룹별로 계산된 최대 값들 중에서 최대 값을 찾습니다.
'''
```

```
# 각 군집의 직경: groupby ~ apply ~ pdist 군집내의
# 데이터 포인트 중 최대값을 구합니다
# 이들의 최대값을 구합니다.
```

```
max_diam = df_dunn.groupby('cat5')[cont_cols].apply(lambda x: np.max(pdist(x))).max()
```

```
max_diam
```

```
Out[36]: 2.2012507003065327
```

```
In [37]: df_dunn['cat5'].value_counts()
```

```
Out[37]: B    1185
D    1086
C     93
A     36
Name: cat5, dtype: int64
```

```
In [38]: '''
`cdist()` 함수는 SciPy 라이브러리의 `scipy.spatial.distance` 모듈에 속해 있으며, 두 집합 간의 쌍별 거리를 계산하는 데 사용됩니다.
`pdist()` 함수와 유사하지만, `cdist()`는 두 개의 입력 배열 간의 거리를 계산합니다.

## cdist() 함수의 인자
1. XA (array_like):
    - m x n 크기의 입력 배열입니다. 여기서 m은 첫 번째 집합의 관측값의 수이고, n은 각 관측값의 차원입니다.
2. XB (array_like):
    - p x n 크기의 입력 배열입니다. 여기서 p는 두 번째 집합의 관측값의 수이고, n은 각 관측값의 차원입니다.
    - `XA`와 `XB`는 동일한 차원(n)을 가져야 합니다.
3. metric (str or function, optional):
    - 거리를 계산할 메트릭을 지정합니다. 기본값은 'euclidean' (유클리드 거리)입니다.
    - 문자열로 메트릭을 지정할 수 있으며, 지원되는 메트릭에는 'euclidean', 'cityblock' (맨해튼 거리),
      'cosine', 'hamming' 등 여러 가지가 있습니다.
    - 사용자 정의 함수도 사용할 수 있습니다.
      이 경우 함수는 두 개의 1차원 배열을 입력으로 받아 거리를 계산하고 하나의 스칼라 값을 반환해야 합니다.

## cdist() 함수의 반환값
- Y (ndarray):
    - m x p 크기의 거리 행렬입니다. 이 행렬의 각 요소 \(\ Y[i, j] \\)는 `XA[i]`와 `XB[j]` 사이의 거리를 나타냅니다.
...
print('')
```

```
In [39]: # 위 설명용 예제 1
import numpy as np
from scipy.spatial.distance import cdist

# 샘플 데이터
XA = np.array([[0, 1], [1, 0], [2, 0]])
XB = np.array([[0, 0], [1, 1], [3, 3]])

# 유클리드 거리 계산
Y = cdist(XA, XB, metric='euclidean')

# 거리 행렬 출력
print("유클리드 거리 행렬:\n", Y)
```

유클리드 거리 행렬:

```
[[1.      1.      3.60555128]
 [1.      1.      3.60555128]
 [2.      1.41421356 3.16227766]]
```

In [40]:

```
# 위 설명용 예제 2
# 샘플 데이터
XA = np.array([[0, 1], [1, 0], [2, 0]])
XB = np.array([[0, 0], [1, 1], [3, 3]])

# 맨해튼 거리 계산
Y_manhattan = cdist(XA, XB, metric='cityblock')
print("맨해튼 거리 행렬:\n", Y_manhattan)

# 코사인 거리 계산
Y_cosine = cdist(XA, XB, metric='cosine')
print("코사인 거리 행렬:\n", Y_cosine)
```

맨해튼 거리 행렬:

```
[[1. 1. 5.]
 [1. 1. 5.]
 [2. 2. 4.]]
```

코사인 거리 행렬:

```
[[ nan 0.29289322 0.29289322]
 [ nan 0.29289322 0.29289322]
 [ nan 0.29289322 0.29289322]]
```

In [41]:

```
# 위 설명용 예제 3
# 샘플 데이터
XA = np.array([[0, 1], [1, 0], [2, 0]])
XB = np.array([[0, 0], [1, 1], [3, 3]])

# 사용자 정의 거리 함수 (예: 두 벡터의 원소 차이의 제곱합)
def custom_metric(u, v):
    return np.sum((u - v) ** 2)

# 사용자 정의 거리 함수로 거리 계산
Y_custom = cdist(XA, XB, metric=custom_metric)
print("사용자 정의 거리 행렬:\n", Y_custom)
```

사용자 정의 거리 행렬:

```
[[ 1.  1. 13.]
 [ 1.  1. 13.]
 [ 4.  2. 10.]]
```

In [42]:

```
'''
# 두 그룹 간의 거리 행렬 크기 확인
`cat5`가 'B'인 그룹과 'C'인 그룹 간의 거리 행렬의 크기를 확인합니다.
## 내부 동작 설명
- `df_dunn.loc[df_dunn['cat5'] == 'B', cont_cols]`: `df_dunn` 데이터프레임에서 `cat5`가 'B'인 행과 연속형 특성 열을 선택합니다.
- `df_dunn.loc[df_dunn['cat5'] == 'C', cont_cols]`: `df_dunn` 데이터프레임에서 `cat5`가 'C'인 행과 연속형 특성 열을 선택합니다.
- `cdist(..., ...)` : 두 데이터 세트 간의 거리 행렬을 계산합니다.
    각 행 간의 거리를 계산하여 NxM 형태의 행렬을 반환합니다.
    여기서 N은 첫 번째 데이터 세트의 행 수, M은 두 번째 데이터 세트의 행 수입니다.
- `.shape`: 거리 행렬의 크기를 반환합니다. 이는 (N, M) 형태의 튜플입니다.
'''

# cdist 두 행렬의 데이터 포인트의 간의 거리를 구합니다.

cdist(
    df_dunn.loc[df_dunn['cat5'] == 'B', cont_cols],
    df_dunn.loc[df_dunn['cat5'] == 'C', cont_cols]
).shape
```

Out[42]: (1185, 93)

In [43]: df_dunn.groupby('cat5').apply(lambda x: print(x.name))

```
A
B
C
D
```

Out[43]: —

In [44]:

```
# 아래 aa의 출력결과
'''
cat5    0
A
"[[0.58717836 1.58296339 1.60420365 ... 1.42868625 1.61155058 0.7716703 ]
 [1.70179351 0.94048491 0.97605291 ... 1.25173579 0.91971822 1.79583324]
 [1.31410905 1.38177768 1.04965036 ... 1.04163162 1.48570929 1.0530518 ]
```

```

...
[1.78746079 0.79948435 0.77531618 ... 1.22995984 1.00195432 1.64595579]
[1.58436012 0.8964494 1.00013013 ... 1.22407665 1.08834789 1.42170805]
[1.55001706 1.00864422 1.11399409 ... 0.88038931 0.88855845 1.47783885]]"
B
"[[1.67872855 1.22627566 1.28801869 ... 1.10295128 0.83450733 1.48386611]
[1.36997505 1.01318659 1.14413238 ... 1.15929632 1.09640529 1.39730898]
[1.8053872 1.21073148 1.20246303 ... 1.21914999 0.72640205 1.59652252]
...
[1.53538022 0.87212857 1.03875897 ... 1.16819038 0.96699797 1.55304972]
[1.61131483 1.33120631 1.42308713 ... 0.96132576 1.26033785 1.42148633]
[1.74132879 0.85189025 1.28682037 ... 1.04942338 1.00217257 1.79540053]]"
C
"[[1.36027664 1.08285177 0.65536688 ... 1.1605751 1.13119318 1.23325638]
[1.01663036 1.3364482 1.10305591 ... 1.12091244 1.18877104 1.11195993]
[1.14441929 1.53117778 1.29229342 ... 1.35139563 1.33862088 1.12575984]
...
[1.61238678 0.96713148 0.79268056 ... 1.30376009 1.07739418 1.48705644]
[0.82250688 1.27383022 1.33354039 ... 1.34730292 1.22506547 0.92543872]
[1.33516388 1.07549497 0.80976135 ... 1.11044519 0.97604246 1.33360525]]"
D
"[[1.67872855 1.36997505 1.8053872 ... 1.53538022 1.61131483 1.74132879]
[1.22627566 1.01318659 1.21073148 ... 0.87212857 1.33120631 0.85189025]
[1.28801869 1.14413238 1.20246303 ... 1.03875897 1.42308713 1.28682037]
...
[1.10295128 1.15929632 1.21914999 ... 1.16819038 0.96132576 1.04942338]
[0.83450733 1.09640529 0.72640205 ... 0.96699797 1.26033785 1.00217257]
[1.48386611 1.39730898 1.59652252 ... 1.55304972 1.42148633 1.79540053]]"
...

aa = df_dunn.groupby('cat5').apply(
    lambda x: print(cdist(x[cont_cols], df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols])
)
)

#aa.to_frame().to_csv('dunn.csv')
#print(type(aa))

```

```
[0.58717836 1.58296339 1.60420365 ... 1.42868625 1.61155058 0.7716703 ]
[1.70179351 0.94048491 0.97605291 ... 1.25173579 0.91971822 1.79583324]
[1.31410905 1.38177768 1.04965036 ... 1.04163162 1.48570929 1.0530518 ]
...
[1.78746079 0.79948435 0.77531618 ... 1.22995984 1.00195432 1.64595579]
[1.58436012 0.8964494 1.00013013 ... 1.22407665 1.08834789 1.42170805]
[1.55001706 1.00864422 1.11399409 ... 0.88038931 0.88855845 1.47783885]]
[[1.67872855 1.22627566 1.28801869 ... 1.10295128 0.83450733 1.48386611]
[1.36997505 1.01318659 1.14413238 ... 1.15929632 1.09640529 1.39730898]
[1.8053872 1.21073148 1.20246303 ... 1.21914999 0.72640205 1.59652252]
...
[1.53538022 0.87212857 1.03875897 ... 1.16819038 0.96699797 1.55304972]
[1.61131483 1.33120631 1.42308713 ... 0.96132576 1.26033785 1.42148633]
[1.74132879 0.85189025 1.28682037 ... 1.04942338 1.00217257 1.79540053]]
[[1.36027664 1.08285177 0.65536688 ... 1.1605751 1.13119318 1.23325638]
[1.01663036 1.3364482 1.10305591 ... 1.12091244 1.18877104 1.11195993]
[1.14441929 1.53117778 1.29229342 ... 1.35139563 1.33862088 1.12575984]
...
[1.61238678 0.96713148 0.79268056 ... 1.30376009 1.07739418 1.48705644]
[0.82250688 1.27383022 1.33354039 ... 1.34730292 1.22506547 0.92543872]
[1.33516388 1.07549497 0.80976135 ... 1.11044519 0.97604246 1.33360525]]
[[1.67872855 1.36997505 1.8053872 ... 1.53538022 1.61131483 1.74132879]
[1.22627566 1.01318659 1.21073148 ... 0.87212857 1.33120631 0.85189025]
[1.28801869 1.14413238 1.20246303 ... 1.03875897 1.42308713 1.28682037]
...
[1.10295128 1.15929632 1.21914999 ... 1.16819038 0.96132576 1.04942338]
[0.83450733 1.09640529 0.72640205 ... 0.96699797 1.26033785 1.00217257]
[1.48386611 1.39730898 1.59652252 ... 1.55304972 1.42148633 1.79540053]]
```

```
In [45]: aa = df_dunn.groupby('cat5').apply(
          lambda x: np.min(cdist(x[cont_cols], df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols]))
        )
aa
```

```
Out[45]: cat5
A      0.262578
B      0.173348
C      0.255460
D      0.173348
dtype: float64
```

```
In [46]: aa = df_dunn.groupby('cat5').apply(  
        lambda x : x.name  
        )  
  
print(type(aa))  
aa
```

```
<class 'pandas.core.series.Series'>
```

```
Out[46]: cat5  
A      A  
B      B  
C      C  
D      D  
dtype: object
```

```
In [47]: aa = df_dunn.groupby('cat5').apply(  
        lambda x : df_dunn['cat5'] != x.name  
        )  
  
print(type(aa))  
aa.T
```

```
<class 'pandas.core.frame.DataFrame'>
```

Out[47]:

cat5	A	B	C	D
id				
309145	True	True	True	False
368647	True	True	True	False
487072	True	False	True	True
490811	True	False	True	True
484687	True	True	True	False
...	...	...	...	...
291689	True	False	True	True
108894	True	True	True	False
277966	True	False	True	True
137263	True	False	True	True
159345	True	True	True	False

2400 rows × 4 columns

```
In [48]: aa = df_dunn.groupby('cat5').apply(
            lambda x: df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols]
        )

print(type(aa))
aa

<class 'pandas.core.frame.DataFrame'>
```

Out[48]:

		cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	cont10	cont11	cont12	cont13
cat5	id														
A	309145	0.93484	0.29862	0.12013	0.71414	0.27825	1.01559	0.54047	0.80249	0.90302	0.80604	0.55281	0.95061	0.86138	0.75343
	368647	0.32245	0.35401	0.41031	0.75546	0.34427	0.25169	0.30402	0.33034	0.20197	0.37512	0.18451	0.25133	0.50361	0.82109
	487072	0.47681	0.74517	0.71498	0.20361	0.49142	0.43309	0.68805	0.23303	0.38456	0.38120	0.44321	0.70845	0.38167	0.19346
	490811	0.46057	0.35253	0.68699	0.43218	0.28081	0.37182	0.38092	0.58439	0.45705	0.47263	0.81078	0.72384	0.28665	0.80193
	484687	0.95435	0.48605	0.33035	0.79471	0.48517	0.23492	0.79788	0.65686	0.63854	0.24914	0.42852	0.20030	0.48748	0.72627
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
D	400667	0.80848	0.77436	0.64202	0.35618	0.28065	0.33842	1.01114	0.50200	0.44454	0.80041	0.48041	0.78780	0.40138	0.66848
	25152	0.57363	0.78028	0.72768	0.37131	0.27919	0.37832	0.36898	0.30138	0.51931	0.46612	0.70681	0.52139	0.63827	0.33826
	291689	0.48443	0.48401	0.46948	0.21202	0.42113	0.38295	0.41983	0.26219	0.47076	0.41719	0.71013	0.32099	0.35481	0.80839
	277966	0.51043	0.29128	0.92336	0.35686	0.50808	0.52674	0.89700	0.33648	0.10523	0.80020	0.79388	0.75213	0.34323	0.68441
	137263	0.32016	0.04494	0.42490	0.17903	0.77849	0.38004	0.39062	0.24725	0.28542	0.48963	0.35199	0.29951	0.33680	0.82353

7200 rows × 14 columns

In [49]:

```
'''
# `min_dist` 계산
`df_dunn`의 `cat5` 그룹별로 다른 그룹과의 최소 거리를 계산하고, 그 중 최소 값을 찾습니다.
## 내부 동작 설명
- `df_dunn.groupby('cat5')`: `cat5`별로 데이터를 그룹화합니다.
- `.apply(lambda x: np.min(cdist(x[cont_cols], df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols])))`:
  각 그룹에 대해 함수(`lambda x: np.min(cdist(x[cont_cols], df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols])))`를 적용합니다.
- `x[cont_cols]`: 현재 그룹의 연속형 특성 데이터를 선택합니다.
- `df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols]`: 현재 그룹을 제외한 나머지 데이터의 연속형 특성 데이터를 선택합니다.
- `cdist(x[cont_cols], df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols])`: 현재 그룹과 다른 모든 그룹 간의 거리 행렬을 계산합니다.
- `np.min(cdist(...))`: 계산된 거리 행렬에서 최소 거리를 찾습니다.
- `.min()`: 모든 그룹별로 계산된 최소 값들 중에서 최소 값을 찾습니다.
'''

# groupby ~ apply ~ cdist를 이용하여 각 군집과 다른 군집간의 거리 중 최소값을 구하고,
# 이들의 최소값을 구합니다. → 군집 간의 거리 중 최소값
```

```
min_dist = df_dunn.groupby('cat5').apply(
    lambda x: np.min(cdist(x[cont_cols], df_dunn.loc[df_dunn['cat5'] != x.name, cont_cols]))
).min()

min_dist
```

Out[49]: 0.17334836111137597

In [50]: min_dist / max_diam

Out[50]: 0.07874994024407876

In [51]: # 위 코드들의 종합 설명

...

이 코드는 클러스터 내 데이터 포인트 간의 거리와 클러스터 간 최소 거리를 계산하고,
이를 통해 Dunn Index를 구하는 작업을 수행합니다.

Dunn Index는 클러스터링 품질을 평가하는 지표 중 하나로, 값이 클수록 클러스터링 품질이 좋음을 나타냅니다.

1. 필요한 모듈 임포트

2. 데이터 준비

`df\_basetable`은 원래의 데이터프레임 `df\_train`의 복사본입니다.

3. 연속형 컬럼과 카테고리컬 컬럼 정의

- `cat\_cols`는 'cat0'부터 'cat9'까지의 10개 카테고리컬 컬럼 리스트입니다.

- `cont\_cols`는 'cont0'부터 'cont13'까지의 14개 연속형 컬럼 리스트입니다.

4. 유클리디안 거리 행렬 계산

- `pdist` 함수는 연속형 컬럼(`cont\_cols`)에 대한 모든 데이터 포인트 사이의 쌍별 거리(유클리디안 거리)를 계산합니다.

- `squareform` 함수는 계산된 거리 벡터를 거리 행렬로 변환합니다.

5. 각 클러스터의 최대 직경 계산

- `df\_dunn`을 'cat5' 컬럼을 기준으로 그룹화합니다.

- cat5 그룹(클러스터)별로 연속형 컬럼(`cont\_cols`)에 대한 모든 쌍별 거리를 계산합니다.

- cat5 그룹(클러스터)별로 최대 거리를 구하고, 그 중 최대값을 `max\_diam`으로 저장합니다.

6. 'cat5'의 값 분포 확인

- `df\_dunn` 데이터프레임의 'cat5' 컬럼 값들의 빈도수를 계산하고 출력합니다.

7. 두 클러스터 간 거리 행렬 계산

- 'cat5' 컬럼 값이 'B'인 데이터 포인트와 'C'인 데이터 포인트 간의 연속형 컬럼(`cont\_cols`)에 대한 쌍별 거리를 계산합니다.
- 계산된 거리 행렬의 형태를 출력합니다.

8. 클러스터 간 최소 거리 계산

- `df\_dunn`을 'cat5' 컬럼을 기준으로 그룹화합니다.
- cat5 그룹(클러스터)별로 데이터 포인트와 다른 모든 클러스터 간의 최소 거리를 계산합니다.
- 모든 클러스터 간 최소 거리 중 최솟값을 `min\_dist`로 저장합니다.

9. Dunn Index 계산

- 클러스터 간 최소 거리(`min\_dist`)를 클러스터 내 최대 거리(`max\_diam`)로 나눠 Dunn Index를 계산합니다.

전체 흐름 정리

1. 데이터 준비: 데이터프레임을 복사하고 필요한 컬럼을 정의합니다.
2. 유클리디안 거리 행렬을 계산하여 클러스터 내 최대 직경을 구합니다.
3. 두 클러스터 간 거리 행렬을 계산하여 형태를 확인합니다.
4. 클러스터 간 최소 거리를 구합니다.
5. Dunn Index를 계산하여 클러스터링 품질을 평가합니다.

Dunn Index가 클수록 클러스터링의 품질이 좋음을 의미합니다.

이 지표는 클러스터 내의 데이터 포인트들이 얼마나 밀집해 있는지(작을수록 좋음)와 클러스터 간의 분리도가 얼마나 큰지를(클수록 좋음) 동시에 고려합니다.

```
'''
```

```
print('')
```

TIP xgb.XGBRegressor custom loss function을 적용해 봅니다.

df_dunn에서 **cont_cols**를 입력변수로 하고 target을 예측하는 모델을

아래 손실함수를 사용하여 xgb.XGBRegressor를 학습시킵니다.

$$L(y_i, \hat{y}_i) = \frac{1}{2}(y_i - \hat{y}_i)^2$$

xgb.XGBRegressor 가이드에 의하면,

```
A custom objective function can be provided for the ``objective``
parameter. In this case, it should have the signature
``objective(y_true, y_pred) -> grad, hess``:
```

```
y_true: array_like of shape [n_samples]
```

```

    The target values
y_pred: array_like of shape [n_samples]
    The predicted values

grad: array_like of shape [n_samples]
    The value of the gradient for each sample point.
hess: array_like of shape [n_samples]
    The value of the second derivative for each sample point

```

이를 위해서는 Gradient와 Hessian을 구해야 합니다.

$$\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = -(y_i - F(x_i))$$

$$\frac{\partial^2 L(y_i, F(x_i))}{\partial^2 F(x_i)} = 1$$

`xgb.XGBRegressor()` 에서 사용하는 `objective` 파라미터에 custom objective 함수를 정의할 때, 이 함수는 특정 형식의 값을 반환해야 합니다.

이 값은 Gradient와 Hessian입니다. 이들은 XGBoost 모델의 학습 과정에서 중요한 역할을 합니다.

Gradient와 Hessian의 역할

XGBoost는 경사 하강법을 사용하여 모델을 학습시킵니다. Gradient와 Hessian은 손실 함수의 1차 및 2차 미분값으로,

각각 모델이 얼마나 틀렸는지와 그 틀림의 변화율을 나타냅니다. 구체적으로:

- **Gradient (g):** 손실 함수의 1차 미분값으로, 현재 예측값이 얼마나 틀렸는지를 나타냅니다. 이를 통해 모델의 가중치를 얼마나 조정해야 하는지 알 수 있습니다.
- **Hessian (h):** 손실 함수의 2차 미분값으로, Gradient의 변화율을 나타냅니다. 이는 모델의 가중치를 얼마나 조정해야 하는지에 대한 안정성을 제공합니다.

Custom Objective 함수의 정의

Custom objective 함수는 예측값과 실제값을 입력으로 받아, Gradient와 Hessian을 반환해야 합니다. 이 함수는 다음과 같은 형식을 가집니다:

```

def custom_objective(y_true, y_pred):
    gradient = ... # 손실 함수의 1차 미분값 계산

```

```
hessian = ... # 손실 함수의 2차 미분값 계산
return gradient, hessian
```

예제

다음은 일반적인 회귀 손실 함수인 평균 제곱 오차(MSE)를 사용한 custom objective 함수의 예제입니다.

```
import xgboost as xgb
import numpy as np

def mse_custom_objective(y_true, y_pred):
    gradient = y_pred - y_true # MSE의 1차 미분값
    hessian = np.ones_like(y_true) # MSE의 2차 미분값은 1 (상수)
    return gradient, hessian

# 데이터 준비
X = np.random.rand(100, 10)
y = np.random.rand(100)

# 모델 학습
model = xgb.XGBRegressor(objective=mse_custom_objective)
model.fit(X, y)
```

Gradient와 Hessian의 계산 예시

1. Gradient 계산:

- MSE의 손실 함수: $L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$
- Gradient: $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$

2. Hessian 계산:

- Hessian: $\frac{\partial^2 L}{\partial \hat{y}^2} = 1$

이처럼, custom objective 함수를 정의할 때, 손실 함수의 1차 및 2차 미분을 계산하여 Gradient와 Hessian을 반환해야 합니다.

이는 XGBoost 모델이 각 단계에서 가중치를 업데이트하는 데 사용됩니다.

더 복잡한 예제

보다 복잡한 손실 함수를 사용하여 Gradient와 Hessian을 계산할 수도 있습니다.

예를 들어, 로그 손실(log loss)을 사용한 binary classification의 custom objective 함수는 다음과 같습니다:

```
import xgboost as xgb
import numpy as np

def log_loss_custom_objective(y_true, y_pred):
    pred = 1.0 / (1.0 + np.exp(-y_pred)) # 시그모이드 함수
    gradient = pred - y_true # Log Loss의 1차 미분값
    hessian = pred * (1.0 - pred) # Log Loss의 2차 미분값
    return gradient, hessian
```

데이터 준비

```
X = np.random.rand(100, 10)
y = np.random.randint(0, 2, 100)
```

모델 학습

```
model = xgb.XGBRegressor(objective=log_loss_custom_objective)
model.fit(X, y)
```

이 예제에서는 로그 손실을 사용하여 이진 분류 문제를 해결합니다.

`custom_objective` 함수는 손실 함수의 1차 및 2차 미분값을 반환하여, XGBoost가 경사 하강법을 통해 가중치를 업데이트할 수 있도록 합니다.

In [52]:

```
'''
`XGBRegressor.fit()` 함수는 XGBoost 라이브러리의 회귀 모델을 훈련시키기 위해 사용됩니다.
이 함수는 주어진 훈련 데이터를 바탕으로 모델의 파라미터를 학습합니다.
내부적으로 수행되는 작업들은 매우 복잡하지만, 주요 단계들을 이해하는 것은 XGBoost의 동작 원리를 파악하는 데 도움이 됩니다.

### `XGBRegressor.fit()`의 내부 동작
#### 1. 데이터 준비
- 입력 데이터 처리:
  - 입력 데이터 `X`와 레이블 `y`는 NumPy 배열이나 Pandas DataFrame 형태로 제공됩니다.
  - `fit()` 함수는 이 데이터를 내부적으로 `DMatrix`라는 XGBoost의 데이터 구조로 변환합니다.
  - `DMatrix`는 데이터를 효율적으로 처리하기 위해 사용되는 특수한 구조로, 특히 희소 행렬(sparse matrix)에 최적화되어 있습니다.
```

- 결측값 처리:
 - `XGBRegressor``는 기본적으로 결측값(예: ``NaN``)을 처리할 수 있습니다.
결측값은 내부적으로 자동 처리되거나 별도로 지정된 방식으로 처리됩니다.

2. 모델 초기화

- 파라미터 설정:
 - ``fit()`` 함수 호출 시 사용자가 전달한 하이퍼파라미터가 모델에 설정됩니다.
예를 들어, 학습률(`learning rate`), 최대 깊이(`max depth`), 손실 함수(`objective`), 부스팅(`rounds`) 횟수 등.
 - `XGBoost`는 기본적으로 다양한 파라미터를 사용하며, 이를 통해 모델의 성능을 조정할 수 있습니다.

3. 부스팅 라운드 시작

- 트리 기반 학습:
 - `XGBoost`는 부스팅(`Boosting`) 알고리즘의 일종인 그래디언트 부스팅(`Gradient Boosting`)을 사용합니다.
 - 모델 학습은 여러 개의 약한 예측 모델(일반적으로 의사결정 트리)의 앙상블로 이루어집니다.
 - 각 라운드에서 새로운 트리가 추가되며, 이전 트리의 오차(잔차)를 줄이는 방향으로 학습합니다.
- 목표 손실 함수 계산:
 - 기본적으로, 회귀 작업에서는 평균 제곱 오차(`Mean Squared Error, MSE`)를 최소화하는 것이 목표입니다.
손실 함수는 학습 중에 각 라운드마다 계산되어, 트리 학습을 위한 그래디언트(1차 미분)와 헤시안(2차 미분) 정보를 제공합니다.
- 트리 생성:
 - 각 트리는 데이터 분할(`split`)을 통해 생성됩니다. 분할은 정보 이득(`gain`)을 극대화하는 방식으로 이루어집니다.
정보 이득은 특정 변수를 기준으로 데이터를 나눴을 때, 얼마나 불순도가 줄어드는지를 측정한 값입니다.
- 가중치 업데이트:
 - 각 트리의 리프 노드에는 예측값이 저장되며, 예측 오차를 줄이기 위해 가중치가 업데이트됩니다.
이 과정이 부스팅 라운드마다 반복됩니다.

4. 정규화 및 규제

- 정규화(`Learning Rate`):
 - 각 트리의 기여도를 조정하기 위해 학습률이 적용됩니다.
이 학습률은 모델이 너무 빠르게 수렴하여 과적합(`overfitting`)되는 것을 방지하기 위해 사용됩니다.
- 규제(`Regulation`):
 - 모델이 너무 복잡해지지 않도록 `L1` 및 `L2` 규제가 적용됩니다.
이 규제는 모델의 가중치가 과도하게 커지는 것을 방지하여 과적합을 줄이는 역할을 합니다.

5. 조기 종료 (Early Stopping)

- ``fit()`` 함수는 조기 종료(`Early Stopping`) 기능을 지원합니다.
검증 데이터를 함께 제공하고, 일정 라운드 동안 성능이 향상되지 않으면 학습을 조기에 중단할 수 있습니다.
이를 통해 불필요한 연산을 줄이고, 모델이 과적합되지 않도록 합니다.

6. 모델 최적화 및 종료

- 최종 모델 선택:
 - 학습이 완료되면, 최적의 모델이 선택됩니다. 만약 조기 종료가 설정된 경우, 가장 성능이 좋았던 모델이 선택됩니다.
- 모델 저장:

- 학습된 모델은 내부적으로 저장되어, 이후 예측 작업에 사용될 수 있습니다.
- 사용자는 ``predict()`` 메소드를 통해 예측을 수행할 수 있습니다.

요약

- ``XGBRegressor.fit()``는 입력 데이터를 처리하고, 모델의 파라미터를 설정한 후, 부스팅 과정을 통해 학습을 진행합니다.
- 부스팅 과정은 트리 기반의 그래디언트 부스팅을 사용하며, 각 라운드에서 오차를 줄이는 방향으로 새로운 트리를 추가합니다.
- 학습 과정에서 정규화, 규제, 조기 종료 등이 사용되어 모델의 성능을 최적화합니다.
- 학습이 완료되면, 최적의 모델이 저장되고 이후 예측에 사용됩니다.

XGBoost의 강력한 성능은 이러한 정교한 학습 과정 덕분에 가능합니다.

'''

print('')

In [53]:

'''

``XGBRegressor``는 XGBoost 라이브러리의 회귀 모델로, 주로 연속형 목표 변수를 예측하는 데 사용됩니다. XGBoost는 성능이 우수하고 확장성이 뛰어난 Gradient Boosting 프레임워크로, 트리 기반 모델을 사용하여 예측을 수행합니다.

``XGBRegressor``의 주요 인자

1. `n_estimators (int, default=100)`:
 - 부스팅 트리의 개수를 설정합니다. 모델이 생성할 약한 학습기의 수입니다.
2. `max_depth (int, default=6)`:
 - 각 개별 회귀 트리의 최대 깊이를 설정합니다. 트리의 깊이가 깊을수록 모델이 더 복잡해지고, 과적합될 가능성이 높아집니다.
3. `learning_rate (float, default=0.3)`:
 - 부스팅 단계에서 각 약한 학습기의 기여를 줄이는 비율입니다. 낮은 학습률을 설정하면 더 많은 트리가 필요합니다.
4. `verbosity (int, default=1)`:
 - 출력 메시지의 상세 수준을 설정합니다. 0은 메시지를 출력하지 않고, 1은 경고 메시지만 출력하고, 2는 정보 메시지도 출력합니다.
5. `objective (str, default='reg:squarederror')`:
 - 최소화할 손실 함수를 정의합니다. 회귀 문제의 경우 'reg:squarederror' (평균 제곱 오차) 등이 있습니다.
6. `booster (str, default='gbtree')`:
 - 부스팅 알고리즘을 선택합니다. 'gbtree' (기본값), 'gblinear', 'dart' 등을 사용할 수 있습니다.
7. `n_jobs (int, default=1)`:
 - 병렬 처리에 사용할 CPU 코어의 수입니다. -1로 설정하면 모든 가용 코어를 사용합니다.
8. `subsample (float, default=1)`:
 - 각 트리를 훈련시키기 위해 사용되는 샘플의 비율입니다. 0에서 1 사이의 값을 가집니다.
9. `colsample_bytree (float, default=1)`:
 - 각 트리를 훈련시킬 때 선택되는 피처의 비율입니다.
10. `gamma (float, default=0)`:
 - 리프 노드를 추가하기 위한 최소 손실 감소를 설정합니다. 이 값이 클수록 모델은 더 보수적이 됩니다.
11. `min_child_weight (float, default=1)`:
 - 리프 노드의 최소 가중치 합입니다. 이 값이 클수록 모델은 더 보수적이 됩니다.
12. `reg_alpha (float, default=0)`:

```

- L1 정규화 항의 가중치입니다.
13. reg_lambda (float, default=1):
- L2 정규화 항의 가중치입니다.

## 리턴값
`XGBRegressor` 객체 자체가 반환되며, 이를 통해 모델 훈련, 예측, 평가 등을 수행할 수 있습니다.

## 주요 속성
- feature_importances_: 각 피처의 중요도를 나타내는 속성입니다.
- best_iteration_: 최적의 반복 수를 나타냅니다.
- best_score_: 최적의 반복 시의 점수를 나타냅니다.
- evals_result_: 훈련 과정에서 평가 메트릭의 결과를 저장한 딕셔너리입니다.

## 주요 함수
- fit(X, y, sample_weight=None, eval_set=None, eval_metric=None, early_stopping_rounds=None, verbose=True,
    xgb_model=None, sample_weight_eval_set=None, callbacks=None):
    - 모델을 훈련시키는 함수입니다.
    - `X`: 훈련 데이터
    - `y`: 목표 변수
    - `sample_weight`: 샘플 가중치
    - `eval_set`: 평가 데이터
    - `eval_metric`: 평가 메트릭
    - `early_stopping_rounds`: 조기 중단 라운드 수
    - `verbose`: 상세 수준
    - `xgb_model`: 기존 XGBoost 모델
    - `sample_weight_eval_set`: 평가 데이터의 샘플 가중치
    - `callbacks`: 콜백 함수 리스트
- predict(X, output_margin=False, ntree_limit=None, validate_features=True, base_margin=None, iteration_range=None, missing=np.na
    - 주어진 입력 데이터에 대한 예측을 수행합니다.
    - `X`: 입력 데이터
    - `output_margin`: 마진 출력 여부
    - `ntree_limit`: 사용 트리의 수
    - `validate_features`: 피처 검증 여부
    - `base_margin`: 초기 마진 점수
    - `iteration_range`: 반복 범위
    - `missing`: 누락된 값
- score(X, y, sample_weight=None): 입력 데이터와 목표 변수를 사용하여 모델의 성능을 평가합니다.
    - `X`: 입력 데이터
    - `y`: 목표 변수
    - `sample_weight`: 샘플 가중치
- get_booster(): XGBoost 부스터를 반환합니다.
- set_params(params): 모델 매개변수를 설정합니다.

```

- `get_params(deep=True)`: 모델 매개변수를 반환합니다.
- `save_model(fname)`: 모델을 파일로 저장합니다.
- `load_model(fname)`: 파일에서 모델을 로드합니다.

아래 예제에서는 `XGBRegressor`를 사용하여 회귀 모델을 훈련시키고, 테스트 데이터에 대해 예측을 수행한 다음, 평균 제곱 오차를 계산하여 모델의 성능을 평가합니다.

```
...
```

```
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_boston
from sklearn.metrics import mean_squared_error
```

```
# 데이터 로드 및 분할
boston = load_boston()
print(type(boston))
```

```
X_train, X_test, y_train, y_test = train_test_split(boston.data, boston.target, test_size=0.2, random_state=42)
```

```
# 모델 초기화 및 훈련
model = xgb.XGBRegressor(n_estimators=100, max_depth=3, learning_rate=0.1)
model.fit(X_train, y_train)
```

```
# 예측
y_pred = model.predict(X_test)
```

```
# 성능 평가
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
```

```
<class 'sklearn.utils.Bunch'>
Mean Squared Error: 7.2667278403836315
```

In [54]: `import numpy as np`

```
...
```

``np.ones_like()``는 NumPy 라이브러리에서 제공하는 함수로, 주어진 배열과 같은 형태와 데이터 타입을 가지면서 모든 요소가 1로 채워진 새로운 배열을 생성하는 데 사용됩니다.
이 함수는 주로 기존 배열의 구조를 그대로 유지하면서, 값만 1로 채우고 싶을 때 유용합니다.

```
### `np.ones_like()` 함수의 동작
#### 인자
`np.ones_like()` 함수는 다음과 같은 인자를 받습니다:
```

1. ``a`` (필수): 기준이 되는 배열. 이 배열의 형태(`shape`)와 데이터 타입(`dtype`)을 따라 새 배열이 생성됩니다.
2. ``dtype`` (선택): 생성할 배열의 데이터 타입. 만약 이 인자가 지정되지 않으면, 기본적으로 ``a`` 배열의 데이터 타입을 따릅니다.
예를 들어, ``a``가 ``int`` 타입이면 생성된 배열도 ``int`` 타입이 됩니다.
3. ``order`` (선택): 배열의 메모리 레이아웃을 지정하는 옵션입니다.
``C`` (C-스타일 행 우선 배열), ``F`` (Fortran-스타일 열 우선 배열), ``A`` (입력 배열의 레이아웃을 따름) 중 하나를 선택할 수 있습니다.
기본값은 ``A``입니다.
4. ``subok`` (선택): `True`로 설정하면, 입력 배열 ``a``의 서브클래스가 유지됩니다. `False`로 설정하면 기본 NumPy 배열을 반환합니다.
기본값은 ``True``입니다.
5. ``shape`` (선택): 원하는 모양으로 배열의 크기를 변경할 수 있는 옵션입니다.
이 인자를 지정하면, 입력 배열 ``a``의 모양을 무시하고, 지정한 모양으로 배열이 생성됩니다.

리턴값

- 새로운 배열: ``np.ones_like()``는 입력 배열 ``a``와 같은 형태 및 데이터 타입을 가지면서, 모든 요소가 1로 채워진 새로운 배열을 반환합니다.
반환된 배열은 원래 배열과 독립적이며, 원본 배열의 내용은 변경되지 않습니다.

요약

- 인자
 - ``a``: 기준이 되는 배열.
 - ``dtype``: (선택) 새 배열의 데이터 타입.
 - ``order``: (선택) 배열의 메모리 레이아웃 (``C``, ``F``, ``A``).
 - ``subok``: (선택) 배열 서브클래스 유지 여부.
 - ``shape``: (선택) 새 배열의 모양.
- 리턴값
 - ``a``와 같은 형태, 데이터 타입을 가지며 모든 요소가 1인 새로운 배열.

``np.ones_like()``는 주어진 배열의 구조를 유지하면서 빠르게 1로 채워진 배열을 생성하고자 할 때 유용합니다.
...

예시 배열

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a)
print('-----')
```

a와 같은 형태의 1로 채워진 배열 생성

여기서 ``b``는 ``a``와 동일한 형태인 2x3 배열이며, 모든 값이 1로 채워져 있습니다.

```
b = np.ones_like(a)
print(b)
print('-----')
```

dtype 인자를 사용하여 부동 소수점 타입의 배열 생성

``dtype=np.float64``로 지정했기 때문에, 결과 배열 ``c``는 모든 요소가 부동 소수점(float) 타입으로 채워집니다.

```
c = np.ones_like(a, dtype=np.float64)
print(c)
print('-----')
```

```
[[1 2 3]
 [4 5 6]]
-----
[[1 1 1]
 [1 1 1]]
-----
[[1. 1. 1.]
 [1. 1. 1.]]
-----
```

In [55]: `from sklearn.metrics import mean_squared_error`

```
def loss_se(y_true, y_pred):
    return -(y_true - y_pred), np.ones_like(y_true)
```

```
reg_xgb = xgb.XGBRegressor(objective=loss_se)
reg_xgb.fit(df_dunn[cont_cols], df_dunn['target'])
```

```
mean_squared_error(df_dunn['target'], reg_xgb.predict(df_dunn[cont_cols]))
```

Out[55]: 0.6040788088560752

In [56]: `reg_xgb = xgb.XGBRegressor()`
`reg_xgb.fit(df_dunn[cont_cols], df_dunn['target'])`

```
mean_squared_error(df_dunn['target'], reg_xgb.predict(df_dunn[cont_cols]))
```

Out[56]: 0.6040788088560752

In [57]: `# 위 2개 코드 비교`

```
...
```

두 코드 블록 모두 `xgb.XGBRegressor`를 사용하여 회귀 모델을 학습하고, 모델의 예측 성능을 평가하기 위해 `mean_squared_error`를 계산하는 작업을 수행합니다. 그러나 두 코드 블록의 차이점은 `XGBRegressor`의 `objective` 파라미터 설정에 있습니다.

```
### 1. 사용자 정의 손실 함수 사용 (`loss_se`)
```

```
#### 동작 설명
```

1. 사용자 정의 손실 함수 정의:
 - 이 함수는 **XGBoost**의 사용자 정의 손실 함수로 사용됩니다.
 - 첫 번째 반환값은 손실의 그라디언트(**gradient**)입니다.
 - 두 번째 반환값은 손실의 헤시안(**hessian**)입니다.
 - 여기서는 단순히 `-(y_true - y_pred)`를 그라디언트로, ``np.ones_like(y_true)``를 헤시안으로 반환하고 있습니다.
2. **XGBRegressor** 인스턴스 생성:
 - ``objective`` 파라미터에 ``loss_se`` 함수가 전달됩니다.
이는 **XGBoost**가 모델을 학습할 때 이 손실 함수를 사용한다는 의미입니다.
3. 모델 학습:
 - ``df_dunn[cont_cols]`` 데이터를 사용하여 ``df_dunn['target']``을 예측하는 모델을 학습합니다.
4. 모델 평가:
 - 학습된 모델을 사용하여 예측값을 생성하고, 실제 값과 예측값 간의 평균 제곱 오차(**MSE**)를 계산합니다.

2. 기본 손실 함수 사용

동작 설명

1. **XGBRegressor** 인스턴스 생성:
 - ``objective`` 파라미터를 설정하지 않으면, **XGBoost**는 기본적으로 회귀 문제에 대해 ``reg:squarederror`` 손실 함수를 사용합니다. 이는 평균 제곱 오차(**MSE**)를 최소화하는 방향으로 모델을 학습합니다.
 2. 모델 학습:
 - 앞서와 동일하게 데이터를 사용하여 모델을 학습합니다.
 3. 모델 평가:
 - 학습된 모델을 사용하여 예측값을 생성하고, 실제 값과 예측값 간의 평균 제곱 오차(**MSE**)를 계산합니다.
- ### 비교
- 손실 함수:
 - 첫 번째 코드 블록에서는 사용자 정의 손실 함수(``loss_se``)를 사용합니다.
이 함수는 그라디언트와 헤시안을 직접 정의하여 모델 학습에 영향을 미칩니다.
 - 두 번째 코드 블록에서는 **XGBoost**의 기본 손실 함수(``reg:squarederror``)를 사용합니다.
 - 모델 학습 방식:
 - 첫 번째 코드 블록은 사용자 정의 손실 함수를 사용하여 더 세밀한 조정이 가능하지만, 정의된 손실 함수가 적절하지 않으면 모델 성능이 떨어질 수 있습니다.
 - 두 번째 코드 블록은 검증된 기본 손실 함수를 사용하여 일반적으로 안정적인 성능을 보입니다.
 - 평가 방식:
 - 두 코드 블록 모두 학습된 모델을 사용하여 예측값을 생성하고, ``mean_squared_error``를 사용하여 모델 성능을 평가합니다.
평가 방식에는 차이가 없습니다.

사용자 정의 손실 함수를 사용하여 얻은 결과가 더 나을 수도 있지만, 그렇지 않을 가능성도 있습니다. 일반적으로 검증된 기본 손실 함수를 사용하는 것이 안전하며, 사용자 정의 손실 함수는 특정 문제나 데이터셋에서 필요할 때 사용하는 것이 좋습니다.

```
'''
print('')
```

문제1

범주형 변수의 조합의 특성을 파악하고자 한다. 다음 과정을 수행하라.

단계 1-1

basetable을 복사하여 prob1을 만들고 cat0 ~ 9 변수 순서대로 문자열 결합하여,

10자리 문자형 파생 변수(cat_c)를 생성하라.

```
In [58]: # 방법 1: 문자열에 대한 sum
df_prob1 = df_basetable.assign(
    cat_c = df_basetable[cat_cols].sum(axis=1)
)

df_prob1.head()
```

Out[58]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont7	cont8	cont9	cont10	cont11	cont12	cont13	target	targetA	
id																					
267387	A	B	B	C	B	B	A	E	G	G	...	0.30912	0.47134	0.53290	0.97139	0.59414	0.36856	0.42951	8.13741	False	A
410037	A	A	A	A	B	B	A	E	C	F	...	0.62596	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.40427	True	A
139373	A	A	B	A	B	B	A	E	C	J	...	0.27010	0.40112	0.53052	0.35635	0.33341	0.33354	0.82542	7.98022	False	A
113765	A	A	A	C	B	B	A	E	C	G	...	0.28331	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.26085	True	A
179915	A	B	A	C	B	B	A	E	E	L	...	0.46316	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.26448	True	A

5 rows × 27 columns

In [59]:

df_prob1['cat_c']

Out[59]:

```
id
267387    ABBCBBAEGG
410037    AAAABBAECF
139373    AABABBAECJ
113765    AAACBBAECG
179915    ABACBBAEEL
...
320975    AAACBBAECI
29591     ABACBBAECF
46717     ABBCBDAEGF
463191    AAACBDAEEF
415856    AAACBDAEGI
Name: cat_c, Length: 240000, dtype: object
```

In [60]:

```
'''
`''.join(x)`는 파이썬에서 문자열을 결합하는 방법 중 하나입니다.
이 표현이 `df_basetable[cat_cols].apply(lambda x: ''.join(x), axis=1)`에서 사용된 맥락을 단계별로 설명해드리겠습니다.

### 맥락 이해: 코드의 구성 요소
먼저, 이 코드의 각 구성 요소를 간략히 설명합니다:

1. `df_basetable[cat_cols]`:
   - `df_basetable`은 Pandas DataFrame이고, `cat_cols`는 이 DataFrame에서 선택된 범주형 열들의 리스트입니다.
```

```

- `df_basetable[cat_cols]`는 해당 범주형 열들만 포함하는 서브 DataFrame을 반환합니다.
2. `apply()` :
- `apply()` 함수는 DataFrame에 대해 특정 연산을 적용하는 함수입니다.
- `axis=1`로 설정되어 있으므로, 각 행에 대해 `lambda` 함수가 적용됩니다. 즉, DataFrame의 각 행이 입력으로 전달됩니다.
3. `lambda x: ''.join(x)` :
- `lambda` 함수는 익명 함수로, 각 행에 대해 `''.join(x)`를 수행합니다.
- 여기서 `x`는 특정 행에 해당하는 값들(이 경우, `cat_cols`에 해당하는 열들의 값들)입니다.

### `''.join(x)`의 동작 단계
1. `x`는 무엇인가?
- `x`는 각 행에서 선택된 범주형 열들의 값으로 이루어진 `Series` 객체입니다.
  예를 들어, 행 하나가 `['A', 'B', 'C']` 같은 값을 가진다면, `x`는 이 리스트와 유사한 형태가 됩니다.
2. `''.join(x)`의 의미
- `join()`은 파이썬의 문자열 메소드로,
  문자열을 요소로 가지는 반복 가능한(iterable) 객체(리스트, 튜플 등)의 요소들을 하나의 문자열로 결합합니다.
- 이때, `join()` 앞에 있는 문자열 `''`는 각 요소 사이에 삽입할 구분자(separator)를 의미합니다.
  여기서는 빈 문자열 `''`이기 때문에, 각 요소가 바로 연결됩니다.
- 예를 들어, `x = ['A', 'B', 'C']`라면 `''.join(x)`의 결과는 `'ABC'`가 됩니다.
  즉, 리스트의 각 요소를 연결하여 하나의 문자열로 만든 것입니다.
3. 최종 결과
- `apply(lambda x: ''.join(x), axis=1)`은 DataFrame의 각 행에서 범주형 열의 값을 결합하여 하나의 문자열로 변환합니다.
- 예를 들어, DataFrame의 한 행이 다음과 같다고 가정해 봅시다:
  col1 col2 col3
  A     B     C
- `cat_cols = ['col1', 'col2', 'col3']`라면, `x`는 `['A', 'B', 'C']`가 되고, `''.join(x)`는 `'ABC'`가 됩니다.

### 코드 전체의 동작
- `df_basetable[cat_cols].apply(lambda x: ''.join(x), axis=1)`은
  DataFrame의 각 행에서 `cat_cols`에 해당하는 열들의 값을 하나의 문자열로 결합한 시리즈(열)를 반환합니다.
- 이 결과로 생성된 시리즈는 각 행의 범주형 값들을 연결한 문자열로 구성됩니다.

이를 활용하면 여러 범주형 변수를 하나의 문자열로 연결하여 새로운 특성을 만들 수 있습니다.
'''
print('')

```

```

In [61]: # 방법 2: apply(axis=1) join
df_prob1 = df_basetable.assign(
    cat_c = df_basetable[cat_cols].apply(lambda x: ''.join(x), axis=1)
)

```

```
df_prob1.head()
```

Out[61]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont7	cont8	cont9	cont10	cont11	cont12	cont13	target	targetA	
id																					
267387	A	B	B	C	B	B	A	E	G	G	...	0.30912	0.47134	0.53290	0.97139	0.59414	0.36856	0.42951	8.13741	False	A
410037	A	A	A	A	B	B	A	E	C	F	...	0.62596	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.40427	True	A
139373	A	A	B	A	B	B	A	E	C	J	...	0.27010	0.40112	0.53052	0.35635	0.33341	0.33354	0.82542	7.98022	False	A
113765	A	A	A	C	B	B	A	E	C	G	...	0.28331	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.26085	True	A
179915	A	B	A	C	B	B	A	E	E	L	...	0.46316	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.26448	True	A

5 rows × 27 columns

In [62]:

```
df_prob1['cat_c']
```

Out[62]:

```
id
267387    ABBCBBAEGG
410037    AAAABBAECF
139373    AABABBAECJ
113765    AAACBBAECG
179915    ABACBBAEEL
...
320975    AAACBBAECI
29591     ABACBBAECF
46717     ABBCBDAEGF
463191    AAACBDAEEF
415856    AAACBDAEGI
Name: cat_c, Length: 240000, dtype: object
```

In [63]:

```
# 방법 3: apply(axis=1) pd.Series.str.cat
df_prob1 = df_basetable.assign(
    cat_c = df_basetable[cat_cols].apply(lambda x: x.str.cat(), axis=1)
)

df_prob1.head()
```

Out[63]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont7	cont8	cont9	cont10	cont11	cont12	cont13	target	targetA	
id																					
267387	A	B	B	C	B	B	A	E	G	G	...	0.30912	0.47134	0.53290	0.97139	0.59414	0.36856	0.42951	8.13741	False	A
410037	A	A	A	A	B	B	A	E	C	F	...	0.62596	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.40427	True	A
139373	A	A	B	A	B	B	A	E	C	J	...	0.27010	0.40112	0.53052	0.35635	0.33341	0.33354	0.82542	7.98022	False	A
113765	A	A	A	C	B	B	A	E	C	G	...	0.28331	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.26085	True	A
179915	A	B	A	C	B	B	A	E	E	L	...	0.46316	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.26448	True	A

5 rows × 27 columns

In [64]: df_prob1['cat_c']

Out[64]: id

267387 ABBCBBAEGG
 410037 AAAABBAECF
 139373 AABABBAECJ
 113765 AAACBBAECG
 179915 ABACBBAEEL

...

320975 AAACBBAECI
 29591 ABACBBAECF
 46717 ABBCBDAEGF
 463191 AAACBDAEEF
 415856 AAACBDAEGI

Name: cat_c, Length: 240000, dtype: object

```
In [65]: import pandas as pd
temp_col = ['cat0', 'cat1', 'cat2']
# 예제 데이터
df_temp = pd.DataFrame({
    'cat0': ['a', 'b', 'c'],
    'cat1': ['d', 'e', 'f'],
    'cat2': ['g', 'h', 'i']
})
```

```
# 결과
df_res = df_temp.assign(
    cat_c = df_temp[temp_col].sum(axis=1)
)

df_res
```

```
Out[65]:
```

	cat0	cat1	cat2	cat_c
0	a	d	g	adg
1	b	e	h	beh
2	c	f	i	cfi

```
In [66]: df_temp[temp_col].apply(lambda x: print(x), axis=0)
```

```
0    a
1    b
2    c
Name: cat0, dtype: object
0    d
1    e
2    f
Name: cat1, dtype: object
0    g
1    h
2    i
Name: cat2, dtype: object
```

```
Out[66]: cat0    None
cat1    None
cat2    None
dtype: object
```

```
In [67]: df_temp[temp_col].apply(lambda x: print(x), axis=1)
```

```

cat0    a
cat1    d
cat2    g
Name: 0, dtype: object
cat0    b
cat1    e
cat2    h
Name: 1, dtype: object
cat0    c
cat1    f
cat2    i
Name: 2, dtype: object

```

```

Out[67]: 0    None
        1    None
        2    None
        dtype: object

```

```

In [68]: df_res = df_temp.assign(
          cat_c = df_temp[temp_col].apply(lambda x: ''.join(x), axis=1)
        )

df_res

```

```

Out[68]:   cat0 cat1 cat2 cat_c
0      a   d   g   adg
1      b   e   h   beh
2      c   f   i   cfi

```

```

In [69]: df_res = df_temp.assign(
          cat_c = df_temp[temp_col].apply(lambda x: x.str.cat(), axis=1)
        )

df_res

```

```
Out[69]:
```

	cat0	cat1	cat2	cat_c
0	a	d	g	adg
1	b	e	h	beh
2	c	f	i	cfi

```
In [70]: # 위 3개 방법의 비교(예제는 바로 위)
```

```
...
```

위 세 가지 방법 모두 `df_basetable` 데이터프레임의 카테고리컬 컬럼(`temp_col`)을 문자열로 결합하여 새로운 컬럼 `cat_c`를 생성하는 작업을 수행합니다. 하지만 각 방법은 문자열을 결합하는 방식이 다릅니다. 세 가지 방법을 비교하여 자세히 설명하겠습니다.

방법 1: 문자열에 대한 `sum`

- 이 방법은 `temp_col`에 해당하는 컬럼들을 행 단위로 더합니다.
- 문자열의 `sum` 연산은 문자열을 단순히 연결하는 역할을 합니다.
- 각 행의 `temp_col` 컬럼에 있는 값을 모두 문자열로 변환한 후, 이를 순서대로 연결합니다.

방법 2: `apply(axis=1) join`

- 이 방법은 `apply` 함수와 `lambda` 함수를 사용하여 각 행에서 `temp_col`에 해당하는 값들을 문자열로 결합합니다.
- `lambda x: ''.join(x)`는 `temp_col`의 값을 리스트로 받아서, 이를 하나의 문자열로 결합합니다.

방법 3: `apply(axis=1) pd.Series.str.cat`

- 이 방법은 `apply` 함수와 `pd.Series.str.cat` 메서드를 사용하여 각 행에서 `temp_col`에 해당하는 값들을 문자열로 결합합니다.
- `x.str.cat()`는 `Series`의 문자열을 결합하는 메서드로, 각 행의 `temp_col` 값들을 문자열로 결합합니다.

비교

1. 속도 및 효율성:

- 방법 1: 가장 간단하고 빠른 방법입니다. `sum(axis=1)`은 내부적으로 최적화되어 있어 효율적입니다.
- 방법 2: `apply`와 `lambda`를 사용하므로 다소 느릴 수 있지만, 명확하고 직관적입니다.
- 방법 3: `pd.Series.str.cat`은 `pandas`에서 제공하는 문자열 결합 메서드로, `apply`를 사용하기 때문에 방법 2와 유사한 속도를 가집니다.

2. 사용 편의성:

- 방법 1: 가장 간결한 구문을 가지고 있지만, `cat_cols`의 값이 모두 문자열이어야 합니다.
- 방법 2: `lambda` 함수를 사용하여 문자열 결합을 명확하게 표현합니다.
- 방법 3: `pandas`의 메서드를 활용하여 직관적이지만, `apply`를 사용하기 때문에 다소 장황합니다.

3. 결과:

- 세 가지 방법 모두 동일한 결과를 생성합니다.

각 행의 `cat\_cols` 값들을 문자열로 결합하여 새로운 컬럼 `cat\_c`를 생성합니다.

결론

- 가장 간단하고 효율적인 방법은 방법 1입니다.
- 보다 명확하게 표현하고 싶다면 방법 2나 방법 3을 사용할 수 있습니다.
- 방법 3은 pandas의 메서드를 활용하여 pandas 스타일에 익숙한 사용자에게는 더 직관적일 수 있습니다.

```
print('')
```

단계 1-2

50개 이상 행으로 이루어지고, target에 대한 정규성을 부정할 수 없는

cat_c를 찾아 cat_c_A와 cat_c_B를 구하라.

정규성 검정은 Kolmogorov-Smirnov 검정을 사용한다. (유의 수준: 5%)

- cat_c의 행의 개수가 많은 순으로 cat_c 3개를 cat_c_A
- cat_c의 행의 개수가 적은 순으로 cat_c 3개를 cat_c_B
- 행의 개수가 같을 경우에는 p-value가 높은 순으로 고른다.

함수가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [71]: df_prob1['cat_c'].value_counts()
```

```
Out[71]: AAAABBAECF      6507
        AAAABDAECF      5915
        ABACBBAEEL      4366
        AAACBBAECF      4145
        AAACBDAECF      3636
        ...
        BAACBBADCN       1
        BAACBBAEDG       1
        BBACBBBEEA       1
        ABACBDCDAL       1
        AABDBDAEGF       1
        Name: cat_c, Length: 6900, dtype: int64
```

```
In [72]: '''
주어진 코드는 데이터프레임 `df_prob1`에서 특정 조건을 만족하는 행들만 선택하여 `df_sub`에 할당하는 작업을 수행합니다.
### 1. `df_prob1['cat_c']`
- `df_prob1` 데이터프레임의 `cat_c` 열을 선택합니다. `cat_c`는 범주형 열로 가정할 수 있습니다.

### 2. `df_prob1['cat_c'].value_counts()`
- `value_counts()` 메서드는 `cat_c` 열에서 각 고유 값이 등장하는 빈도수를 계산하여,
  빈도수를 내림차순으로 정렬한 Series를 반환합니다.
- 예를 들어, `cat_c` 열에 'A', 'B', 'A', 'C', 'B', 'B'와 같은 값이 있다면, 이 단계는 다음과 같은 Series를 생성합니다:
  B      3
  A      2
  C      1

### 3. `.pipe(lambda x: x.loc[x >= 50])`
- `pipe()` 메서드는 함수형 프로그래밍의 스타일을 지원하며, 앞의 결과를 `lambda` 함수로 전달합니다.
- `lambda x: x.loc[x >= 50]`는 빈도수가 50 이상인 값만을 선택합니다.
- 즉, `value_counts()`의 결과에서 빈도수가 50 이상인 인덱스와 값들만을 필터링합니다.
- 예를 들어, `value_counts()`의 결과가 다음과 같다면:
  A      100
  B       60
  C       40

  필터링 후에는 다음과 같은 결과가 됩니다:

  A      100
  B       60

### 4. `.index`
- `index` 속성은 위에서 필터링된 Series의 인덱스 값을 반환합니다.
```

이 인덱스 값들은 빈도수가 50 이상인 `cat\_c`의 고유 값들입니다.

- 예를 들어, 위 필터링된 결과의 인덱스는:
`Index(['A', 'B'], dtype='object')`

5. `df\_prob1['cat\_c'].isin(...)`

- `isin()` 메서드는 `cat\_c` 열의 각 값이 주어진 리스트나 인덱스에 포함되어 있는지 여부를 확인하여, 각 행에 대해 `True` 또는 `False` 값을 반환하는 **Boolean Series**를 생성합니다.
- 예를 들어, `cat\_c` 열이 `['A', 'B', 'C', 'A', 'B']`와 같고, 주어진 인덱스가 `['A', 'B']`라면, `isin()`의 결과는:

```
0    True
1    True
2   False
3    True
4    True
dtype: bool
```

6. `df\_prob1.loc[...]`

- `loc[]` 메서드는 인덱싱을 통해 데이터프레임에서 행을 선택합니다.
 여기서는 `isin()` 메서드의 **Boolean Series**를 사용하여 `True`인 행들만 선택합니다.
- 예를 들어, `df\_prob1` 데이터프레임이 다음과 같다면:

	cat_c	value
0	A	10
1	B	20
2	C	30
3	A	40
4	B	50

`isin()` 메서드의 결과를 사용하여 `loc[]`로 필터링하면, `cat\_c`가 'A' 또는 'B'인 행들만 선택됩니다:

	cat_c	value
0	A	10
1	B	20
3	A	40
4	B	50

최종 결과

- 필터링된 데이터프레임은 빈도수가 50 이상인 `cat\_c` 값들을 포함하는 행들만을 포함합니다.
- 예를 들어, `df\_prob1`에서 빈도수가 50 이상인 값들이 'A'와 'B'라면,
 최종적으로 `cat\_c`가 'A' 또는 'B'인 행들만 `df\_sub`에 저장됩니다.

이러한 과정으로 `df\_sub`는 `df\_prob1`에서 `cat\_c` 열의 빈도수가 50 이상인 값들만 포함하는 행들로 이루어진 데이터프레임이 됩니다.

```

...

# 50개 이상의 빈도를 지닌 cat_c의 샘플들만 가져오는 루틴입니다.
# 이러한 루틴은 groupby ~ filter 로 간소화 시킬 수 있습니다.
df_sub = df_prob1.loc[
    df_prob1['cat_c'].isin(
        df_prob1['cat_c'].value_counts().pipe(
            lambda x: x.loc[x >= 50]
        ).index
    )
]

df_sub.head()

```

Out[72]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont7	cont8	cont9	cont10	cont11	cont12	cont13	target	targetA	
id																					
410037	A	A	A	A	B	B	A	E	C	F	...	0.62596	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.40427	True	A
113765	A	A	A	C	B	B	A	E	C	G	...	0.28331	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.26085	True	A
179915	A	B	A	C	B	B	A	E	E	L	...	0.46316	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.26448	True	A
166456	A	A	A	C	B	B	A	E	E	F	...	0.26890	0.08374	0.26894	0.13828	0.34825	0.29079	0.28819	7.79397	False	A
29503	A	B	A	C	B	B	A	E	E	F	...	0.28413	0.30163	0.32551	0.26982	0.19329	0.39034	0.30002	6.66255	True	A

5 rows × 27 columns

In [73]: df_sub['cat_c']

```
Out[73]: id
410037    AAAABBAECF
113765    AAACBBAECG
179915    ABACBBAEEL
166456    AAACBBAEEF
29503     ABACBBAEEF
...
320975    AAACBBAECI
29591     ABACBBAECF
46717     ABBBCBDAEGF
463191    AAACBDAEEF
415856    AAACBDAEGI
Name: cat_c, Length: 202695, dtype: object
```

```
In [74]: '''
`kstest()` 함수는 SciPy 라이브러리에서 제공하는 Kolmogorov-Smirnov 테스트를 수행하는 함수입니다.
이 함수는 주어진 데이터가 특정 분포를 따르는지 여부를 검정하기 위해 사용됩니다.
Kolmogorov-Smirnov(K-S) 테스트는 분포 간의 차이를 비교하는 비모수적 통계 검정 방법 중 하나입니다.

### `kstest()` 함수의 주요 구성 요소
#### 1. 인자 (Parameters)
`kstest()` 함수는 여러 인자를 받지만, 주요 인자는 다음과 같습니다:
- `rvs` (필수):
  - 테스트할 데이터 또는 함수.
  - 데이터 배열 (1D array)일 경우, 주어진 데이터가 특정 분포를 따르는지 테스트합니다.
  - 함수일 경우, 이 함수는 무작위 변수의 샘플링을 위한 함수이며,
    주어진 함수가 생성한 무작위 샘플이 특정 분포를 따르는지 테스트합니다.
- `cdf` (필수):
  - 비교할 누적 분포 함수(CDF) 또는 분포의 이름.
  - `cdf`는 주어진 데이터가 비교될 누적 분포 함수로, 이를 문자열로 전달하면 SciPy의 내장 분포에서 CDF를 자동으로 가져옵니다.
    예를 들어, `norm`은 정규분포를 의미합니다.
  - 사용자 정의 함수로도 제공할 수 있으며, 이 함수는 CDF 값을 반환해야 합니다.
- `args` (선택):
  - 분포 함수에 필요한 추가적인 파라미터. 예를 들어, `cdf`로 `norm`을 사용하면, `args`에 평균과 표준편차를 제공할 수 있습니다.
- `alternative` (선택):
  - 대립 가설을 지정합니다. 사용할 수 있는 값은 `two-sided`, `less`, `greater`입니다.
    - `two-sided`: 기본값. 데이터가 CDF와 다른지 여부를 테스트합니다.
    - `less`: 데이터가 CDF보다 작은지 여부를 테스트합니다.
    - `greater`: 데이터가 CDF보다 큰지 여부를 테스트합니다.
- `mode` (선택):
  - 테스트의 정확도와 속도 간의 균형을 조정합니다. `auto` (기본값), `exact`, `asympt` 중 선택 가능합니다.
    - `auto`: 샘플 크기에 따라 자동으로 선택합니다.
```

- `'exact'`: 작은 샘플에 대해 정확한 p-값을 계산합니다.
- `'asympt'`: 큰 샘플에 대해 비대칭적 근사치(`asymptotic approximation`)를 사용합니다.

2. 리턴값 (Returns)

``kstest()`` 함수는 두 개의 값을 반환하는 튜플을 반환합니다:

- ``statistic``:
 - K-S 검정 통계량(`Kolmogorov-Smirnov statistic`)입니다. 이는 데이터 분포와 비교 분포 간의 최대 차이입니다.
 - 값이 클수록 데이터와 비교 분포 간의 차이가 크다는 의미입니다.
- ``pvalue``:
 - 검정 통계량에 대응하는 p-값입니다.
 - p-값이 작을수록 (일반적으로 0.05보다 작으면) 데이터가 해당 분포를 따른다는 귀무 가설을 기각할 수 있습니다.

3. 속성 및 메소드 (Attributes and Methods)

SciPy의 ``kstest()`` 함수 자체는 객체가 아닌 단순 함수로 속성과 메소드를 가지지 않습니다. 그러나 반환된 튜플의 각 요소에 대해 다양한 통계적 해석이 가능합니다.

요약

- 인자:
 - ``rvs``: 테스트할 데이터 또는 무작위 변수 샘플링 함수.
 - ``cdf``: 비교할 누적 분포 함수(CDF) 또는 분포의 이름.
 - ``args``: CDF에 전달할 추가 파라미터.
 - ``alternative``: 대립 가설의 형태 (`'two-sided'`, `'less'`, `'greater'`).
 - ``mode``: 검정 모드 (`'auto'`, `'exact'`, `'asympt'`).
- 리턴값:
 - ``statistic``: K-S 검정 통계량.
 - ``pvalue``: p-값.

Kolmogorov-Smirnov 검정은 주어진 데이터가 특정 분포를 따르는지 확인하는 데 유용한 도구입니다. ``kstest()`` 함수는 이를 수행하는 기본 함수이며, 다양한 분포에 대해 이 검정을 수행할 수 있습니다.

1. 데이터가 정규 분포를 따르는지 테스트하기:

```
from scipy.stats import kstest, norm
import numpy as np
```

```
# 예제 데이터 생성 (정규 분포)
```

```
data = np.random.normal(0, 1, 1000)
```

```
# 데이터가 정규 분포를 따르는지 검정
```

```
statistic, pvalue = kstest(data, 'norm')
```

```

print(f"K-S Statistic: {statistic}")
print(f"P-value: {pvalue}")
print('-----')

##### 2. 사용자 정의 분포 함수와의 비교:
import numpy as np

# 수정된 사용자 정의 CDF
def custom_cdf(x):
    return np.where(x < 1, x**2, 1)

# 데이터 생성
data = np.random.uniform(0, 1, 100)

# 데이터가 사용자 정의 분포를 따르는지 검정
statistic, pvalue = kstest(data, custom_cdf)

print(f"K-S Statistic: {statistic}")
print(f"P-value: {pvalue}")
print('-----')

```

```

K-S Statistic: 0.034855313277845634
P-value: 0.17193652255370417
-----

```

```

K-S Statistic: 0.3539658041793654
P-value: 1.0297487875558886e-11
-----

```

In [75]:

```

'''
아래 코드는 Kolmogorov-Smirnov (K-S) 검정을 사용하여 데이터프레임 `df_prob1`의 `target` 열이 정규 분포를 따르는지 확인합니다.
세 가지 방법으로 K-S 검정을 수행하고 있으며, 각각의 방법이 어떻게 동작하는지 설명하겠습니다.

### 1. Kolmogorov-Smirnov 검정 (K-S 검정)
K-S 검정은 두 개의 누적 분포 함수(CDF)를 비교하여, 두 분포가 동일한지 여부를 검정하는 비모수적 방법입니다.
여기서는 `target` 열이 정규 분포를 따르는지 확인하기 위해 K-S 검정을 사용합니다.

### 2. K-S 검정을 위한 `scipy.stats.kstest` 함수
- `kstest(rvs, cdf, args=(), N=20)`:
  - `rvs`: 데이터 샘플 또는 값들.
  - `cdf`: 비교할 누적 분포 함수.
  - `args`: `cdf`에 전달할 추가적인 매개변수.
  - `N`: 샘플 수 (디폴트 20).
'''

```

3. 코드 설명

- 필요한 라이브러리를 임포트합니다. ``kstest``는 K-S 검정 함수입니다.
- ``norm``은 정규 분포 객체, ``zscore``는 Z-점수를 계산하는 함수입니다.
- ``partial``은 함수의 일부 인수를 고정하여 새로운 함수를 생성하는 도구입니다.

첫 번째 방법: 표준화된 값에 대한 K-S 검정

- ``zscore(df_prob1['target'], ddof=1)``: ``target`` 열을 표준화하여 Z-점수를 계산합니다. ``ddof=1``은 샘플 표준 편차를 계산할 때 사용됩니다.
- ``norm.cdf``: 표준 정규 분포의 누적 분포 함수입니다.
- ``kstest``는 표준화된 ``target`` 열의 값들이 표준 정규 분포를 따르는지 검정합니다.

두 번째 방법: 원래 값에 대한 K-S 검정

- ``df_prob1['target']``: ``target`` 열의 원래 값들입니다.
- ``norm.cdf``: 정규 분포의 누적 분포 함수입니다.
- ``args=[df_prob1['target'].mean(), df_prob1['target'].std()]``: ``norm.cdf`` 함수에 평균과 표준 편차를 인수로 전달합니다.
- ``kstest``는 ``target`` 열의 원래 값들이 평균과 표준 편차를 갖는 정규 분포를 따르는지 검정합니다.

세 번째 방법: ``partial``을 사용한 K-S 검정

- ``df_prob1['target']``: ``target`` 열의 원래 값들입니다.
- ``partial(norm.cdf, loc=df_prob1['target'].mean(), scale=df_prob1['target'].std())``: ``norm.cdf`` 함수의 ``loc``와 ``scale`` 인수를 고정하여 새로운 함수를 만듭니다.
- ``kstest``는 ``target`` 열의 값들이 평균과 표준 편차를 갖는 정규 분포를 따르는지 검정합니다.

비교와 결론

- 세 가지 방법 모두 ``target`` 열이 정규 분포를 따르는지 검정합니다.
- 첫 번째 방법은 ``target`` 열을 표준화한 후 표준 정규 분포와 비교합니다.
- 두 번째와 세 번째 방법은 ``target`` 열의 원래 값들을 평균과 표준 편차를 갖는 정규 분포와 비교합니다.
- 세 번째 방법은 두 번째 방법과 거의 동일하지만, ``partial``을 사용하여 인수를 고정한 누적 분포 함수로 비교합니다.

코드는 세 가지 방법 모두 정규성을 검정하지만, 두 번째와 세 번째 방법은 원래 데이터의 평균과 표준 편차를 사용하여 정규성을 검정하고, 첫 번째 방법은 데이터를 표준화하여 정규성을 검정합니다.

결과는 세 가지 방법 모두 동일하게 나올 가능성이 높습니다.

...

kstest를 진행하는 방법을 정리합니다.

방법1: 입력 샘플을 zscore를 통한 표준화

방법2: kstest의 args를 통해 norm.cdf에 파라미터 전달

방법3: functools.partial로 norm.cdf에 파라미터 설정

from **functools** **import** **partial**

```
from scipy.stats import kstest, norm, zscore

(
    kstest(zscore(df_prob1['target'], ddof=1), norm.cdf),
    kstest(df_prob1['target'], norm.cdf, args = [df_prob1['target'].mean(), df_prob1['target'].std()]),
    kstest(df_prob1['target'], partial(norm.cdf, loc=df_prob1['target'].mean(), scale=df_prob1['target'].std()))
)
```

```
Out[75]: (KstestResult(statistic=0.048494479307812144, pvalue=0.0),
KstestResult(statistic=0.048494479307812144, pvalue=0.0),
KstestResult(statistic=0.048494479307812144, pvalue=0.0))
```

```
In [76]: '''
```

아래 예제에서 df_prob1은 160개의 행을 가지고 있으며, cat_c 열에는 'A', 'B', 'C'의 값들이 있습니다.
그룹 'A'는 60개, 그룹 'B'는 30개, 그룹 'C'는 70개의 행을 가집니다.

groupby('cat_c')에 의해 그룹이 만들어지면, filter(lambda x: len(x) >= 50)가 각 그룹에 대해 적용됩니다:

첫 번째 그룹 x는 'A' 값을 가지는 60개의 행을 포함하는 데이터프레임입니다. len(x)는 60이므로 lambda x: len(x) >= 50은 True를 반환함
두 번째 그룹 x는 'B' 값을 가지는 30개의 행을 포함하는 데이터프레임입니다. len(x)는 30이므로 lambda x: len(x) >= 50은 False를 반환함
세 번째 그룹 x는 'C' 값을 가지는 70개의 행을 포함하는 데이터프레임입니다. len(x)는 70이므로 lambda x: len(x) >= 50은 True를 반환함
결과적으로, 그룹 'A'와 'C'는 필터링 후 데이터프레임에 남게 되고, 그룹 'B'는 제거됩니다.

```
'''
```

```
# 예제
import pandas as pd
```

```
# 샘플 데이터 생성
```

```
data = {
    'cat_c': ['A']*60 + ['B']*30 + ['C']*70,
    'value': range(160)
}
df_t = pd.DataFrame(data)
df_t
print(df_t['cat_c'].value_counts())
```

```
C    70
A    60
B    30
Name: cat_c, dtype: int64
```

```
In [77]: # 필터링 - 'cat_c' 컬럼으로 필터링한 row의 갯수(len())가 50개 이상인 것 -> 'B'는 30개이므로 포함 안됨
filtered_df = df_t.groupby('cat_c').filter(lambda x: len(x) >= 50)
```

```
filtered_df
print(filtered_df['cat_c'].value_counts())
```

```
C    70
A    60
Name: cat_c, dtype: int64
```

```
In [78]: aa = df_prob1.groupby('cat_c').filter(lambda x: len(x) >= 50).groupby('cat_c')['target'].apply(lambda x : x)
print(aa.to_frame())
print(type(aa))
```

```
      target
id
410037  7.40427
113765  7.26085
179915  7.26448
166456  7.79397
29503   6.66255
...      ...
320975  7.60537
29591   8.10343
46717   6.93285
463191  6.64854
415856  7.43852
```

```
[202695 rows x 1 columns]
<class 'pandas.core.series.Series'>
```

```
In [79]: '''
`zscore()` 함수는 주어진 데이터의 각 값에 대해 Z-점수를 계산하여 표준화된 값을 반환합니다.
Z-점수는 데이터의 평균에서 얼마나 떨어져 있는지를 표준편차의 단위로 나타낸 값입니다. 주로 데이터 전처리 과정에서 사용됩니다.
```

```
### `zscore()` 함수의 인자
`zscore()` 함수는 주로 SciPy 라이브러리의 `scipy.stats` 모듈에서 사용됩니다.
`scipy.stats.zscore(a, axis=0, ddof=0, nan_policy='propagate')
```

1. a (array_like):
 - Z-점수를 계산할 입력 데이터입니다. 일반적으로 리스트, 배열, 또는 데이터프레임의 열 등이 사용됩니다.
2. axis (int or None, optional, default=0):

```

- Z-점수를 계산할 축을 지정합니다. 기본값은 0으로, 각 열에 대해 계산합니다.
- `axis=0`: 각 열에 대해 Z-점수를 계산합니다.
- `axis=1`: 각 행에 대해 Z-점수를 계산합니다.
- `axis=None`: 전체 배열에 대해 Z-점수를 계산합니다.
3. ddof (int, optional, default=0):
- 표준편차 계산 시 자유도 조정을 위한 값입니다. 기본값은 0입니다.
- 표본 표준편차를 사용할 경우 `ddof=1`로 설정할 수 있습니다.
4. nan_policy ({'propagate', 'raise', 'omit'}, optional, default='propagate'):
- 입력 데이터에 NaN 값이 포함된 경우의 처리 방식을 지정합니다.
- `propagate`: NaN 값을 그대로 유지합니다.
- `raise`: NaN 값이 포함된 경우 에러를 발생시킵니다.
- `omit`: NaN 값을 무시하고 계산합니다.

### 리턴값
- zscore (ndarray):
- 입력 데이터와 동일한 형태의 배열로, 각 값이 Z-점수로 변환된 결과입니다.
...

import numpy as np
from scipy.stats import zscore

# 샘플 데이터 생성
data = np.array([10, 20, 30, 40, 50])

# Z-점수 계산
z_scores = zscore(data)

print(z_scores)

[-1.41421356 -0.70710678  0.          0.70710678  1.41421356]

```

In [80]:

```

'''
각 Z-점수는 데이터 값이 평균에서 얼마나 떨어져 있는지를 표준편차의 단위로 나타낸 값입니다.

### Z-점수 계산 방법
Z-점수는 다음 수식을 사용하여 계산됩니다:


$$z = (x - \mu) / \sigma$$


여기서:
- x : 원본 데이터 값
- mu : 데이터의 평균

```

```

- sigma : 데이터의 표준편차
...

import numpy as np
from scipy.stats import zscore

# 샘플 데이터 생성 (2D 배열)
data = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

# 각 열에 대해 Z-점수 계산 (기본)
z_scores_col = zscore(data, axis=0)
print("각 열에 대해 Z-점수 계산 (기본):\n", z_scores_col)
print('-----')

# 각 행에 대해 Z-점수 계산
z_scores_row = zscore(data, axis=1)
print("각 행에 대해 Z-점수 계산:\n", z_scores_row)
print('-----')

# 전체 데이터에 대해 Z-점수 계산
z_scores_all = zscore(data, axis=None)
print("전체 데이터에 대해 Z-점수 계산:\n", z_scores_all)
print('-----')

# 자유도 1로 표본 표준편차 사용하여 Z-점수 계산
z_scores_ddof = zscore(data, ddof=1)
print("자유도 1로 표본 표준편차 사용하여 Z-점수 계산:\n", z_scores_ddof)
print('-----')

# NaN 값 처리 예제
data_with_nan = np.array([[10, np.nan, 30], [40, 50, 60], [70, 80, np.nan]])
z_scores_nan = zscore(data_with_nan, nan_policy='omit')
print("NaN 값 무시하고 Z-점수 계산:\n", z_scores_nan)

```

각 열에 대해 Z-점수 계산 (기본):

```
[[-1.22474487 -1.22474487 -1.22474487]
 [ 0.          0.          0.          ]
 [ 1.22474487  1.22474487  1.22474487]]
```

각 행에 대해 Z-점수 계산:

```
[[-1.22474487  0.          1.22474487]
 [-1.22474487  0.          1.22474487]
 [-1.22474487  0.          1.22474487]]
```

전체 데이터에 대해 Z-점수 계산:

```
[[-1.54919334 -1.161895   -0.77459667]
 [-0.38729833  0.          0.38729833]
 [ 0.77459667  1.161895    1.54919334]]
```

자유도 1로 표본 표준편차 사용하여 Z-점수 계산:

```
[[-1. -1. -1.]
 [ 0.  0.  0.]
 [ 1.  1.  1.]]
```

NaN 값 무시하고 Z-점수 계산:

```
[[-1.22474487      nan -1.          ]
 [ 0.          -1.          1.          ]
 [ 1.22474487  1.          nan]]
```

In [81]: '''

`kstest()` 함수는 Kolmogorov-Smirnov (KS) 검정을 수행하여 주어진 표본 데이터가 특정 분포를 따르는지 여부를 테스트합니다.

`kstest()` 함수의 인자

`scipy.stats.kstest(rvs, cdf, args=(), N=20, alternative='two-sided', mode='auto', method='auto')`

1. `rvs (array_like or callable):`

- 테스트할 데이터입니다. 배열 형태의 표본 데이터이거나, 난수를 생성하는 함수일 수 있습니다.

2. `cdf (str or callable):`

- 비교할 누적분포함수 (CDF)입니다. 문자열로 분포의 이름을 지정하거나, 함수를 사용할 수 있습니다.
- 예를 들어, `'norm'`은 정규분포를 나타냅니다.

3. `args (tuple, optional):`

- `cdf`에 추가적으로 전달할 매개변수들입니다. 기본값은 빈 튜플 `()`입니다.

4. `N (int, optional):`

- 두 번째 표본의 크기입니다. `rvs`가 배열이 아닌 경우에만 사용됩니다. 기본값은 20입니다.

5. `alternative (str, optional):`

- 대립가설의 형태를 지정합니다. 기본값은 `'two-sided'`입니다.

```

- ``two-sided``: 양측 검정
- ``less``: 데이터의 분포가 `cdf`보다 작다는 대립가설
- ``greater``: 데이터의 분포가 `cdf`보다 크다는 대립가설
6. mode (str, optional):
- 검정 통계량의 모드를 지정합니다. 기본값은 ``auto``입니다.
- ``auto``, ``exact``, ``asympt``: 각각 자동 선택, 정확한 검정, 비대칭 검정을 나타냅니다.
7. method (str, optional):
- 사용될 알고리즘을 지정합니다. 기본값은 ``auto``입니다.
- ``auto``, ``exact``, ``asympt``: 각각 자동 선택, 정확한 방법, 비대칭 방법을 나타냅니다.

### 리턴값
`kstest()` 함수는 `KstestResult` 객체를 반환합니다. 이 객체는 다음 두 가지 속성을 포함합니다:
- statistic (float):
  - KS 검정 통계량입니다. 표본 데이터와 누적분포함수 사이의 최대 차이를 나타냅니다.
- pvalue (float):
  - 검정의 p-값입니다. 귀무가설이 참이라는 가정 하에 관측된 데이터보다 극단적인 결과가 나타날 확률을 나타냅니다.
...

...

### `kstest`와 관련된 추가 메서드
`kstest`는 Kolmogorov-Smirnov 검정에만 사용됩니다. 그러나 SciPy의 `stats` 모듈에는 이와 관련된 다른 검정 함수들도 있습니다:
- `scipy.stats.kstest`:
  - 단일 표본 Kolmogorov-Smirnov 검정
- `scipy.stats.ks_2samp`:
  - 두 표본 Kolmogorov-Smirnov 검정. 두 표본 데이터의 분포가 동일한지 여부를 검증합니다.
- `scipy.stats.normaltest`:
  - 데이터가 정규분포를 따르는지 여부를 검증합니다.
- `scipy.stats.shapiro`:
  - Shapiro-Wilk 검정을 사용하여 데이터의 정규성을 검증합니다.
- `scipy.stats.anderson`:
  - Anderson-Darling 검정을 사용하여 데이터의 정규성을 검증합니다.
...

import numpy as np
from scipy.stats import kstest, norm

# 아래 예제에서는 `data`가 정규분포를 따른다고 가정하고, `kstest()`를 사용하여 이 가정을 검증합니다.
# 샘플 데이터 생성 (정규분포를 따름)
data = np.random.normal(loc=0, scale=1, size=1000)

# Kolmogorov-Smirnov 검정 수행

```

```
result = kstest(data, 'norm')

print(result)
```

```
KstestResult(statistic=0.04205484107450552, pvalue=0.05654333968946601)
```

In [82]: `from scipy.stats import expon`

```
# 아래 예제에서는 `data`가 지수분포를 따른다고 가정하고, `kstest()`를 사용하여 이를 검증합니다.
# 샘플 데이터 생성 (지수분포를 따름)
data = np.random.exponential(scale=1, size=1000)

# Kolmogorov-Smirnov 검정 수행 (지수분포와 비교)
result = kstest(data, 'expon')

print(result)
```

```
KstestResult(statistic=0.03930894140294139, pvalue=0.08857419311484993)
```

In [83]: `from scipy.stats import kstest, norm`

```
df_prob1.groupby('cat_c').filter(lambda x: len(x) >= 50).groupby('cat_c')['target'].apply(
    lambda x: pd.Series(kstest(zscore(x), ddof=1), norm.cdf), index=['stat', 'pvalue'])
)
```

Out[83]:

cat_c		stat	pvalue
AAAABBADAF	stat	0.093664	
	pvalue		0.548731
AAAABBADCF	stat	0.075332	
	pvalue		0.100458
AAAABBADCI	stat	0.084878	
	...		
BBACBDAEEI	pvalue		0.490084
BBACBDAEEL	stat	0.087865	
	pvalue		0.530286
BBACBDAEGF	stat	0.067768	
	pvalue		0.863032

Name: target, Length: 1252, dtype: float64

In [84]: `df_prob1.groupby('cat_c').filter(lambda x: len(x) >= 50).groupby('cat_c')['target'].apply(
 lambda x: pd.Series(kstest(zscore(x), ddof=1), norm.cdf), index=['stat', 'pvalue'])
).unstack()`

Out[84]:

	stat	pvalue
cat_c		
AAAABBADAF	0.093664	0.548731
AAAABBADCF	0.075332	0.100458
AAAABBADCI	0.084878	0.705019
AAAABBADFE	0.093602	0.586496
AAAABBAEAA	0.051012	0.922963
...	...	...
BBACBDAECF	0.075426	0.611944
BBACBDAEEF	0.085839	0.583426
BBACBDAEEI	0.107658	0.490084
BBACBDAEEL	0.087865	0.530286
BBACBDAEGF	0.067768	0.863032

626 rows × 2 columns

In [85]:

```
'''
### 첫 번째 코드 설명

이 코드는 특정 조건을 만족하는 행들을 필터링하고, 그룹별로 Kolmogorov-Smirnov (K-S) 검정을 수행한 후,
결과를 정렬하고 최종적으로 특정 범주 `cat_c`의 상위 및 하위 3개 값을 선택합니다.

1. 필터링 및 그룹화:
- `df_prob1` 데이터프레임을 `cat_c` 열을 기준으로 그룹화하고, 각 그룹의 크기가 50 이상인 그룹만 선택합니다.

2. K-S 검정 수행:
- 필터링된 데이터프레임을 다시 `cat_c` 열을 기준으로 그룹화하고, 각 그룹의 `target` 열에 대해 K-S 검정을 수행합니다.
- `zscore(x, ddof=1)`을 사용하여 `target` 열을 표준화한 후, 표준 정규 분포와 비교합니다.
- K-S 검정 결과(statistic과 pvalue)를 `pd.Series`로 반환하고, 이를 언스택하여 데이터프레임 형태로 변환합니다.

3. 빈도수 추가 및 정렬:
- `cat_c` 열의 빈도수를 계산하여 `df_kstest`에 추가합니다.
- p-value가 0.05 이상인 행들만 선택하고, 빈도수(cnt)와 p-value를 기준으로 정렬합니다.
'''
```

4. 상위 및 하위 3개 범주 선택:

- 정렬된 데이터프레임에서 상위 3개와 하위 3개의 `cat\_c` 값을 선택합니다.

```
...
```

추가 설명

```
...
```

아래 코드는 데이터프레임 `df\_prob1`에서 특정 조건을 만족하는 `cat\_c` 그룹에 대해 Kolmogorov-Smirnov 검정을 수행하고, 그 결과를 정리하여 최종적으로 각 그룹의 빈도수와 함께 `df\_kstest` 데이터프레임에 저장합니다.

```
### 1. `df_prob1.groupby('cat_c')`
```

- `df\_prob1` 데이터프레임을 `cat\_c` 열을 기준으로 그룹화합니다.
- 이 작업은 동일한 `cat\_c` 값을 가지는 행들을 그룹으로 묶습니다.

```
### 2. `.filter(lambda x: len(x) >= 50)`
```

- `filter()` 메서드는 각 그룹에 대해 주어진 조건을 만족하는 그룹만을 남깁니다.
- 여기서는 `lambda x: len(x) >= 50` 조건을 사용하여, 행의 개수가 50 이상인 그룹들만 필터링합니다.
- 즉, `cat\_c` 값이 50개 이상인 그룹들만 남깁니다.

```
### 3. `.groupby('cat_c')['target']`
```

- 필터링된 데이터프레임을 다시 `cat\_c` 열을 기준으로 그룹화하고, 각 그룹에서 `target` 열을 선택합니다.
- 이제 각 그룹의 `target` 열에 대해 작업을 수행할 준비가 됩니다.

```
### 4. `.apply(lambda x: pd.Series(kstest(zscore(x, ddof=1), norm.cdf), index=['stat', 'pvalue']))`
```

- `apply()` 메서드는 각 그룹의 `target` 열에 대해 함수를 적용합니다.
- `lambda x: ...`는 그룹의 `target` 열을 입력으로 받아 다음 작업을 수행합니다:
 - `zscore(x, ddof=1)` : 그룹의 `target` 값들을 표준화합니다.
`ddof=1`은 표준화 계산 시 자유도를 1로 설정하여 표본 표준편차를 사용하도록 합니다.
 - `kstest(zscore(x, ddof=1), norm.cdf)` : 표준화된 `target` 값들에 대해 Kolmogorov-Smirnov 검정을 수행합니다.
여기서 `norm.cdf`는 정규분포의 누적분포함수입니다.
 - `pd.Series(..., index=['stat', 'pvalue'])` : KS 검정의 결과 (검정 통계량과 p-value)를 Series 객체로 변환합니다.
이 객체는 'stat'과 'pvalue'라는 두 개의 인덱스를 가집니다.
- 결과적으로 각 그룹의 `target` 열에 대해 KS 검정의 통계량과 p-value를 계산한 Series를 반환합니다.

```
### 5. `.unstack()`
```

- `unstack()` 메서드는 멀티인덱스 Series를 데이터프레임으로 변환합니다.
- 여기서는 각 그룹의 'stat'과 'pvalue' 값을 열로 갖는 데이터프레임이 생성됩니다.

```
### 6. `.join(df_prob1['cat_c'].value_counts().rename('cnt'))`
```

- `df\_prob1['cat\_c'].value\_counts()`는 `cat\_c` 열에서 각 값의 빈도수를 계산하여 Series로 반환합니다.
- `rename('cnt')`는 이 Series의 이름을 'cnt'로 변경합니다.

- ``join()`` 메서드는 이 `Series`를 앞서 만든 데이터프레임에 조인하여, 각 그룹의 빈도수를 추가합니다.
- 조인은 ``cat_c`` 값을 기준으로 수행됩니다.

최종 결과

- ``df_kstest`` 데이터프레임은 각 ``cat_c`` 그룹에 대해 다음 열들을 포함합니다:
 - `'stat'`: KS 검정 통계량
 - `'pvalue'`: KS 검정 p-value
 - `'cnt'`: 각 ``cat_c`` 값의 빈도수

예제

예를 들어, ``df_prob1`` 데이터프레임이 다음과 같다고 가정합니다:

	cat_c	target
0	A	1.2
1	B	2.3
2	A	1.8
3	C	2.5
4	B	2.1
...		

이 경우, ``cat_c`` 값이 50개 이상인 그룹들에 대해 KS 검정을 수행하고, 그 결과와 각 그룹의 빈도수를 포함하는 데이터프레임이 생성됩니다.

	stat	pvalue	cnt
cat_c			
A	0.123	0.456	100
B	0.234	0.567	75
...			

이러한 방식으로 ``df_kstest``는 조건을 만족하는 ``cat_c`` 그룹들에 대한 KS 검정 결과와 빈도수를 포함하게 됩니다.

방법 1

- # `groupby ~ filter`를 통해 50개 이상의 샘플을 지닌 `cat_c`의 샘플들만 골라냅니다.
- # `groupby ~ apply`를 통해 `cat_c` 각각의 `target` 샘플에 대한
- # Kolmogorov Smirnov 검정을합니다.
- # KS-test 시 `norm.cdf`의 기본설정이 `loc=0`, `scale(표준편차)=1`인 만큼
- # 분포의 형태를 검증하는 만큼 대상 샘플을 표준화를 하여 평균과
- # 표준편차(표본표준편차, `ddof=1`)를 `norm.cdf` 형태와 맞추어 줍니다.
- # 빈도수도 결합해줍니다.

```
df_kstest = df_prob1.groupby('cat_c').filter(lambda x: len(x) >= 50).groupby('cat_c')['target'].apply(
    lambda x: pd.Series(kstest(zscore(x, ddof=1), norm.cdf), index=['stat', 'pvalue']))
```

```
).unstack().join(  
    df_prob1['cat_c'].value_counts().rename('cnt')  
)  
  
df_kstest.head()
```

Out[85]:

	stat	pvalue	cnt
cat_c			
AAAABBADAF	0.093664	0.548731	69
AAAABBADCF	0.075332	0.100458	259
AAAABBADCI	0.084878	0.705019	65
AAAABBADef	0.093602	0.586496	65
AAAABBAEAA	0.051012	0.922963	110

```
In [86]: df_kstest = df_kstest.query('pvalue >= 0.05').sort_values(['cnt', 'pvalue'], ascending=[False, True])  
  
df_kstest
```

Out[86]:

cat_c	stat	pvalue	cnt
ABACBBAEEI	0.030799	0.054494	1888
AAAABBAEEI	0.040326	0.099566	914
AAACBDAEGI	0.044676	0.057831	880
AAACBBAEEA	0.045374	0.066533	819
AAACBBAEEH	0.036724	0.216905	814
...	...	...	...
ABBCBDAEEM	0.077785	0.899734	50
AAACBCAEEI	0.071563	0.943845	50
ABADBDAEAL	0.067553	0.964781	50
BBAABDAEEF	0.060599	0.987656	50
ABBABBAECI	0.048159	0.999475	50

500 rows × 3 columns

In [87]:

```
'''
`shapiro()` 함수는 Shapiro-Wilk 검정을 수행하여 주어진 표본 데이터가 정규 분포를 따르는지 여부를 검증합니다.
이 함수는 주로 SciPy 라이브러리의 `scipy.stats` 모듈에서 사용됩니다.

### `shapiro()` 함수의 인자
scipy.stats.shapiro(x)
1. x (array_like):
    - 정규성을 검정할 데이터입니다. 배열 또는 리스트 형태로 입력됩니다.
2. axis (int or None, optional, default=0):
    - 데이터를 따라 정규성 검정을 수행할 축입니다.
    - 기본값은 0으로, 각 열에 대해 검정을 수행합니다.
    - `axis=None`으로 설정하면 데이터의 플래트닝된 1D 버전에 대해 검정을 수행합니다.
3. ret (bool, optional, default=False):
    - `True`로 설정하면 검정 통계량과 p-값 외에 추가적인 정보를 반환합니다.

### 리턴값
`shapiro()` 함수는 다음 두 가지 값을 포함하는 `ShapiroResult` 객체를 반환합니다:
```

```

- statistic (float):
  - Shapiro-Wilk 검정 통계량입니다. 이 값은 데이터의 정규성 정도를 나타냅니다.
- pvalue (float):
  - 검정의 p-값입니다. 귀무가설이 참이라는 가정 하에 관측된 데이터보다 극단적인 결과가 나타날 확률을 나타냅니다.

### `ShapiroResult` 객체의 속성과 메서드
`ShapiroResult` 객체는 `namedtuple` 형식을 기반으로 하며, 결과 값을 쉽게 접근할 수 있는 속성과 메서드를 제공합니다.
- statistic:
  - Shapiro-Wilk 검정 통계량
- pvalue:
  - p-값

### 주의사항
- Shapiro-Wilk 검정은 데이터의 크기가 너무 크거나 작은 경우 적절하지 않을 수 있습니다.
  일반적으로  $3 < n < 5000$ 인 경우에 유효합니다.
- 데이터에 NaN 값이 포함된 경우, 함수는 오류를 발생시킵니다. NaN 값을 제거하거나 다른 처리를 해야 합니다.

### SciPy의 다른 정규성 검정 함수들
SciPy의 `stats` 모듈에는 Shapiro-Wilk 검정 외에도 다양한 정규성 검정 함수들이 있습니다:
- `scipy.stats.normaltest`:
  - D'Agostino's  $K^2$  검정을 사용하여 데이터의 정규성을 검증합니다.
- `scipy.stats.anderson`:
  - Anderson-Darling 검정을 사용하여 데이터의 정규성을 검증합니다. 여러 가지 분포에 대해 검정할 수 있습니다.
- `scipy.stats.ks_1samp`:
  - Kolmogorov-Smirnov 검정을 사용하여 주어진 데이터가 특정 분포를 따르는지 여부를 검증합니다.

Shapiro-Wilk 검정은 특히 작은 표본에 대해 정규성을 검증하는 데 효과적입니다.
...

# 예제에서는 `data`가 정규분포를 따른다고 가정하고, `shapiro()`를 사용하여 이 가정을 검증합니다.
import numpy as np
from scipy.stats import shapiro

# 샘플 데이터 생성 (정규분포를 따름)
data = np.random.normal(loc=0, scale=1, size=100)

# Shapiro-Wilk 검정 수행
result = shapiro(data)

print(result)

```

```
ShapiroResult(statistic=0.9841891527175903, pvalue=0.27718207240104675)
```

In [88]: *# 예제에서는 `data\_uniform` 이 균등분포를 따른다고 가정하고, `shapiro()`를 사용하여 이를 검증합니다.*

```
# 샘플 데이터 생성 (균등분포를 따름)
data_uniform = np.random.uniform(low=-1, high=1, size=100)

# Shapiro-Wilk 검정 수행
result_uniform = shapiro(data_uniform)

print(result_uniform)
```

```
ShapiroResult(statistic=0.963693380355835, pvalue=0.007438260596245527)
```

In [89]: *# 아래 코드가 에러를 발생하므로 다음과 같이 수정*

```
from scipy.stats import shapiro

# 데이터 필터링
data1 = df_prob1.loc[df_prob1['cat_c'] == 'ABBABBAECI', 'target']
print(len(data1))

# 데이터 길이가 3 이상인지 확인
if len(data1) >= 3:
    shapiro_result1 = shapiro(data1)
    shapiro_result2 = shapiro(data1 * 5)
    print(shapiro_result1, shapiro_result2)
else:
    print("Not enough data for Shapiro-Wilk test.")
```

```
50
```

```
ShapiroResult(statistic=0.9897967576980591, pvalue=0.9413185119628906) ShapiroResult(statistic=0.9897967576980591, pvalue=0.9413185119628906)
```

In [90]: *# Q: shapiro도 kstest 처럼 표준화가 필요한가요?*

```
# VS shapiro
from scipy.stats import shapiro
(
    shapiro(df_prob1.loc[df_prob1['cat_c'] == 'ABBABBAECI', 'target']),

    # 임의의 수 5를 곱해 표준편차가 5배가 되도록해봅니다
    shapiro(df_prob1.loc[df_prob1['cat_c'] == 'ABBABBAECI', 'target'] * 5)
)
```

```
Out[90]: (ShapiroResult(statistic=0.9897967576980591, pvalue=0.9413185119628906),
         ShapiroResult(statistic=0.9897967576980591, pvalue=0.9413185119628906))
```

```
In [91]: cat_c_A = df_kstest.iloc[:3].index.tolist()
         cat_c_B = df_kstest.iloc[-3:].index.tolist()

         cat_c_A, cat_c_B
```

```
Out[91]: ([ 'ABACBBAAEEI', 'AAAABBAEEI', 'AAACBDAEGI'],
         [ 'ABADBDAEAL', 'BBAABDAEEF', 'ABBABBAECI'])
```

```
In [92]: df_prob1['cat_c'].value_counts()
```

```
Out[92]: AAAABBAECF      6507
         AAAABDAECF      5915
         ABACBBAAEEL      4366
         AAACBBAECF      4145
         AAACBDAECF      3636
         ...
         BAACBBADCN         1
         BAACBBAEDG         1
         BBACBBBEEA         1
         ABACBDCDAL         1
         AABDBDAEGF         1
         Name: cat_c, Length: 6900, dtype: int64
```

```
In [93]: '''
         ### 두 번째 코드 설명

         코드는 `df_prob1` 데이터프레임의 각 범주 `cat_c`에 대해 K-S 검정을 수행하고, 결과를 리스트에 저장한 후
         데이터프레임으로 변환하여 특정 조건을 만족하는 행들을 정렬하고 최종적으로 상위 및 하위 3개 값을 선택합니다.

         1. K-S 검정 결과 저장을 위한 리스트 초기화:

         2. 빈도수 계산 및 조건에 따른 필터링:
            - `cat_c` 열의 빈도수를 계산하고, 빈도수가 50 미만인 경우 루프를 중단합니다.
            - 각 `cat_c` 값에 대해 `target` 열을 선택하고, 이를 표준화하여 K-S 검정을 수행합니다.
            - 검정 결과(statistic, pvalue)와 빈도수(v)를 리스트에 추가합니다.

         3. 리스트를 데이터프레임으로 변환 및 정렬:
            - 리스트를 데이터프레임으로 변환하고, `cat_c`를 인덱스로 설정합니다.
            - p-value가 0.05 이상인 행들만 선택하고, 빈도수(cnt)와 p-value를 기준으로 정렬합니다.
```

4. 상위 및 하위 3개 범주 선택:

- 정렬된 데이터프레임에서 하위 3개와 상위 3개의 `cat\_c` 값을 선택합니다.

...

방법 2: 반복문

from scipy.stats import kstest, norm

결과를 저장하기 위한 list입니다.

ks_result = []

value_counts로 cat_c의 빈도수를 가져옵니다.

k(key)는 cat_c, v(value)는 빈도수가 됩니다.

for k, v in df_prob1['cat_c'].value_counts().items():

빈도수에 대하여 내림 차순으로 넘어오기 때문에

50 미만일 경우에 요건에 맞추어 멈춥니다.

if v < 50:

break

현재 라운드에 대한 kstest를 수행합니다.

s_target = df_prob1.loc[df_prob1['cat_c'] == k, 'target']

result = kstest((s_target - s_target.mean()) / s_target.std(), norm.cdf)

(cat_c, 검정통계량, pvalue, 빈도수) 튜플을 ks_result_에 추가합니다.

ks_result.append((k, result.statistic, result.pvalue, v))

df_kstest = pd.DataFrame(ks_result, columns=['cat_c', 'stat', 'p value', 'cnt']).set_index('cat_c')

df_kstest

Out[93]:

cat_c	stat	p value	cnt
AAAABBAECF	0.060856	2.165306e-21	6507
AAAABDAECF	0.089300	1.709063e-41	5915
ABACBBAEEL	0.051439	1.763877e-10	4366
AAACBBAECF	0.053636	8.357369e-11	4145
AAACBDAECF	0.077939	1.172657e-19	3636
...	...	...	...
AAAABDAEAM	0.090387	7.749260e-01	50
ABADBDAEAL	0.067553	9.647810e-01	50
AAACBCAEEL	0.071563	9.438446e-01	50
ABBCBDAEAL	0.133650	3.057621e-01	50
ABBCBDAECG	0.098339	6.821673e-01	50

626 rows × 3 columns

```
In [94]: df_kstest = df_kstest.query('`p value` >= 0.05').sort_values(['cnt', 'p value'], ascending=[True, False])
cat_c_a = df_kstest.index[-3:].tolist()
cat_c_b = df_kstest.index[:3].tolist()

df_kstest
```

Out[94]:

cat_c	stat	p value	cnt
ABBABBAECI	0.048159	0.999475	50
BBAABDAEEF	0.060599	0.987656	50
ABADBDAEAL	0.067553	0.964781	50
AAACBCAEEI	0.071563	0.943845	50
ABBCBDAEEM	0.077785	0.899734	50
...	...	...	...
AAACBBAEEH	0.036724	0.216905	814
AAACBBAEEA	0.045374	0.066533	819
AAACBDAEGI	0.044676	0.057831	880
AAAABBAEEI	0.040326	0.099566	914
ABACBBAEEI	0.030799	0.054494	1888

500 rows × 3 columns

In [95]: # 위 2가지 방법 비교 설명

...

두 코드 비교

- 데이터 필터링:

- 첫 번째 코드는 `groupby`와 `filter`를 사용하여 그룹의 크기가 50 이상인 행들을 선택합니다.
- 두 번째 코드는 빈도수를 계산하여 조건에 맞지 않는 경우 루프를 중단합니다.

- K-S 검정 수행:

- 첫 번째 코드는 `apply`를 사용하여 각 그룹에 대해 K-S 검정을 수행합니다.
- 두 번째 코드는 루프를 통해 각 `cat\_c` 값에 대해 K-S 검정을 수행합니다.

- 결과 저장 및 변환:

- 첫 번째 코드는 `apply`와 `unstack`을 사용하여 K-S 검정 결과를 데이터프레임으로 변환합니다.
- 두 번째 코드는 리스트에 결과를 저장한 후 데이터프레임으로 변환합니다.

- 정렬 및 선택:
 - 첫 번째 코드는 빈도수(cnt)와 p-value를 기준으로 정렬하고, 상위 3개와 하위 3개의 `cat\_c` 값을 선택합니다.
 - 두 번째 코드는 동일한 방식으로 정렬하지만, 결과를 반대로 상위 3개와 하위 3개를 선택합니다.

요약

- 첫 번째 코드는 `groupby`와 `apply`를 사용하여 더 간결하고 직관적인 방식으로 동작합니다.
 - 두 번째 코드는 루프를 사용하여 조건에 따라 더 세밀하게 제어할 수 있지만, 코드가 더 길어지고 복잡해집니다.
 - 두 코드 모두 최종적으로 p-value가 0.05 이상인 `cat\_c` 값을 선택하고, 상위 3개와 하위 3개 값을 반환합니다.
- ```
...
```
- ```
print('')
```

In [96]:

```
'''
`functools.partial()` 함수는 기존 함수를 부분적으로 적용하여 새로운 함수를 생성하는 데 사용됩니다.
즉, 특정 인자에 고정 값을 미리 설정한 새로운 함수를 생성할 수 있습니다.
이는 고차 함수나 콜백 함수 등을 다룰 때 유용하게 사용됩니다.

### `functools.partial()` 함수의 인자
functools.partial(func, /, *args, keywords)
1. func (callable):
    - 부분적으로 적용할 원본 함수입니다.
2. *args:
    - 원본 함수에 전달할 위치 인자들입니다. 고정할 인자들을 순서대로 전달합니다.
3. keywords:
    - 원본 함수에 전달할 키워드 인자들입니다. 고정할 키워드 인자들을 키-값 쌍으로 전달합니다.

### 리턴값
- partial 객체 (callable):
    - 원본 함수의 일부 인자가 고정된 새로운 함수입니다. 이 함수는 나머지 인자들을 전달받아 호출될 수 있습니다.

`functools.partial()` 함수는 함수형 프로그래밍 패러다임에서 강력한 도구로,
함수 호출을 유연하게 하고 코드의 재사용성을 높이는 데 도움을 줍니다.
'''

import functools

# 기본 함수 정의
def add(a, b):
    return a + b

# 부분적으로 적용된 함수 생성
```

```

add_five = functools.partial(add, 5)

# 부분 함수 호출
result = add_five(10)
print(result) # 출력: 15

import functools

# 기본 함수 정의
def greet(greeting, name):
    return f"{greeting}, {name}!"

# 부분적으로 적용된 함수 생성
greet_hello = functools.partial(greet, greeting="Hello")

# 부분 함수 호출
result = greet_hello(name="Alice")
print(result) # 출력: Hello, Alice!

```

```

15
Hello, Alice!

```

In [97]:

```

'''
### 내부 동작
`functools.partial()`로 생성된 부분 함수는 원본 함수의 일부 인자를 미리 설정하여 저장하고,
호출 시 나머지 인자들을 전달받아 원본 함수를 호출합니다. 이는 함수 호출을 간소화하고 재사용성을 높이는 데 유용합니다.

### 활용 예제

부분 함수는 다양한 상황에서 유용하게 사용할 수 있습니다:

1. 콜백 함수:
    - GUI나 비동기 프로그래밍에서 콜백 함수로 사용하기에 적합합니다.

2. 고차 함수:
    - 고차 함수(다른 함수를 인자로 받거나 반환하는 함수)와 함께 사용하면 코드의 가독성을 높일 수 있습니다.

3. 매개변수 사전 설정:
    - 자주 사용되는 함수 호출 패턴에서 특정 인자들을 미리 설정하여 사용하면 코드의 중복을 줄일 수 있습니다.

아래 예제에서 `power` 함수는 `base`와 `exponent`를 받아 거듭제곱을 계산합니다.
`functools.partial()`을 사용하여 `exponent` 인자를 각각 2와 3으로 고정한 `square`와 `cube` 함수를 생성합니다.
'''

```

```
이를 통해 제곱과 세제곱을 쉽게 계산할 수 있습니다.
...
```

```
# 고차 함수 예제
```

```
def power(base, exponent):
    return base ** exponent
```

```
# 부분 함수 생성
```

```
square = functools.partial(power, exponent=2)
cube = functools.partial(power, exponent=3)
```

```
print(square(4)) # 출력: 16
print(cube(2))   # 출력: 8
```

```
16
```

```
8
```

In [98]: # 방법 3.

```
...
```

```
`functools.partial`을 사용하여 `norm.cdf`에 파라미터를 설정하는 방식으로 K-S 검정을 수행하는 코드를 작성해 보겠습니다.
`partial`을 사용하면 `cdf` 함수에 필요한 파라미터를 고정할 수 있으므로,
이를 활용하여 각 그룹에 대해 K-S 검정을 수행할 수 있습니다.
```

```
### 코드 설명
```

1. 필터링 및 그룹화:

- `df\_prob1` 데이터프레임을 `cat\_c` 열을 기준으로 그룹화하고, 각 그룹의 크기가 50 이상인 그룹만 선택합니다.

2. K-S 검정 결과 저장을 위한 리스트 초기화:

3. 그룹별 K-S 검정 수행:

- 각 그룹의 `target` 값을 표준화하여 Z-점수를 계산합니다.
- `partial`을 사용하여 `norm.cdf` 함수의 `loc`와 `scale` 파라미터를 고정합니다.
- 각 그룹에 대해 K-S 검정을 수행하고, 결과를 리스트에 저장합니다.

4. K-S 검정 결과 데이터프레임 생성 및 정렬:

5. 상위 및 하위 3개 cat_c 값 선택:

이 코드에서는 `functools.partial`을 사용하여 `norm.cdf`에 필요한 파라미터를 설정하고, 이를 활용하여 각 그룹별로 K-S 검정을 수행합니다.

```
결과적으로, p-value가 0.05 이상인 상위 3개와 하위 3개의 `cat_c` 값을 선택할 수 있습니다.
'''
print('')
```

```
In [99]: '''
`df_filtered.groupby('cat_c')` 코드에서 `name`과 `group`에 할당되는 값들을 설명하겠습니다.

### 1. `groupby('cat_c')`의 의미:
- `groupby('cat_c')`는 `df_filtered` 데이터프레임의 `cat_c` 열을 기준으로 데이터를 그룹화하는 기능을 수행합니다.
  즉, `cat_c` 열에서 동일한 값을 가지는 행들끼리 묶어서 그룹을 형성합니다.
- 이 과정에서 각 그룹은 동일한 `cat_c` 값을 가진 행들로 구성됩니다.

### 2. `for name, group in df_filtered.groupby('cat_c'):` 루프의 동작:
- 이 구문은 `groupby` 메서드로 생성된 그룹들에 대해 반복 작업을 수행하기 위한 `for` 루프입니다.
- `groupby('cat_c')`는 그룹 객체를 생성하며, 이 객체는 각 그룹에 대해 `(그룹 이름, 그룹 데이터)`의 쌍을 순차적으로 반환합니다.

### 3. `name`과 `group`에 할당되는 값:
- `name`:
  - `name`에는 각 그룹의 이름(즉, `cat_c` 열의 고유한 값)이 할당됩니다.
  - 예를 들어, `cat_c` 열에 'A', 'B', 'C'라는 세 가지 고유 값이 있다면, 루프가 순회할 때마다 `name`에는 'A', 'B', 'C' 중 하나가 차례로 할당됩니다.
- `group`:
  - `group`에는 해당 그룹에 속하는 데이터가 할당됩니다.
  - 이는 원래 데이터프레임에서 `cat_c`가 `name`과 동일한 값을 가지는 행들로 구성된 미니 데이터프레임입니다.
  - 예를 들어, `name`이 'A'라면, `group`에는 `cat_c` 값이 'A'인 행들로 이루어진 데이터프레임이 들어갑니다.

아래 예에서 볼 수 있듯이, `name`에는 'A', 'B', 'C'가 순서대로 할당되고,
`group`에는 해당 그룹에 속하는 데이터프레임이 할당됩니다.
'''

import pandas as pd

# 예제 데이터프레임 생성
data = {'cat_c': ['A', 'A', 'B', 'B', 'C', 'C'],
        'value': [10, 20, 30, 40, 50, 60]}
df_filtered = pd.DataFrame(data)

# 그룹화하고 반복문 실행
for name, group in df_filtered.groupby('cat_c'):
    print(f"Group Name: {name}")
```

```
print(group)
print() # 빈 줄로 그룹 구분
```

Group Name: A

	cat_c	value
0	A	10
1	A	20

Group Name: B

	cat_c	value
2	B	30
3	B	40

Group Name: C

	cat_c	value
4	C	50
5	C	60

In [100...

```
'''
`partial()` 함수는 Python의 `functools` 모듈에 포함된 함수로,
기존 함수에 일부 인자를 고정하여 새로운 함수를 생성하는 데 사용됩니다.
이렇게 생성된 함수는 고정된 인자를 사용하면서도 다른 인자를 추가로 받을 수 있습니다.

### `partial()` 함수의 동작 원리
#### 1. 함수 정의
from functools import partial

#### 2. 사용법
partial(func, *args, kwargs)
- `func`: 새로 생성할 함수의 원본이 되는 함수입니다.
- `*args`: 원본 함수의 위치 인자 중에서 고정할 인자들을 지정합니다.
- `kwargs`: 원본 함수의 키워드 인자 중에서 고정할 인자들을 지정합니다.

#### 3. 리턴값
- `partial()` 함수는 원래의 함수 `func`에 대해 지정된 인자들을 고정한 새로운 함수를 반환합니다.
- 새로 반환된 함수는 `func`와 동일한 함수처럼 동작하지만, 일부 인자들은 이미 고정된 상태로 동작합니다.

### 예제 설명
#### 1. 원본 함수 `norm.cdf`:
- `norm.cdf(x, loc=0, scale=1)`는 정규 분포의 누적 분포 함수(CDF)를 계산하는 함수입니다.
- `x`는 계산할 값, `loc`는 평균(기본값 0), `scale`는 표준편차(기본값 1)입니다.
```

2. `partial(norm.cdf, loc=0, scale=1)`의 의미:

- `partial()` 함수를 이용해 `norm.cdf`의 `loc` 인자를 `0`으로, `scale` 인자를 `1`로 고정 한 새로운 함수를 생성합니다.
- 이제 `norm_cdf` 함수는 `x` 값만 받아서 누적 분포 값을 계산할 수 있는 함수가 됩니다.

3. `norm_cdf` 함수의 동작:

- `norm_cdf`는 `norm.cdf(x, loc=0, scale=1)`와 동일하게 동작합니다.
- 이 함수는 `x`만을 인자로 받으며, `loc`과 `scale`이 이미 고정되어 있으므로 이를 지정할 필요가 없습니다.

정리

- `partial()` 함수는 원본 함수의 일부 인자를 미리 지정하여 새로운 함수를 만드는 데 사용됩니다.
- 이 새로운 함수는 나머지 인자들만을 필요로 하며, 고정된 인자들은 이미 지정된 값으로 동작합니다.
- 이를 통해 코드의 재사용성을 높이고, 반복적으로 인자를 지정할 필요를 줄일 수 있습니다.

아래 코드는 `1.96`이 표준 정규 분포에서 어디에 위치하는지 누적 분포 값을 출력합니다.

`norm_cdf` 함수는 이미 `loc=0`, `scale=1`로 고정되어 있으므로, 단순히 `x` 값만 입력하면 됩니다.

```
...
```

```
from scipy.stats import norm
from functools import partial
```

```
# norm.cdf에 loc=0, scale=1을 고정한 새로운 함수 생성
norm_cdf = partial(norm.cdf, loc=0, scale=1)
```

```
result = norm_cdf(1.96)
print(result) # 1.96을 표준정규분포에서 계산한 CDF 값 출력
```

```
0.9750021048517795
```

In [101...

```
...
```

아래 코드에서는 데이터프레임을 생성하고, 조건을 만족하는 행들을 필터링한 후 그룹별로 Kolmogorov-Smirnov (K-S) 검정을 수행하여 결과를 저장하고, 이를 정렬 및 필터링하여 상위 및 하위 그룹을 출력하는 과정을 거칩니다.

데이터프레임 생성 (df_prob1)

```
np.random.seed(0)
df_prob1 = pd.DataFrame({
    'cat_c': np.random.choice(['A', 'B', 'C', 'D'], size=500),
    'target': np.random.randn(500)
})
```

1. `np.random.seed(0)`: 난수 생성의 시드 값을 설정하여 재현 가능한 결과를 얻도록 합니다.

```

2. `df_prob1` 데이터프레임 생성:
- 'cat_c': 'A', 'B', 'C', 'D' 중 하나를 랜덤하게 500번 선택한 값.
- 'target': 표준 정규 분포에서 랜덤하게 500개의 값을 선택한 값.

### 조건을 만족하는 행들 필터링
df_filtered = df_prob1.groupby('cat_c').filter(lambda x: len(x) >= 50)
3. `df_prob1`를 'cat_c' 기준으로 그룹화한 후, 각 그룹의 크기가 50 이상인 그룹만을 필터링하여 `df_filtered` 데이터프레임에 저장합니다.

### K-S 검정 결과 저장을 위한 리스트 초기화
ks_result = []
4. `ks_result` 리스트를 초기화하여 K-S 검정 결과를 저장할 준비를 합니다.

### 그룹별 K-S 검정 수행
for name, group in df_filtered.groupby('cat_c'):
    # target 값을 표준화 (Z-score)
    z_scores = zscore(group['target'], ddof=1)

    # norm.cdf에 파라미터 설정
    norm_cdf = partial(norm.cdf, loc=0, scale=1)

    # K-S 검정 수행
    stat, pvalue = kstest(z_scores, norm_cdf)

    # 결과 저장 (cat_c, stat, pvalue, count)
    ks_result.append((name, stat, pvalue, len(group)))

5. `df_filtered`를 'cat_c' 기준으로 그룹화하고, 각 그룹에 대해 다음을 수행합니다:
- `zscore(group['target'], ddof=1)` : 각 그룹의 'target' 값을 Z-score로 표준화합니다.
- `partial(norm.cdf, loc=0, scale=1)` : 정규 분포의 누적 분포 함수 (CDF)를 평균 0, 표준편차 1로 설정합니다.
- `kstest(z_scores, norm_cdf)` : 표준화된 데이터와 정규 분포 CDF를 사용하여 K-S 검정을 수행하고, 통계량과 p-값을 반환합니다.
- `ks_result.append((name, stat, pvalue, len(group)))` : 그룹 이름, 통계량, p-값, 그룹 크기를 `ks_result` 리스트에 저장합니다.

### K-S 검정 결과 데이터프레임 생성
df_kstest = pd.DataFrame(ks_result, columns=['cat_c', 'stat', 'p value', 'cnt']).set_index('cat_c')
6. `ks_result` 리스트를 사용하여 K-S 검정 결과를 포함하는 `df_kstest` 데이터프레임을 생성하고, 'cat_c'를 인덱스로 설정합니다.

### p-value가 0.05 이상인 값만 선택하고 정렬
df_kstest = df_kstest.query('`p value` >= 0.05').sort_values(['cnt', 'p value'], ascending=[False, True])
7. `df_kstest`에서 p-값이 0.05 이상인 행들만 선택하고, 'cnt'를 기준으로 내림차순으로, 'p value'를 기준으로 오름차순으로 정렬합니다.

### 상위 및 하위 3개 cat_c 값 선택
cat_c_A = df_kstest.iloc[:3].index.tolist()

```

```
cat_c_B = df_kstest.iloc[-3:].index.tolist()
```

8. `df\_kstest`의 상위 3개 `cat\_c` 값을 `cat\_c\_A` 리스트에, 하위 3개 `cat\_c` 값을 `cat\_c\_B` 리스트에 저장합니다.

```
### 결과 출력
```

```
print("cat_c_A (상위 3개):", cat_c_A)
```

```
print("cat_c_B (하위 3개):", cat_c_B)
```

9. `cat\_c\_A`와 `cat\_c\_B` 리스트를 출력합니다.

이 코드는 주어진 데이터프레임에서 특정 조건을 만족하는 그룹을 필터링하고, 각 그룹에 대해 K-S 검정을 수행하여 결과를 저장한 후, 이를 정렬하고 필터링하여 상위 및 하위 그룹을 선택하는 전체 과정을 포함하고 있습니다.

```
### 구현 코드
```

```
import pandas as pd
```

```
import numpy as np
```

```
from scipy.stats import kstest, norm
```

```
from scipy.stats import zscore
```

```
from functools import partial
```

```
# 데이터프레임 생성 예제 (df_prob1)
```

```
np.random.seed(0)
```

```
df_prob1 = pd.DataFrame({
    'cat_c': np.random.choice(['A', 'B', 'C', 'D'], size=500),
    'target': np.random.randn(500)
})
```

```
print('df_prob1')
```

```
print(df_prob1)
```

```
print('-----')
```

```
print(df_prob1['cat_c'].value_counts())
```

```
print('-----')
```

```
# 조건을 만족하는 행들 필터링
```

```
df_filtered = df_prob1.groupby('cat_c').filter(lambda x: len(x) >= 50)
```

```
print('df_filtered')
```

```
print(df_filtered)
```

```
print('++++')
```

```

# K-S 검정 결과 저장을 위한 리스트 초기화
ks_result = []

# 그룹별 K-S 검정 수행
for name, group in df_filtered.groupby('cat_c'):
    print('name = {}, group = {}'.format(name, group))
    print(type(group))
    print('~~~~~')
    # target 값을 표준화 (Z-score)
    z_scores = zscore(group['target'], ddof=1)

    # norm.cdf에 파라미터 설정
    norm_cdf = partial(norm.cdf, loc=0, scale=1)

    # K-S 검정 수행
    stat, pvalue = kstest(z_scores, norm_cdf)

    # 결과 저장 (cat_c, stat, pvalue, count)
    ks_result.append((name, stat, pvalue, len(group)))

# K-S 검정 결과 데이터프레임 생성
df_kstest = pd.DataFrame(ks_result, columns=['cat_c', 'stat', 'p value', 'cnt']).set_index('cat_c')

# p-value가 0.05 이상인 값만 선택하고 정렬
df_kstest = df_kstest.query('`p value` >= 0.05').sort_values(['cnt', 'p value'], ascending=[False, True])

# 상위 및 하위 3개 cat_c 값 선택
cat_c_A = df_kstest.iloc[:3].index.tolist()
cat_c_B = df_kstest.iloc[-3:].index.tolist()

# 결과 출력
print("cat_c_A (상위 3개):", cat_c_A)
print("cat_c_B (하위 3개):", cat_c_B)

```

```
df_prob1
   cat_c  target
0      A  0.523277
1      D -0.171546
2      B  0.771791
3      A  0.823504
4      D  2.163236
..     ...     ...
495     C  2.380745
496     D  0.330577
497     C  0.949246
498     B -1.502397
499     C -1.777667
```

```
[500 rows x 2 columns]
```

```
-----
D    140
A    127
B    122
C    111
```

```
Name: cat_c, dtype: int64
```

```
-----
df_filtered
   cat_c  target
0      A  0.523277
1      D -0.171546
2      B  0.771791
3      A  0.823504
4      D  2.163236
..     ...     ...
495     C  2.380745
496     D  0.330577
497     C  0.949246
498     B -1.502397
499     C -1.777667
```

```
[500 rows x 2 columns]
```

```
+++++
name = A, group = [   cat_c  target
0      A  0.523277
3      A  0.823504
12     A -0.024326
```

```

15      A -0.098150
16      A  0.910179
..      ...      ...
474     A -2.004216
475     A  0.376877
480     A  0.219510
482     A -0.938982
486     A -0.591403

```

```
[127 rows x 2 columns]]
```

```
<class 'pandas.core.frame.DataFrame'>
```

```

~~~~~
name = B, group = [   cat_c   target
2      B  0.771791
8      B  1.099660
10     B  0.640132
19     B -0.466419
25     B -0.042257
..      ...      ...
483     B  1.017021
488     B  0.755396
492     B  2.116791
494     B -0.035768
498     B -1.502397

```

```
[122 rows x 2 columns]]
```

```
<class 'pandas.core.frame.DataFrame'>
```

```

~~~~~
name = C, group = [   cat_c   target
11     C -1.616956
14     C  0.279925
18     C  0.786328
20     C -0.944446
23     C  0.379152
..      ...      ...
487     C  1.124419
493     C -1.610878
495     C  2.380745
497     C  0.949246
499     C -1.777667

```

```
[111 rows x 2 columns]]
```

```
<class 'pandas.core.frame.DataFrame'>
~~~~~
name = D, group = [   cat_c   target
1      D -0.171546
4      D  2.163236
5      D  1.336528
6      D -0.369182
7      D -0.239379
..    ...    ...
481    D  0.393063
489    D  0.867407
490    D -0.656464
491    D -2.834555
496    D  0.330577

[140 rows x 2 columns]]
<class 'pandas.core.frame.DataFrame'>
~~~~~
cat_c_A (상위 3개): ['D', 'A', 'B']
cat_c_B (하위 3개): ['A', 'B', 'C']
```

단계 1-3

bartlett 검정을 사용하여 cat_c_A에 해당하는 범주간에

target의 등분산성을 조사했을 때 검정 통계량을 A라 한다.

그리고 bartlett 검정을 사용하여 cat_c_B에 해당하는 범주간에

target의 등분산성을 조사했을 때 검정 통계량을 B라고 한다.

함수가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

In [102...

```
'''
`bartlett()` 함수는 Python의 `scipy.stats` 모듈에 포함된 함수로, Bartlett's test를 수행하는 데 사용됩니다.
Bartlett's test는 여러 그룹의 데이터가 동일한 분산을 가지는지(즉, 등분산성) 여부를 검정하는 가설 검정 방법입니다.

### `bartlett()` 함수의 설명
#### 1. 함수 정의
from scipy.stats import bartlett

#### 2. 함수 시그니처
scipy.stats.bartlett(*args)

#### 3. 인자
- `*args`:
    - 두 개 이상의 배열이나 시퀀스 데이터를 인자로 전달합니다. 각 인자는 그룹별 데이터 집합을 나타냅니다.
    - 최소 두 개 이상의 그룹이 필요하며, 각 그룹은 1차원 배열 형태로 제공됩니다.

    예시:
    group1 = [data1, data2, data3, ...]
    group2 = [data1, data2, data3, ...]
    ...
    bartlett(group1, group2, ...)

#### 4. 리턴값
- `statistic`:
    - `Bartlett's test`에서 계산된 검정 통계량입니다.
    - 이 값은 분산의 차이를 평가하기 위한 값으로, Chi-square 분포를 따릅니다.
- `p-value`:
    - 검정 통계량에 해당하는 p-값입니다.
    - p-값은 귀무가설(즉, 모든 그룹이 동일한 분산을 가진다)을 기각할 수 있는지 여부를 결정하는 데 사용됩니다.
    - 일반적으로 p-값이 0.05보다 작으면 귀무가설을 기각하고, 그룹 간에 분산 차이가 있다고 판단합니다.

리턴 예시:
(statistic, p-value)

### Bartlett's Test에 대한 개요
- 목적:
    - 여러 그룹 간의 분산이 동일한지(등분산성) 여부를 검정하는 것.
- 귀무가설:
    - 모든 그룹의 분산이 동일하다.
- 대립가설:
    - 적어도 하나의 그룹이 다른 그룹과 분산이 다르다.
```

- 검정 방식:

- **Bartlett's test**는 주어진 데이터 그룹 간의 분산을 비교하여 **Chi-square** 통계량을 계산하고, 이 통계량이 해당 **Chi-square** 분포에서 얼마나 유의미한지를 평가합니다.

주의 사항

- **Bartlett's test**는 데이터가 정규 분포를 따른다는 가정 하에 사용됩니다.
만약 데이터가 정규 분포를 따르지 않는다면, **Levene's test**와 같은 다른 등분산성 검정을 고려할 수 있습니다.

아래 예제 해석:

- 예시에서 **p-값**이 **0.05**보다 작으므로, 귀무가설을 기각할 수 있으며, 이는 그룹 간의 분산이 동일하지 않다는 것을 의미합니다.

```
from scipy.stats import bartlett
```

두 그룹의 데이터 예시

```
group1 = [9.1, 8.2, 8.6, 9.0, 9.5]
```

```
group2 = [7.8, 8.0, 8.3, 7.9, 8.1]
```

```
group3 = [6.7, 7.2, 7.0, 7.5, 7.8]
```

Bartlett's test 수행

```
statistic, p_value = bartlett(group1, group2, group3)
```

```
print("Bartlett's test statistic:", statistic)
```

```
print("p-value:", p_value)
```

```
Bartlett's test statistic: 2.9280906175994628
```

```
p-value: 0.2312987049422069
```

In [103...

```
'''
```

아래 코드는 **Bartlett** 검정을 사용하여 'cat_c' 열의 특정 그룹에 대한 'target' 열의 분산 동질성을 검정하는 과정을 포함합니다.

Bartlett 검정 수행

```
result_1_3_1 = bartlett(
    *df_prob1.loc[df_prob1['cat_c'].isin(cat_c_A)].groupby('cat_c')['target'].agg(list)
)
```

1. `df\_prob1.loc[df\_prob1['cat\_c'].isin(cat\_c\_A)`:

- `df\_prob1`에서 'cat_c' 열의 값이 `cat\_c\_A` 리스트에 포함된 행들만 선택합니다.

2. `.groupby('cat\_c')['target'].agg(list)`:

- 선택된 데이터프레임을 'cat_c' 열로 그룹화하고, 각 그룹의 'target' 열의 값을 리스트로 집계합니다.

3. `bartlett(\*grouped\_data)`:

- `groupby`와 `agg(list)`로 만들어진 시리즈의 각 리스트를 **Bartlett** 검정의 인자로 전달합니다.

```

`*`는 리스트의 각 요소를 개별 인자로 전달하는 unpacking 연산자입니다.
- Bartlett 검정을 수행하여, 분산의 동질성을 검정합니다. 결과는 `result_1_3_1`에 저장됩니다.

result_1_3_2 = bartlett(
    *df_prob1.loc[df_prob1['cat_c'].isin(cat_c_B)].groupby('cat_c')['target'].agg(list)
)

4. `df_prob1.loc[df_prob1['cat_c'].isin(cat_c_B)]:`
- `df_prob1`에서 'cat_c' 열의 값이 `cat_c_B` 리스트에 포함된 행들만 선택합니다.
5. `.groupby('cat_c')['target'].agg(list):`
- 선택된 데이터프레임을 'cat_c' 열로 그룹화하고, 각 그룹의 'target' 열의 값을 리스트로 집계합니다.
6. `bartlett(*grouped_data):`
- `groupby`와 `agg(list)`로 만들어진 시리즈의 각 리스트를 Bartlett 검정의 인자로 전달합니다.
- Bartlett 검정을 수행하여, 분산의 동질성을 검정합니다. 결과는 `result_1_3_2`에 저장됩니다.

### Bartlett 검정 통계량 저장
A = result_1_3_1.statistic
B = result_1_3_2.statistic

7. `result_1_3_1.statistic`:
- `result_1_3_1` 객체에서 Bartlett 검정 통계량을 추출하여 `A` 변수에 저장합니다.
8. `result_1_3_2.statistic`:
- `result_1_3_2` 객체에서 Bartlett 검정 통계량을 추출하여 `B` 변수에 저장합니다.

### 전체 동작 설명
1. `cat_c_A` 리스트에 포함된 'cat_c' 값들을 가진 그룹의 'target' 열의 값들에 대해 Bartlett 검정을 수행하고,
   결과를 `result_1_3_1`에 저장합니다.
2. `cat_c_B` 리스트에 포함된 'cat_c' 값들을 가진 그룹의 'target' 열의 값들에 대해 Bartlett 검정을 수행하고,
   결과를 `result_1_3_2`에 저장합니다.
3. 두 Bartlett 검정 결과에서 통계량을 추출하여 각각 `A`와 `B` 변수에 저장합니다.

이를 통해, 'cat_c_A'와 'cat_c_B'에 해당하는 그룹들 간의 'target' 열의 분산이 동질적인지 여부를 검정할 수 있습니다.
Bartlett 검정은 분산이 동일하다는 귀무가설을 검정하며,
p-값이 작으면 귀무가설을 기각하고 분산이 다르다고 결론지을 수 있습니다.
'''

from scipy.stats import bartlett

result_1_3_1 = bartlett(
    *df_prob1.loc[df_prob1['cat_c'].isin(cat_c_A)].groupby('cat_c')['target'].agg(list)
)

```

```

result_1_3_2 = bartlett(
    *df_prob1.loc[df_prob1['cat_c'].isin(cat_c_B)].groupby('cat_c')['target'].agg(list)
)

A = result_1_3_1.statistic
B = result_1_3_2.statistic

result_1_3_1, A, result_1_3_2, B

```

```

Out[103]: (BartlettResult(statistic=1.2287174474777187, pvalue=0.5409877066703417),
1.2287174474777187,
BartlettResult(statistic=0.4912286195875672, pvalue=0.7822238628353085),
0.4912286195875672)

```

단계 1-4

등분산성을 부정할 수 없는 경우는 경우는 단 하나 있다.(유의 수준: 5%)

이 경우에 해당하는 행들을 대상으로 해당 범주 3개에 대한

분산분석(ANOVA)를 수행하고 p-value를 C라고 한다.

함수가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

In [104...

```

'''
`f_oneway()` 함수는 Python의 `scipy.stats` 모듈에 포함된 함수로,
일원분산분석(ANOVA, Analysis of Variance)를 수행하는 데 사용됩니다.
이 함수는 여러 그룹의 평균이 통계적으로 유의미하게 다른지 검정하기 위한 방법입니다.

### `f_oneway()` 함수의 설명

#### 1. 함수 정의
from scipy.stats import f_oneway

```

2. 함수 시그니처

```
scipy.stats.f_oneway(*args)
```

3. 인자

```
- `*args`:
```

- 두 개 이상의 배열이나 시퀀스 데이터를 인자로 전달합니다. 각 인자는 그룹별 데이터 집합을 나타냅니다.
- 최소 두 개 이상의 그룹이 필요하며, 각 그룹은 1차원 배열 형태로 제공됩니다.
- 각 배열은 독립적인 그룹의 샘플을 나타내며, 이 함수는 이러한 그룹의 평균이 서로 다른지를 검정합니다.

예시:

```
group1 = [data1, data2, data3, ...]
group2 = [data1, data2, data3, ...]
...
f_oneway(group1, group2, ...)
```

4. 리턴값

```
- `statistic`:
```

- F 통계량입니다. 이 값은 그룹 간의 분산과 그룹 내 분산을 비교하여 계산된 값입니다.
- F 통계량이 클수록 그룹 간 평균 차이가 크다고 볼 수 있습니다.

```
- `p-value`:
```

- F 통계량에 해당하는 p-값입니다.
- p-값은 귀무가설(즉, 모든 그룹의 평균이 동일하다)을 기각할 수 있는지 여부를 결정하는 데 사용됩니다.
- 일반적으로 p-값이 0.05보다 작으면 귀무가설을 기각하고, 그룹 간에 평균 차이가 있다고 판단합니다.

리턴 예시:

```
(statistic, p-value)
```

일원분산분석(ANOVA)에 대한 개요

- 목적:
 - 여러 그룹 간의 평균이 동일한지 여부를 검정하는 것.
- 귀무가설:
 - 모든 그룹의 평균이 동일하다.
- 대립가설:
 - 적어도 하나의 그룹이 다른 그룹과 평균이 다르다.
- 검정 방식:
 - ANOVA는 그룹 간의 분산(변동)과 그룹 내의 분산을 비교하여 F-통계량을 계산하고, 이 통계량이 해당 F-분포에서 얼마나 유의미한지를 평가합니다.

주의사항

- ANOVA는 각 그룹의 데이터가 정규 분포를 따르고, 각 그룹의 분산이 동일하다는 가정을 바탕으로 합니다. 이러한 가정이 충족되지 않을 경우, 결과가 왜곡될 수 있으므로, 사전 검정을 통해 가정을 확인하는 것이 좋습니다.
- 만약 두 개 이상의 그룹 간에 유의미한 차이가 발견되면, 추가적인 사후 검정(Post-hoc test)을 통해

어떤 그룹들 간에 차이가 존재하는지 구체적으로 확인할 수 있습니다.

- 아래 예제 해석:

- 예시에서 p-값이 0.05보다 작으므로, 귀무가설을 기각할 수 있으며, 이는 그룹 간의 평균이 유의미하게 다르다는 것을 의미합니다.

...

```
from scipy.stats import f_oneway
```

세 그룹의 데이터 예시

```
group1 = [8.5, 9.2, 9.0, 9.6, 9.1]
```

```
group2 = [7.8, 8.0, 8.3, 7.9, 8.2]
```

```
group3 = [6.5, 6.9, 7.0, 7.3, 7.4]
```

일원분산분석(ANOVA) 수행

```
statistic, p_value = f_oneway(group1, group2, group3)
```

```
print("F-statistic:", statistic)
```

```
print("p-value:", p_value)
```

```
F-statistic: 48.66666666666665
```

```
p-value: 1.7481228208271677e-06
```

In [105...

...

아래 코드는 `cat\_c\_B` 리스트에 포함된 'cat_c' 값들에 대해 'target' 열의 값을 사용하여 일원분산분석 (ANOVA)을 수행하는 과정을 보여줍니다.

일원분산분석은 여러 그룹 간의 평균 차이를 검정하는 통계적 방법입니다.

```
result_1_4 = f_oneway(
    *df_prob1.loc[df_prob1['cat_c'].isin(cat_c_B)].groupby('cat_c')['target'].agg(list)
)
```

동작 과정

1. `df\_prob1.loc[df\_prob1['cat\_c'].isin(cat\_c\_B)`:

- `df\_prob1` 데이터프레임에서 'cat_c' 열의 값이 `cat\_c\_B` 리스트에 포함된 행들만 선택합니다.

- `cat\_c\_B`는 이전 코드에서 필터링된 하위 3개의 'cat_c' 값을 포함하는 리스트입니다.

2. `.groupby('cat\_c')['target'].agg(list)`:

- 선택된 데이터프레임을 'cat_c' 열로 그룹화하고, 각 그룹의 'target' 열의 값을 리스트로 집계합니다.

- 이 결과는 'cat_c' 값을 인덱스로 하고, 각 'cat_c' 값에 대해 'target' 값들의 리스트를 값으로 갖는 시리즈가 됩니다.

3. `f\_oneway(\*grouped\_data)`:

- `agg(list)`로 만들어진 시리즈의 각 리스트를 ANOVA 검정의 인자로 전달합니다.

- ``\*``는 리스트의 각 요소를 개별 인자로 전달하는 **unpacking** 연산자입니다.
- 따라서, `f_oneway` 함수는 각 그룹의 `'target'` 값 리스트를 개별 인자로 받아 처리합니다.
- `f_oneway` 함수는 일원분산분석을 수행하여 각 그룹 간의 평균 차이가 유의미한지 검정합니다.

결과 저장

4. `result_1_4`:

- `f_oneway` 함수의 결과는 `result_1_4` 객체에 저장됩니다. 이 객체는 F-통계량과 p-값을 포함합니다.

전체 동작 설명

1. `df_prob1` 데이터프레임에서 `cat_c_B` 리스트에 포함된 `'cat_c'` 값들을 가진 행들만 선택합니다.
2. 선택된 데이터프레임을 `'cat_c'` 기준으로 그룹화하고, 각 그룹의 `'target'` 값을 리스트로 집계합니다.
3. 집계된 리스트들을 `f_oneway` 함수에 개별 인자로 전달하여 일원분산분석을 수행합니다.
4. ANOVA 검정 결과인 F-통계량과 p-값을 `result_1_4` 객체에 저장합니다.

이를 통해, `cat_c_B`에 포함된 그룹들 간의 `'target'` 값들의 평균 차이가 유의미한지 검정할 수 있습니다. ANOVA 검정은 귀무가설이 모든 그룹의 평균이 같다는 가설을 검정하며, p-값이 작으면 귀무가설을 기각하고 그룹 간 평균 차이가 유의미하다고 결론지을 수 있습니다.

```
from scipy.stats import f_oneway
```

```
result_1_4 = f_oneway(
    *df_prob1.loc[df_prob1['cat_c'].isin(cat_c_B)].groupby('cat_c')['target'].agg(list)
)
```

```
C = result_1_4.pvalue
```

```
result_1_4, C
```

```
Out[105]: (F_onewayResult(statistic=0.9835616919481085, pvalue=0.37498778787469267),
0.37498778787469267)
```

```
In [106... A, B, C, A + B + C, round(A + B + C, 3)
```

```
Out[106]: (1.2287174474777187,
0.4912286195875672,
0.37498778787469267,
2.0949338549399785,
2.095)
```

A + B + C 를 소수점 넷째 자리에서 반올림하여 셋째 자리까지 표시하라.

10.740

In [107... `# 단계 1-3, 1-4 설명``...`

코드는 특정 조건을 만족하는 데이터에 대해 **Bartlett's** 테스트와 ANOVA (F-test)를 수행하여 분산의 동질성 및 평균의 차이를 검정합니다. 이를 통해 데이터의 통계적 특성을 파악할 수 있습니다.

1. `bartlett` 테스트 수행

`result\_1\_3\_1`

- `df\_prob1` 데이터프레임에서 `cat\_c` 값이 `cat\_c\_A` 목록에 포함된 행들을 선택합니다.
- 선택된 데이터프레임을 `cat\_c` 값으로 그룹화하고, 각 그룹의 `target` 값을 리스트로 변환합니다.
- 변환된 리스트들을 `bartlett` 함수에 전달하여 **Bartlett's** 테스트를 수행합니다.
- **Bartlett's** 테스트는 여러 그룹의 데이터에 대해 분산이 동일한지 (동질적 분산성) 여부를 검정합니다.

`result\_1\_3\_2`

- `df\_prob1` 데이터프레임에서 `cat\_c` 값이 `cat\_c\_B` 목록에 포함된 행들을 선택합니다.
- 선택된 데이터프레임을 `cat\_c` 값으로 그룹화하고, 각 그룹의 `target` 값을 리스트로 변환합니다.
- 변환된 리스트들을 `bartlett` 함수에 전달하여 **Bartlett's** 테스트를 수행합니다.

2. 테스트 결과에서 통계량 추출

- `result\_1\_3\_1`과 `result\_1\_3\_2` 객체에서 **Bartlett's** 테스트의 통계량(**statistic**)을 추출하여 변수 `A`와 `B`에 저장합니다.

3. ANOVA (F-test) 수행

`result\_1\_4`

- `df\_prob1` 데이터프레임에서 `cat\_c` 값이 `cat\_c\_B` 목록에 포함된 행들을 선택합니다.
- 선택된 데이터프레임을 `cat\_c` 값으로 그룹화하고, 각 그룹의 `target` 값을 리스트로 변환합니다.
- 변환된 리스트들을 `f\_oneway` 함수에 전달하여 ANOVA (F-test)를 수행합니다.
- ANOVA는 여러 그룹의 평균이 동일한지 (동질적 평균성) 여부를 검정합니다.

4. ANOVA 테스트 결과에서 p-value 추출

- `result\_1\_4` 객체에서 ANOVA 테스트의 p-value를 추출하여 변수 `C`에 저장합니다.

요약

- **Bartlett's** 테스트 (`result\_1\_3\_1`, `result\_1\_3\_2`):
 - `df\_prob1` 데이터프레임의 특정 그룹 (`cat\_c\_A` 및 `cat\_c\_B`)에 대해 분산의 동질성을 검정합니다.
 - 테스트 결과에서 통계량(**statistic**)을 변수 `A`와 `B`에 저장합니다.

```
- ANOVA (F-test) (`result_1_4`):
- `df_prob1` 데이터프레임의 특정 그룹 (`cat_c_B`)에 대해 평균의 차이를 검정합니다.
- 테스트 결과에서 p-value를 변수 `C`에 저장합니다.

### 코드의 주요 목표
- Bartlett's 테스트를 통해 선택된 그룹들의 분산이 동일한지 확인합니다.
- ANOVA (F-test)를 통해 선택된 그룹들의 평균이 동일한지 확인합니다.
- 이 테스트들은 데이터의 통계적 특성을 파악하는 데 도움을 줍니다.
  Bartlett's 테스트는 분산의 동질성, ANOVA는 평균의 동질성을 검정합니다.
...
print('')
```

문제 2

연속형 변수 cont0 ~ 13 중에서, 서로 피어슨 상관도(Pearson correlation)가 높은 변수들 찾고,

이들을 PCA를 통한 차원 축소를 하여 성능 향상이 있는지 확인하고자 한다.

단계 2-1

basetable 에서 인덱스와 컬럼 모두 cont0 ~ 13이고,

인덱스와 컬럼에 해당하는 변수 간에 피어슨 상관도(Pearson correlation)를

값으로 하는 대각 대칭형인 데이터프레임을 corr을 만든다.

```
In [108... df_corr = df_basetable[cont_cols].corr()
df_corr
```

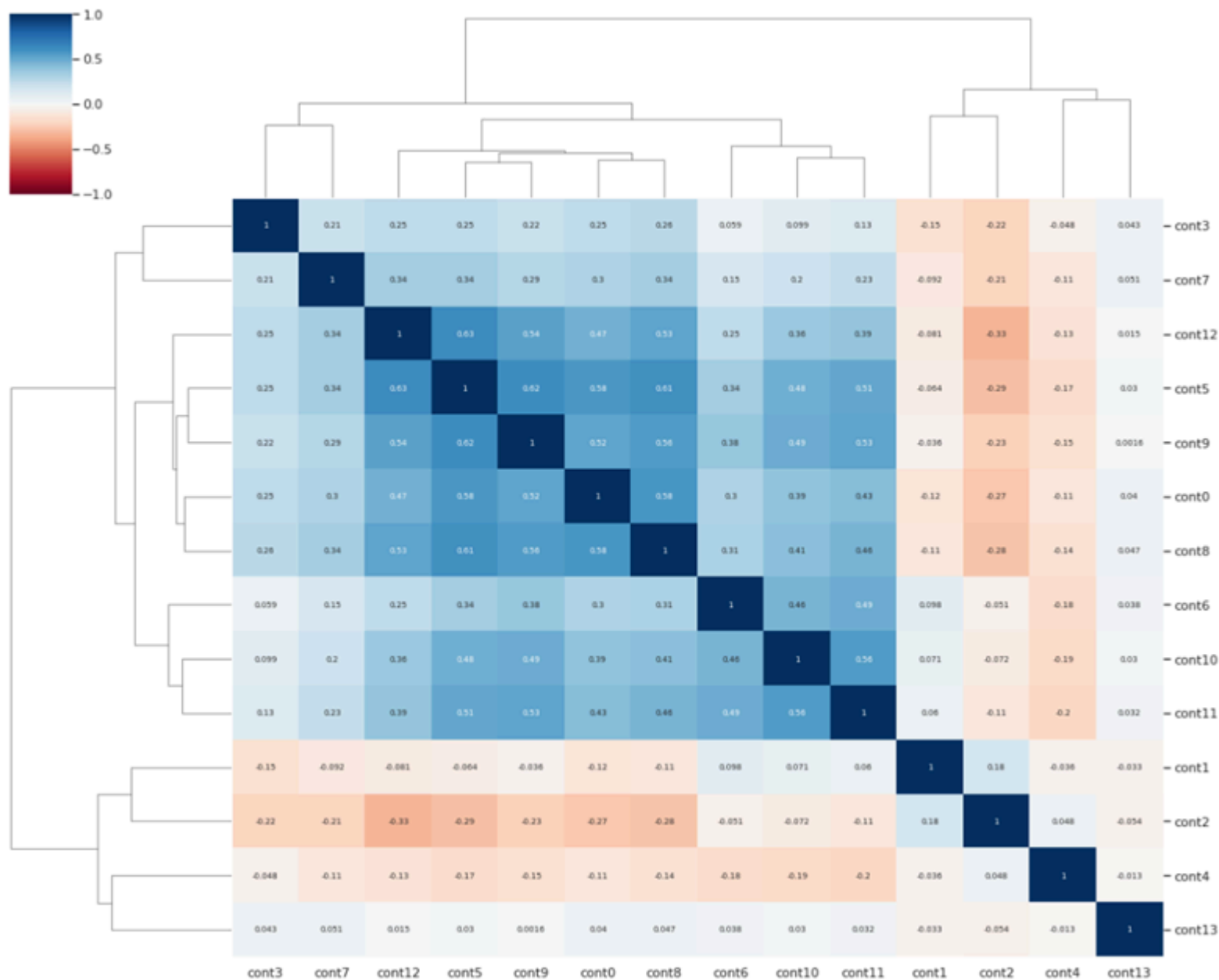
Out[108]:

	cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	cont10	cont11	cont12	cor
cont0	1.000000	-0.123344	-0.268621	0.255262	-0.108197	0.573894	0.301672	0.300353	0.579618	0.524481	0.394401	0.425350	0.473249	0.039
cont1	-0.123344	1.000000	0.174625	-0.153110	-0.035455	-0.064297	0.099449	-0.091482	-0.105811	-0.035944	0.070926	0.060877	-0.080143	-0.034
cont2	-0.268621	0.174625	1.000000	-0.216238	0.047065	-0.291732	-0.050488	-0.213688	-0.278496	-0.227655	-0.071688	-0.107875	-0.329195	-0.054
cont3	0.255262	-0.153110	-0.216238	1.000000	-0.048677	0.253592	0.059515	0.215057	0.264717	0.220375	0.099694	0.131967	0.253720	0.043
cont4	-0.108197	-0.035455	0.047065	-0.048677	1.000000	-0.166160	-0.175280	-0.105320	-0.136829	-0.150984	-0.187024	-0.199560	-0.127953	-0.012
cont5	0.573894	-0.064297	-0.291732	0.253592	-0.166160	1.000000	0.341552	0.341689	0.606329	0.615449	0.482025	0.509316	0.631540	0.029
cont6	0.301672	0.099449	-0.050488	0.059515	-0.175280	0.341552	1.000000	0.151666	0.307044	0.381141	0.455662	0.485435	0.246285	0.037
cont7	0.300353	-0.091482	-0.213688	0.215057	-0.105320	0.341689	0.151666	1.000000	0.334679	0.283921	0.200623	0.228630	0.337916	0.049
cont8	0.579618	-0.105811	-0.278496	0.264717	-0.136829	0.606329	0.307044	0.334679	1.000000	0.558560	0.413619	0.458089	0.530215	0.047
cont9	0.524481	-0.035944	-0.227655	0.220375	-0.150984	0.615449	0.381141	0.283921	0.558560	1.000000	0.484661	0.529252	0.538278	0.000
cont10	0.394401	0.070926	-0.071688	0.099694	-0.187024	0.482025	0.455662	0.200623	0.413619	0.484661	1.000000	0.557161	0.361102	0.028
cont11	0.425350	0.060877	-0.107875	0.131967	-0.199560	0.509316	0.485435	0.228630	0.458089	0.529252	0.557161	1.000000	0.385753	0.031
cont12	0.473249	-0.080143	-0.329195	0.253720	-0.127953	0.631540	0.246285	0.337916	0.530215	0.538278	0.361102	0.385753	1.000000	0.013
cont13	0.039613	-0.034036	-0.054582	0.043121	-0.012127	0.029530	0.037730	0.049162	0.047070	0.000957	0.028532	0.031311	0.013784	1.000

단계 2-2

계층적 군집화를 활용하여 다음 개념도(실제 결과와는 상관 없음)와 같이

연관성이 높은 변수 군집을 찾아 보고자 한다.



초기에는 cont0~13 변수 각각을 하나의 군집으로 배정한 상태에서 시작하여,

군집 간 결합 조건으로 두 군집에 포함된 변수간의 피어슨 상관도 절대값의 평균이 0.5이상 일 때,

피어슨 상관도의 절대값이 높은 것을 우선으로 군집들을 결합한다. 군집의 크기가 가장 큰 변수들을 구하라.

Hint] cont0 ~ 13간의 거리는 [단계 2-1] corr을 이용하여 군집 결합 조건을 고려하여 구한다.

(두 변수간에 피어슨 상관도의 절대값이 높을 수록 거리는 가깝다.)

함수가이드

sklearn.cluster.AgglomerativeClustering

문제 지시사항 외 Default 값 사용

In [109...

```
# 역수를 취하는 접근법은 평균을 구하는 과정에서의 왜곡이 발생하여 성립하지 않습니다.
# df_dist = 1 / df_corr.abs() - 1

# 상관도를 문제에서 제시한 요건에 맞게 거리로 바꿉니다.
# -1, 1에 가까울수록 유사 → 거리가 가까움
# 군집간 거리의 평균을 임계값으로 잡고 있다는 점에서 -abs를 취합니다.
# 자신의 거리는 0이라는 점에서. 1을 더해줍니다.
# 절대값의 평균 0.5 이상일때 → 거리가 0.5 이하일 때

df_dist = 1 - df_corr.abs()
df_dist
```

Out[109]:

	cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	cont10	cont11	cont12	cont13
cont0	0.000000	0.876656	0.731379	0.744738	0.891803	0.426106	0.698328	0.699647	0.420382	0.475519	0.605599	0.574650	0.526751	0.960387
cont1	0.876656	0.000000	0.825375	0.846890	0.964545	0.935703	0.900551	0.908518	0.894189	0.964056	0.929074	0.939123	0.919857	0.965964
cont2	0.731379	0.825375	0.000000	0.783762	0.952935	0.708268	0.949512	0.786312	0.721504	0.772345	0.928312	0.892125	0.670805	0.945418
cont3	0.744738	0.846890	0.783762	0.000000	0.951323	0.746408	0.940485	0.784943	0.735283	0.779625	0.900306	0.868033	0.746280	0.956879
cont4	0.891803	0.964545	0.952935	0.951323	0.000000	0.833840	0.824720	0.894680	0.863171	0.849016	0.812976	0.800440	0.872047	0.987873
cont5	0.426106	0.935703	0.708268	0.746408	0.833840	0.000000	0.658448	0.658311	0.393671	0.384551	0.517975	0.490684	0.368460	0.970470
cont6	0.698328	0.900551	0.949512	0.940485	0.824720	0.658448	0.000000	0.848334	0.692956	0.618859	0.544338	0.514565	0.753715	0.962270
cont7	0.699647	0.908518	0.786312	0.784943	0.894680	0.658311	0.848334	0.000000	0.665321	0.716079	0.799377	0.771370	0.662084	0.950838
cont8	0.420382	0.894189	0.721504	0.735283	0.863171	0.393671	0.692956	0.665321	0.000000	0.441440	0.586381	0.541911	0.469785	0.952930
cont9	0.475519	0.964056	0.772345	0.779625	0.849016	0.384551	0.618859	0.716079	0.441440	0.000000	0.515339	0.470748	0.461722	0.999043
cont10	0.605599	0.929074	0.928312	0.900306	0.812976	0.517975	0.544338	0.799377	0.586381	0.515339	0.000000	0.442839	0.638898	0.971468
cont11	0.574650	0.939123	0.892125	0.868033	0.800440	0.490684	0.514565	0.771370	0.541911	0.470748	0.442839	0.000000	0.614247	0.968689
cont12	0.526751	0.919857	0.670805	0.746280	0.872047	0.368460	0.753715	0.662084	0.469785	0.461722	0.638898	0.614247	0.000000	0.986216
cont13	0.960387	0.965964	0.945418	0.956879	0.987873	0.970470	0.962270	0.950838	0.952930	0.999043	0.971468	0.968689	0.986216	0.000000

In [110...]

```
'''
`AgglomerativeClustering`은 scikit-learn 라이브러리에서 제공하는 계층적 클러스터링 알고리즘입니다.
이 알고리즘은 데이터 포인트들을 점진적으로 병합하여 계층적 클러스터를 생성합니다.

### `AgglomerativeClustering` 클래스의 인자
class sklearn.cluster.AgglomerativeClustering(
    n_clusters=2,
    *,
    affinity='euclidean',
    memory=None,
    connectivity=None,
    compute_full_tree='auto',
    linkage='ward',
    distance_threshold=None
)
```

1. `n_clusters (int, default=2):`
 - 클러스터의 수를 지정합니다. ``distance_threshold``가 지정되지 않은 경우에만 사용됩니다.
2. `affinity (str or callable, default='euclidean'):`
 - 데이터 간의 거리를 계산하는 방식입니다. 사용 가능한 값으로는 `'euclidean', 'l1', 'l2', 'manhattan', 'cosine', 'precomputed'` 등이 있습니다. ``linkage='ward'``인 경우 `'euclidean'`만 사용할 수 있습니다.
3. `memory (str or object, default=None):`
 - 캐시 디렉토리를 지정하거나 `joblib.Memory` 객체를 사용할 수 있습니다. 디스크 캐시를 통해 계산을 저장할 수 있습니다.
4. `connectivity (array-like or callable, default=None):`
 - 데이터의 연결을 정의하는 커넥티비티 행렬입니다. 데이터 간의 연결성을 나타냅니다.
5. `compute_full_tree ('auto' or bool, default='auto'):`
 - True로 설정하면 모든 데이터 포인트에 대해 전체 트리를 계산합니다. 클러스터의 수가 지정되지 않은 경우에만 사용됩니다.
6. `linkage (str, default='ward'):`
 - 클러스터 병합 기준입니다. 사용 가능한 값으로는 `'ward', 'complete', 'average', 'single'`이 있습니다.
 - `'ward'`: 클러스터 내 분산을 최소화합니다.
 - `'complete'`: 두 클러스터 간 최대 거리 (최대 연결).
 - `'average'`: 두 클러스터 간 평균 거리 (평균 연결).
 - `'single'`: 두 클러스터 간 최소 거리 (최소 연결).
7. `distance_threshold (float, default=None):`
 - 클러스터링을 멈출 거리 임계값입니다. 설정되면 클러스터의 수는 ``distance_threshold``에 따라 결정됩니다.

리턴값

- `labels_ (array, shape [n_samples]):`
 - 각 데이터 포인트의 클러스터 레이블을 나타냅니다.

속성

- `n_clusters_:`
 - 최종 클러스터의 수를 나타냅니다. ``distance_threshold``가 설정된 경우 사용됩니다.
- `labels_:`
 - 각 데이터 포인트의 클러스터 레이블을 나타냅니다.
- `n_leaves_:`
 - 클러스터링의 트리 리프 수를 나타냅니다.
- `n_connected_components_:`
 - 연결된 구성 요소의 수를 나타냅니다.
- `children_:`
 - 각 병합 단계에서 병합된 클러스터의 인덱스를 나타내는 배열입니다.

메서드

- `fit(X[, y]):`
 - 모델을 데이터 X에 맞춥니다.
- `fit_predict(X[, y]):`
 - 모델을 데이터 X에 맞추고, 클러스터 레이블을 반환합니다.

- `get_params([deep]):`
- 추정기의 매개변수를 반환합니다.
- `set_params(params):`
- 추정기의 매개변수를 설정합니다.

사용 예제

아래 예제에서는 `AgglomerativeClustering` 객체를 생성하고 데이터에 맞춘 후, 클러스터 레이블과 병합된 클러스터 정보를 출력합니다. ``AgglomerativeClustering``은 계층적 클러스터링의 강력한 도구로, 다양한 거리 측정 방법과 병합 기준을 통해 유연하게 클러스터링을 수행할 수 있습니다.

```
import numpy as np
from sklearn.cluster import AgglomerativeClustering

# 데이터 생성
X = np.array([[1, 2], [1, 4], [1, 0],
              [4, 2], [4, 4], [4, 0]])

# AgglomerativeClustering 객체 생성
clustering = AgglomerativeClustering(n_clusters=2, affinity='euclidean', linkage='ward')

# 데이터에 적합
clustering.fit(X)

# 결과 출력
print("클러스터 레이블:", clustering.labels_)
print("병합된 클러스터:")
print(clustering.children_)
print("리프 수:", clustering.n_leaves_)
print("연결된 구성 요소 수:", clustering.n_connected_components_)
```

클러스터 레이블: [1 1 1 0 0 0]

병합된 클러스터:

[[0 1]

[3 5]

[2 6]

[4 7]

[8 9]]

리프 수: 6

연결된 구성 요소 수: 1

```
In [111... from sklearn.cluster import AgglomerativeClustering

clu_agg = AgglomerativeClustering(
    n_clusters = None, # distance_threshold에 의해 결정
    affinity = 'precomputed', # 거리행렬입력 받도록 설정
    linkage = 'average', # 피어슨 상관도 절대값의 평균
    distance_threshold = 0.5 # 0.5이상 일 때→ 이하
)

clu_agg.fit(df_dist)

# 결과 출력
print("클러스터 레이블:", clu_agg.labels_)
print("병합된 클러스터:")
print(clu_agg.children_)
print("리프 수:", clu_agg.n_leaves_)
print("연결된 구성 요소 수:", clu_agg.n_connected_components_)
```

클러스터 레이블: [0 5 6 4 7 0 8 2 0 0 1 1 0 3]

병합된 클러스터:

```
[[ 5 12]
 [ 0  8]
 [ 9 14]
 [10 11]
 [15 16]
 [ 6 17]
 [18 19]
 [ 7 20]
 [ 2  3]
 [21 22]
 [ 4 23]
 [ 1 24]
 [13 25]]
```

리프 수: 14

연결된 구성 요소 수: 1

```
In [112... pd.Series(clu_agg.labels_).value_counts().to_frame().T
```

```
Out[112]:
```

	0	1	8	7	6	5	4	3	2
0	5	2	1	1	1	1	1	1	1

```
In [113... pd.Series(clu_agg.labels_).value_counts().idxmax()
```

```
Out[113]: 0
```

```
In [114... X_corr = df_dist.loc[
    clu_agg.labels_ == pd.Series(clu_agg.labels_).value_counts().idxmax()
].index.tolist()

X_corr
```

```
Out[114]: ['cont0', 'cont5', 'cont8', 'cont9', 'cont12']
```

```
In [115... '''
`scipy.cluster.hierarchy.linkage()` 함수는 계층적 클러스터링(Hierarchical Clustering)을 수행하기 위해 사용되는 함수입니다.
이 함수는 데이터 간의 거리를 기반으로 클러스터를 생성하고, 클러스터 간의 계층적 구조를 구축합니다.

### `linkage()` 함수의 설명

#### 1. 함수 정의
from scipy.cluster.hierarchy import linkage

#### 2. 함수 시그니처
scipy.cluster.hierarchy.linkage(y, method='single', metric='euclidean', optimal_ordering=False)

#### 3. 인자
- `y`:
    - 클러스터링할 데이터로, `n x m` 크기의 2차원 배열 또는 `(n * (n-1)) / 2` 크기의 1차원 배열을 입력으로 받습니다.
    - 만약 2차원 배열이라면, 각 행은 하나의 관측치를 나타내며, `linkage()` 함수는 이를 통해 거리 행렬을 계산합니다.
    - 1차원 배열인 경우, 이미 거리 행렬의 상삼각 부분이 계산된 상태(즉, `pdist`와 같은 함수의 결과)로 입력됩니다.
- `method`:
    - 클러스터 간의 거리를 계산하는 방법을 지정합니다.
    - 가능한 값으로는 `'single'`, `'complete'`, `'average'`, `'weighted'`, `'centroid'`, `'median'`, `'ward'` 등이 있습니다.
    - 기본값은 `'single'`이며, 이는 최단 연결법을 사용하여 클러스터 간의 최소 거리를 기준으로 병합합니다.
    주요 클러스터링 방법들:
    - `'single'`: 최단 연결법, 두 클러스터 간의 가장 가까운 점들 사이의 거리를 기준으로 병합.
    - `'complete'`: 최장 연결법, 두 클러스터 간의 가장 먼 점들 사이의 거리를 기준으로 병합.
    - `'average'`: 평균 연결법, 두 클러스터 간의 모든 점들의 평균 거리를 기준으로 병합.
    - `'ward'`: Ward의 최소 분산 방법, 클러스터의 내적 분산의 증가를 최소화하는 방향으로 병합.
- `metric`:
    - 거리를 계산하는 방법을 지정합니다. 기본값은 `'euclidean'` (유클리드 거리)입니다.
    - `metric`에는 `pdist`에서 사용할 수 있는 다양한 거리 계산 방법들이 포함됩니다
```

(예: `'cityblock'`, `'cosine'`, `'hamming'`, `'jaccard'`, `'minkowski'` 등).

- `optimal_ordering``:
 - 최적의 클러스터 순서를 찾을지 여부를 지정합니다. 기본값은 `False`입니다.
 - `True`로 설정하면 덴드로그램에서 리프 노드들이 최적화된 순서로 배치되어, 같은 클러스터에 속하는 항목들이 인접하게 표시됩니다.
 - 이 옵션을 사용하면 덴드로그램의 시각적 해석이 더 명확해질 수 있습니다.

4. 리턴값

- `Z``:
 - `linkage()` 함수는 계층적 클러스터링의 결과를 나타내는 행렬 `Z``를 반환합니다.
 - `Z``는  $(n-1) \times 4$  크기의 2차원 배열로, 클러스터 병합의 단계별 정보를 포함하고 있습니다. 여기서 `n``은 입력 데이터의 샘플 수입니다.

각 행은 클러스터 병합의 단계를 나타내며, 각 열은 다음 정보를 포함합니다:

1. 첫 번째 클러스터의 인덱스: 병합된 첫 번째 클러스터의 인덱스.
2. 두 번째 클러스터의 인덱스: 병합된 두 번째 클러스터의 인덱스.
3. 병합된 클러스터 사이의 거리: 병합 시 두 클러스터 간의 거리.
4. 병합된 클러스터의 크기: 병합된 클러스터에 속하는 원소의 수.

예시:

```
Z = linkage(data, method='single')
```

반환된 `Z``의 첫 번째 행이 `[3. , 4. , 1. , 2.]``이라면, 이는 인덱스 3과 4에 해당하는 클러스터가 거리 1에서 병합되었으며, 병합 후 클러스터 크기는 2임을 의미합니다.

5. 속성 및 함수

`linkage()` 함수 자체에는 속성이나 메서드가 따로 존재하지 않지만, 반환된 `Z`` 행렬을 이용해 다양한 분석을 수행할 수 있습니다.

- `scipy.cluster.hierarchy.dendrogram()`:
 - `Z`` 행렬을 이용해 덴드로그램(계층적 클러스터링의 시각적 표현)을 그릴 수 있습니다.

예시:

```
from scipy.cluster.hierarchy import dendrogram
import matplotlib.pyplot as plt
```

```
# 덴드로그램 그리기
dendrogram(Z)
plt.show()
```

- `scipy.cluster.hierarchy.fcluster()`:
 - `Z`` 행렬을 기반으로 클러스터를 형성하는 데 사용됩니다. 주어진 임계값을 넘는 클러스터들로 데이터를 나눌 수 있습니다.

```
예시 :
from scipy.cluster.hierarchy import fcluster

# 클러스터 형성
clusters = fcluster(Z, t=1.5, criterion='distance')

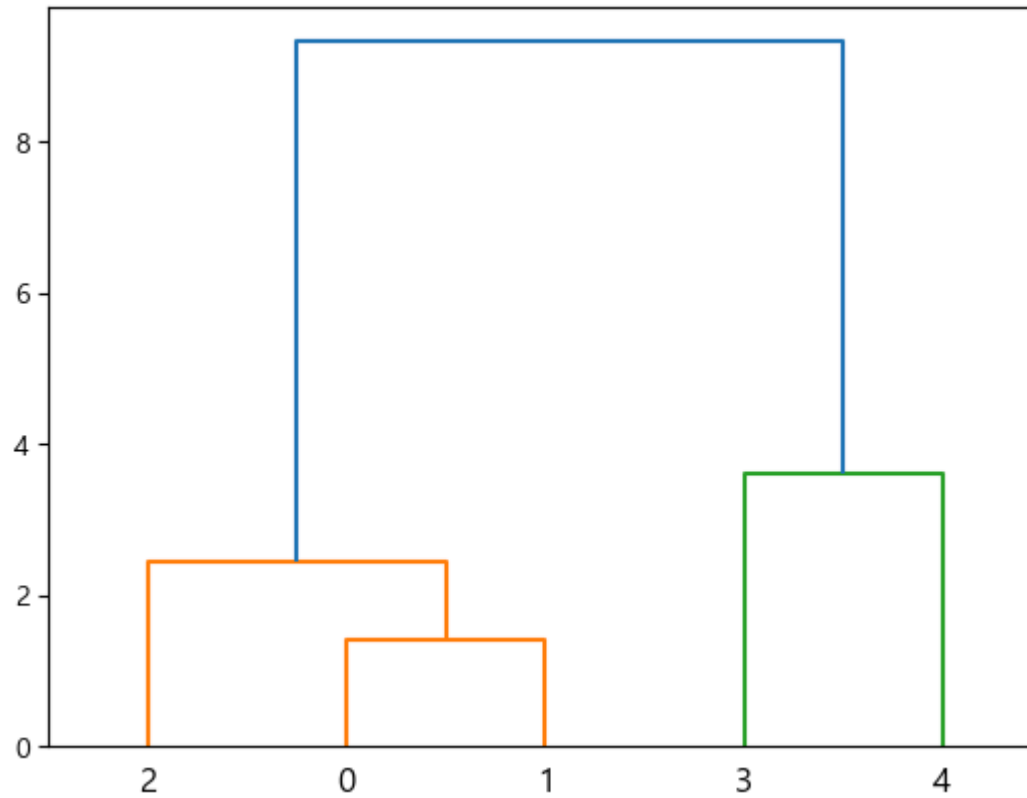
### 요약
- `scipy.cluster.hierarchy.linkage()` 함수는 계층적 클러스터링을 수행하여 데이터 간의 계층적 구조를 표현하는 함수입니다.
- 다양한 방법(method)과 거리(metric)를 통해 클러스터링을 수행할 수 있으며, 결과로 계층적 병합 정보가 담긴 행렬 `Z`를 반환합니다.
- 반환된 `Z`를 이용해 덴드로그램을 그리거나, 클러스터를 형성할 수 있습니다.
...

import numpy as np
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt

# 예제 데이터 생성
data = np.array([[1, 2], [2, 3], [3, 4], [5, 6], [8, 8]])

# 계층적 클러스터링 수행
Z = linkage(data, method='ward')

# 덴드로그램 시각화
dendrogram(Z)
plt.show()
```



```
In [116... # 속성에 대한 연관성으로 사용하고 있는 방법입니다.
# scipy에 있는 cluster 모듈로 구현해보고
# 속성간에 연관성을 시각화 해봅니다.
from scipy.cluster import hierarchy
from scipy.spatial.distance import squareform

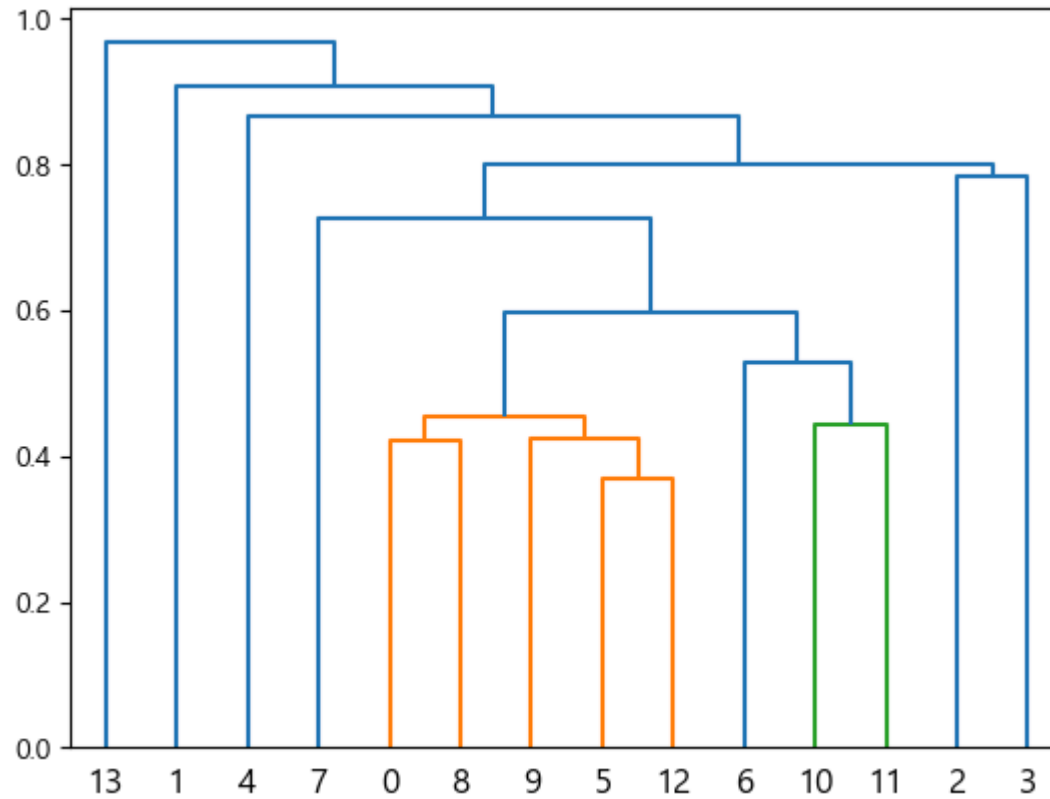
df_corr = df_basetable[['cont{}'.format(i) for i in range(14)]].corr()
df_dist = 1 - df_corr.abs()

# scipy.cluster.hierarchy를 통해 군집화를 합니다.
Z = hierarchy.linkage(squareform(df_dist), method='average')
print('Z = ')
print(Z)
print('-----')

plt.figure()
```

```
# 군집 결합 임계점을 0.5로 하여 색상으로 군집들을 표시합니다.  
hierarchy.dendrogram(Z, color_threshold=0.5)  
plt.show()
```

```
Z =  
[[ 5.         12.         0.36845954  2.         ]  
 [ 0.          8.         0.42038217  2.         ]  
 [ 9.         14.         0.42313695  3.         ]  
 [10.         11.         0.44283934  2.         ]  
 [15.         16.         0.45554535  5.         ]  
 [ 6.         17.         0.52945163  3.         ]  
 [18.         19.         0.59858252  8.         ]  
 [ 7.         20.         0.72756524  9.         ]  
 [ 2.          3.         0.78376232  2.         ]  
 [21.         22.         0.80037013 11.         ]  
 [ 4.         23.         0.86790466 12.         ]  
 [ 1.         24.         0.90871139 13.         ]  
 [13.         25.         0.96757262 14.         ]]
```



In [117...

단계 2-2까지 정리

...

코드는 연속형 변수들 간의 상관계수를 이용하여 군집화를 수행하고, 가장 큰 군집의 변수들을 식별한 후, 이를 시각화합니다.

1. 상관계수 계산 및 거리 행렬 생성

```
df_corr = df_basetable[cont_cols].corr()
```

- `df\_basetable` 데이터프레임에서 연속형 변수(`cont\_cols`)들 간의 상관계수 행렬을 계산합니다.

```
df_dist = 1 - df_corr.abs()
```

- 상관계수의 절대값을 이용하여 거리 행렬을 생성합니다.

상관계수가 높을수록 거리가 짧아지며, 상관계수가 낮을수록 거리가 길어집니다.

2. 병합 군집화 (Agglomerative Clustering) 수행

- `AgglomerativeClustering`을 사용하여 병합 군집화를 수행합니다.

- `n\_clusters=None`:

클러스터의 수는 `distance\_threshold`에 의해 결정됩니다.

- `affinity='precomputed'`:
거리 행렬을 직접 입력받도록 설정합니다.
- `linkage='average'`:
평균 연결법을 사용하여 클러스터 간 거리를 계산합니다.
- `distance\_threshold=0.5`:
이 값 이하에서 클러스터를 형성합니다.

3. 각 클러스터의 크기 확인

```
pd.Series(clu_agg.labels_).value_counts().to_frame().T
```

- 각 클러스터에 속한 변수의 개수를 확인합니다.
- 클러스터 레이블을 시리즈로 변환하고, 각 클러스터의 빈도수를 계산하여 데이터프레임 형식으로 출력합니다.

4. 가장 큰 클러스터의 변수들 식별

```
X_corr = df_dist.loc[
    clu_agg.labels_ == pd.Series(clu_agg.labels_).value_counts().idxmax()
].index.tolist()
```

- 가장 큰 클러스터의 레이블을 식별하고, 그 클러스터에 속한 변수들의 목록을 추출합니다.
- `clu\_agg.labels\_`와 가장 큰 클러스터의 레이블이 일치하는 변수들의 인덱스를 리스트로 변환합니다.

5. 덴드로그램을 통한 군집 시각화

```
df_corr = df_basetable[['cont{}'.format(i) for i in range(14)]].corr()
df_dist = 1 - df_corr.abs()
```

- 상관관계수 행렬을 다시 계산하고 거리 행렬을 생성합니다.

```
Z = hierarchy.linkage(squareform(df_dist), method='average')
```

- `scipy.cluster.hierarchy`를 사용하여 평균 연결법을 통해 거리 행렬을 기반으로 군집화를 수행합니다.
- `squareform`을 사용하여 거리 행렬을 계층적 군집화 함수가 사용할 수 있는 형태로 변환합니다.

```
plt.figure()
hierarchy.dendrogram(Z, color_threshold=0.5)
plt.show()
```

- 덴드로그램을 생성하여 군집 구조를 시각화합니다.
- `color\_threshold=0.5`를 설정하여 특정 거리 임계값에서 클러스터를 색상으로 표시합니다.

코드의 전체 목표

- 연속형 변수들 간의 상관관계를 이용하여 군집화를 수행합니다.
- 병합 군집화를 통해 변수들을 클러스터로 그룹화하고, 가장 큰 클러스터에 속한 변수들을 식별합니다.
- 덴드로그램을 통해 군집화 결과를 시각화하여 변수 간의 관계를 시각적으로 이해할 수 있도록 합니다.

이 과정은 변수 간의 상관성을 기반으로 변수들을 그룹화하여, 데이터의 구조적 관계를 파악하는 데 도움을 줍니다.

```
'''
print('')
```

단계 2-3

basetable에서 cat0 ~ 9을 각 변수의 첫 번째 범주(알파벳 순)를 제외하고

원핫 인코딩(One-hot encoding)하여 prob2를 만든다.

In [118...

```
# 방법 1.
df_prob2 = pd.get_dummies(df_basetable, columns=cat_cols, drop_first=True)

df_prob2.head()
```

Out[118]:

	cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	...	cat9_F	cat9_G	cat9_H	cat9_I	cat9_J	cat9_K	cat9_L
id																		
267387	0.65226	0.55771	0.25398	0.28196	0.82400	0.22443	0.69220	0.30912	0.47134	0.53290	...	0	1	0	0	0	0	
410037	0.36970	0.68159	0.30038	0.20156	0.46113	0.43278	0.97199	0.62596	0.40193	0.49181	...	1	0	0	0	0	0	
139373	0.47881	0.63976	0.40024	0.24153	0.27128	0.36238	0.41310	0.27010	0.40112	0.53052	...	0	0	0	0	1	0	
113765	0.43438	0.77575	0.54887	0.21935	0.27944	0.32670	0.42301	0.28331	0.46425	0.38302	...	0	1	0	0	0	0	
179915	0.47866	0.64326	0.37255	0.21004	0.41188	0.20121	0.37466	0.46316	0.75578	0.55676	...	0	0	0	0	0	0	

5 rows × 45 columns

In [119...

```
df_prob2.columns
```

```
Out[119]: Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7',
                'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target',
                'targetA', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C',
                'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E',
                'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F',
                'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M',
                'cat9_N', 'cat9_O'],
                dtype='object')
```

In [120...

```
'''
`OneHotEncoder`는 `sklearn.preprocessing` 모듈에 포함된 클래스입니다.
이 클래스는 범주형 데이터를 숫자 형태의 원-핫 인코딩(One-Hot Encoding)으로 변환하는 데 사용됩니다.
원-핫 인코딩은 범주형 변수를 이진 벡터로 변환하여, 머신러닝 모델에 적합한 입력 데이터로 만듭니다.

#### `OneHotEncoder` 클래스의 설명
#### 1. 클래스 정의
from sklearn.preprocessing import OneHotEncoder

#### 2. 생성자 시그니처
OneHotEncoder(*, categories='auto', drop=None, sparse_output=True, dtype=<class 'numpy.float64'>,
               handle_unknown='error', min_frequency=None, max_categories=None, feature_name_combiner='concat')

#### 3. 주요 인자
- `categories`:
  - 각 특성(feature)에 대해 인코딩할 범주를 지정합니다.
  - 기본값은 `'auto'`로, 자동으로 데이터를 학습하여 범주를 결정합니다.
  - 사용자가 명시적으로 범주의 리스트를 제공할 수도 있습니다.
  예시:
  categories=[['A', 'B', 'C'], [1, 2, 3]]

- `drop`:
  - 일부 범주를 드롭하여 다중공선성을 줄일 수 있습니다.
  - `'first'`: 첫 번째 범주를 드롭.
  - `'if_binary'`: 이진 범주형 변수의 경우 첫 번째 범주를 드롭.
  - `'none'` (기본값): 범주를 드롭하지 않음.
  - 사용자가 명시적으로 드롭할 범주의 리스트를 제공할 수도 있습니다.

- `sparse_output`:
  - `True`로 설정하면 희소 행렬(sparse matrix) 형태로 결과를 반환합니다. 기본값은 `True`입니다.
  - `False`로 설정하면 밀집 행렬(dense matrix)로 결과를 반환합니다.

- `dtype`:
```

- 인코딩된 결과의 데이터 타입을 지정합니다.
- 기본값은 ``numpy.float64``입니다.
- ``handle_unknown``:
 - 인코딩 시 학습되지 않은 범주를 어떻게 처리할지 지정합니다.
 - ``'error'`` (기본값): 알 수 없는 범주가 나타나면 오류를 발생시킵니다.
 - ``'ignore'``: 알 수 없는 범주를 무시하고, 해당 범주의 원-핫 인코딩은 모두 0으로 처리합니다.
- ``min_frequency`` 및 ``max_categories`` (버전 1.2부터 추가):
 - ``min_frequency``: 최소 빈도를 설정해 이보다 작은 빈도의 범주는 드롭할 수 있습니다.
 - ``max_categories``: 최대 범주 수를 설정해 이보다 많은 범주를 가진 경우 일부를 드롭할 수 있습니다.
- ``feature_name_combiner`` (버전 1.3부터 추가):
 - 기본값은 ``'concat'``으로, 원-핫 인코딩된 피처의 이름을 결합하여 형성합니다.

4. 리턴값

- ``fit_transform`` 또는 ``transform`` 메서드를 사용하면 원-핫 인코딩된 결과를 반환합니다.
- 이 결과는 기본적으로 희소 행렬(``scipy.sparse.csr_matrix``) 형태이며, ``sparse_output=False``로 설정하면 밀집 행렬(``numpy.ndarray``)로 반환됩니다.

5. 주요 속성

- ``categories_``:
 - 인코더가 학습한 범주들의 리스트입니다.
 - 각 입력 변수에 대한 범주의 목록을 포함하는 리스트가 반환됩니다.

예시:

```
encoder.categories_
```

- ``drop_idx_``:
 - 드롭된 범주의 인덱스를 나타냅니다.
 - ``drop`` 옵션이 설정된 경우에만 이 속성이 유효합니다.

6. 주요 메서드

- ``fit(X[, y])``:
 - ``X``를 학습하여 인코딩할 범주를 결정합니다.
 - ``X``는 2차원 배열, 데이터프레임 또는 리스트의 리스트 형태로 제공됩니다.
- ``fit_transform(X[, y])``:
 - ``fit``을 수행한 후, ``X``를 변환하여 원-핫 인코딩된 결과를 반환합니다.
 - ``fit``과 ``transform``을 연달아 수행할 때 사용됩니다.
- ``transform(X)``:

- 학습된 범주에 따라 `X`를 원-핫 인코딩된 결과로 변환합니다.
- `X`는 `fit`과 동일한 형태의 데이터여야 합니다.
- `inverse\_transform(X)` :
 - 원-핫 인코딩된 데이터 `X`를 원래 범주형 데이터로 변환합니다.
 - 인코딩 이전의 데이터로 복원하는 데 사용됩니다.
- `get\_feature\_names\_out([input\_features])` :
 - 원-핫 인코딩된 피처들의 이름을 반환합니다.
 - `input\_features`는 원본 피처 이름의 리스트로, 이를 이용해 새로운 피처 이름을 생성할 수 있습니다.

예시 :

```
encoder.get_feature_names_out(['feature1', 'feature2'])
...
```

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder

# 예제 데이터
data = np.array([[ 'Male', 1], [ 'Female', 3], [ 'Female', 2]])

# OneHotEncoder 인스턴스 생성 (sparse_output -> sparse)
encoder = OneHotEncoder(sparse=False) # sparse_output을 sparse로 변경

# 데이터 학습 및 변환
encoded_data = encoder.fit_transform(data)

# 결과 출력
print(encoded_data)
print(encoder.categories_)

# get_feature_names로 피처 이름 출력
print(encoder.get_feature_names(['gender', 'category']))

# print(encoder.get_feature_names_out(['gender', 'category']))

[[0.  1.  1.  0.  0.]
 [1.  0.  0.  0.  1.]
 [1.  0.  0.  1.  0.]]
[array(['Female', 'Male'], dtype='<U6'), array(['1', '2', '3'], dtype='<U6')]
['gender_Female' 'gender_Male' 'category_1' 'category_2' 'category_3']
```

In [121... df_basetable[cat_cols]

Out[121]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9
id										
267387	A	B	B	C	B	B	A	E	G	G
410037	A	A	A	A	B	B	A	E	C	F
139373	A	A	B	A	B	B	A	E	C	J
113765	A	A	A	C	B	B	A	E	C	G
179915	A	B	A	C	B	B	A	E	E	L
...	...	...	...	...	...	...	...	...	...	...
320975	A	A	A	C	B	B	A	E	C	I
29591	A	B	A	C	B	B	A	E	C	F
46717	A	B	B	C	B	D	A	E	G	F
463191	A	A	A	C	B	D	A	E	E	F
415856	A	A	A	C	B	D	A	E	G	I

240000 rows × 10 columns

In [122...

```
'''
df_basetable[cat_cols].apply(lambda x: print(x), axis=0)
```

[결과]

id

267387 A

410037 A

139373 A

113765 A

179915 A

..

320975 A

29591 A

46717 A

```
463191    A
415856    A
Name: cat0, Length: 240000, dtype: object
id
267387    B
410037    A
139373    A
113765    A
179915    B
..
320975    A
29591     B
46717     B
463191    A
415856    A
Name: cat1, Length: 240000, dtype: object
id
267387    B
410037    A
139373    B
113765    A
179915    A
..
320975    A
29591     A
46717     B
463191    A
415856    A
Name: cat2, Length: 240000, dtype: object
id
267387    C
410037    A
139373    A
113765    C
179915    C
..
320975    C
29591     C
46717     C
463191    C
415856    C
Name: cat3, Length: 240000, dtype: object
```

```
id
267387    B
410037    B
139373    B
113765    B
179915    B
..
320975    B
29591     B
46717     B
463191    B
415856    B
Name: cat4, Length: 240000, dtype: object
id
267387    B
410037    B
139373    B
113765    B
179915    B
..
320975    B
29591     B
46717     D
463191     D
415856     D
Name: cat5, Length: 240000, dtype: object
id
267387    A
410037    A
139373    A
113765    A
179915    A
..
320975    A
29591     A
46717     A
463191    A
415856    A
Name: cat6, Length: 240000, dtype: object
id
267387    E
410037    E
```

```
139373      E
113765      E
179915      E
..
320975      E
29591       E
46717       E
463191      E
415856      E
Name: cat7, Length: 240000, dtype: object
id
267387      G
410037      C
139373      C
113765      C
179915      E
..
320975      C
29591       C
46717       G
463191      E
415856      G
Name: cat8, Length: 240000, dtype: object
id
267387      G
410037      F
139373      J
113765      G
179915      L
..
320975      I
29591       F
46717       F
463191      F
415856      I
Name: cat9, Length: 240000, dtype: object
cat0      None
cat1      None
cat2      None
cat3      None
cat4      None
cat5      None
```

```
cat6      None
cat7      None
cat8      None
cat9      None
dtype: object
'''
print('')
```

In [123...

```
# 방법 2.

from sklearn.preprocessing import OneHotEncoder

# drop 매개변수를 통해 제외할 가변수화에서 제외할 수준을 지정할 수 있습니다.
ohe = OneHotEncoder(
    # 사전순으로 첫번째 범주값을 제외합니다.
    drop=df_basetable[cat_cols].apply(lambda x: np.sort(x.unique())[0], axis=0),
    sparse=False, # pd.DataFrame에 넣을 수 있도록, Dense Matrix로 받습니다.
    dtype=np.int64
)

ohe.fit(df_basetable[cat_cols])
```

```
Out[123]: OneHotEncoder(categorical_features=None, categories=None,
                        drop=array(['A', 'A', 'A', 'A', 'B', 'A', 'A', 'B', 'A', 'A'], dtype=object),
                        dtype=<class 'numpy.int64'>, handle_unknown='error',
                        n_values=None, sparse=False)
```

In [124...

```
# 변수명을 만들어 냅니다.
dum_cols = ohe.get_feature_names(cat_cols)

print('dum_cols = ', dum_cols)
print('-----')
print('cat_cols = ', cat_cols)
print('-----')
print('df_basetable.columns = ', df_basetable.columns)
print('-----')

df_prob2 = pd.concat([
    df_basetable.drop(columns=cat_cols),
    pd.DataFrame(ohe.transform(df_basetable[cat_cols]),
        index=df_basetable.index, columns=dum_cols)
```

```

], axis=1)

print('df_prob2.columns = ', df_prob2.columns)
print('-----')

dum_cols = ['cat0_B' 'cat1_B' 'cat2_B' 'cat3_C' 'cat3_D' 'cat4_C' 'cat5_B' 'cat5_C'
            'cat5_D' 'cat6_B' 'cat6_C' 'cat7_D' 'cat7_E' 'cat7_G' 'cat8_C' 'cat8_D'
            'cat8_E' 'cat8_G' 'cat9_B' 'cat9_F' 'cat9_G' 'cat9_H' 'cat9_I' 'cat9_J'
            'cat9_K' 'cat9_L' 'cat9_M' 'cat9_N' 'cat9_O']

-----
cat_cols = ['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8', 'cat9']
-----
df_basetable.columns = Index(['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8',
                              'cat9', 'cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6',
                              'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13',
                              'target', 'targetA'],
                              dtype='object')
-----
df_prob2.columns = Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7',
                          'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target',
                          'targetA', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C',
                          'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E',
                          'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F',
                          'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M',
                          'cat9_N', 'cat9_O'],
                          dtype='object')
-----

```

In [125...

단계 2-3 설명

...

코드는 범주형 변수를 원-핫 인코딩(One-Hot Encoding)을 통해 더미 변수로 변환하여 모델링에 사용할 수 있는 형태로 만드는 과정입니다.
구체적으로, `pd.get\_dummies`와 `OneHotEncoder`를 사용하여 범주형 변수를 처리하고, 결과를 기존 데이터프레임에 결합하는 것을 포함합니다.

1. `pd.get\_dummies`를 이용한 원-핫 인코딩

- `pd.get\_dummies`를 사용하여 `df\_basetable`의 범주형 변수(`cat\_cols`)들을 원-핫 인코딩합니다.
- `drop\_first=True`를 설정하여 각 범주형 변수의 첫 번째 수준을 제외합니다.
이는 다중공선성을 피하기 위해 사용됩니다.
- 결과는 원-핫 인코딩된 데이터프레임 `df\_prob2`입니다.

```

#### 2. `OneHotEncoder` 설정 및 학습
- `OneHotEncoder` 객체를 생성합니다.
- `drop` 매개변수: 각 범주형 변수의 사전순 첫 번째 범주값을 제외합니다.
  `np.sort(x.unique())[0]`를 사용하여 각 범주형 변수의 고유값을 정렬하고 첫 번째 값을 선택합니다.
- `sparse=False`: 희소 행렬이 아닌 밀집 행렬로 출력합니다. 이는 `pd.DataFrame`으로 변환할 때 유용합니다.
- `dtype=np.int64`: 결과 데이터의 타입을 `int64`로 설정합니다.
- `ohe.fit`을 사용하여 `df_basetable[cat_cols]`의 범주형 변수들에 대해 `OneHotEncoder`를 학습시킵니다.

#### 3. 원-핫 인코딩된 컬럼 이름 추출
- `OneHotEncoder` 객체에서 원-핫 인코딩된 컬럼 이름을 추출합니다.
  이는 나중에 새로운 데이터프레임을 생성할 때 사용됩니다.

#### 4. 데이터 변환 및 결합
- `df_basetable`에서 원래의 범주형 변수(`cat_cols`)들을 드롭합니다.
- `ohe.transform`을 사용하여 `df_basetable[cat_cols]`의 범주형 변수들을 원-핫 인코딩합니다.
- `pd.DataFrame`을 사용하여 원-핫 인코딩된 결과를 데이터프레임으로 변환하고,
  `dum_cols`를 컬럼 이름으로 사용합니다.
- `pd.concat`을 사용하여 원래의 데이터프레임(`df_basetable`에서 범주형 변수를 드롭한 것)과
  원-핫 인코딩된 데이터프레임을 결합합니다. 결합은 열 방향(`axis=1`)으로 수행됩니다.

#### 코드의 전체 목표
코드는 범주형 변수를 원-핫 인코딩하여 모델링에 사용할 수 있도록 변환하는 것입니다.
원-핫 인코딩은 범주형 데이터를 수치형 데이터로 변환하여 머신러닝 모델에서 사용할 수 있도록 합니다.
여기서는 두 가지 방법(`pd.get_dummies`와 `OneHotEncoder`)을 사용하여 원-핫 인코딩을 수행하였으며,
각 방법은 다중공선성을 피하기 위해 첫 번째 범주값을 드롭하는 방법을 사용했습니다.
결과적으로, 범주형 변수들이 수치형 더미 변수로 변환되어 기존의 데이터프레임과 결합됩니다.
'''
print('')

```

단계 2-4

prob2에서 80%는 학습용 prob2_train으로 20%는 테스트용 prob2_test으로 분리한다.

함수가이드

sklearn.model_selection.train_test_split, random_state=123

문제 지시사항 외 Default 값 사용

```
In [126... from sklearn.model_selection import train_test_split

df_prob2_train, df_prob2_test = train_test_split(df_prob2, train_size=0.8, random_state=123)
df_prob2_train, df_prob2_test = df_prob2_train.copy(), df_prob2_test.copy()
```

```
In [127... '''
`PCA` (Principal Component Analysis, 주성분 분석)는 `sklearn.decomposition` 모듈에 포함된 클래스입니다.
이 클래스는 고차원의 데이터를 저차원으로 변환하는 차원 축소 기법을 제공합니다.
PCA는 데이터의 분산을 최대한 설명하는 주성분(Principal Components)을 찾고, 이를 기반으로 데이터를 투영합니다.

### `PCA` 클래스의 설명

#### 1. 클래스 정의
from sklearn.decomposition import PCA

#### 2. 생성자 시그니처
PCA(n_components=None, *, copy=True, whiten=False, svd_solver='auto', tol=0.0, iterated_power='auto', random_state=None)

#### 3. 주요 인자
- `n_components`:
  - 변환 후 유지할 차원의 수 또는 주성분의 수를 지정합니다.
  - 기본값은 `None`이며, 이 경우 모든 주성분이 반환됩니다.
  - `n_components`를 다음과 같이 설정할 수 있습니다:
    - 정수값: 특정 차원 수로 축소.
    - 실수값 ( $0.0 < n\_components \leq 1.0$ ): 누적 분산 비율이 지정된 값만큼 되는 최소한의 주성분 수.
    - `mle`: 분산을 최적화하는 차원 수를 자동으로 선택 (단, `svd_solver='full'`일 때만 사용 가능).
    - `None`: 모든 주성분을 반환.
- `copy`:
  - `True`로 설정하면 입력 데이터를 복사하여 사용합니다. `False`로 설정하면 데이터를 그대로 덮어씁니다.
  - 기본값은 `True`입니다.
- `whiten`:
  - `True`로 설정하면 주성분을 구할 때 각 주성분 벡터의 크기를 고정(정규화)하여 데이터가 구면 형태로 변환됩니다.
  - `False`로 설정하면 원본 분산을 유지합니다.
  - 회귀 분석이나 신경망 등에서 주성분을 사용할 때 유용할 수 있습니다.
  - 기본값은 `False`입니다.
- `svd_solver`:
  - SVD(특이값 분해)를 계산하는 방법을 지정합니다.
  - 가능한 값:
    - `auto`: 입력 데이터에 따라 자동으로 최적의 방법을 선택합니다.
```

- ``full``: 기본적인 SVD 방법을 사용합니다.
- ``arpack``: 희소 행렬의 경우 ARPACK 알고리즘을 사용합니다.
- ``randomized``: 큰 데이터셋의 경우 빠르게 계산할 수 있는 확률적 알고리즘을 사용합니다.
- ``tol``:
 - ``arpack`` 방식에서 사용하는 수렴 허용 오차를 지정합니다.
 - 기본값은 ``0.0``입니다.
- ``iterated_power``:
 - ``randomized`` 방식에서 사용되는 거듭제곱(iterative) 횟수를 지정합니다.
 - 기본값은 ``auto``입니다.
- ``random_state``:
 - 난수 발생기의 시드 값을 지정합니다. 이 값이 고정되면 동일한 결과를 재현할 수 있습니다.
 - 기본값은 ``None``입니다.

4. 리턴값

- ``fit`` 메서드와 ``fit_transform`` 메서드는 PCA 모델을 학습시키고 데이터를 변환합니다.
- ``transform`` 메서드는 주어진 데이터를 학습된 PCA 모델에 따라 변환합니다.
- ``inverse_transform`` 메서드는 주성분 공간에서 원래 공간으로 데이터를 변환합니다.

5. 주요 속성

- ``components_``:
 - 주성분을 나타내는 벡터들의 배열입니다.
 - 각 행이 하나의 주성분을 나타내며, 크기는 ``(n_components, n_features)``입니다.
 - 이 속성은 주성분이 데이터의 각 차원에 대한 기여도를 나타냅니다.
- ``explained_variance_``:
 - 각 주성분에 의해 설명되는 분산의 크기입니다.
 - 크기가 큰 주성분일수록 더 많은 데이터의 분산을 설명합니다.
- ``explained_variance_ratio_``:
 - 각 주성분에 의해 설명되는 분산 비율입니다. 전체 분산 중 해당 주성분이 설명하는 비율을 나타냅니다.
 - 이 값들을 모두 더하면 1이 됩니다.
- ``singular_values_``:
 - 각 주성분에 해당하는 특이값(Singular Values)입니다.
 - SVD를 통해 계산된 특이값들이며, 데이터 행렬의 ``L2`` 노름(norm)과 관련됩니다.
- ``mean_``:
 - 입력 데이터의 평균입니다.
 - PCA는 데이터를 중앙 정렬(centered)하기 때문에, 이 평균은 나중에 데이터를 원래 공간으로 변환하는 데 사용될 수 있습니다.
- ``n_components_``:
 - 선택된 주성분의 수입니다.
 - 이는 ``n_components`` 인자에 따라 달라집니다.
- ``n_features_``:
 - 입력 데이터의 피처 수입니다.
- ``n_samples_``:
 - 입력 데이터의 샘플 수입니다.

```

- `noise_variance`:
  - MLE 방식에서 추정된 잡음의 분산입니다. 다른 SVD 방식에서는 정의되지 않습니다.

#### 6. 주요 메서드
- `fit(X[, y])`:
  - 데이터 `X`를 사용하여 PCA 모델을 학습합니다. 주성분들을 계산하고 필요한 속성들을 설정합니다.
- `fit_transform(X[, y])`:
  - `fit`과 `transform`을 연달아 수행하며, 데이터를 바로 변환합니다. 학습된 PCA 모델에 따라 데이터를 변환한 결과를 반환합니다.
- `transform(X)`:
  - 학습된 PCA 모델을 사용하여 데이터 `X`를 주성분 공간으로 변환합니다. 원래의 고차원 데이터를 저차원으로 투영합니다.
- `inverse_transform(X)`:
  - 주성분 공간에서 원래의 고차원 공간으로 데이터를 변환합니다.
  - `transform`으로 변환된 데이터를 다시 원래의 형태로 복원하는 데 사용됩니다.
- `fit_predict(X)`:
  - `fit`과 `predict`를 연달아 수행합니다. 주성분 분석 후 데이터를 예측하여 클러스터링을 수행하는 등에서 사용할 수 있습니다.

### 요약
- `PCA`는 데이터를 저차원으로 축소하면서 최대한의 분산을 유지하는 데 사용됩니다.
- `n_components`는 유지할 차원의 수를 결정하며, `fit`, `transform`, `fit_transform`, `inverse_transform` 등의 메서드를 사용하여 PCA를 수행할 수 있습니다.
- 주성분, 설명된 분산 비율 등 여러 속성을 통해 데이터의 변환 및 분석을 이해할 수 있습니다.
...

import numpy as np
from sklearn.decomposition import PCA

# 예제 데이터 생성
X = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9],
              [1.9, 2.2], [3.1, 3.0], [2.3, 2.7],
              [2, 1.6], [1, 1.1], [1.5, 1.6], [1.1, 0.9]])

# PCA 인스턴스 생성 및 학습
pca = PCA(n_components=2)
pca.fit(X)

print("X:\n", X)
print('-----')

# 주성분들
print("주성분들:\n", pca.components_)
print('-----')

```

```
# 설명된 분산 비율
print("설명된 분산 비율:\n", pca.explained_variance_ratio_)
print('-----')

# 데이터 변환 (차원 축소)
X_pca = pca.transform(X)
print("변환된 데이터:\n", X_pca)
print('-----')

# 원래 공간으로 복원
X_original = pca.inverse_transform(X_pca)
print("복원된 데이터:\n", X_original)
print('-----')
```

X:

```
[[2.5 2.4]
 [0.5 0.7]
 [2.2 2.9]
 [1.9 2.2]
 [3.1 3. ]
 [2.3 2.7]
 [2.  1.6]
 [1.  1.1]
 [1.5 1.6]
 [1.1 0.9]]
```

주 성분들 :

```
[[ -0.6778734  -0.73517866]
 [ -0.73517866   0.6778734 ]]
```

설명된 분산 비율 :

```
[0.96318131 0.03681869]
```

변환된 데이터 :

```
[[ -0.82797019 -0.17511531]
 [ 1.77758033  0.14285723]
 [ -0.99219749  0.38437499]
 [ -0.27421042  0.13041721]
 [ -1.67580142 -0.20949846]
 [ -0.9129491   0.17528244]
 [ 0.09910944 -0.3498247 ]
 [ 1.14457216  0.04641726]
 [ 0.43804614  0.01776463]
 [ 1.22382056 -0.16267529]]
```

복원된 데이터 :

```
[[2.5 2.4]
 [0.5 0.7]
 [2.2 2.9]
 [1.9 2.2]
 [3.1 3. ]
 [2.3 2.7]
 [2.  1.6]
 [1.  1.1]
 [1.5 1.6]
```

[1.1 0.9]]

In [128...

```
'''
`PCA` (Principal Component Analysis) 함수의 `n_components` 인자는 PCA 수행 시 주성분을 몇 개 선택할 것인지
또는 주성분의 분산 비율을 얼마나 설명할 것인지를 결정하는 중요한 파라미터입니다.
이 인자는 여러 가지 방식으로 설정될 수 있으며, 이에 따라 PCA의 결과와 목적이 달라집니다.

1. 정수 값 (1 이상의 정수)로 전달할 때:
    - `n_components`가 1 이상의 정수일 때, 이는 데이터에서 추출할 주성분의 개수를 의미합니다.
    - 예를 들어, `n_components=3`이라고 설정하면, PCA는 데이터의 차원을 3개의 주성분으로 축소합니다.
      이 주성분들은 원래 데이터의 분산을 최대한 설명하는 방향으로 정해집니다.
    - 이러한 설정은 주로 데이터의 차원을 특정 개수의 주성분으로 축소하고자 할 때 사용됩니다.
      예를 들어, 시각화 목적으로 2개 또는 3개의 주성분으로 차원을 줄이거나,
      모델의 입력 특성을 줄이기 위해 특정 개수의 주성분을 선택할 수 있습니다.

2. 0과 1 사이의 실수 값으로 전달할 때:
    - `n_components`가 0과 1 사이의 실수일 때, 이는 전체 분산 중 얼마나 많은 비율을 설명할 주성분을 선택할 것인지를 의미합니다.
    - 예를 들어, `n_components=0.8`이라고 설정하면, PCA는 원래 데이터의 분산의 80% 이상을 설명하는 최소 개수의 주성분을 선택합니다.
    - 이러한 설정은 주로 데이터의 분산을 최대한 보존하면서 차원을 축소하고자 할 때 사용됩니다.
      이 방법은 데이터의 구조와 분산을 보다 잘 보존할 수 있는 주성분을 선택하는데 유용합니다.

### 차이점 요약
- 정수 값 설정:
    - 명시적으로 선택한 주성분의 개수로 차원을 축소합니다.
    - 특정 개수의 주성분으로 데이터를 표현하고자 할 때 사용됩니다.
    - 주로 차원 축소의 결과를 예측 가능하게 하기 위해 사용됩니다.

- 실수 값 설정:
    - 데이터의 분산 비율을 기준으로 주성분을 선택합니다.
    - 선택된 주성분들이 원래 데이터의 분산을 얼마나 잘 설명하는지를 기준으로 합니다.
    - 주로 데이터의 정보를 최대한 보존하면서 차원을 축소하고자 할 때 사용됩니다.

아래 예시에서 `n_components=2`로 설정하면 결과 데이터의 차원이 (100, 2)가 되고,
`n_components=0.8`로 설정하면 분산의 80%를 설명하는 주성분을 선택하므로
결과 데이터의 차원은 (100, k)로 설정되며 k는 데이터에 따라 다릅니다.
'''

from sklearn.decomposition import PCA
import numpy as np

# 샘플 데이터 생성
X = np.random.rand(100, 5)
```

```
# n_components를 정수로 설정
pca_int = PCA(n_components=2)
X_reduced_int = pca_int.fit_transform(X)
print(f"Shape with integer n_components: {X_reduced_int.shape}")

# n_components를 실수로 설정
pca_float = PCA(n_components=0.8)
X_reduced_float = pca_float.fit_transform(X)
print(f"Shape with float n_components: {X_reduced_float.shape}")
```

```
Shape with integer n_components: (100, 2)
Shape with float n_components: (100, 4)
```

In [129...

```
'''
PCA에서 언급되는 분산 비율은 데이터의 총 분산 중에서 각 주성분이 차지하는 비율을 의미합니다.
이를 통해 주성분들이 원본 데이터의 정보를 얼마나 잘 설명하는지 평가할 수 있습니다.

### 분산 비율의 의미
1. 총 분산:
    - 원본 데이터의 모든 특성(차원)에 대한 분산의 합을 말합니다.
      분산은 데이터가 평균으로부터 얼마나 퍼져 있는지를 나타내는 척도입니다.
2. 주성분의 분산:
    - PCA는 원본 데이터의 분산을 최대한 보존하면서 차원을 축소하는 기법입니다.
      PCA를 수행하면 주성분(Principal Components)이 생성되고,
      각 주성분은 원본 데이터의 방향을 나타내며 그 방향으로의 분산을 설명합니다.
    - 각 주성분이 차지하는 분산은 고유값(eigenvalue)으로 표현됩니다.

3. 분산 비율:
    - 각 주성분의 분산이 전체 분산에서 차지하는 비율을 분산 비율(Explained Variance Ratio)이라고 합니다.
    - 예를 들어, 첫 번째 주성분이 전체 분산의 40%를 설명하고, 두 번째 주성분이 30%를 설명한다면,
      두 개의 주성분이 총 분산의 70%를 설명하는 셈입니다.

### 예시로 설명
데이터를 예로 들어 설명하겠습니다. 다음은 PCA를 통해 얻은 주성분과 그 분산 비율입니다:
```

주성분	분산 (고유값)	분산 비율
PC1	5.0	50%
PC2	3.0	30%
PC3	2.0	20%

이 경우, 첫 번째 주성분(PC1)이 전체 데이터의 분산 중 50%를 설명하고, 두 번째 주성분(PC2)이 30%를 설명하며, 세 번째 주성분(PC3)이 20%를 설명합니다. 분산 비율은 각 주성분의 분산을 총 분산으로 나눈 값입니다.

코드 예시

아래 예시에서 첫 번째 주성분이 40%의 분산을, 두 번째 주성분이 30%의 분산을, 세 번째 주성분이 20%의 분산을 설명하며, 누적 분산 비율을 보면 세 번째 주성분까지 합쳐서 90%의 분산을 설명하는 것을 알 수 있습니다.

따라서, `n\_components`를 0과 1 사이의 값으로 설정할 때 이 값은 설명하려는 총 분산 비율을 의미합니다. 예를 들어, `n\_components=0.8`로 설정하면, PCA는 원래 데이터의 80% 이상의 분산을 설명하는 최소 개수의 주성분을 선택합니다.

```
from sklearn.decomposition import PCA
import numpy as np

# 샘플 데이터 생성
X = np.random.rand(100, 5)

# PCA 수행
pca = PCA()
pca.fit(X)

# 주성분의 분산 비율 출력
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# 누적 분산 비율 출력
print("Cumulative Explained Variance Ratio:", np.cumsum(pca.explained_variance_ratio_))
```

```
Explained Variance Ratio: [0.25701429 0.23015573 0.19439485 0.18689409 0.13154104]
Cumulative Explained Variance Ratio: [0.25701429 0.48717002 0.68156487 0.86845896 1.          ]
```

단계 2-5

prob2_train으로 [단계 2-2]에서 도출된 변수들로 PCA 모델을 만들고

분산 설명률이 높은 순으로 누적 분산 설명률이 0.8을 넘기는 최소의 컴포넌트를 prob2_train에 추가 한다.

동일한 PCA 모델로 prob2_test를 변환하여 같은 수의 컴포넌트를 prob2_test에 추가 한다.

함수가이드

sklearn.decomposition.PCA (입력의 순서는 cont 뒤의 번호가 낮은 것에서 높은 것 순으로 한다, random_state=123)

문제 지시사항 외 Default 값 사용

```
In [130... from sklearn.decomposition import PCA
```

```
# 분산 설명률이 높은 순으로 누적 분산 설명률이 0.8
```

```
pca = PCA(n_components = 0.8, random_state=123)
```

```
pca.fit(df_prob2_train[X_corr])
```

```
Out[130]: PCA(copy=True, iterated_power='auto', n_components=0.8, random_state=123,
          svd_solver='auto', tol=0.0, whiten=False)
```

```
In [131... print('pca.n_components_ = ', pca.n_components_)
```

```
pca.n_components_ = 3
```

```
In [132... X_comp = ['pca_{i}'.format(i) for i in range(pca.n_components_)]
```

```
print('X_comp = ', X_comp)
```

```
df_prob2_train[X_comp] = pd.DataFrame(pca.transform(df_prob2_train[X_corr]), index=df_prob2_train.index, columns=X_comp)
```

```
df_prob2_test[X_comp] = pd.DataFrame(pca.transform(df_prob2_test[X_corr]), index=df_prob2_test.index, columns=X_comp)
```

```
X_comp = ['pca_0', 'pca_1', 'pca_2']
```

```
In [133... df_prob2_train[X_comp]
```

Out[133]:

	pca_0	pca_1	pca_2
id			
487693	-0.363381	0.018541	-0.051800
307175	-0.103216	0.294394	-0.096653
493409	0.853154	0.047363	0.104571
414858	-0.079167	0.021580	-0.084363
408161	0.149872	-0.073583	0.197499
...	...	...	...
486483	0.520503	-0.250642	0.132757
397343	-0.536040	-0.013147	0.156720
119442	0.270999	-0.172300	-0.083993
127964	-0.345073	0.095621	-0.046980
128591	0.103191	-0.117542	-0.011144

192000 rows × 3 columns

In [134...

df_prob2_test[X_comp]

Out[134]:

	pca_0	pca_1	pca_2
id			
309145	0.891113	0.078173	0.042182
368647	-0.402723	-0.154452	0.026537
487072	-0.216491	0.056313	0.024912
490811	-0.226155	0.157114	-0.021635
484687	-0.018692	0.390843	0.329713
...	...	...	...
11528	-0.302192	0.092255	-0.002482
484506	-0.353851	0.035517	-0.045551
465250	0.183479	-0.179650	-0.002625
361168	0.518040	0.007619	0.107655
268321	0.157793	-0.196783	0.075800

48000 rows × 3 columns

In [135...

단계 2-5 설명

...

코드는 PCA (주성분 분석)를 사용하여 원래 변수들의 차원을 축소하고,
 훈련 데이터와 테스트 데이터에 대해 주성분을 계산하여 데이터프레임에 추가하는 작업을 수행합니다.

1. 데이터 분할 및 복사

- `train\_test\_split`을 사용하여 `df\_prob2` 데이터를 훈련 세트(80%)와 테스트 세트(20%)로 분할합니다.
- `df\_prob2\_train`과 `df\_prob2\_test`을 각각 복사합니다.
 이는 원본 데이터프레임을 보호하고 이후의 변환 작업이 독립적으로 수행되도록 하기 위함입니다.

2. PCA 객체 생성 및 학습

- `PCA` 객체를 생성합니다.
 - `n\_components=0.8`: 누적 분산 설명률이 80%가 될 때까지 주성분을 선택합니다.
 즉, 데이터의 분산을 80% 설명할 수 있는 주성분의 수를 자동으로 결정합니다.
 - `random\_state=123`: 랜덤 상태를 설정하여 결과가 재현 가능하도록 합니다.

```

- `pca.fit`을 사용하여 `df_prob2_train`의 `X_corr` 열에 해당하는 데이터에 대해 PCA를 학습시킵니다.
  `X_corr`은 이전에 식별된 가장 큰 클러스터의 변수들을 포함합니다.

### 3. 주성분 변수 생성 및 훈련 데이터 변환
- `pca.n_components`는 누적 분산 설명률이 80%에 도달할 때까지 선택된 주성분의 수를 나타냅니다.
- 주성분 변수 이름을 생성합니다. 예를 들어, `pca_0`, `pca_1` 등으로 이름을 지정합니다.

- `pca.transform`을 사용하여 `df_prob2_train`의 `X_corr` 열에 해당하는 데이터를 주성분 공간으로 변환합니다.
- 변환된 주성분 데이터를 데이터프레임으로 변환하고, `X_comp`를 열 이름으로 사용하며, 원래의 인덱스를 유지합니다.
- 변환된 주성분 데이터를 `df_prob2_train` 데이터프레임에 추가합니다.

### 4. 테스트 데이터 변환
- `pca.transform`을 사용하여 `df_prob2_test`의 `X_corr` 열에 해당하는 데이터를 주성분 공간으로 변환합니다.
- 변환된 주성분 데이터를 데이터프레임으로 변환하고, `X_comp`를 열 이름으로 사용하며, 원래의 인덱스를 유지합니다.
- 변환된 주성분 데이터를 `df_prob2_test` 데이터프레임에 추가합니다.

### 전체 코드 동작 요약
1. 데이터 분할: 원본 데이터프레임 `df_prob2`를 훈련 세트와 테스트 세트로 분할합니다.
2. PCA 학습: 훈련 세트의 특정 변수들(`X_corr`)에 대해 PCA를 수행하여 누적 분산 설명률이 80%가 될 때까지 주성분을 선택합니다.
3. 주성분 변수 생성 및 추가:
   - 훈련 세트와 테스트 세트 각각에 대해 선택된 주성분으로 데이터를 변환하고, 이를 데이터프레임에 추가합니다.
   - 결과적으로, `df_prob2_train`과 `df_prob2_test`는 원래 변수들 외에 주성분 변수들도 포함하게 됩니다.

이 과정을 통해 데이터의 차원을 축소하고, 중요한 정보를 유지하면서 데이터의 구조를 단순화할 수 있습니다.
이는 모델 학습 시 과적합을 줄이고 계산 효율성을 높이는 데 도움이 됩니다.
'''
print('')

```

단계 2-6

prob2_train에서 target을 선형회귀모델로 학습하고

prob2_test를 다음 성능 지표로 평가한 값을 A라고 한다.

입력변수: cont0 ~ 13, cat0 ~ 9 원핫-인코딩 후 나온

파생 변수 (변수명, 범주명 알파벳 순서)

대상변수: target

성능지표: $MSE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Hint] 학습데이터의 형태는 (192,000, 43) 이다.

함수가이드

sklearn.linear_model.LinearRegression

문제 지시사항 외 Default 값 사용

In [136...

```
'''
코드는 `LinearRegression` 모델을 사용하여 예측을 수행하고,
훈련 데이터와 테스트 데이터에 대해 평균 제곱 오차(MSE)를 계산하는 작업을 수행합니다.

### 1. 독립 변수 리스트 생성
- `cont_cols`: 연속형 변수들의 리스트입니다.
- `dum_cols.tolist()`: 범주형 변수를 원-핫 인코딩한 결과로 생성된 더미 변수들의 리스트입니다.
- `X_lr`는 이 두 리스트를 합친 것으로, 회귀 모델에 사용할 모든 독립 변수들의 리스트입니다.

### 2. 선형 회귀 모델 생성 및 학습
- `LinearRegression` 객체를 생성합니다.
- `reg_lr.fit`을 사용하여 `df_prob2_train` 데이터프레임의 독립 변수(`X_lr`)와
  종속 변수(`target`)를 사용하여 회귀 모델을 학습시킵니다.

### 3. 테스트 데이터에 대한 예측 및 MSE 계산
- `reg_lr.predict`을 사용하여 `df_prob2_test` 데이터프레임의 독립 변수(`X_lr`)를 기반으로 `target` 값을 예측합니다.
- `mean_squared_error`를 사용하여 실제 `target` 값과 예측된 `target` 값 간의 평균 제곱 오차(MSE)를 계산합니다.
- 이 값은 테스트 데이터에 대한 예측 성능을 나타내며, 변수 `A`에 저장됩니다.

### 4. 훈련 데이터에 대한 예측 및 MSE 계산
- 훈련 데이터에 대해서도 `reg_lr.predict`을 사용하여 독립 변수(`X_lr`)를 기반으로 `target` 값을 예측합니다.
- `mean_squared_error`를 사용하여 훈련 데이터의 실제 `target` 값과 예측된 `target` 값 간의 평균 제곱 오차(MSE)를 계산합니다.
- 결과는 튜플 형태로 반환됩니다. 첫 번째 값은 테스트 데이터의 MSE(`A`), 두 번째 값은 훈련 데이터의 MSE입니다.

### 전체 코드 동작 요약
1. 독립 변수 리스트 생성: 모델에 사용할 연속형 변수들과 원-핫 인코딩된 범주형 변수들을 합쳐서 `X_lr` 리스트를 생성합니다.
2. 선형 회귀 모델 학습: `df_prob2_train` 데이터프레임을 사용하여 `LinearRegression` 모델을 학습시킵니다.
3. 테스트 데이터 예측 및 MSE 계산: 테스트 데이터에 대해 예측을 수행하고, 평균 제곱 오차를 계산하여 `A` 변수에 저장합니다.
'''
```

4. 훈련 데이터 예측 및 MSE 계산: 훈련 데이터에 대해 예측을 수행하고, 평균 제곱 오차를 계산합니다.

5. 결과 반환: 테스트 데이터의 MSE와 훈련 데이터의 MSE를 튜플 형태로 반환합니다.

이 과정은 모델의 성능을 평가하는 데 사용됩니다. 테스트 데이터에 대한 MSE는 모델의 일반화 성능을 나타내고, 훈련 데이터에 대한 MSE는 모델이 훈련 데이터에 얼마나 잘 맞는지 나타냅니다.

이 두 값의 비교를 통해 모델이 과적합(overfitting)되었는지 또는 과소적합(underfitting)되었는지를 판단할 수 있습니다.

...

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
X_lr = cont_cols + dum_cols.tolist()
print('X_lr = ', X_lr)
```

```
reg_lr = LinearRegression()
reg_lr.fit(df_prob2_train[X_lr], df_prob2_train['target'])
```

```
A = mean_squared_error(
    df_prob2_test['target'], reg_lr.predict(df_prob2_test[X_lr]))
(
    A,
    mean_squared_error(df_prob2_train['target'], reg_lr.predict(df_prob2_train[X_lr]))
)
```

```
X_lr = ['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12',
'cont13', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_
D', 'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat
9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

Out[136]: (0.7455022756348629, 0.7446868277591954)

단계 2-7

prob2_train에서 cont0 ~ 13 중에서 PCA에서 사용했던 변수를 제외하고

단계 2-5에서 추가한 PCA 컴포넌트를 포함하여 target로 선형회귀모델을 학습한다.

prob2_test로 동일한 지표로 대한 성능 평가한 값을 B라 한다.

입력변수: cont0 ~ 13중 PCA에 사용한 것을 제외한 나머지 변수,

PCA를 통한 파생 변수를 분산 설명력이 높은 순으로,

cat0 ~ 9까지 원핫-인코딩 후 나온 파생 변수 (변수명, 범주명 알파벳 순서)

대상변수: target

성능지표: $MSE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

함수가이드

sklearn.linear_model.LinearRegression

문제 지시사항 외 Default 값 사용

In [137...

```
'''
코드는 이전 코드와 비슷한 작업을 수행하지만, 사용하는 독립 변수가 다릅니다.
새로운 독립 변수 리스트를 생성하고, 그 리스트를 사용하여 선형 회귀 모델을 학습
및 평가합니다.

### 1. 새로운 독립 변수 리스트 생성
- `cont_cols`: 연속형 변수들의 리스트입니다.
- `X_corr`: PCA에서 사용된 연속형 변수들의 리스트입니다.
- `[i for i in cont_cols if i not in X_corr]`:
    PCA에 사용되지 않은 연속형 변수들을 선택합니다.
- `X_comp`: PCA 변환 후 생성된 주성분 변수들의 리스트입니다.
- `dum_cols.tolist()`: 범주형 변수를 원-핫 인코딩한 결과로 생성된 더미 변수들의 리스트입니다.
- 이 세 리스트를 합쳐서 최종 독립 변수 리스트 `X_lr`를 생성합니다.
    이는 PCA에서 사용되지 않은 연속형 변수들, 주성분 변수들, 그리고 더미 변수들을 포함합니다.

### 2. 선형 회귀 모델 생성 및 학습
reg_lr = LinearRegression()
reg_lr.fit(df_prob2_train[X_lr], df_prob2_train['target'])
- `LinearRegression` 객체를 생성합니다.
- `reg_lr.fit`을 사용하여 `df_prob2_train` 데이터프레임의 독립 변수(`X_lr`)와 종속 변수(`target`)를 사용하여
    회귀 모델을 학습시킵니다.

### 3. 테스트 데이터에 대한 예측 및 MSE 계산
B = mean_squared_error(
```

```
df_prob2_test['target'], reg_lr.predict(df_prob2_test[X_lr]))
)
```

- `reg_lr.predict`을 사용하여 `df_prob2_test` 데이터프레임의 독립 변수(`X_lr`)를 기반으로 `target` 값을 예측합니다.
- `mean_squared_error`를 사용하여 실제 `target` 값과 예측된 `target` 값 간의 평균 제곱 오차(MSE)를 계산합니다.
- 이 값은 테스트 데이터에 대한 예측 성능을 나타내며, 변수 `B`에 저장됩니다.

4. 훈련 데이터에 대한 예측 및 MSE 계산

- 훈련 데이터에 대해서도 `reg_lr.predict`을 사용하여 독립 변수(`X_lr`)를 기반으로 `target` 값을 예측합니다.
- `mean_squared_error`를 사용하여 훈련 데이터의 실제 `target` 값과 예측된 `target` 값 간의 평균 제곱 오차(MSE)를 계산합니다.
- 결과는 튜플 형태로 반환됩니다. 첫 번째 값은 테스트 데이터의 MSE(`B`), 두 번째 값은 훈련 데이터의 MSE입니다.

전체 코드 동작 요약

1. 새로운 독립 변수 리스트 생성: PCA에서 사용되지 않은 연속형 변수들, 주성분 변수들, 그리고 원-핫 인코딩된 범주형 변수들을 합쳐서 `X_lr` 리스트를 생성합니다.
2. 선형 회귀 모델 학습: `df_prob2_train` 데이터프레임을 사용하여 `LinearRegression` 모델을 학습시킵니다.
3. 테스트 데이터 예측 및 MSE 계산: 테스트 데이터에 대해 예측을 수행하고, 평균 제곱 오차를 계산하여 `B` 변수에 저장합니다.
4. 훈련 데이터 예측 및 MSE 계산: 훈련 데이터에 대해 예측을 수행하고, 평균 제곱 오차를 계산합니다.
5. 결과 반환: 테스트 데이터의 MSE와 훈련 데이터의 MSE를 튜플 형태로 반환합니다.

이 과정은 모델의 성능을 평가하는 데 사용됩니다. 테스트 데이터에 대한 MSE는 모델의 일반화 성능을 나타내고, 훈련 데이터에 대한 MSE는 모델이 훈련 데이터에 얼마나 잘 맞는지를 나타냅니다. 두 값의 비교를 통해 모델이 과적합(overfitting)되었는지 또는 과소적합(underfitting)되었는지를 판단할 수 있습니다.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X_lr = [i for i in cont_cols if i not in X_corr] + X_comp + dum_cols.tolist()
print('X_lr = ', X_lr)

reg_lr = LinearRegression()
reg_lr.fit(df_prob2_train[X_lr], df_prob2_train['target'])

B = mean_squared_error(df_prob2_test['target'], reg_lr.predict(df_prob2_test[X_lr]))
(
    B,
    mean_squared_error(df_prob2_train['target'], reg_lr.predict(df_prob2_train[X_lr]))
)
```

```
X_lr = ['cont1', 'cont2', 'cont3', 'cont4', 'cont6', 'cont7', 'cont10', 'cont11', 'cont13', 'pca_0', 'pca_1', 'pca_2', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

Out[137]: (0.7499049886498875, 0.7487486877700789)

In [138... A, B, A - B, round(A - B, 3)

Out[138]: (0.7455022756348629, 0.7499049886498875, -0.004402713015024595, -0.004)

A-B 를 소수점 넷째 자리에서 반올림하여 셋째 자리까지 구하라.

-0.004

In [139... Image('3-6.png')

Out[139]:

1. Gaussian Mixture Model의 개념

GMM은 다음과 같은 방식으로 동작합니다:

- **혼합 분포(Mixture Distribution):** GMM은 여러 개의 가우시안 분포를 가중합한 혼합 분포로 데이터를 설명합니다. 이때 각 가우시안 분포는 특정 그룹이나 군집을 나타낸다고 볼 수 있습니다.
- **가우시안 분포(Gaussian Distribution):** 각 구성 요소(컴포넌트)는 가우시안 분포로 표현됩니다. 각 분포는 특정 평균(μ)과 공분산(σ^2)을 가지며, 이러한 파라미터는 GMM의 구성 요소를 정의합니다.
- **가중치(Weight):** 각 가우시안 분포는 특정 가중치(π)를 가지며, 이 가중치는 전체 데이터에서 해당 분포가 차지하는 비율을 나타냅니다. 모든 가중치의 합은 1이 됩니다.

수학적으로, GMM은 다음과 같이 정의됩니다:

$$p(x) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x|\mu_k, \Sigma_k)$$

여기서:

- K : 혼합 모델의 가우시안 분포의 개수
- π_k : 각 가우시안 분포의 가중치 ($\sum_{k=1}^K \pi_k = 1$)
- $\mathcal{N}(x|\mu_k, \Sigma_k)$: 평균이 μ_k , 공분산이 Σ_k 인 가우시안 분포
- x : 관찰된 데이터

In [140...

Image('3-7.png')

Out[140]:

2. Gaussian Mixture Model의 주요 특징

- **모수적 방법:** GMM은 모델에 대한 명시적인 가정을 가지고 있으며, 이러한 가정에 따라 데이터를 설명하는 파라미터(평균, 공분산, 가중치)를 추정합니다.
- **EM 알고리즘:** GMM의 파라미터는 일반적으로 EM(Expectation-Maximization) 알고리즘을 통해 추정됩니다. 이 알고리즘은 주어진 데이터에 대해 각 가우시안 분포에 속할 확률(기대 단계)을 계산한 다음, 그 확률을 바탕으로 가우시안 분포의 파라미터를 업데이트하는 방식(최대화 단계)으로 동작합니다.
- **군집화와 밀도 추정:** GMM은 데이터를 자연스럽게 여러 군집으로 나눌 수 있습니다. 각 데이터 포인트는 특정 가우시안 분포에 속할 확률을 가지며, 이 확률을 바탕으로 데이터 포인트를 군집화할 수 있습니다. 또한, GMM은 데이터를 생성할 수 있는 확률 밀도 함수를 제공하여, 새로운 데이터의 생성 가능성을 평가할 수 있습니다.
- **이상치 탐지:** GMM은 이상치 탐지에도 사용될 수 있습니다. 모델의 특정 군집에 대한 확률이 매우 낮은 데이터 포인트를 이상치로 간주할 수 있습니다.

3. Gaussian Mixture Model의 사용 사례

- **군집화:** GMM은 k-means와 유사한 군집화 방법으로 사용할 수 있습니다. k-means는 구형의 클러스터만 찾을 수 있지만, GMM은 타원형의 클러스터도 모델링할 수 있어 더 유연합니다.
- **밀도 추정:** 데이터의 밀도를 추정하여, 데이터가 특정 범위에 속할 확률을 계산할 수 있습니다.
- **이상치 탐지:** 데이터 포인트가 GMM에서 유도된 분포에 잘 맞지 않을 경우, 이를 이상치로 식별할 수 있습니다.
- **데이터 생성:** GMM에서 학습된 분포를 사용하여 새로운 데이터를 생성할 수 있습니다. 이는 특히 데이터 샘플링이나 몬테카를로 시뮬레이션에서 유용합니다.

4. GMM의 장단점

- 장점:
 - 데이터의 다양한 패턴을 유연하게 모델링할 수 있습니다.
 - k-means보다 복잡한 분포를 표현할 수 있어, 비구형의 군집화가 가능합니다.
 - 통계적 확률 모델이므로 결과에 대한 불확실성을 평가할 수 있습니다.
- 단점:
 - EM 알고리즘이 지역 최적해에 수렴할 수 있으므로 초기화에 민감합니다.
 - 가우시안 분포의 개수 K 를 미리 정해야 하며, 이 값을 잘못 설정하면 모델 성능이 떨어질 수 있습니다.
 - 높은 차원의 데이터에서는 계산이 복잡해질 수 있습니다.

문제3

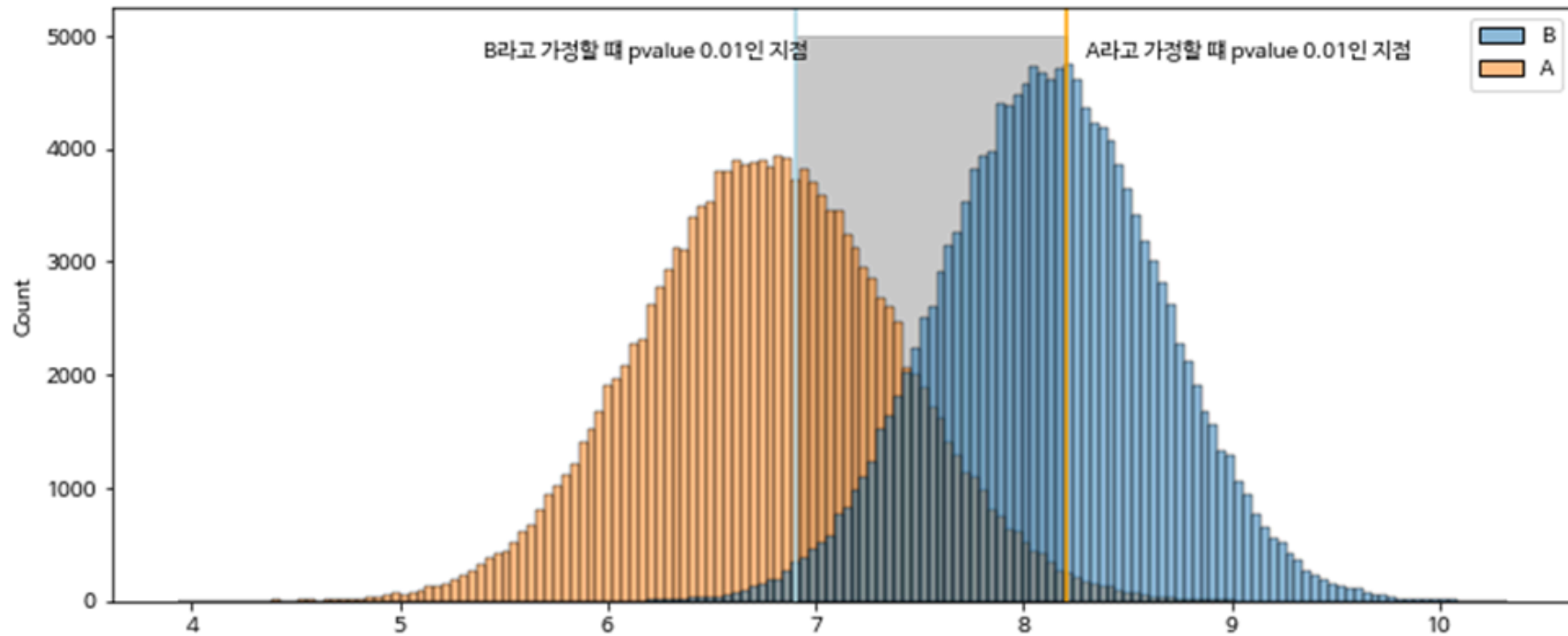
전처리 단계에서 target이 두 개의 정규 분포의 혼합한 형태 보이고 있음 확인했고,

target을 7.45 경계값을 기준으로 A와 B범주로 나누었다.

Gaussian Mixture라는 모델을 사용하여 두 개 정규 분포의 매개 변수를 측정했고, target의 분포를 시각화했다.

아래 시각화 그래프에서 A, B가 혼재하여 A 또는 B라고 확신할 수는 없는 구간이 존재한다.

A 또는 B라고 확신할 수 있는 관측치만을 가지고 A / B를 분류 모델을 만들고자 한다. 다음 단계들을 수행하라.



In []:

단계 3-1

A 범주의 평균은 6.769 표준편차는 0.616이고 범주 B일 때 평균은 8.123 표준편차는 0.527이다.

측정한 값을 범주 A, B의 모집단의 target의 평균과 표준편차라고 가정한다.

두 개의 정규 분포를 이용하여, 귀무가설이 'target은 A 범주에 해당한다'일 때

p-value를 구하여 파생변수(prob_A)를 만든다.

반대로, 귀무가설이 'target은 B 범주에 해당한다'일 때 p-value를 구하여

파생변수(prob_B)를 만든다.

basetable에 두 개의 파생 변수 prob_A, prob_B를 추가하여 prob3 데이터프레임을 만든다.

함수가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

In [141...

```
'''
`norm.cdf()` 함수는 SciPy 라이브러리에서 제공하는 정규 분포(가우시안 분포)의
누적 분포 함수(Cumulative Distribution Function, CDF)를 계산하는 함수입니다.
이 함수는 주어진 값이 정규 분포에서 어느 정도의 확률 누적 값을 갖는지를 반환합니다.

### 1. 함수의 사용법
scipy.stats.norm.cdf(x, loc=0, scale=1)

### 2. 인자 (Parameters)
- `x` (float or array-like):
  - 필수 인자입니다.
  - CDF를 계산할 지정(데이터 포인트)입니다. 이 값은 정규 분포의 누적 확률을 구하고자 하는 지점에 해당합니다.
  - 단일 값 또는 배열 형태로 전달할 수 있습니다.
- `loc` (float, optional, default=0):
  - 선택적 인자입니다.
  - 정규 분포의 평균( $\mu$ )을 지정합니다. `loc`는 정규 분포의 위치 파라미터로, 분포의 중심이 되는 값을 의미합니다.
  - 기본값은 0입니다.
- `scale` (float, optional, default=1):
  - 선택적 인자입니다.
  - 정규 분포의 표준 편차( $\sigma$ )를 지정합니다. `scale`은 분포의 폭을 조절하는 파라미터로, 분산을 조절합니다.
  - 기본값은 1입니다.

### 3. 리턴값 (Return)
- `cdf_value` (float or ndarray):
  - 주어진 `x` 값에 대해 정규 분포의 CDF 값을 반환합니다.
  - `x`가 단일 값이면 단일 `float` 값을 반환하고, `x`가 배열이면 각 요소에 대해 계산된 CDF 값이 담긴 `ndarray`를 반환합니다.

### 4. 설명
- 단일 값에 대한 CDF: `1.96`이라는 값이 표준 정규 분포(평균이 0, 표준 편차가 1인 분포)에서 97.5%의 누적 확률에 해당합니다.
  이는 표준 정규 분포에서 오른쪽 꼬리에 해당하는 확률이 약 2.5%라는 것을 의미합니다.
- 평균과 표준 편차가 다른 정규 분포: 평균이 `1`, 표준 편차가 `2`인 정규 분포에서 `1.96`이라는 값의 CDF는 약 `0.668`입니다.
```

- 이는 이 분포에서 `1.96` 이하의 값이 나타날 확률이 약 66.8%임을 의미합니다.
- 배열에 대한 CDF: 여러 값을 가진 배열 `x\_values`에 대해 각각의 CDF 값을 계산하여 배열로 반환합니다.

5. 추가 정보

- `norm.cdf()`는 기본적으로 표준 정규 분포(평균이 0, 표준 편차가 1)에서 CDF를 계산하지만, `loc`과 `scale` 파라미터를 통해 다른 정규 분포를 정의할 수 있습니다.
- CDF 값은 0에서 1 사이의 값을 가지며, 이는 특정 값 이하의 확률을 나타냅니다.

이 함수는 정규 분포와 관련된 통계적 분석이나 확률 계산에서 자주 사용되며, 가설 검정, 신뢰 구간 계산 등 다양한 응용에서 중요한 역할을 합니다.

```
'''
```

```
import scipy.stats as stats
```

```
# 단일 값에 대한 CDF 계산
```

```
result_single = stats.norm.cdf(1.96)
print('result_single = ', result_single)
print('-----')
```

```
# 평균과 표준 편차를 변경한 경우
```

```
result_shifted = stats.norm.cdf(1.96, loc=1, scale=2)
print('result_shifted = ', result_shifted)
print('-----')
```

```
# 배열에 대한 CDF 계산
```

```
import numpy as np
x_values = np.array([-1.0, 0.0, 1.0, 2.0])
result_array = stats.norm.cdf(x_values)
print('result_array = ', result_array)
print('-----')
```

```
result_single = 0.9750021048517795
-----
result_shifted = 0.6843863034837774
-----
result_array = [0.15865525 0.5          0.84134475 0.97724987]
-----
```

In [142...

```
from scipy.stats import norm

df_prob3 = df_basetable.assign(
    # H0: target은 A 범주, H1: target B 범주,
```

```

# target ~ N(mu = 6.769, std = 0.616) 우측 꼬리
prob_A = 1 - norm.cdf(df_basetable['target'], loc=6.769, scale=0.616),
# H0: target은 B 범주, H1: target A 범주,
# target ~ N(mu = 8.123, std= 0.527) 좌측 꼬리
prob_B = norm.cdf(df_basetable['target'], loc=8.123, scale=0.527)
)

print('prob_A = \n', df_prob3['prob_A'])
print('prob_B = \n', df_prob3['prob_B'])

```

df_prob3

prob_A =

id

267387	0.013160
410037	0.151204
139373	0.024634
113765	0.212302
179915	0.210597

...

320975	0.087273
29591	0.015145
46717	0.395123
463191	0.577520
415856	0.138544

Name: prob_A, Length: 240000, dtype: float64

prob_B =

id

267387	0.510907
410037	0.086313
139373	0.393223
113765	0.050924
179915	0.051649

...

320975	0.162996
29591	0.485189
46717	0.011962
463191	0.002572
415856	0.097002

Name: prob_B, Length: 240000, dtype: float64

Out[142]:

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont8	cont9	cont10	cont11	cont12	cont13	target	targetA	prob_A
id																				
267387	A	B	B	C	B	B	A	E	G	G	...	0.47134	0.53290	0.97139	0.59414	0.36856	0.42951	8.13741	False	0.013160
410037	A	A	A	A	B	B	A	E	C	F	...	0.40193	0.49181	0.56479	0.39386	0.36247	0.71541	7.40427	True	0.151204
139373	A	A	B	A	B	B	A	E	C	J	...	0.40112	0.53052	0.35635	0.33341	0.33354	0.82542	7.98022	False	0.024634
113765	A	A	A	C	B	B	A	E	C	G	...	0.46425	0.38302	0.38054	0.68827	0.34895	0.73375	7.26085	True	0.212302
179915	A	B	A	C	B	B	A	E	E	L	...	0.75578	0.55676	0.31308	0.43157	0.32302	0.18028	7.26448	True	0.210597
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
320975	A	A	A	C	B	B	A	E	C	I	...	0.31427	0.43718	0.40494	0.69718	0.34519	0.67524	7.60537	False	0.087273
29591	A	B	A	C	B	B	A	E	C	F	...	0.46395	0.83396	0.71724	0.69450	0.33008	0.28460	8.10343	False	0.015145
46717	A	B	B	C	B	D	A	E	G	F	...	0.87541	0.63918	0.58994	0.77751	0.86388	0.74568	6.93285	True	0.395123
463191	A	A	A	C	B	D	A	E	E	F	...	0.06595	0.61991	0.62167	0.25545	0.33329	0.28366	6.64854	True	0.577520
415856	A	A	A	C	B	D	A	E	G	I	...	0.90318	0.81729	0.57672	0.46053	0.91726	0.25874	7.43852	True	0.138544

240000 rows x 20 columns

In [143]:

```
df_prob3.columns
```

Out[143]:

```
Index(['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8',  
      'cat9', 'cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6',  
      'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13',  
      'target', 'targetA', 'prob_A', 'prob_B'],  
      dtype='object')
```

단계 3-2

prob3 중에서 target이 A 또는 B라고 신뢰할 수 있는 행들만을 골라내고자 한다.

prob_B가 0.01 미만이거나, prob_A가 0.01 미만인 것만 남긴다. (즉, 위 그림에서 회색 영역에 해당하는 부분은 제외)

Hint] 행의 개수는 124,600가 된다.

```
In [144... df_prob3 = df_prob3.query('prob_B < 0.01 or prob_A < 0.01')
len(df_prob3)
```

Out[144]: 124600

```
In [145... df_prob3.head()
```

```
Out[145]:
```

	cat0	cat1	cat2	cat3	cat4	cat5	cat6	cat7	cat8	cat9	...	cont8	cont9	cont10	cont11	cont12	cont13	target	targetA	prob_A
id																				
29503	A	B	A	C	B	B	A	E	E	F	...	0.30163	0.32551	0.26982	0.19329	0.39034	0.30002	6.66255	True	0.568599
113326	A	B	A	A	B	D	A	E	C	H	...	0.65004	0.63958	0.60535	0.70951	0.71664	0.78782	5.98141	True	0.899473
199507	A	B	B	A	B	B	A	E	A	L	...	0.22571	0.48717	0.33999	0.51461	0.37367	0.69462	6.60870	True	0.602656
346281	A	A	A	D	B	D	A	E	C	F	...	0.45999	0.79100	0.62269	0.83624	0.78785	0.30893	5.42339	True	0.985535
137333	A	A	A	C	B	D	A	D	C	K	...	0.49416	0.89742	0.59422	0.72298	0.72535	0.82115	8.34177	False	0.005337

5 rows × 28 columns

```
In [146... '''
`explode()` 함수는 Pandas 라이브러리에서 제공하는 함수로, Series와 DataFrame에서 사용됩니다.
이 함수는 리스트나 딕셔너리 형태의 값을 개별적인 행으로 확장(explode)시키는 데 사용됩니다.
```

``explode()``는 특히 데이터프레임에서 각 셀에 리스트가 포함된 경우 유용합니다.

``explode()`` 함수

인자 (Parameters)

- ``column``: str 또는 list-like, 선택 사항 (DataFrame에서만 사용)
 - 분할할 열의 이름. 여러 열을 한 번에 분할하려면 열 이름의 리스트를 제공합니다.
- ``ignore_index``: bool, 기본값 False
 - ``True``로 설정하면 결과 데이터프레임의 인덱스를 무시하고 새 인덱스를 생성합니다. 기본적으로 원래의 인덱스가 유지됩니다.

리턴값 (Returns)

- DataFrame 또는 Series: 원래 Series나 DataFrame과 같은 유형으로 반환됩니다. 지정된 열의 각 요소가 개별 행으로 확장됩니다.

사용 방법

Series에서의 사용

Series의 각 요소가 리스트인 경우, ``explode()``를 사용하면 각 리스트의 요소가 개별 행으로 확장됩니다.

```
import pandas as pd
```

```
s = pd.Series([[1, 2, 3], [4, 5], [], [6, 7]])
print(s)
```

결과

```
# 0    [1, 2, 3]
# 1      [4, 5]
# 2         []
# 3      [6, 7]
# dtype: object
```

Explode

```
exploded_s = s.explode()
print(exploded_s)
```

결과

```
# 0    1
# 0    2
# 0    3
# 1    4
# 1    5
# 2   NaN
# 3    6
# 3    7
```

```
# dtype: object

#### DataFrame에서의 사용
DataFrame의 특정 열이 리스트인 경우, `explode()`를 사용하여 해당 열의 리스트 요소를 개별 행으로 확장할 수 있습니다.

import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3], 'B': [[1, 2], [3, 4, 5], [6]]})
print(df)

# 결과
#   A      B
# 0  1  [1, 2]
# 1  2 [3, 4, 5]
# 2  3      [6]

# Explode column 'B'
exploded_df = df.explode('B')
print(exploded_df)

# 결과
#   A  B
# 0  1  1
# 0  1  2
# 1  2  3
# 1  2  4
# 1  2  5
# 2  3  6

#### 여러 열을 동시에 분할
DataFrame에서 여러 열을 동시에 분할할 수 있습니다. 각 열의 리스트가 같은 길이여야 합니다.

df = pd.DataFrame({'A': [[1, 2], [3, 4, 5]], 'B': [['a', 'b'], ['c', 'd', 'e']]})
print(df)

# 결과
#           A      B
# 0    [1, 2]  [a, b]
# 1  [3, 4, 5] [c, d, e]

# Explode both columns 'A' and 'B'
exploded_df = df.explode(['A', 'B'])
```

```
print(exploded_df)
```

```
# 결과
```

```
#   A  B
# 0  1  a
# 0  2  b
# 1  3  c
# 1  4  d
# 1  5  e
# ..
```

```
#### `ignore_index` 인자 사용
```

기본적으로 원래의 인덱스가 유지됩니다. `ignore\_index=True`로 설정하면 새로운 인덱스가 생성됩니다.

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [[1, 2], [3, 4, 5], [6]]})
exploded_df = df.explode('B', ignore_index=True)
print(exploded_df)
```

```
# 결과
```

```
#   A  B
# 0  1  1
# 1  1  2
# 2  2  3
# 3  2  4
# 4  2  5
# 5  3  6
```

이렇게 `explode()` 함수는 리스트나 딕셔너리 형태의 데이터를 쉽게 분할하여 분석할 수 있도록 도와줍니다.

```
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [[1, 2], [3, 4, 5], [6]]})
print(df1)
print('-----')
```

```
exploded_df1 = df1.explode('B')
print(exploded_df1)
print('-----')
```

ignore_index 인자는 Pandas 버전 1.3.0부터 도입되었습니다. 따라서, 이 기능을 사용하려면 Pandas의 최신 버전으로 업그레이드해야 합니다.

```
#exploded_df2 = df1.explode('B', ignore_index=True)
#print(exploded_df2)
```

```
# 대안으로 reset_index 사용
exploded_df2 = df1.explode('B').reset_index(drop=True)
print(exploded_df2)
```

```
   A      B
0  1  [1, 2]
1  2  [3, 4, 5]
2  3      [6]
-----
   A  B
0  1  1
0  1  2
1  2  3
1  2  4
1  2  5
2  3  6
-----
   A  B
0  1  1
1  1  2
2  2  3
3  2  4
4  2  5
5  3  6
```

단계 3-3

prob3에 cat0 ~ 9를 제외하는 범주 없이 원핫 인코딩(One-Hot Encoding) 한다.

In [147...

```
'''
# 아래 코드 실행 결과

cat_cols = ['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8', 'cat9']
-----
df_prob3[cat_cols] =
      cat0 cat1 cat2 cat3 cat4 cat5 cat6 cat7 cat8 cat9
id
29503    A   B   A   C   B   B   A   E   E   F
```

```

113326    A    B    A    A    B    D    A    E    C    H
199507    A    B    B    A    B    B    A    E    A    L
346281    A    A    A    D    B    D    A    E    C    F
137333    A    A    A    C    B    D    A    D    C    K
...      ...    ...    ...    ...    ...    ...    ...    ...    ...
76711     A    B    A    C    B    D    A    D    E    F
215151    A    B    B    C    B    B    A    E    E    F
244056    A    B    A    C    B    B    A    E    E    F
324013    A    B    B    C    B    D    A    E    G    F
463191    A    A    A    C    B    D    A    E    E    F

```

```
[124600 rows x 10 columns]
```

```
-----
```

```
df_prob3[cat_cols].apply(lambda x: print(x,name), axis=0)
```

```
id
```

```
29503      A
```

```
113326      A
```

```
199507      A
```

```
346281      A
```

```
137333      A
```

```
..
```

```
76711      A
```

```
215151      A
```

```
244056      A
```

```
324013      A
```

```
463191      A
```

```
Name: cat0, Length: 124600, dtype: object D
```

```
id
```

```
29503      B
```

```
113326      B
```

```
199507      B
```

```
346281      A
```

```
137333      A
```

```
..
```

```
76711      B
```

```
215151      B
```

```
244056      B
```

```
324013      B
```

```
463191      A
```

```
Name: cat1, Length: 124600, dtype: object D
```

```
id
```

```
29503      A
```

```
113326    A
199507    B
346281    A
137333    A
..
76711     A
215151    B
244056    A
324013    B
463191    A
Name: cat2, Length: 124600, dtype: object D
id
29503     C
113326    A
199507    A
346281    D
137333    C
..
76711     C
215151    C
244056    C
324013    C
463191    C
Name: cat3, Length: 124600, dtype: object D
id
29503     B
113326    B
199507    B
346281    B
137333    B
..
76711     B
215151    B
244056    B
324013    B
463191    B
Name: cat4, Length: 124600, dtype: object D
id
29503     B
113326    D
199507    B
346281    D
```

```
137333    D
..
76711     D
215151    B
244056    B
324013    D
463191    D
Name: cat5, Length: 124600, dtype: object D
id
29503     A
113326    A
199507    A
346281    A
137333    A
..
76711     A
215151    A
244056    A
324013    A
463191    A
Name: cat6, Length: 124600, dtype: object D
id
29503     E
113326     E
199507     E
346281     E
137333     D
..
76711     D
215151     E
244056     E
324013     E
463191     E
Name: cat7, Length: 124600, dtype: object D
id
29503     E
113326     C
199507     A
346281     C
137333     C
..
76711     E
```

```

215151      E
244056      E
324013      G
463191      E
Name: cat8, Length: 124600, dtype: object D
id
29503       F
113326      H
199507      L
346281      F
137333      K
..
76711       F
215151      F
244056      F
324013      F
463191      F
Name: cat9, Length: 124600, dtype: object D
-----
df_prob3[cat_cols].apply(lambda x: print(x.unique()), axis=0)
['A' 'B']
['B' 'A']
['A' 'B']
['C' 'A' 'D']
['B' 'C']
['B' 'D' 'C' 'A']
['A' 'B' 'C']
['E' 'D' 'G' 'B']
['E' 'C' 'A' 'G' 'D']
['F' 'H' 'L' 'K' 'G' 'I' 'B' 'J' 'M' 'A' 'O' 'N']
-----
df_prob3[cat_cols].apply(lambda x: [{x.name, i} for i in np.sort(x.unique())], axis=0)
cat0      [cat0_A, cat0_B]
cat1      [cat1_A, cat1_B]
cat2      [cat2_A, cat2_B]
cat3      [cat3_A, cat3_C, cat3_D]
cat4      [cat4_B, cat4_C]
cat5      [cat5_A, cat5_B, cat5_C, cat5_D]
cat6      [cat6_A, cat6_B, cat6_C]
cat7      [cat7_B, cat7_D, cat7_E, cat7_G]
cat8      [cat8_A, cat8_C, cat8_D, cat8_E, cat8_G]
cat9      [cat9_A, cat9_B, cat9_F, cat9_G, cat9_H, cat9_...

```

```

dtype: object
-----
dum_cols = ['cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C',
            'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G',
            'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I',
            'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
-----
df_prob3.columns = Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7',
                          'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target',
                          'targetA', 'prob_A', 'prob_B', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B',
                          'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C',
                          'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C',
                          'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D',
                          'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H',
                          'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O'],
                          dtype='object')
-----
...

# cat으로 시작하는 변수를 뽑을 수도 있지만, 보다 일반적으로 적용할 수 있는 방법을 보여드립니다.
# dum_cols = [i for i in df_prob3_train.columns if i.startswith('cat')]

# 가변수명이 만들어지는 규칙을 이용해서 가변수명을 구합니다.
# 이를 통해 첫번째, 마지막 변수를 제외하는 로직으로 확장할 수 있습니다.
# explode 는 Series 요소가 list 구조로 되어 있을 때, 이를 풀어내어 Series 구조로 만듭니다.

print('cat_cols = ', cat_cols)
print('-----')

print('df_prob3[cat_cols] =')
print(df_prob3[cat_cols])
print('-----')

print('df_prob3[cat_cols].apply(lambda x: print(x,name), axis=0)')
df_prob3[cat_cols].apply(lambda x: print(x,name), axis=0)
print('-----')

print('df_prob3[cat_cols].apply(lambda x: print(x.unique()), axis=0)')
df_prob3[cat_cols].apply(lambda x: print(x.unique()), axis=0)
print('-----')

print('df_prob3[cat_cols].apply(lambda x: [{_}.format(x.name, i) for i in np.sort(x.unique())], axis=0)')

```

```
temp = df_prob3[cat_cols].apply(lambda x: ['{}_{}'.format(x.name, i) for i in np.sort(x.unique())], axis=0)
print(temp)
print('-----')

dum_cols = df_prob3[cat_cols].apply(
    lambda x: ['{}_{}'.format(x.name, i) for i in np.sort(x.unique())], axis=0).explode().tolist()

print('dum_cols = ', dum_cols)
print('-----')

df_prob3 = pd.get_dummies(df_prob3, columns=cat_cols)
print('df_prob3.columns = ', df_prob3.columns)
print('-----')
```

```
cat_cols = ['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8', 'cat9']
```

```
-----
```

```
df_prob3[cat_cols] =
```

```
    cat0 cat1 cat2 cat3 cat4 cat5 cat6 cat7 cat8 cat9
```

```
id
```

```
29503      A      B      A      C      B      B      A      E      E      F
```

```
113326      A      B      A      A      B      D      A      E      C      H
```

```
199507      A      B      B      A      B      B      A      E      A      L
```

```
346281      A      A      A      D      B      D      A      E      C      F
```

```
137333      A      A      A      C      B      D      A      D      C      K
```

```
...
```

```
76711      A      B      A      C      B      D      A      D      E      F
```

```
215151      A      B      B      C      B      B      A      E      E      F
```

```
244056      A      B      A      C      B      B      A      E      E      F
```

```
324013      A      B      B      C      B      D      A      E      G      F
```

```
463191      A      A      A      C      B      D      A      E      E      F
```

```
[124600 rows x 10 columns]
```

```
-----
```

```
df_prob3[cat_cols].apply(lambda x: print(x,name), axis=0)
```

```
id
```

```
29503      A
```

```
113326      A
```

```
199507      A
```

```
346281      A
```

```
137333      A
```

```
..
```

```
76711      A
```

```
215151      A
```

```
244056      A
```

```
324013      A
```

```
463191      A
```

```
Name: cat0, Length: 124600, dtype: object D
```

```
id
```

```
29503      B
```

```
113326      B
```

```
199507      B
```

```
346281      A
```

```
137333      A
```

```
..
```

```
76711      B
```

```
215151      B
```

```
244056    B
324013    B
463191    A
Name: cat1, Length: 124600, dtype: object D
id
29503     A
113326    A
199507    B
346281    A
137333    A
..
76711     A
215151    B
244056    A
324013    B
463191    A
Name: cat2, Length: 124600, dtype: object D
id
29503     C
113326    A
199507    A
346281    D
137333    C
..
76711     C
215151    C
244056    C
324013    C
463191    C
Name: cat3, Length: 124600, dtype: object D
id
29503     B
113326    B
199507    B
346281    B
137333    B
..
76711     B
215151    B
244056    B
324013    B
463191    B
```

Name: cat4, Length: 124600, dtype: object D

id

29503 B

113326 D

199507 B

346281 D

137333 D

..

76711 D

215151 B

244056 B

324013 D

463191 D

Name: cat5, Length: 124600, dtype: object D

id

29503 A

113326 A

199507 A

346281 A

137333 A

..

76711 A

215151 A

244056 A

324013 A

463191 A

Name: cat6, Length: 124600, dtype: object D

id

29503 E

113326 E

199507 E

346281 E

137333 D

..

76711 D

215151 E

244056 E

324013 E

463191 E

Name: cat7, Length: 124600, dtype: object D

id

29503 E

```

113326    C
199507    A
346281    C
137333    C
..
76711     E
215151    E
244056    E
324013    G
463191    E
Name: cat8, Length: 124600, dtype: object D
id
29503     F
113326    H
199507    L
346281    F
137333    K
..
76711     F
215151    F
244056    F
324013    F
463191    F
Name: cat9, Length: 124600, dtype: object D

```

```

-----
df_prob3[cat_cols].apply(lambda x: print(x.unique()), axis=0)

```

```

['A' 'B']
['B' 'A']
['A' 'B']
['C' 'A' 'D']
['B' 'C']
['B' 'D' 'C' 'A']
['A' 'B' 'C']
['E' 'D' 'G' 'B']
['E' 'C' 'A' 'G' 'D']
['F' 'H' 'L' 'K' 'G' 'I' 'B' 'J' 'M' 'A' 'O' 'N']

```

```

-----
df_prob3[cat_cols].apply(lambda x: [{}_{}.format(x.name, i) for i in np.sort(x.unique())], axis=0)

```

```

cat0          [cat0_A, cat0_B]
cat1          [cat1_A, cat1_B]
cat2          [cat2_A, cat2_B]
cat3          [cat3_A, cat3_C, cat3_D]

```

```

cat4                                [cat4_B, cat4_C]
cat5                                [cat5_A, cat5_B, cat5_C, cat5_D]
cat6                                [cat6_A, cat6_B, cat6_C]
cat7                                [cat7_B, cat7_D, cat7_E, cat7_G]
cat8                                [cat8_A, cat8_C, cat8_D, cat8_E, cat8_G]
cat9    [cat9_A, cat9_B, cat9_F, cat9_G, cat9_H, cat9_...
dtype: object
-----
dum_cols = ['cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
-----
df_prob3.columns = Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target', 'targetA', 'prob_A', 'prob_B', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O'],
dtype='object')
-----

```

In [148...

```
# 단계 3-1 ~ 3-3 정리
...
```

코드는 `df\_basetable` 데이터프레임에 새로운 열을 추가하고 특정 조건을 만족하는 행들을 필터링하여 `df\_prob3` 데이터프레임을 생성한 후, 범주형 변수를 원-핫 인코딩하는 과정을 수행합니다.

1. 새로운 확률 열 추가

```
- `prob_A`:
- `norm.cdf(df_basetable['target'], loc=6.769, scale=0.616)`은 평균 6.769, 표준편차 0.616을 갖는 정규분포에서 `target` 값에 해당하는 누적 분포 함수 값을 계산합니다.
- 이를 `1 -`로 감싸서, 우측 꼬리 확률(1에서 누적 분포 값을 뺀 값)을 구합니다. 즉, `target` 값이 평균보다 더 클 확률을 계산합니다.
- `prob_B`:
- `norm.cdf(df_basetable['target'], loc=8.123, scale=0.527)`은 평균 8.123, 표준편차 0.527을 갖는 정규분포에서 `target` 값에 해당하는 누적 분포 함수 값을 계산합니다. 이는 좌측 꼬리 확률입니다.
```

이 두 확률 값을 `df\_basetable`에 추가하여 `df\_prob3`를 만듭니다.

2. 특정 조건을 만족하는 행 필터링

- ``prob_B``가 `0.01`보다 작거나, ``prob_A``가 `0.01`보다 작은 행들만 필터링하여 ``df_prob3``에 저장합니다.
- 이는 특정 확률 조건을 만족하는 행들만 남기겠다는 의미입니다.

3. 더미 변수 열 이름 생성

- ``df_prob3[cat_cols]``:
 - ``cat_cols`` 리스트에 있는 열들만 추출합니다.
- ``apply``:
 - 각 열에 대해, 열 이름과 열의 고유 값을 조합하여 새로운 열 이름을 생성합니다.
 - 예를 들어, 열 이름이 ``cat1``이고 고유 값이 ``[a, b, c]``라면 ``['cat1_a', 'cat1_b', 'cat1_c']``와 같은 리스트를 생성합니다.
- ``explode``:
 - 리스트 구조를 풀어서 시리즈 구조로 만듭니다.
- ``tolist``:
 - 리스트로 변환합니다.

이 과정을 통해 각 범주형 변수의 원-핫 인코딩된 열 이름 리스트인 ``dum_cols``를 생성합니다.

4. 원-핫 인코딩 수행

- ``pd.get_dummies``를 사용하여 범주형 변수를 원-핫 인코딩합니다.
- ``cat_cols``에 있는 열들을 더미 변수로 변환하여 ``df_prob3``에 추가합니다.

5. 결과

- ``dum_cols`` 리스트를 `numpy` 배열로 변환하여 결과를 출력합니다.

전체 코드 동작 요약

1. 확률 열 추가: ``df_basetable`` 데이터프레임에 ``prob_A``와 ``prob_B`` 열을 추가합니다.
 2. 행 필터링: ``prob_B``가 `0.01`보다 작거나 ``prob_A``가 `0.01`보다 작은 행들만 남깁니다.
 3. 더미 변수 열 이름 생성: 범주형 변수의 고유 값에 따라 원-핫 인코딩된 열 이름 리스트를 생성합니다.
 4. 원-핫 인코딩 수행: 범주형 변수를 원-핫 인코딩하여 ``df_prob3`` 데이터프레임에 추가합니다.
 5. 결과 출력: 원-핫 인코딩된 열 이름 리스트를 `numpy` 배열로 변환하여 출력합니다.
- ```
...
```
- ```
print('')
```

단계 3-4

prob3를 학습용 80%, 그 외 20%는 테스트용으로 나누어 prob3_train, prob3_test로 한다.

함수가이드

```
sklearn.model_selection.train_test_split, random_state=123
```

문제 지시사항 외 Default 값 사용

```
In [149... from sklearn.model_selection import train_test_split  
  
df_prob3_train, df_prob3_test = train_test_split(df_prob3, train_size=0.8, random_state=123)
```

```
In [150... Image('3-4.png')
```

Out[150]:

단계 3-5

prob3_train으로 targetA를 예측하는 xgboost 분류 모델을 다음과 같은 설정으로 prob3_train를 사용하여 학습한다.

prob3_test으로 다음 성능지표로 평가한 값을 C 라고 한다.

입력변수: cont0 ~ 13, cat0 ~ 9까지 원핫-인코딩 파생 변수 (변수명, 범주순으로 알파벳 순서)

대상변수: targetA

$$\text{성능지표: } \text{Accuracy}(Y, \hat{Y}) = \frac{TP+TN}{TP+FP+TN+FN}$$

하이퍼 파라미터(Hyper-Parameter, 초매개변수) 설정:

항목	값
트리의 최대 깊이	2
L1 규제	0.1
L2 규제	0.1
트리 당 컬럼 샘플링 비율	0.25
트리의 수	500
random_state	123

함수가이드

xgboost 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [151... import xgboost as xgb
from sklearn.metrics import accuracy_score# Accuracy

X_xgb = cont_cols + dum_cols
print('X_xgb = ', X_xgb)
```

```

clf_xgb = xgb.XGBClassifier(
    max_depth=2, # 트리의 최대 깊이: 2
    reg_alpha=0.1, # L1 규제: 0.1
    reg_lambda=0.1, # L2 규제: 0.1
    colsample_bytree=0.25, # 트리 당 컬럼 샘플링 비율: 0.25
    n_estimators=500, # 트리의 수: 500
    random_state=123, # random_state: 123
)

clf_xgb.fit(df_prob3_train[X_xgb], df_prob3_train['targetA'])

```

```

X_xgb = ['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']

```

```

Out[151]: XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
    colsample_bytree=0.25, gamma=0, learning_rate=0.1,
    max_delta_step=0, max_depth=2, min_child_weight=1, missing=None,
    n_estimators=500, n_jobs=1, nthread=None,
    objective='binary:logistic', random_state=123, reg_alpha=0.1,
    reg_lambda=0.1, scale_pos_weight=1, seed=None, silent=True,
    subsample=1)

```

```

In [152... df_prob3_train['targetA'].value_counts(normalize=True).max()

```

```

Out[152]: 0.5531099518459069

```

```

In [153... C = accuracy_score(df_prob3_test['targetA'], clf_xgb.predict(df_prob3_test[X_xgb]))
(
    C,
    accuracy_score(df_prob3_train['targetA'], clf_xgb.predict(df_prob3_train[X_xgb]))
)

```

```

Out[153]: (0.6547351524879614, 0.6661918138041734)

```

```

In [154... C, round(C, 3)

```

```

Out[154]: (0.6547351524879614, 0.655)

```

C 를 소수점 넷째 자리에서 반올림하여 셋째 자리까지 출력하라.

0.655

In [155...

```
# 3-4 ~ 3-6 정리

...

이 코드는 `XGBClassifier`를 사용하여 이진 분류 모델을 학습시키고, 훈련 데이터와 테스트 데이터에 대한 예측 정확도를 계산합니다.

### 1. 데이터 분할
- `train_test_split`을 사용하여 `df_prob3` 데이터프레임을 80%의 훈련 데이터 (`df_prob3_train`)와 20%의 테스트 데이터 (`df_prob3_test`)로 분할합니다.

### 2. 독립 변수 설정
- `X_xgb`는 모델 학습에 사용할 독립 변수들의 리스트입니다.
- `cont_cols`는 연속형 변수들의 리스트이고, `dum_cols`는 원-핫 인코딩된 범주형 변수들의 리스트입니다.
- 이 두 리스트를 결합하여 최종 독립 변수 리스트를 생성합니다.

### 3. XGBoost 분류기 설정 및 학습
- `max_depth=2`: 트리의 최대 깊이를 2로 설정하여 모델의 복잡도를 제어합니다.
- `reg_alpha=0.1`: L1 규제를 0.1로 설정하여 모델의 가중치 일부를 0으로 만듭니다.
- `reg_lambda=0.1`: L2 규제를 0.1로 설정하여 가중치의 크기를 제한합니다.
- `colsample_bytree=0.25`: 각 트리를 구성할 때 샘플링할 컬럼의 비율을 0.25로 설정합니다.
- `n_estimators=500`: 500개의 트리를 사용하여 모델을 학습시킵니다.
- `random_state=123`: 난수 생성 시드를 설정하여 결과의 재현성을 보장합니다.

- `clf_xgb.fit`을 사용하여 `df_prob3_train` 데이터프레임의 독립 변수(`X_xgb`)와 종속 변수(`targetA`)를 사용하여 모델을 학습시킵니다.

### 4. 훈련 데이터의 클래스 불균형 확인
- `value_counts(normalize=True)`를 사용하여 `targetA`의 클래스 분포를 확인합니다.
- `.max()`를 사용하여 가장 빈번한 클래스의 비율을 반환합니다.
- 이는 클래스 불균형 정도를 확인하기 위한 것입니다.

### 5. 테스트 데이터에 대한 예측 정확도 계산
- `clf_xgb.predict`을 사용하여 테스트 데이터의 독립 변수(`X_xgb`)를 기반으로 `targetA` 값을 예측합니다.
- `accuracy_score`를 사용하여 실제 `targetA` 값과 예측된 `targetA` 값 간의 정확도를 계산하고, 이를 변수 `C`에 저장합니다.

### 6. 결과 반환
(
    C,
    accuracy_score(df_prob3_train['targetA'],
                    clf_xgb.predict(df_prob3_train[X_xgb]))
)
```

```
)
- `accuracy_score`를 사용하여 훈련 데이터의 실제 `targetA` 값과 예측된 `targetA` 값 간의 정확도를 계산합니다.
- 결과는 튜플 형태로 반환됩니다. 첫 번째 값은 테스트 데이터의 정확도(`C`), 두 번째 값은 훈련 데이터의 정확도입니다.
```

```
### 전체 코드 동작 요약
```

1. 데이터 분할:
`df\_prob3`를 80% 훈련 데이터와 20% 테스트 데이터로 분할합니다.
2. 독립 변수 설정:
연속형 변수들과 원-핫 인코딩된 범주형 변수들을 결합하여 독립 변수 리스트를 만듭니다.
3. XGBoost 분류기 설정 및 학습:
XGBoost 분류기를 설정하고 훈련 데이터를 사용하여 학습시킵니다.
4. 클래스 불균형 확인: 훈련 데이터에서 가장 빈번한 클래스의 비율을 확인합니다.
5. 예측 정확도 계산: 테스트 데이터와 훈련 데이터에 대해 예측을 수행하고 정확도를 계산합니다.
6. 결과 반환: 테스트 데이터와 훈련 데이터의 정확도를 튜플 형태로 반환합니다.

이 과정은 모델의 성능을 평가하는 데 사용됩니다.

테스트 데이터에 대한 정확도는 모델의 일반화 성능을 나타내고, 훈련 데이터에 대한 정확도는

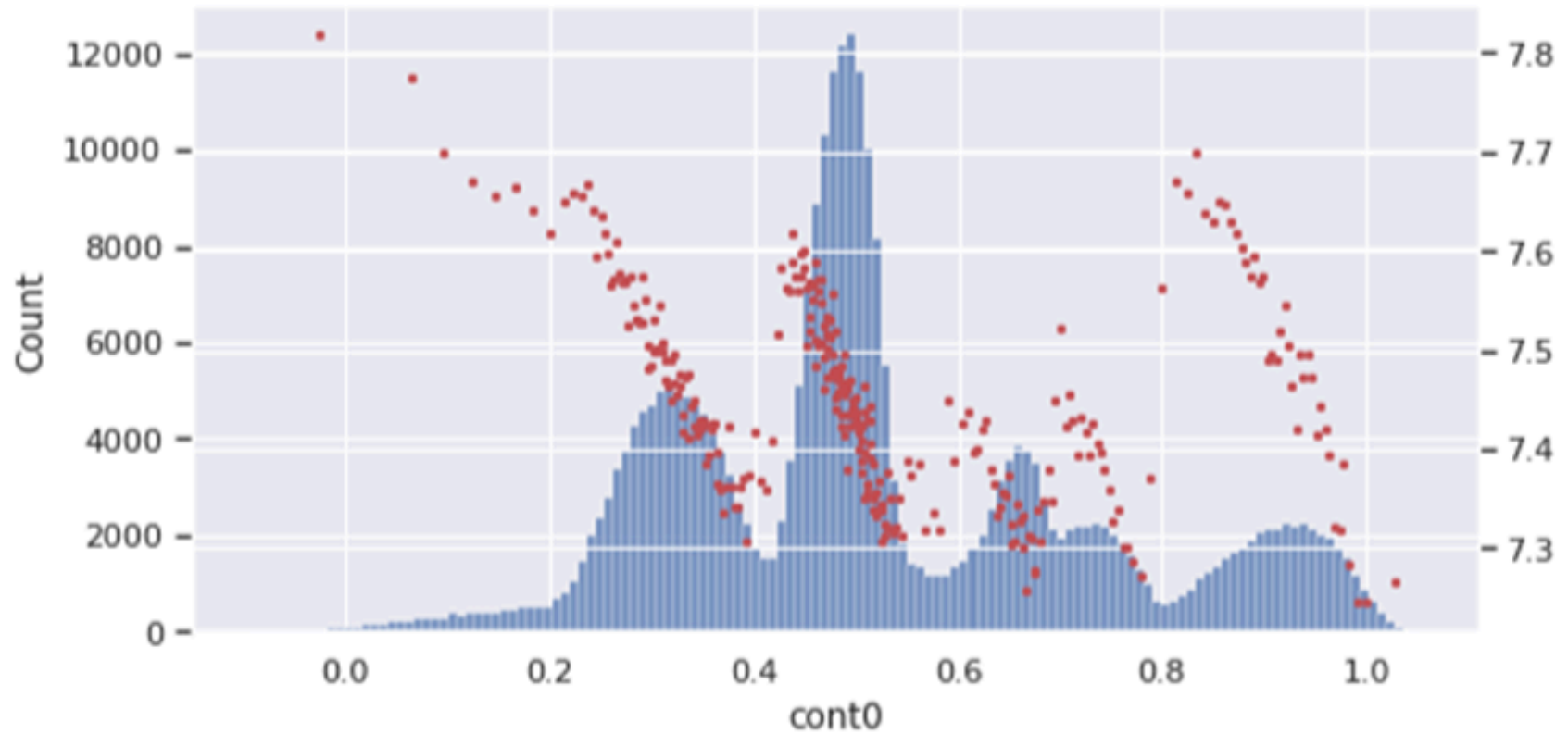
모델이 훈련 데이터에 얼마나 잘 맞는지를 나타냅니다. 두 값의 비교를 통해

모델이 과적합(overfitting)되었는지 또는 과소적합(underfitting)되었는지를 판단할 수 있습니다.

```
...
```

```
print('')
```

문제 4



위 차트는 cont0 의 히스토그램과 cont0를 300개의 분위 구간으로 나누어 x값은 분위 구간의 중간값,

y값은 분위 구간에서 target의 평균을 산포도 그래프로 동시에 출력했다.

좌측 y축은 히스토그램의 빈도수이고, 우측 y축은 target의 평균이다.

히스토그램에서 동일한 분위 구간 내에서 cont0와 target간에 선형적 관계가 있음이 확인되었다.

이 점을 이용하여 파생 변수를 만들어서 회귀모델을 만들어 간다.

In []:

단계 4-1

basetable에서 cat0 ~ 9을 첫 번째 범주(알파벳 순)는 제외하고

원핫 인코딩(One-Hot Encoding)하여 prob4데이터셋을 만든다.

```
In [156... # 첫 번째 범주는 제외하라고 했으므로 srop_first = True가 필요?
#df_prob4 = pd.get_dummies(df_basetable, columns=cat_cols, srop_first = True)
df_prob4 = pd.get_dummies(df_basetable, columns=cat_cols)
df_prob4.head()
```

```
Out[156]:
```

	cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	...	cat9_F	cat9_G	cat9_H	cat9_I	cat9_J	cat9_K	cat9_...
id																		
267387	0.65226	0.55771	0.25398	0.28196	0.82400	0.22443	0.69220	0.30912	0.47134	0.53290	...	0	1	0	0	0	0	
410037	0.36970	0.68159	0.30038	0.20156	0.46113	0.43278	0.97199	0.62596	0.40193	0.49181	...	1	0	0	0	0	0	
139373	0.47881	0.63976	0.40024	0.24153	0.27128	0.36238	0.41310	0.27010	0.40112	0.53052	...	0	0	0	0	1	0	
113765	0.43438	0.77575	0.54887	0.21935	0.27944	0.32670	0.42301	0.28331	0.46425	0.38302	...	0	1	0	0	0	0	
179915	0.47866	0.64326	0.37255	0.21004	0.41188	0.20121	0.37466	0.46316	0.75578	0.55676	...	0	0	0	0	0	0	

5 rows × 55 columns

```
In [157... df_prob4.columns
```

```
Out[157]: Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7',
                'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target',
                'targetA', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B',
                'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B',
                'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D',
                'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G',
                'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J',
                'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O'],
                dtype='object')
```

```
In [158... '''
아래 코드의 각 라인은 Pandas와 NumPy를 사용하여 데이터프레임의 특정 열에서 더미 컬럼(dum_cols)을 생성하는 과정입니다.
각 줄의 동작을 자세히 설명하면 다음과 같습니다.
```

```

### 1. `df_basetable[cat_cols]`
- 동작: `df_basetable`이라는 데이터프레임에서 `cat_cols`에 해당하는 열(컬럼)들만 선택합니다.
- `df_basetable`은 전체 데이터프레임이고, `cat_cols`는 범주형 변수(카테고리형 변수)의 열 이름 리스트로 가정합니다.
  이 부분에서 특정 열들을 추출합니다.
- 출력: `cat_cols`에 해당하는 열들만 포함된 새로운 데이터프레임이 반환됩니다.

### 2. `.apply(lambda x: ['{}_{}'.format(x.name, i) for i in np.sort(x.unique())[1:]])`
- 동작: `apply` 메서드는 선택된 열들에 대해 각 열을 `x`로 받아 `lambda` 함수를 적용합니다.
  - `x.name`: 현재 열의 이름입니다.
  - `x.unique()`: 해당 열에서 중복을 제거한 고유값들을 반환합니다.
  - `np.sort(x.unique())[1:]`: 고유값을 정렬한 후, 첫 번째 요소를 제외한 나머지 요소들로 구성된 배열을 반환합니다.
    이 작업은 더미 변수 생성 시 기준값(예: 원-핫 인코딩의 첫 번째 범주)을 제외하는 데 유용합니다.
  - `['{}_{}'.format(x.name, i) for i in np.sort(x.unique())[1:]]`:
    각 열의 이름과 각 고유값을 결합하여 문자열 리스트를 생성합니다.
    예를 들어, 열 이름이 `color`이고 고유값이 `['red', 'blue', 'green']`이라면,
    `['color_blue', 'color_green']`와 같은 리스트를 생성합니다.
- 출력: 각 열에 대해 생성된 문자열 리스트가 포함된 시리즈가 반환됩니다.

### 3. `.explode()`
- 동작: `explode` 메서드는 리스트나 시리즈의 각 요소를 개별 행으로 확장합니다.
- 출력: 리스트의 각 요소가 행으로 나열된 시리즈가 반환됩니다.
  예를 들어, `['color_blue', 'color_green']`라는 리스트가 하나의 열에 있었다면, 이 리스트는 두 개의 행으로 확장됩니다.

### 4. `.tolist()`
- 동작: `tolist` 메서드는 시리즈를 리스트로 변환합니다.
- 출력: 열 이름과 고유값이 결합된 문자열 리스트가 반환됩니다.

### 5. `np.sort(dum_cols)`
- 동작: `dum_cols` 리스트를 NumPy의 `sort` 메서드를 사용하여 정렬합니다.
- 출력: 문자열 리스트가 알파벳 순으로 정렬됩니다.

### 최종 동작 요약
- 이 코드의 목적은 주어진 범주형 열들에 대해 첫 번째 고유값을 제외한 나머지 고유값을 각 열의 이름과 결합하여
  새로운 문자열 리스트를 만드는 것입니다.
- 그 리스트는 각 고유값에 해당하는 더미 변수의 이름으로 사용할 수 있습니다.
  마지막으로 이 리스트를 알파벳 순으로 정렬합니다.
...
print('')

```

```
In [159... dum_cols = \
    df_basetable[cat_cols].apply(lambda x: ['{}_{}'.format(x.name, i) for i in np.sort(x.unique()[1:]).explode().tolist()
    np.sort(dum_cols)
```

```
Out[159]: array(['cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C',
        'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D',
        'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G',
        'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J',
        'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O'], dtype='<U6')
```

단계 4-2

prob4의 80%를 prob4_train으로 20% prob4_test로 나눈다.

함수가이드

sklearn.model_selection.train_test_split, random_state=123

```
In [160... from sklearn.model_selection import train_test_split

# df_prob4.copy()를 train_test_split을 할 때 해주어도 됩니다.
df_prob4_train, df_prob4_test = train_test_split(df_prob4.copy(), train_size=0.8, random_state=123)
#df_prob4_train, df_prob4_test = df_prob4_train.copy(), df_prob4_test.copy()
```

```
In [161... '''
`pandas.cut()` 함수는 연속형 변수를 구간(bins)으로 나누어 범주형 변수로 변환하는 데 사용됩니다.
예를 들어, 연속적인 나이 데이터를 특정 연령대 그룹으로 나눌 때 유용합니다.

### 1. 함수 정의
pd.cut(x, bins, right=True, labels=None, retbins=False, precision=3, include_lowest=False, duplicates='raise', ordered=True)

### 2. 주요 인자
- `x`:
    - 설명: 데이터를 구간으로 나누기 위해 사용되는 1차원 배열(또는 시리즈)입니다.
    - 유형: 배열, 시리즈, 또는 리스트 형태의 데이터.
- `bins`:
    - 설명: 구간의 경계를 지정합니다.
```

- 유형: 정수나 배열.
- 정수: 지정된 수의 동일한 크기의 구간으로 나누어집니다.
- 배열: 구간의 경계를 명시적으로 지정할 수 있습니다. 이 경우, 배열의 길이는 구간의 수보다 1개 많아야 합니다.
- ``right``:
 - 설명: 각 구간의 오른쪽 경계를 포함할지 여부를 결정합니다.
 - 유형: ``bool``.
 - 기본값: ``True`` (오른쪽 경계 포함).
- ``labels``:
 - 설명: 반환된 구간을 나타내는 레이블입니다.
 - 유형: 배열 또는 ``False``.
 - 배열: 구간에 대응되는 레이블을 지정할 수 있습니다.
 - ``False``: 구간 번호가 반환됩니다.
 - 기본값: ``None`` (구간 자체가 레이블로 사용됨).
- ``retbins``:
 - 설명: 구간 경계를 함께 반환할지 여부를 결정합니다.
 - 유형: ``bool``.
 - 기본값: ``False``.
- ``precision``:
 - 설명: 구간 경계를 반올림하는 소수점 이하 자릿수입니다.
 - 유형: ``int``.
 - 기본값: ``3``.
- ``include_lowest``:
 - 설명: 첫 번째 구간의 왼쪽 경계를 포함할지 여부를 결정합니다.
 - 유형: ``bool``.
 - 기본값: ``False``.
- ``duplicates``:
 - 설명: 중복된 구간 경계가 있을 때 처리 방법을 결정합니다.
 - 유형: ``{'raise', 'drop'}``.
 - ``raise``: 중복된 경계가 있을 경우 에러를 발생시킵니다.
 - ``drop``: 중복된 경계를 제거하고 구간을 생성합니다.
 - 기본값: ``raise``.
- ``ordered``:
 - 설명: 반환된 구간이 순서가 있는 범주형 데이터인지 여부를 결정합니다.
 - 유형: ``bool``.
 - 기본값: ``True``.

3. 리턴값

- ``cut`` 함수는 기본적으로 범주형 데이터가 포함된 ``pandas.Categorical`` 객체를 반환합니다. ``retbins=True``로 설정하면 두 개의 값을 반환합니다.
 - ``cat``: 입력 데이터의 각 값이 어느 구간에 속하는지를 나타내는 범주형 데이터.
 - ``bins``: 실제 구간 경계 값들의 배열.

4. 예제

```
import pandas as pd
```

```
# 예제 데이터
```

```
data = [1, 7, 5, 4, 6, 3, 8, 2, 9, 5]
```

```
# 3개의 구간으로 나누기
```

```
result = pd.cut(data, bins=3)
```

```
print(result)
```

- 출력:

```
[(0.992, 3.667], (6.333, 9.0], (3.667, 6.333], (3.667, 6.333], (3.667, 6.333],  
 (0.992, 3.667], (6.333, 9.0], (0.992, 3.667], (6.333, 9.0], (3.667, 6.333]]  
Categories (3, interval[float64, right]): [(0.992, 3.667] < (3.667, 6.333] < (6.333, 9.0]]
```

5. 예제 2: 사용자 지정 레이블

```
import pandas as pd
```

```
# 예제 데이터
```

```
data = [1, 7, 5, 4, 6, 3, 8, 2, 9, 5]
```

```
# 3개의 구간으로 나누고 레이블 지정
```

```
result = pd.cut(data, bins=[0, 3, 6, 9], labels=['low', 'medium', 'high'])
```

```
print(result)
```

- 출력:

```
[low, high, medium, medium, medium, low, high, low, high, medium]  
Categories (3, object): ['low' < 'medium' < 'high']
```

6. 예제 3: 구간 경계값 반환

```
import pandas as pd

# 예제 데이터
data = [1, 7, 5, 4, 6, 3, 8, 2, 9, 5]

# 구간 경계와 함께 반환
result, bins = pd.cut(data, bins=3, retbins=True)

print(result)
print(bins)

- 출력:

[(0.992, 3.667], (6.333, 9.0], (3.667, 6.333], (3.667, 6.333], (3.667, 6.333],
 (0.992, 3.667], (6.333, 9.0], (0.992, 3.667], (6.333, 9.0], (3.667, 6.333]]
Categories (3, interval[float64, right]): [(0.992, 3.667] < (3.667, 6.333] < (6.333, 9.0]]
[0.992 3.667 6.333 9.    ]

### 요약

- `pd.cut()`은 연속적인 데이터를 구간으로 나누어 범주형 데이터로 변환하는 데 사용됩니다.
- 여러 인자를 통해 구간의 개수, 구간 경계, 레이블 등을 세밀하게 조정할 수 있습니다.
- 반환된 결과는 범주형 데이터이며, 구간 경계 값을 함께 반환할 수도 있습니다.
...

print('')
```

단계 4-3

prob4_train에서 cont0를 100개의 우측이 닫힌

(ex. i 번째 분위수 $< \text{cont0} \leq i+1$ 번째 분위수) 분위 구간으로 나누고

각각의 분위 구간에서 target의 평균을 구해 파생 변수 cont0_q를 만든다.

prob4_test에는 prob4_train을 나누었던 구간 설정과 구간의 평균으로 cont0_q를 만든다.

이 때, prob4_test의 cont0값이 train의 첫 번째 구간보다 작다면, 첫 번째 구간으로 간주하고,

마지막 구간보가 크다면, 마지막 구간으로 간주한다.

함수가이드

Hint 1] pandas.Series.quantile, pandas.cut 활용을 권장한다.

Hint 2] 두 번째 분위 구간은 $0.111 < \text{cont0} \leq 0.176$ 이고 target의 평균은 7.672 (셋째 자리까지 반올림)

Hint 3] cont0_q의 평균은 7.46, 표준 편차는 0.10 (소수점 둘째 자리까지 반올림)

In [162...

```
'''
아래 코드는 Pandas와 NumPy를 사용하여 데이터프레임에서 분위수를 계산하고,
이를 바탕으로 특정 열에 대해 구간을 나누고, 각 구간에 대한 평균을 계산하여 새로운 열을 생성하는 과정을 다룹니다.

q = np.arange(0, 101) / 100 # np.linspace(0, 1, 101)
이 라인은 0과 1 사이에 101개의 균일한 간격의 수를 생성합니다.
`np.arange(0, 101) / 100`는 0.00, 0.01, 0.02, ..., 1.00과 같은 값을 생성합니다.
`np.linspace(0, 1, 101)`을 사용해도 같은 결과를 얻을 수 있습니다.

q_val = df_prob4_train['cont0'].quantile(q)
이 라인은 `df_prob4_train` 데이터프레임의 `cont0` 열에 대해 위에서 생성한 분위점 `q`를 사용하여 분위수를 계산합니다.
이 결과는 `q_val`에 저장됩니다. `q_val`는 `cont0` 열의 분위수 값을 포함하는 시리즈가 됩니다.

q_val.iloc[[0, -1]] = (-np.inf, np.inf)
이 라인은 `q_val`의 첫 번째 값과 마지막 값을 각각 `-np.inf`와 `np.inf`로 설정하여
첫 번째 분위수보다 작은 값은 첫 번째 분위구간에, 마지막 분위수보다 큰 값은 마지막 분위구간에 속하도록 합니다.

q_range = pd.cut(df_prob4_train['cont0'], bins=q_val, right=True)
이 라인은 `df_prob4_train` 데이터프레임의 `cont0` 열을 `q_val`에 따라 구간으로 나눕니다.
`pd.cut` 함수는 `cont0`의 값을 `q_val`의 분위구간에 맞춰 구간을 나누고, 그 결과를 `q_range`에 저장합니다.
`right=True`는 각 구간의 오른쪽 경계를 포함한다는 의미입니다.

q_mean = df_prob4_train.groupby(q_range)['target'].agg('mean')
이 라인은 `q_range`에 따라 `df_prob4_train` 데이터를 그룹화하고, 각 구간에 속하는 `target` 열의 평균을 계산합니다.
```

결과는 `q\_mean`에 저장됩니다. `q\_mean`은 각 구간의 `target` 평균값을 포함하는 시리즈가 됩니다.

```
df_prob4_train['cont0_q'] = q_range.map(q_mean).astype('float')
```

이 라인은 `df\_prob4\_train` 데이터프레임에 새로운 열 `cont0\_q`를 생성합니다.

`q\_range` 값을 `q\_mean`의 값으로 매핑합니다. `map` 함수는 `replace`보다 빠르게 동작하고, 매핑 정보가 없는 값은 결측치로 처리합니다.

결과적으로, `cont0\_q`는 각 `cont0` 값이 속하는 구간의 `target` 평균값을 갖게 됩니다.

`astype('float')`은 결과를 float 형식으로 변환합니다.

```
df_prob4_test['cont0_q'] = pd.cut(df_prob4_test['cont0'], bins=q_val, right=True).map(q_mean).astype('float')
```

이 라인은 `df\_prob4\_test` 데이터프레임에 대해 동일한 작업을 수행합니다.

`df\_prob4\_test`의 `cont0` 열을 `q\_val`에 따라 구간으로 나누고,

각 구간에 대해 `q\_mean`의 값을 매핑하여 `cont0\_q` 열을 생성합니다.

결과적으로, `df\_prob4\_test`의 각 `cont0` 값도 대응하는 구간의 `target` 평균값으로 대체됩니다.

이렇게 전체적인 과정은 `cont0` 열의 값을 분위구간으로 나누고, 각 구간에 대해 `target` 값의 평균을 계산하여 새로운 열을 생성하는 것입니다. 이 과정을 통해 각 데이터 포인트가 속하는 분위구간의 평균 `target` 값을 구할 수 있습니다.

...

```
# 0, 1 사이(경계 포함)하여 101개의 간격이 균일한 수를 구합니다.
```

```
q = np.arange(0, 101) / 100 # np.linspace(0, 1, 101)
```

```
# 만들어낸 분위점으로 분위수를 구합니다.
```

```
q_val = df_prob4_train['cont0'].quantile(q)
```

```
# 첫 번째 분위보다 작은 수는 첫 번째 분위구간, 마지막 분위수보다 큰 수는
```

```
# 마지막 분위구간으로 매칭되게 합니다.
```

```
q_val.iloc[[0, -1]] = (-np.inf, np.inf)
```

```
# 분위구간을 구합니다.
```

```
q_range = pd.cut(df_prob4_train['cont0'], bins=q_val, right=True)
```

```
# 분위구간의 평균을 구합니다.
```

```
q_mean = df_prob4_train.groupby(q_range)['target'].agg('mean')
```

```
# Train에서 도출한 분위구간 설정으로 분위구간 도출후,
```

```
# Train에서 구한 구간에 평균으로 매핑합니다.
```

```
# map은 replace 빠른 처리가 가능합니다. map에서는 replace와 달리
```

```
# mapping 정보를 제공하지 않은 값은 결측이 되게 됩니다.
```

```
# map은 원래 데이터의 형식을 지니게 되어 category가 되고,
```

```
# replace는 치환할 데이터의 형식을 따르게 되어 float이 됩니다.
```

```
df_prob4_train['cont0_q'] = q_range.map(q_mean).astype('float')
```

```
df_prob4_test['cont0_q'] = pd.cut(df_prob4_test['cont0'], bins=q_val, right=True).map(q_mean).astype('float')
```

In [163...

```
'''
아래 코드는 Pandas와 NumPy를 사용하여 `df_prob4_train` 데이터프레임의 `cont0` 열에 대해 분위수를 계산하고,
각 분위구간에 대해 평균 `target` 값을 구하여 새로운 열로 생성하는 과정을 다룹니다.

# 0과 1 사이를 균일하게 나누는 101개의 점을 생성합니다.
q = np.linspace(0, 1, 101)
이 라인은 0과 1 사이를 균일하게 나누는 101개의 점을 생성합니다. `q`는 0.00, 0.01, 0.02, ..., 1.00의 값을 가지는 배열이 됩니다.

# 만들어진 분위점으로 분위수를 구합니다.
q_val = df_prob4_train['cont0'].quantile(q)
이 라인은 `df_prob4_train` 데이터프레임의 `cont0` 열에 대해 위에서 생성한 분위점 `q`를 사용하여 분위수를 계산합니다.
`q_val`은 `cont0` 열의 분위수 값을 포함하는 시리즈가 됩니다.

# 첫 번째 분위보다 작은 수는 첫 번째 분위구간, 마지막 분위수보다 큰 수는 마지막 분위구간으로 매칭되게 합니다.
q_val.iloc[[0, -1]] = [-np.inf, np.inf]
이 라인은 `q_val`의 첫 번째 값과 마지막 값을 각각 `-np.inf`와 `np.inf`로 설정하여
첫 번째 분위수보다 작은 값은 첫 번째 분위구간에, 마지막 분위수보다 큰 값은 마지막 분위구간에 속하도록 합니다.

# 분위구간을 구합니다.
q_range = pd.cut(df_prob4_train['cont0'], bins=q_val, right=True)
이 라인은 `df_prob4_train` 데이터프레임의 `cont0` 열을 `q_val`에 따라 구간으로 나눕니다.
`pd.cut` 함수는 `cont0`의 값을 `q_val`의 분위구간에 맞춰 구간을 나누고, 그 결과를 `q_range`에 저장합니다.
`right=True`는 각 구간의 오른쪽 경계를 포함한다는 의미입니다.

# 분위구간의 평균을 구합니다.
q_mean = df_prob4_train.groupby(q_range)['target'].mean()
이 라인은 `q_range`에 따라 `df_prob4_train` 데이터를 그룹화하고, 각 구간에 속하는 `target` 열의 평균을 계산합니다.
결과는 `q_mean`에 저장됩니다. `q_mean`은 각 구간의 `target` 평균값을 포함하는 시리즈가 됩니다.

# 첫 5개의 원래 값, 분위구간 라벨, 분위구간의 평균을 가진 데이터프레임을 생성합니다.
pd.concat([
    df_prob4_train['cont0'].iloc[:5],
    q_range.iloc[:5].rename('cont0_lab'),
    q_range.iloc[:5].map(q_mean).astype('float').rename('cont0_q')
], axis=1
)
이 라인은 `df_prob4_train`의 첫 5개의 데이터에 대해 다음 세 가지 정보를 포함하는 데이터프레임을 생성합니다:
1. 원래 `cont0` 값 (`df_prob4_train['cont0'].iloc[:5]`)
2. `pd.cut`을 통해 구한 구간 라벨 (`q_range.iloc[:5]`)
3. 각 구간에 대한 평균 `target` 값 (`q_range.iloc[:5].map(q_mean).astype('float')`)
`pd.concat` 함수를 사용하여 이 세 가지 정보를 합쳐 새로운 데이터프레임을 생성합니다.
'''
```

`axis=1`은 열 방향으로 데이터를 합친다는 의미입니다.

전체 과정 요약

1. 0과 1 사이의 101개의 균일한 점을 생성합니다.
2. `cont0` 열의 값들에 대해 이 점들을 기준으로 분위수를 계산합니다.
3. 첫 번째 분위수보다 작은 값은 첫 번째 구간에, 마지막 분위수보다 큰 값은 마지막 구간에 속하도록 설정합니다.
4. `cont0` 열을 이 분위수에 따라 구간으로 나눕니다.
5. 각 구간에 속하는 `target` 값의 평균을 계산합니다.
6. `df\_prob4\_train`의 첫 5개의 데이터에 대해 원래 값, 구간 라벨, 구간의 평균 `target` 값을 포함하는 데이터프레임을 생성합니다.

이 과정을 통해 데이터의 분포를 이해하고, 각 구간의 `target` 값 평균을 기반으로 새로운 특성을 생성할 수 있습니다.

변수의 처리과정을 상세히 보여드립니다.

```
q = np.linspace(0, 1, 101)
q_val = df_prob4_train['cont0'].quantile(q)
q_val.iloc[[0, -1]] = [-np.inf, np.inf]
q_range = pd.cut(df_prob4_train['cont0'], bins=q_val, right=True)
q_mean = df_prob4_train.groupby(q_range)['target'].mean()
pd.concat([
    df_prob4_train['cont0'].iloc[:5],
    q_range.iloc[:5].rename('cont0_lab'),
    q_range.iloc[:5].map(q_mean).astype('float').rename('cont0_q')
], axis=1
)
```

Out[163]:

	cont0	cont0_lab	cont0_q
id			
487693	0.49355	(0.492, 0.494]	7.449876
307175	0.66260	(0.662, 0.669]	7.306663
493409	0.87161	(0.871, 0.885]	7.612856
414858	0.51951	(0.517, 0.52]	7.352864
408161	0.86293	(0.854, 0.871]	7.638234

In [164...

```
# 단계 4-1 ~ 4-3 정리
...
```

코드는 ``df_basetable`` 데이터프레임의 범주형 변수들을 원-핫 인코딩하고, 연속형 변수 ``cont0``을 분위수 기반으로 구간화하여 각 구간의 평균 값을 사용해 새로운 변수를 생성합니다. 이를 통해 ``cont0``의 값을 구간별로 평균값으로 변환하는 과정을 구현합니다.

1. 데이터 전처리 및 분할

- ``cat_cols``에 있는 범주형 변수들을 원-핫 인코딩하여 새로운 데이터프레임 ``df_prob4``를 생성합니다. 원-핫 인코딩된 변수들은 기존 범주형 변수 이름과 수준이 결합된 새로운 변수로 생성됩니다.
- 각 범주형 변수에 대해 원-핫 인코딩으로 생성된 변수 이름을 리스트로 만듭니다. 여기서 첫 번째 범주(사전순으로 가장 앞에 오는 범주)를 제외합니다.
- ``dum_cols`` 리스트에는 원-핫 인코딩된 변수 이름들이 포함됩니다.
- ``np.sort(dum_cols)``는 ``dum_cols`` 리스트를 정렬합니다.
- ``df_prob4`` 데이터프레임을 80%의 훈련 데이터(``df_prob4_train``)와 20%의 테스트 데이터(``df_prob4_test``)로 분할합니다.
- ``random_state=123``를 사용하여 결과의 재현성을 보장합니다.

2. 분위수 기반 구간화 및 평균값 매핑

- ``0``부터 ``1``까지 ``101``개의 균등한 간격을 가진 숫자 배열 ``q``를 생성합니다. 이 배열은 분위수 계산에 사용됩니다.
- ``df_prob4_train['cont0']``의 ``q`` 분위수를 계산하여 ``q_val``에 저장합니다. 이는 ``cont0`` 값을 100개의 분위 구간으로 나눕니다.
- ``q_val``의 첫 번째 값과 마지막 값을 각각 ``-np.inf``와 ``np.inf``로 설정하여 모든 ``cont0`` 값이 구간에 포함될 수 있도록 합니다.
- ``df_prob4_train['cont0']``의 값을 ``q_val``을 기준으로 구간화하여 ``q_range``에 저장합니다. ``pd.cut``은 데이터를 구간으로 나누는 함수입니다.
- ``q_range``로 구간화된 ``cont0`` 값들을 기준으로 ``target``의 평균값을 계산하여 ``q_mean``에 저장합니다.
- ``q_range``로 구간화된 ``cont0`` 값을 ``q_mean``의 평균값으로 매핑하여 ``df_prob4_train['cont0_q']``에 저장합니다.
- 동일한 ``q_val`` 구간을 사용하여 ``df_prob4_test['cont0']``를 구간화하고, ``q_mean``의 평균값으로 매핑하여 ``df_prob4_test['cont0_q']``에 저장합니다.

3. 결과 확인

- 동일한 과정으로 분위수를 구하고, ``df_prob4_train``의 ``cont0`` 변수의 처음 5개 값에 대해 구간화된 레이블과 평균값을 확인합니다.
- ``pd.concat``을 사용하여 ``cont0``, ``q_range``, ``q_range.map(q_mean)``의 값을 결합하여 결과를 확인합니다.

요약

이 코드는 다음과 같은 작업을 수행합니다:

1. 원-핫 인코딩을 통해 범주형 변수들을 변환합니다.
2. ``cont0`` 변수에 대해 100개의 분위 구간을 생성합니다.
3. 각 구간의 ``target`` 평균값을 계산하고, 이를 ``cont0``의 값에 매핑하여 새로운 변수를 생성합니다.
4. 훈련 데이터와 테스트 데이터에 대해 동일한 구간화를 적용하여 변환된 값을 계산합니다.
5. ``cont0`` 변수의 처음 5개 값에 대해 구간화된 레이블과 평균값을 확인합니다.

```
...
```

```
print('')
```

In [165...

```
'''
`axes.ravel()`은 Matplotlib에서 `plt.subplots()` 함수로 생성된 `axes` 객체를 1차원 배열로 변환하는 데 사용됩니다.
### `axes.ravel()`의 동작
- `axes` 객체: `plt.subplots(4, 4, figsize=(18, 9))`에서 반환된 `axes` 객체는 4x4 배열,
  즉 16개의 개별 플롯(축)을 포함하는 2차원 배열입니다.
  이 배열은 `axes[0, 0]`부터 `axes[3, 3]`까지의 인덱스로 개별 축에 접근할 수 있습니다.
- `ravel()` 함수: `axes.ravel()` 함수는 이 2차원 배열을 1차원 배열로 평탄화합니다.
  예를 들어, 원래의 2차원 배열이 아래와 같다면:
  [[ax1, ax2, ax3, ax4],
   [ax5, ax6, ax7, ax8],
   [ax9, ax10, ax11, ax12],
   [ax13, ax14, ax15, ax16]]

  `axes.ravel()`은 이를 다음과 같은 1차원 배열로 변환합니다:

  [ax1, ax2, ax3, ax4, ax5, ax6, ax7, ax8, ax9, ax10, ax11, ax12, ax13, ax14, ax15, ax16]

### 리턴값
- 리턴값: `axes.ravel()`은 1차원 `ndarray` 객체를 반환합니다.
  이 배열은 원래의 4x4 `axes` 배열에 포함된 개별 플롯을 일렬로 나열한 것입니다.

### 사용 예시
이 함수는 각 축에 대해 반복 작업을 할 때 유용합니다.
예를 들어, 각 서브플롯의 속성을 한 번에 설정하려는 경우, 다음과 같이 사용할 수 있습니다:

fig, axes = plt.subplots(4, 4, figsize=(18, 9))
for ax in axes.ravel():
    ax.plot([1, 2, 3], [4, 5, 6]) # 각 서브플롯에 동일한 그래프를 그리기

이렇게 하면 4x4 배열의 각 서브플롯에서 동일한 작업을 수행할 수 있습니다.
'''

'''
`enumerate()` 함수는 Python에서 반복 가능한(iterable) 객체를 순회할 때,
각 요소에 대한 인덱스(index)를 함께 제공하는 데 사용됩니다.
이 함수는 리스트, 튜플, 문자열, 또는 다른 반복 가능한 객체를 입력으로 받아,
각 요소와 그 요소의 인덱스를 포함하는 튜플을 반환하는 이터레이터를 생성합니다.

### `enumerate()` 함수의 동작
#### 인자
```

- ``iterable`` (필수): 인덱스와 함께 순회하려는 반복 가능한 객체입니다. 예를 들어, 리스트, 튜플, 문자열 등이 될 수 있습니다.
- ``start`` (선택): 인덱스의 시작 값을 지정합니다. 기본값은 ``0``이지만, 이 값을 변경하여 인덱스를 원하는 숫자부터 시작하게 할 수 있습니다. 예를 들어, ``start=1``로 설정하면 인덱스가 ``1``부터 시작합니다.

리턴값

- 이터레이터 객체: ``enumerate()`` 함수는 (인덱스, 요소) 쌍을 포함하는 이터레이터를 반환합니다. 이 이터레이터는 반복 가능한 객체의 각 요소와 해당 요소의 인덱스를 튜플 형태로 제공합니다.

사용 예시

기본 사용 예

```
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

이 코드는 다음과 같은 출력을 생성합니다:

```
0 apple
1 banana
2 cherry
```

- 여기서 ``enumerate(fruits)``는 ``(0, 'apple')``, ``(1, 'banana')``, ``(2, 'cherry')``와 같은 튜플을 생성합니다. ``index`` 변수에는 인덱스가, ``fruit`` 변수에는 해당 요소가 할당됩니다.

start 인자를 사용한 예

```
for index, fruit in enumerate(fruits, start=1):
    print(index, fruit)
```

이 코드는 다음과 같은 출력을 생성합니다:

```
1 apple
2 banana
3 cherry
```

- ``start=1``로 설정했기 때문에 인덱스가 **1**부터 시작합니다.

요약

- 인자
 - ``iterable``: 순회할 반복 가능한 객체.
 - ``start`` (옵션): 인덱스의 시작 값을 지정하는 정수.
- 리턴값
 - ``(인덱스, 요소)`` 쌍을 생성하는 이터레이터 객체. 이터레이터를 통해 반복 시, 각 요소와 해당 요소의 인덱스를 튜플로 제공합니다.

```
`enumerate()` 함수는 반복문 내에서 인덱스와 요소를 동시에 다뤄야 할 때 매우 유용한 도구입니다.
...
print('')
```

단계 4-4

단계 4-3에서 cont0_q를 만든 것처럼 나머지 변수 cont1~13에 대해서도 파생변수를 생성한다.

생성한 파생 변수명은 원래의 변수명에 _q를 결합한다. (ex. cont1 -> cont1_q)

In [166...

```
'''
아래 코드는 `df_train` 데이터프레임에서 연속형 변수 `cont0`부터 `cont13`까지의 히스토그램과 각 변수의 구간별 `target` 평균을 시각화
각 변수에 대해 히스토그램과 `target` 평균을 같은 그래프에 나타내어, 변수 분포와 `target` 값 간의 관계를 시각적으로 분석합니다.

### 1. 그래프 설정 및 초기화
- 4x4 격자의 서브플롯을 생성하여 총 16개의 서브플롯을 `axes` 배열에 저장합니다.
- 그래프의 크기를 `figsize=(18, 9)`로 설정합니다.

### 2. 반복문을 통한 변수별 시각화
- `enumerate(axes.ravel()[:14])`는 `axes` 배열을 1차원으로 펼쳐 `cont0`부터 `cont13`까지의 14개 서브플롯에 대해 인덱스와 축(`ax`)을 순회합니다.
- 각 반복에서 변수명을 `col = 'cont{}'.format(i)`로 설정합니다.
- `sns.histplot(data=df_train, x=col, ax=ax)`는 `df_train` 데이터프레임에서 해당 변수 `col`의 히스토그램을 생성하여 `ax` 서브플롯에 그립니다.

### 3. 구간별 `target` 평균 계산 및 시각화
- `pd.qcut(df_train[col], q=300)`은 변수 `col`을 300개의 분위 구간으로 나누고, 이 구간을 기준으로 `df_train` 데이터프레임을 그룹화합니다.
- 각 구간별 `target` 값의 평균을 계산하여 `tmp` 데이터프레임에 저장합니다.

- `ax.twinx()`는 기존 `ax` 서브플롯과 동일한 x축을 공유하는 새로운 y축을 생성합니다.
- 각 구간의 중간값을 x축 좌표로, 해당 구간의 `target` 평균값을 y축 좌표로 설정하여 빨간색 점(`c='r'`, `s=5`)으로 산점도를 그립니다.

### 4. 그래프 레이아웃 및 출력
- `plt.tight_layout()`는 서브플롯 간의 간격을 자동으로 조정하여 그래프들이 겹치지 않도록 합니다.
- `plt.show()`는 그래프를 화면에 출력합니다.
'''
```

```

### 요약
- 이 코드는 4x4 격자의 서브플롯을 생성하여 `cont0`부터 `cont13`까지의
  14개 연속형 변수에 대해 히스토그램과 구간별 `target` 평균을 시각화합니다.
- 각 서브플롯에는 변수의 히스토그램과 해당 변수의 300개의 분위 구간별 `target` 평균이 빨간색 점으로 표시됩니다.
- 이 시각화를 통해 각 변수의 분포와 `target` 값 간의 관계를 시각적으로 분석할 수 있습니다.
...

...

아래 코드는 `pandas`의 `qcut` 기능을 사용하여 데이터를 특정 구간으로 나누고, 그 결과를 사용해
히스토그램과 `target` 변수의 평균을 시각화합니다. 여기서 `tmp.index.categories.left`는 구간의 왼쪽 경계를 의미합니다.

### 1. `index`
`index`는 `pandas`의 데이터프레임이나 시리즈에서 행(row)의 라벨을 가리킵니다.
`tmp`는 `df_train.groupby()`로 만들어진 그룹화된 결과의 평균을 담고 있는 데이터프레임이므로,
`tmp.index`는 이 데이터프레임의 행 라벨을 가리킵니다. 이 경우, `tmp.index`는 `qcut`으로 생성된 구간들로 구성됩니다.

### 2. `categories`
`qcut`은 데이터를 특정 개수의 구간으로 나누는 함수입니다.
예를 들어, `qcut`의 결과는 각 값이 속한 구간을 나타내는 `Categorical` 데이터로 반환됩니다.
`Categorical` 데이터는 여러 범주(categories)를 가지는데, 각 범주는 하나의 구간을 나타냅니다.
`tmp.index.categories`는 `qcut`에 의해 생성된 구간들을 의미합니다.
각 구간은 `pd.Interval` 객체로 표현되며, 범주형 데이터의 모든 구간이 `categories` 속성으로 저장됩니다.

### 3. `left`
각 구간은 `pd.Interval` 객체로 표현되며, 이 객체는 `left`와 `right` 속성을 가집니다.
`left`는 구간의 왼쪽 경계값을 나타내고, `right`는 구간의 오른쪽 경계값을 나타냅니다.
따라서 `tmp.index.categories.left`는 `qcut`으로 생성된 모든 구간의 왼쪽 경계값들을 나열한 배열입니다.
이 값들은 구간의 중간값을 계산할 때 사용됩니다.

### 전체 흐름 요약
- `qcut`으로 데이터 구간을 나누면, 각 값은 특정 구간에 속하게 됩니다.
- `tmp.index`는 이 구간들을 나타내는 `CategoricalIndex`이며, `tmp.index.categories`는 각 구간의 범주를 나타냅니다.
- `tmp.index.categories.left`는 각 구간의 왼쪽 경계값을 모아놓은 배열입니다.

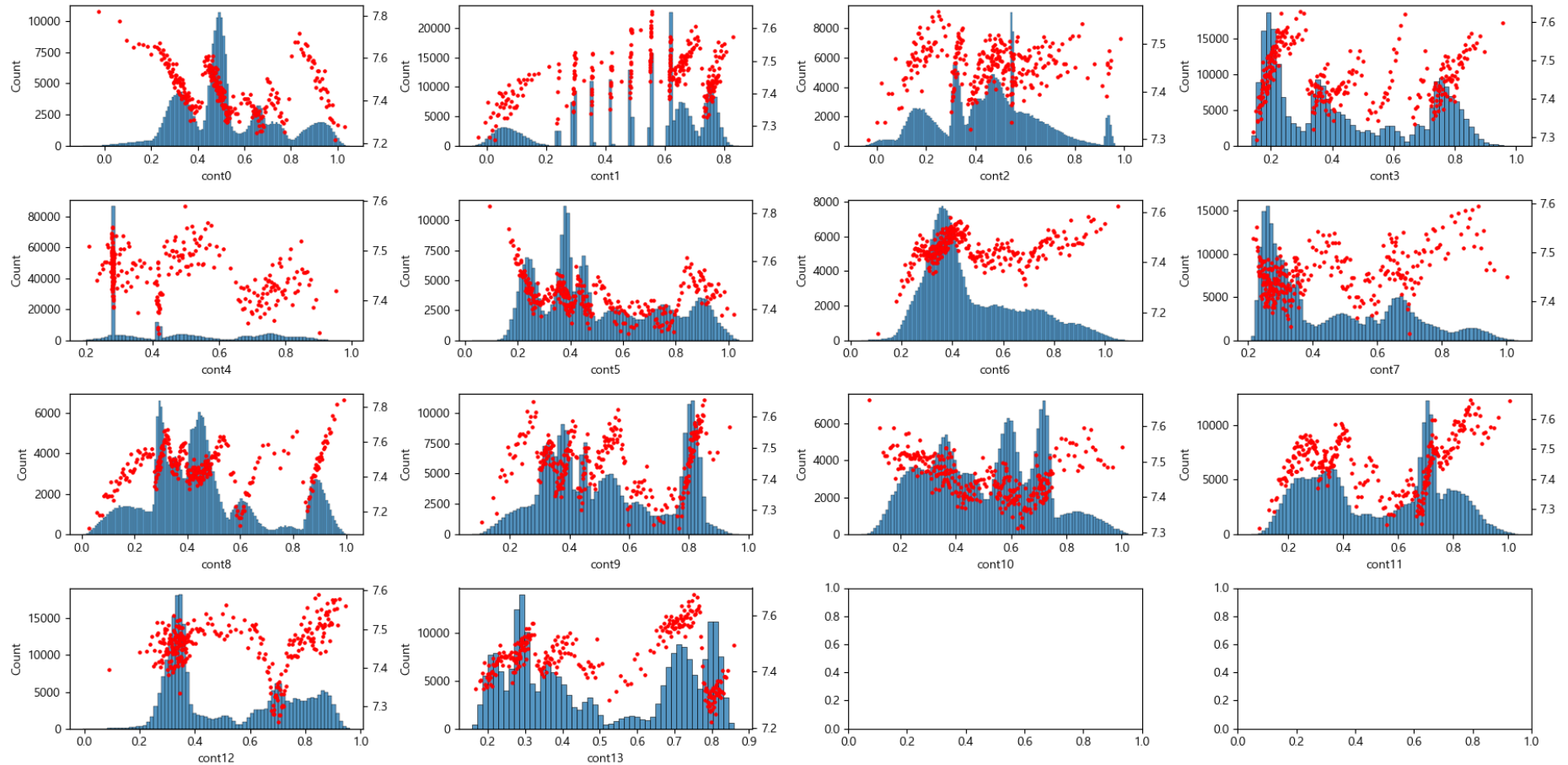
코드에서 `tmp.index.categories.left`와 `tmp.index.categories.right`의 평균을 계산해 구간의 중간값을 구한 다음,
이 중간값과 해당 구간의 `target` 변수 평균을 산점도로 그려 시각화하고 있습니다.
...

# 다른 변수에 대해서도 그래프를 출력합니다.

fig, axes = plt.subplots(4, 4, figsize=(18, 9))
for i, ax in enumerate(axes.ravel()[:14]):

```

```
col = 'cont{}'.format(i)
sns.histplot(data=df_train, x=col, ax=ax)
tmp = df_train.groupby(pd.qcut(df_train[col], q=300))['target'].mean()
ax.twinx().scatter((tmp.index.categories.left + tmp.index.categories.right) / 2, tmp['target'], c='r', s=5)
plt.tight_layout()
plt.show()
```



In [167...

...

아래 코드는 여러 열(`cont\_cols` 리스트에 저장된 열 이름들)을 포함한 데이터프레임에 대해 분위수를 계산하고, 각 분위구간에 대해 평균 `target` 값을 구하여 새로운 열을 생성하는 과정을 다룹니다.

0과 1 사이의 101개의 균일한 간격을 생성합니다.

`q = np.arange(0, 101) / 100`

이 라인은 0과 1 사이를 101개의 균일한 간격으로 나눈 배열을 생성합니다. `q`는 0.00, 0.01, 0.02, ..., 1.00의 값을 가지는 배열이 됩니다.

```
# cont_cols에 포함된 각 열에 대해 반복 작업을 수행합니다.
for i in cont_cols:
    이 라인은 `cont_cols` 리스트에 포함된 각 열 이름에 대해 반복 작업을 시작합니다.
    `cont_cols`는 `df_prob4_train` 데이터프레임의 열 이름들을 포함한 리스트입니다.

    # 각 열에 대한 분위수를 계산합니다.
    qt = df_prob4_train[i].quantile(q)
    이 라인은 현재 열 `i`에 대해 위에서 생성한 분위점 `q`를 사용하여 분위수를 계산합니다.
    `qt`는 현재 열 `i`의 분위수 값을 포함하는 시리즈가 됩니다.

    # 첫 번째 분위수보다 작은 값은 첫 번째 구간, 마지막 분위수보다 큰 값은 마지막 구간에 속하게 합니다.
    qt.iloc[[0, -1]] = (-np.inf, np.inf)
    이 라인은 `qt`의 첫 번째 값과 마지막 값을 각각 `-np.inf`와 `np.inf`로 설정하여
    첫 번째 분위수보다 작은 값은 첫 번째 구간에, 마지막 분위수보다 큰 값은 마지막 구간에 속하도록 합니다.

    # 현재 열에 대해 분위구간을 생성합니다.
    q_range = pd.cut(df_prob4_train[i], bins=qt, right=True)
    이 라인은 현재 열 `i`의 값을 `qt`에 따라 구간으로 나눕니다.
    `pd.cut` 함수는 현재 열 `i`의 값을 `qt`의 분위구간에 맞춰 구간을 나누고, 그 결과를 `q_range`에 저장합니다.
    `right=True`는 각 구간의 오른쪽 경계를 포함한다는 의미입니다.

    # 각 분위구간에 대해 target 값의 평균을 계산합니다.
    q_mean = df_prob4_train.groupby(q_range)['target'].agg('mean')
    이 라인은 `q_range`에 따라 `df_prob4_train` 데이터를 그룹화하고, 각 구간에 속하는 `target` 열의 평균을 계산합니다.
    결과는 `q_mean`에 저장됩니다. `q_mean`은 각 구간의 `target` 평균값을 포함하는 시리즈가 됩니다.

    # 현재 열에 대한 새로운 분위구간 평균값 열을 생성합니다.
    df_prob4_train[i + '_q'] = q_range.map(q_mean).astype('float')
    이 라인은 `df_prob4_train` 데이터프레임에 새로운 열을 생성합니다. 새로운 열의 이름은 현재 열 이름에 `_q`를 붙인 형태이며,
    `q_range` 값을 `q_mean` 값으로 매핑한 결과를 저장합니다. `astype('float')`은 결과를 float 형식으로 변환합니다.

    # 테스트 데이터에 대해서도 동일한 작업을 수행하여 새로운 분위구간 평균값 열을 생성합니다.
    df_prob4_test[i + '_q'] = pd.cut(df_prob4_test[i], bins=qt, right=True).map(q_mean).astype('float')
    이 라인은 `df_prob4_test` 데이터프레임에 대해 동일한 작업을 수행합니다.
    `df_prob4_test`의 현재 열 `i` 값을 `qt`에 따라 구간으로 나누고,
    각 구간에 대해 `q_mean`의 값을 매핑하여 새로운 열을 생성합니다.
    새로운 열의 이름은 `df_prob4_train`과 동일하게 현재 열 이름에 `_q`를 붙인 형태입니다.

### 전체 과정 요약
1. 0과 1 사이의 101개의 균일한 점을 생성합니다.
2. `cont_cols` 리스트에 포함된 각 열에 대해 반복 작업을 수행합니다.
```

3. 현재 열의 분위수를 계산합니다.
4. 첫 번째 분위수보다 작은 값은 첫 번째 구간, 마지막 분위수보다 큰 값은 마지막 구간에 속하도록 설정합니다.
5. 현재 열의 값을 분위수에 따라 구간으로 나눕니다.
6. 각 구간에 속하는 `target` 값의 평균을 계산합니다.
7. `df\_prob4\_train` 데이터프레임에 새로운 열을 생성하고, 각 값이 속하는 구간의 `target` 평균값으로 대체합니다.
8. `df\_prob4\_test` 데이터프레임에 대해 동일한 작업을 수행하여 새로운 열을 생성합니다.

이 과정을 통해 각 열에 대해 분위구간을 기반으로 한 새로운 특성을 생성할 수 있습니다.
이는 모델의 성능을 향상시키기 위해 유용한 전처리 방법입니다.

```
q = np.arange(0, 101) / 100
for i in cont_cols:
    qt = df_prob4_train[i].quantile(q)
    qt.iloc[[0, -1]] = (-np.inf, np.inf)
    q_range = pd.cut(df_prob4_train[i], bins=qt, right=True)
    q_mean = df_prob4_train.groupby(q_range)['target'].agg('mean')

    df_prob4_train[i + '_q'] = q_range.map(q_mean).astype('float')
    df_prob4_test[i + '_q'] = pd.cut(df_prob4_test[i], bins=qt, right=True).map(q_mean).astype('float')

print('df_prob4_train.columns = ', df_prob4_train.columns)
print('df_prob4_test.columns = ', df_prob4_test.columns)
```

```

df_prob4_train.columns = Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7',
                                'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target',
                                'targetA', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B',
                                'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B',
                                'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D',
                                'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G',
                                'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J',
                                'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O', 'cont0_q', 'cont1_q',
                                'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q',
                                'cont8_q', 'cont9_q', 'cont10_q', 'cont11_q', 'cont12_q', 'cont13_q'],
                                dtype='object')
df_prob4_test.columns = Index(['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7',
                                'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'target',
                                'targetA', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B',
                                'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B',
                                'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D',
                                'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G',
                                'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J',
                                'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O', 'cont0_q', 'cont1_q',
                                'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q',
                                'cont8_q', 'cont9_q', 'cont10_q', 'cont11_q', 'cont12_q', 'cont13_q'],
                                dtype='object')

```

In [168...

```

'''
`pandas`의 `between()` 함수는 데이터프레임이나 시리즈의 값들이 특정 범위 내에 속하는지 여부를 판별하는 데 사용됩니다.
이 함수는 범위를 설정하고, 해당 범위 내에 포함되는 값들을 불리언 값(`True` 또는 `False`)으로 반환합니다.

### `between()` 함수의 주요 요소들

#### 1. 인자 (Parameters)
`between()` 함수는 다음과 같은 인자들을 가집니다:
- `left`:
    - 범위의 하한값 (inclusive).
    - 이 값 이상인 요소들이 `True`로 표시됩니다.
    - 기본값은 필수 입력값입니다.
- `right`:
    - 범위의 상한값 (inclusive).
    - 이 값 이하인 요소들이 `True`로 표시됩니다.
    - 기본값은 필수 입력값입니다.
- `inclusive`:
    - 경계값 포함 여부를 설정합니다.
    - 기본값은 `'both'`로, 하한값(`left`)과 상한값(`right`) 모두 포함합니다.

```

- 설정 가능한 값:
 - ``both``: 하한값과 상한값 모두 포함 (기본값)
 - ``neither``: 하한값과 상한값 모두 제외
 - ``left``: 하한값만 포함하고 상한값은 제외
 - ``right``: 상한값만 포함하고 하한값은 제외

2. 리턴값 (Return Value)

- ``Series`` (또는 ``DataFrame``, 원본이 ``DataFrame``인 경우):
 - ``left``와 ``right`` 사이에 있는 값에 대해 ``True``, 그렇지 않은 값에 대해 ``False`` 값을 가지는 ``Series`` 객체를 반환합니다.
 - 원본의 인덱스를 유지하며, 각 요소는 해당 범위 내에 속하는지 여부를 나타냅니다.

3. 속성 (Attributes)

``between()`` 함수 자체는 속성을 가지지 않습니다. 그러나 이 함수가 적용되는 대상(``Series`` 또는 ``DataFrame``)에는 속성이 존재할 수 있습니다.

4. 함수 (Methods)

``between()`` 함수는 다른 ``pandas`` 함수와 결합하여 사용될 수 있으며, 이를 통해 다양한 데이터 조작이 가능합니다. 예를 들어, ``DataFrame.loc[]``와 결합하여 특정 조건에 맞는 행을 선택할 수 있습니다.

예제

```
import pandas as pd
```

```
# 데이터 생성
```

```
data = {'A': [1, 2, 3, 4, 5],
        'B': [10, 20, 30, 40, 50]}
```

```
df = pd.DataFrame(data)
```

```
# 'A' 열의 값이 2 이상 4 이하인 행 선택
```

```
mask = df['A'].between(left=2, right=4)
```

```
filtered_df = df[mask]
```

```
print(filtered_df)
```

위 코드에서는 ``df['A']``의 값들이 2 이상 4 이하인 행들만 ``filtered\_df``에 저장됩니다.

inclusive 사용 예제:

```
mask = df['A'].between(left=2, right=4, inclusive='neither')
```

위 예제에서는 2보다 크고 4보다 작은 값만 ``True``로 판별됩니다.

요약

- `between()` 함수는 `Series` 또는 `DataFrame`의 값들이 특정 범위 내에 있는지 확인할 때 사용됩니다.
 - `left`와 `right` 인자를 통해 범위를 설정하고, `inclusive` 인자를 통해 경계값 포함 여부를 지정할 수 있습니다.
 - `Series` 객체를 반환하며, 각 요소에 대해 `True` 또는 `False` 값을 가집니다.

```
'''
```

```
import pandas as pd
```

```
# 데이터 생성
```

```
data = {'A': [1, 2, 3, 4, 5],
        'B': [10, 20, 30, 40, 50]}
```

```
df = pd.DataFrame(data)
```

```
# 'A' 열의 값이 2 이상 4 이하인 행 선택
```

```
mask = df['A'].between(left=2, right=4)
```

```
filtered_df = df[mask]
```

```
print(filtered_df)
```

```
print('-----')
```

```
mask2 = df['A'].between(left=2, right=4, inclusive='neither')
```

```
filtered_df2 = df[mask2]
```

```
# 만약 여러분의 코드가 inclusive='neither'로 설정되어 있는데, 그럼에도 불구하고 양 끝 값이 포함된다면,
```

```
# 사용 중인 pandas 버전이 1.3.0보다 낮을 가능성이 있습니다.
```

```
print(filtered_df2)
```

```
   A  B
1  2 20
2  3 30
3  4 40
```

```
-----
```

```
   A  B
1  2 20
2  3 30
3  4 40
```

In [169...

```
'''
```

아래 코드는 여러 열(`cont\_cols` 리스트에 저장된 열 이름들)을 포함한 데이터프레임에 대해 분위수를 계산하고, 테스트 데이터의 값이 훈련 데이터의 분위구간에 포함되지 않는 경우를 확인하는 과정을 다룹니다.

```

# 0과 1 사이의 101개의 균일한 간격을 생성합니다.
q = np.arange(0, 101) / 100
이 라인은 0과 1 사이를 101개의 균일한 간격으로 나눈 배열을 생성합니다. `q`는 0.00, 0.01, 0.02, ..., 1.00의 값을 가지는 배열이 됩니다.

# cont_cols에 포함된 각 열에 대해 반복 작업을 수행합니다.
for i in cont_cols:
이 라인은 `cont_cols` 리스트에 포함된 각 열 이름에 대해 반복 작업을 시작합니다.
`cont_cols`는 `df_prob4_train` 데이터프레임의 열 이름들을 포함한 리스트입니다.

    # 각 열에 대한 분위수를 계산합니다.
    qt = df_prob4_train[i].quantile(q)
    이 라인은 현재 열 `i`에 대해 위에서 생성한 분위점 `q`를 사용하여 분위수를 계산합니다.
    `qt`는 현재 열 `i`의 분위수 값을 포함하는 시리즈가 됩니다.

    # 현재 열에 대해 분위구간을 생성합니다.
    q_range = pd.cut(df_prob4_train[i], bins=qt, right=True)
    이 라인은 현재 열 `i`의 값을 `qt`에 따라 구간으로 나눕니다.
    `pd.cut` 함수는 현재 열 `i`의 값을 `qt`의 분위구간에 맞춰 구간을 나누고, 그 결과를 `q_range`에 저장합니다.
    `right=True`는 각 구간의 오른쪽 경계를 포함한다는 의미입니다.

    # 테스트 데이터에서 분위구간에 포함되지 않는 값들을 확인합니다.
    q_range_t = pd.cut(df_prob4_test[i], bins=qt, right=True).isna()
    이 라인은 `df_prob4_test` 데이터프레임의 현재 열 `i` 값을 `qt`에 따라 구간으로 나누고,
    각 값이 구간에 포함되지 않는지를 확인합니다.
    `pd.cut(df_prob4_test[i], bins=qt, right=True)`는 `df_prob4_test`의 현재 열 값을 `qt`의 분위구간에 맞춰 구간을 나누고,
    `isna()`는 해당 값이 구간에 포함되지 않으면 `True`, 포함되면 `False`를 반환합니다. 결과는 `q_range_t`에 저장됩니다.

    # 테스트 데이터의 값이 훈련 데이터의 최소값과 최대값 사이에 있는지 확인합니다.
    notin_range_t = ~df_prob4_test[i].between(*df_prob4_train[i].agg(['min', 'max']))
    이 라인은 `df_prob4_test`의 현재 열 `i` 값이 `df_prob4_train`의 현재 열 `i` 값의 최소값과 최대값 사이에 있는지 확인합니다.
    `between(*df_prob4_train[i].agg(['min', 'max']))`는 값이 최소값과 최대값 사이에 있으면 `True`,
    그렇지 않으면 `False`를 반환합니다. `~` 연산자는 `True`와 `False`를 반대로 바꿉니다. 결과는 `notin_range_t`에 저장됩니다.

    # 열 이름, 분위구간에 포함되지 않는 값의 개수, 최소/최대값 사이에 포함되지 않는 값의 개수,
    # 두 기준이 다른 값의 개수를 출력합니다.
    print(i, q_range_t.sum(), notin_range_t.sum(), (q_range_t != notin_range_t).sum())
    이 라인은 현재 열 이름, `q_range_t`에서 `True`인 값의 개수 (즉, 분위구간에 포함되지 않는 값의 개수),
    `notin_range_t`에서 `True`인 값의 개수 (즉, 최소값과 최대값 사이에 포함되지 않는 값의 개수),
    `q_range_t`와 `notin_range_t`가 다른 값의 개수를 출력합니다.

### 전체 과정 요약
1. 0과 1 사이의 101개의 균일한 점을 생성합니다.

```

2. `cont_cols` 리스트에 포함된 각 열에 대해 반복 작업을 수행합니다.
3. 현재 열의 분위수를 계산합니다.
4. 현재 열의 값을 분위수에 따라 구간으로 나눕니다.
5. 테스트 데이터에서 각 값이 구간에 포함되지 않는지를 확인합니다.
6. 테스트 데이터의 값이 훈련 데이터의 최소값과 최대값 사이에 있는지를 확인합니다.
7. 현재 열 이름, 분위구간에 포함되지 않는 값의 개수, 최소/최대값 사이에 포함되지 않는 값의 개수, 두 기준이 다른 값의 개수를 출력합니다.

이 과정을 통해 테스트 데이터의 값이 훈련 데이터의 분위구간이나 최소/최대값 범위를 벗어나는 경우를 확인할 수 있습니다. 이를 통해 데이터의 분포를 이해하고, 모델링 전에 데이터의 이상치를 파악할 수 있습니다.

```
...
```

Q: 범위를 확장하지 않을 때, Test에서 nan 값은 train의 최소값 또는 최대값의 큰값이 맞나요?

A: 보여드립니다.

```
q = np.arange(0, 101) / 100
```

```
for i in cont_cols:
    qt = df_prob4_train[i].quantile(q)
    q_range = pd.cut(df_prob4_train[i], bins=qt, right=True)
    q_range_t = pd.cut(df_prob4_test[i], bins=qt, right=True).isna()
    notin_range_t = ~df_prob4_test[i].between(*df_prob4_train[i].agg(['min', 'max']))
    print(i, q_range_t.sum(), notin_range_t.sum(), (q_range_t != notin_range_t).sum())
    print('-----')
```

```
cont0 0 0 0
```

```
-----
cont1 0 0 0
```

```
-----
cont2 0 0 0
```

```
-----
cont3 0 0 0
```

```
-----
cont4 0 0 0
```

```
-----
cont5 1 1 0
```

```
-----
cont6 1 1 0
```

```
-----
cont7 0 0 0
```

```
-----
cont8 1 1 0
```

```
-----
cont9 0 0 0
```

```
-----
cont10 1 1 0
```

```
-----
cont11 0 0 0
```

```
-----
cont12 2 2 0
```

```
-----
cont13 0 0 0
```

In [170...

```
# 위 2개 코드 비교
```

```
...
```

두 코드 모두 `df\_prob4\_train`과 `df\_prob4\_test` 데이터프레임에서 연속형 변수(`cont\_cols`)의 분위수를 구간으로 나누고, 각 구간의 `target` 값의 평균을 구하여 새로운 변수를 생성하는 작업을 수행합니다. 그러나 두 코드의 목적과 동작 방식에는 차이가 있습니다.

```
### 첫 번째 코드
```

코드는 각 연속형 변수에 대해 100개의 분위 구간을 만들고, 해당 구간의 `target` 값의 평균을 새로운 변수로 추가합니다.

```
### 동작 설명
```

1. `np.arange(0, 101) / 100`을 사용하여 0부터 1까지 101개의 균등한 값을 생성합니다.

2. 각 연속형 변수 `i`에 대해:

- `qt`에 해당 변수의 분위수를 저장합니다.

- 첫 번째와 마지막 분위수를 ``-np.inf``와 ``np.inf``로 설정하여 모든 값을 포함하도록 합니다.
- ``pd.cut``을 사용하여 `df_prob4_train[i]`의 값을 100개의 분위 구간으로 나눕니다.
- 구간별로 ``target`` 값의 평균을 계산합니다.
- ``df_prob4_train``과 ``df_prob4_test`` 데이터프레임에 새로운 변수로 구간별 평균 값을 추가합니다.

두 번째 코드

코드는 ``df_prob4_test`` 데이터의 값들이 ``df_prob4_train`` 데이터의 분위 구간 밖에 있는지를 검사하는 용도로 사용됩니다.

동작 설명

1. 첫 번째 코드와 동일하게 ``np.arange(0, 101) / 100``을 사용하여 0부터 1까지 101개의 균등한 값을 생성합니다.
2. 각 연속형 변수 ``i``에 대해:
 - ``qt``에 해당 변수의 분위수를 저장합니다.
 - ``pd.cut``을 사용하여 ``df_prob4_train`` 데이터프레임의 변수를 분위 구간으로 나눕니다.
 - ``df_prob4_test`` 데이터에서 분위 구간에 속하지 않는 값들을 ``isna()``를 통해 확인합니다.
 - ``df_prob4_test`` 데이터에서 ``df_prob4_train`` 데이터의 최소값과 최대값 범위를 벗어난 값들을 ``between`` 메서드를 통해 확인합니다.
 - 변수 ``i``에 대해 ``df_prob4_test`` 데이터에서 분위 구간 밖의 값과 최소-최대 범위를 벗어난 값의 수를 출력합니다.

두 코드의 차이점 비교

1. 목적:
 - 첫 번째 코드는 ``target`` 값의 구간별 평균을 계산하여 새로운 변수를 생성합니다. 이 변수를 모델에 사용할 수 있도록 준비하는 과정입니다.
 - 두 번째 코드는 ``df_prob4_test`` 데이터의 값들이 ``df_prob4_train`` 데이터의 분위 구간 밖에 있는지를 확인하고, 이를 통해 데이터의 범위가 적절한지 검사하는 용도입니다.
2. 동작 방식:
 - 첫 번째 코드는 각 변수의 분위 구간을 생성하고, 구간별 ``target`` 값의 평균을 새로운 변수로 추가합니다.
 - 두 번째 코드는 각 변수의 분위 구간을 생성하고, ``df_prob4_test`` 데이터에서 해당 분위 구간 밖에 있는 값을 검사하여 출력합니다.
3. 출력 결과:
 - 첫 번째 코드는 새로운 변수를 ``df_prob4_train``과 ``df_prob4_test`` 데이터프레임에 추가합니다.
 - 두 번째 코드는 각 변수에 대해 ``df_prob4_test`` 데이터에서 분위 구간 밖의 값과 최소-최대 범위를 벗어난 값의 수를 출력합니다.

결론적으로, 첫 번째 코드는 모델링을 위한 데이터 전처리 작업을 수행하며, 두 번째 코드는 테스트 데이터의 분포가 학습 데이터와 일치하는지 확인하는 작업을 수행합니다.

```
...
print('')
```

```
In [171]: df_prob4_train.head()
```

	cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	...	cont4_q	cont5_q	cont6_q	cont7_q	cont8_q	cc
id																	
487693	0.49355	0.69104	0.66940	0.46424	0.78825	0.34951	0.30596	0.69445	0.19411	0.35497	...	7.444439	7.507718	7.418108	7.424195	7.466343	7.3
307175	0.66260	0.55180	0.31117	0.83165	0.22354	0.34808	0.65781	0.30914	0.48613	0.59179	...	7.464061	7.507718	7.447277	7.431217	7.424309	7.3
493409	0.87161	0.69037	0.33452	0.69755	0.28069	0.93415	0.63591	0.87033	0.93617	0.79620	...	7.462176	7.471819	7.454580	7.537384	7.684391	7.3
414858	0.51951	0.03523	0.32366	0.38940	0.41430	0.73061	0.37822	0.62359	0.34185	0.38249	...	7.371190	7.354649	7.498205	7.464006	7.428804	7.4
408161	0.86293	0.61935	0.70905	0.82617	0.41898	0.43902	0.31892	0.58007	0.37662	0.37795	...	7.406167	7.465528	7.446922	7.391109	7.546018	7.4

5 rows × 69 columns

```
In [172]: df_prob4_test.head()
```

	cont0	cont1	cont2	cont3	cont4	cont5	cont6	cont7	cont8	cont9	...	cont4_q	cont5_q	cont6_q	cont7_q	cont8_q	cc
id																	
309145	0.93484	0.29862	0.12013	0.71414	0.27825	1.01559	0.54047	0.80249	0.90302	0.80604	...	7.464230	7.412962	7.417744	7.516305	7.529983	7.4
368647	0.32245	0.35401	0.41031	0.75546	0.34427	0.25169	0.30402	0.33034	0.20197	0.37512	...	7.471364	7.451470	7.418108	7.397414	7.466594	7.4
487072	0.47681	0.74517	0.71498	0.20361	0.49142	0.43309	0.68805	0.23303	0.38456	0.38120	...	7.492398	7.485158	7.363892	7.469959	7.572538	7.4
490811	0.46057	0.35253	0.68699	0.43218	0.28081	0.37182	0.38092	0.58439	0.45705	0.47263	...	7.463842	7.503739	7.483065	7.391109	7.388975	7.3
484687	0.95435	0.48605	0.33035	0.79471	0.48517	0.23492	0.79788	0.65686	0.63854	0.24914	...	7.473854	7.473077	7.523299	7.494512	7.365279	7.5

5 rows × 69 columns

단계 4-5

prob4_train에서 target을 대상으로 선형회귀모델을 학습하고,

prob4_test를 다음 지표로 성능 평가한 값을 D라고 한다.

선형회귀 모델은 아래와 같은 손실함수로 학습을 한다.

$$\|Xw - y\|_2^2 + \alpha \|w\|_2^2, \alpha = 0.1$$

Hint] sklearn.linear_model.Ridge 또는 Lasso 중 요건에 맞는 것을 사용한다.

입력 변수: cont0_q ~ cont13_q, cat0 ~ 9까지 원핫-인코딩 후 나온 파생 변수

(변수명 다음 범주명으로 알파벳 순서이다)

대상 변수: target

$$\text{성능 지표: } MSE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

In [173...

```
'''
`Ridge`는 `scikit-learn` 라이브러리에서 제공하는 선형 회귀 모델로, L2 정규화를 적용하여 모델의 복잡도를 제어합니다.
정규화는 과적합을 방지하고, 특성이 많은 데이터셋에서 모델의 일반화 성능을 향상시킵니다.

### 1. 인자 (Parameters)
- `alpha`:
  - 정규화 강도를 조절하는 매개변수로, 기본값은 `1.0`입니다.
  - 값이 클수록 더 강한 정규화가 적용되며, 회귀 계수의 크기가 작아집니다.
  - `alpha=0`이면 일반 선형 회귀와 동일하게 동작합니다.
- `fit_intercept`:
  - 절편을 학습할지 여부를 결정하는 플래그입니다. 기본값은 `True`입니다.
  - `True`이면 절편을 학습하고, `False`이면 절편을 사용하지 않습니다.
- `normalize` (Deprecated since version 1.0):
  - 이 인자는 `fit_intercept`가 `True`일 때, 특성을 정규화할지 여부를 결정합니다.
  - 기본값은 `False`입니다.
  - 현재는 사용되지 않으며, `preprocessing.StandardScaler`를 사용하는 것이 권장됩니다.
- `copy_X`:
  - `True`이면 원본 데이터 `X`의 복사본을 사용하고, `False`이면 원본 데이터를 덮어쓰지 않습니다.
  - 기본값은 `True`입니다.
- `max_iter`:
  - 알고리즘이 수렴할 때까지의 최대 반복 횟수입니다.
  - 기본값은 `None`이며, 기본 반복 횟수는 `scikit-learn`의 내부 설정에 따릅니다.
- `tol`:
```

- 반복 과정의 종료 기준이 되는 허용 오차입니다. 기본값은 ``1e-3``입니다.
- ``solver``:
 - 최적화에 사용할 알고리즘을 선택합니다.
 - 옵션: ``'auto'``, ``'svd'``, ``'cholesky'``, ``'lsqr'``, ``'sparse_cg'``, ``'sag'``, ``'saga'``
 - 기본값은 ``'auto'``이며, 데이터셋에 따라 적절한 솔버를 자동으로 선택합니다.
- ``random_state``:
 - 재현성을 위해 난수 발생기의 시드를 설정합니다. ``solver``가 ``'sag'`` 또는 ``'saga'``일 때만 사용됩니다.
- ``positive``:
 - ``True``로 설정하면 회귀 계수가 양수로 제한됩니다.
 - 기본값은 ``False``입니다.

2. 리턴값 (Return Value)

``Ridge`` 클래스는 자체적으로 리턴값을 가지지 않지만, 모델 객체를 생성하여 리턴합니다.
이 모델 객체는 학습 후 회귀 계수, 예측값 등을 제공합니다.

3. 속성 (Attributes)

- ``coef_``:
 - 학습된 회귀 계수 벡터입니다.
 - 모델이 학습된 후 사용할 수 있습니다.
- ``intercept_``:
 - 학습된 절편입니다. ``fit_intercept=True``일 때만 사용됩니다.
- ``n_iter_``:
 - ``solver``가 반복 알고리즘(``'sag'``, ``'saga'``, ``'lsqr'``, ``'sparse_cg'``)일 때, 반복 횟수를 나타냅니다.

4. 함수 (Methods)

``Ridge`` 클래스는 다음과 같은 주요 메서드를 제공합니다:

- ``fit(X, y)``:
 - 모델을 학습시킵니다.
 - ``X``는 특성 행렬, ``y``는 타겟 벡터입니다.
 - 모델이 학습된 후 ``coef_``와 ``intercept_``가 결정됩니다.
 - ``predict(X)``:
 - 학습된 모델을 사용하여 새로운 데이터 ``X``에 대한 예측을 수행합니다.
 - ``X``는 특성 행렬이며, 예측된 타겟 벡터를 반환합니다.
 - ``score(X, y)``:
 - 모델의 성능을 평가합니다. R^2 점수를 계산하여 반환합니다.
 - 1에 가까울수록 모델이 타겟 값을 잘 설명합니다.
 - ``get_params(deep=True)``:
 - 모델의 파라미터를 딕셔너리로 반환합니다.
 - ``deep=True``이면 하위 객체의 파라미터도 포함합니다.
 - ``set_params(params)``:
 - 모델의 파라미터를 설정합니다. ``params``는 키-값 쌍으로 전달됩니다.
- ...

```

#### 예제
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# 데이터 생성
X, y = make_regression(n_samples=100, n_features=2, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Ridge 회귀 모델 생성 및 학습
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train, y_train)

# 예측 및 평가
predictions = ridge_model.predict(X_test)
score = ridge_model.score(X_test, y_test)

print(f"Ridge 회귀 계수: {ridge_model.coef_}")
print(f"R² 점수: {score}")

```

Ridge 회귀 계수: [86.36361962 73.19145993]
R² 점수: 0.9998060779667833

In [174...

```

'''
코드는 Ridge 회귀 모델을 사용하여 학습 및 예측을 수행하고, 모델의 성능을 평가합니다.

#### 코드 설명

1. Ridge 회귀 모델 생성
- `Ridge` 클래스는 L2 정규화를 사용한 회귀 모델을 생성합니다.
- `alpha=0.1`은 정규화 강도를 조절하는 하이퍼파라미터입니다. 값이 클수록 더 강한 정규화를 적용합니다.

2. 특징 변수 선택
- `X_rd`는 모델에 사용할 특징 변수 목록을 정의합니다.
- `[i + '_q' for i in cont_cols]`는 모든 연속형 변수(`cont_cols`)에 대한 분위 구간별 평균 값을 포함합니다.
- `dum_cols`는 범주형 변수의 원-핫 인코딩된 열 이름을 포함합니다.

3. 모델 학습
- `df_prob4_train[X_rd]` 데이터프레임의 `X_rd` 변수들을 사용하여 `target` 변수를 예측하도록 모델을 학습시킵니다.

4. 테스트 데이터에 대한 예측 및 평가

```

- `reg_rd.predict(df_prob4_test[X_rd])`는 테스트 데이터 `df_prob4_test[X_rd]`에 대해 학습된 Ridge 모델을 사용하여 예측 값을 생성합니다.
- `mean_squared_error(df_prob4_test['target'], ...)`는 테스트 데이터의 실제 `target` 값과 예측 값 간의 평균 제곱 오차(MSE)를 계산합니다.
- `D`는 테스트 데이터에 대한 MSE 값을 저장합니다.

5. 모델 성능 평가

- `D`는 테스트 데이터의 MSE 값입니다.
- `mean_squared_error(df_prob4_train['target'], reg_rd.predict(df_prob4_train[X_rd]))`는 학습 데이터의 실제 `target` 값과 예측 값 간의 MSE를 계산합니다.
- 두 값을 튜플 형태로 반환하여 학습 데이터와 테스트 데이터에 대한 MSE를 비교합니다.

코드가 동작하는 내용 요약

- Ridge 회귀 모델을 생성하고 정규화 강도를 설정합니다 (`alpha=0.1`).
- 모델에 사용할 특징 변수를 선택합니다 (`X_rd`).
- 학습 데이터를 사용하여 모델을 학습시킵니다.
- 테스트 데이터에 대한 예측을 수행하고, 예측 값과 실제 값 간의 평균 제곱 오차를 계산하여 모델의 성능을 평가합니다.
- 학습 데이터와 테스트 데이터에 대한 MSE를 비교하여 모델의 성능을 확인합니다.

```
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
```

```
reg_rd = Ridge(alpha=0.1)
X_rd = [i + '_q' for i in cont_cols] + dum_cols
print('X_rd = ', X_rd)
print('-----')
```

```
reg_rd.fit(df_prob4_train[X_rd], df_prob4_train['target'])
```

```
D = mean_squared_error(df_prob4_test['target'], reg_rd.predict(df_prob4_test[X_rd]))
(
    D,
    mean_squared_error(df_prob4_train['target'], reg_rd.predict(df_prob4_train[X_rd]))
)
```

```
X_rd = ['cont0_q', 'cont1_q', 'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q', 'cont8_q', 'cont9_q', 'cont10_q', 'cont11_q', 'cont12_q', 'cont13_q', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

Out[174]: (0.7211658399136691, 0.7133971363485151)

In [175... D, round(D, 2)

Out[175]: (0.7211658399136691, 0.72)

D를 소수점 셋째 자리에서 반올림하여 둘째 자리까지 출력하라.

0.72

In [176...

```
'''
`cross_val_score()` 함수는 Scikit-learn 라이브러리에서 제공하는 함수로,
교차 검증(cross-validation)을 통해 모델의 성능을 평가할 때 사용됩니다.
이 함수는 주어진 데이터셋을 여러 개의 폴드(folds)로 나누고, 각 폴드에 대해 모델을 훈련 및 평가하여 성능을 반환합니다.

### `cross_val_score()` 함수의 주요 인자
1. `estimator`:
    - 학습에 사용될 모델 객체입니다. `estimator`는 `fit` 메서드를 가지고 있어야 하며,
      예측을 위해 `predict` 또는 `predict_proba` 메서드를 지원해야 합니다.
    - 예: `estimator=LinearRegression()`, `estimator=RandomForestClassifier()`
2. `X`:
    - 모델을 훈련시키기 위한 피처(features) 데이터입니다. 일반적으로 2차원 배열 또는 판다스 데이터프레임으로 제공됩니다.
    - 예: `X = df[['feature1', 'feature2']]`
3. `y`:
    - 모델의 타겟 값 또는 레이블(label) 데이터입니다. 1차원 배열이나 판다스 시리즈로 제공됩니다.
    - 예: `y = df['target']`
4. `scoring` (선택 사항):
    - 모델의 성능을 평가할 지표를 정의합니다. 문자열(예: `'accuracy'`, `'f1'`, `'roc_auc'`) 또는 함수로 지정할 수 있습니다.
    - 지정하지 않으면 기본적으로 `estimator`의 `score` 메서드가 사용됩니다.
    - 예: `scoring='accuracy'`, `scoring='neg_mean_squared_error'`
5. `cv` (선택 사항):
    - 교차 검증에서 사용할 폴드의 개수 또는 교차 검증 분할 전략입니다. 기본값은 `5`입니다.
    - `cv`는 정수, `KFold` 객체, `StratifiedKFold` 객체, 또는 사용자 정의 분할 전략일 수 있습니다.
    - 예: `cv=5`, `cv=StratifiedKFold(n_splits=10)`
6. `n_jobs` (선택 사항):
    - 병렬 처리를 위해 사용할 CPU 코어 수를 지정합니다. `-1`로 설정하면 모든 가용 코어를 사용합니다.
    - 예: `n_jobs=-1`
7. `verbose` (선택 사항):
    - 실행 중 출력되는 정보의 양을 조절합니다. 큰 값일수록 더 많은 정보가 출력됩니다.
    - 예: `verbose=1`
'''
```

```

8. `fit_params` (선택 사항):
    - 모델 학습 시 추가적으로 전달할 매개변수를 사전 형태로 제공합니다.
    - 예: `fit_params={'early_stopping_rounds': 10}`
9. `pre_dispatch` (선택 사항):
    - 작업을 분할하여 미리 할당할 수 있는 수를 조절하는 매개변수입니다. 기본값은 `'2*n_jobs'`입니다.

### `cross_val_score()` 함수의 리턴값
- 리턴값:
    - 모델의 각 폴드에서 계산된 점수의 리스트가 담긴 1차원 NumPy 배열을 반환합니다.
    - 예: `array([0.75, 0.82, 0.78, 0.80, 0.77])`

### 주요 속성 및 함수
`cross_val_score()`는 리턴값을 통해 주어진 모델의 성능을 평가하는 데 사용되며, 다음과 같은 속성 및 함수와 함께 사용될 수 있습니다:
1. `mean()` :
    - 교차 검증 결과로 반환된 점수 배열의 평균을 계산합니다.
    - 예: `scores.mean()`
2. `std()` :
    - 점수 배열의 표준 편차를 계산합니다. 이는 모델의 성능 변동성을 확인하는 데 유용합니다.
    - 예: `scores.std()`
3. `fit()` :
    - 모델을 주어진 데이터에 대해 학습시킵니다. `cross_val_score`는 내부적으로 각 폴드마다 `fit()`을 호출합니다.
4. `predict()` :
    - 학습된 모델로 새로운 데이터를 예측합니다. `cross_val_score`는 각 폴드의 테스트 셋에 대해 `predict()`를 호출하여 성능을 평가합니다.

### 요약
`cross_val_score()` 함수는 모델 성능을 평가하는 데 매우 유용하며, 다양한 설정을 통해 여러 교차 검증 전략을 쉽게 적용할 수 있습니다. 이 함수는 다양한 파라미터와 함께 사용될 수 있으며, 리턴값으로 모델 성능의 일관성을 평가할 수 있습니다.
...

from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

# 데이터셋 로드
data = load_iris()
X = data.data
y = data.target

# 모델 정의
model = RandomForestClassifier(random_state=42)

# 교차 검증 실행

```

```
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')

print("Cross-validation scores:", scores)
print("Mean accuracy:", scores.mean())
```

```
Cross-validation scores: [0.96666667 0.96666667 0.93333333 0.96666667 1.          ]
Mean accuracy: 0.9666666666666668
```

In [177...

코드는 K-Fold 교차 검증을 사용하여 Ridge 회귀 모델의 성능을 평가합니다. 그러나 코드에는 데이터 누출(leakage)이 발생할 수 있습니다.

코드 설명

1. K-Fold 교차 검증 설정

- `KFold` 클래스는 데이터를 `n\_splits`로 나누어 교차 검증을 수행합니다.
- `random\_state=123`은 재현성을 위해 난수 생성기의 시드 값을 설정합니다.
- `shuffle=True`는 데이터를 섞은 후에 분할합니다.

2. 특징 변수 선택

- `X\_rd`는 모델에 사용할 특징 변수 목록을 정의합니다.
- `[i + '\_q' for i in cont\_cols]`는 모든 연속형 변수(`cont\_cols`)에 대한 분위 구간별 평균 값을 포함합니다.
- `dum\_cols`는 범주형 변수의 원-핫 인코딩된 열 이름을 포함합니다.

3. 교차 검증을 통한 성능 평가

- `cross\_val\_score` 함수는 `cv`에서 지정한 교차 검증 전략을 사용하여 모델의 성능을 평가합니다.
- `scoring='neg\_mean\_squared\_error'`는 성능 평가 기준으로 음수의 평균 제곱 오차를 사용합니다.
- `reg\_rd` 모델을 `df\_prob4\_train[X\_rd]` 데이터와 `df\_prob4\_train['target']`을 사용하여 5번 학습 및 평가합니다.
- 결과는 각 분할의 음수 MSE 값으로 반환됩니다.

4. 평균 음수 MSE 계산 및 부호 변경

- `score\_`는 각 분할의 음수 MSE 값을 담고 있습니다.
- `np.mean(score\_)`는 이 값들의 평균을 계산합니다.
- 최종적으로 부호를 바꾸어 양수의 MSE 값을 반환합니다.

...

target의 값을 가지고 파생변수를 만든다면, Leak을 유의하셔야 합니다.
 # 만일 df_prob4_train을 가지고 교차 검증을 통해 모델을 성능을 측정하고자 한다면,
 # df_prob4_train에 있는 cont{}_q 들은 df_prob4_train['target']을
 # 가지고 파생변수를 만들었는데요, 교차 검증을 한다면,
 # 이는 겹의 셋의 target 값이 사전에 cont{}_q에 반영이 된 겁니다.
 # 이점이 바로 Leak으로 작용하는 겁니다.

```
# 이 Leak으로 교차 검증 결과에 왜곡이 생깁니다.
# 검증은 실제 테스트를 잘 반영해야 하는데요,
# 측정 결과는 위 df_prob4_test를 반영한다고 보기 힘듭니다.
```

```
from sklearn.model_selection import cross_val_score, KFold
```

```
cv = KFold(n_splits=5, random_state=123, shuffle=True)
```

```
X_rd = [i + '_q' for i in cont_cols] + dum_cols
```

```
print('X_rd = ', X_rd)
```

```
print('-----')
```

```
score_ = cross_val_score(
    reg_rd,
    df_prob4_train[X_rd],
    df_prob4_train['target'],
    cv=cv, scoring='neg_mean_squared_error'
)
```

```
print('score_ = ', score_)
```

```
-np.mean(score_)
```

```
X_rd = ['cont0_q', 'cont1_q', 'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q', 'cont8_q', 'cont9_q', 'cont10_q', 'cont11_q', 'cont12_q', 'cont13_q', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

```
-----
score_ = [-0.70446689 -0.71580155 -0.71701696 -0.71912073 -0.71239611]
```

Out[177]: 0.7137604473647998

In [178...

```
'''
```

```
### 데이터 누출(Leakage)이란?
```

데이터 누출(Leakage)은 모델 학습 과정에서 테스트 데이터의 정보가 훈련 데이터에 포함되어,
모델의 성능이 실제보다 과대평가되는 현상을 말합니다.

데이터 누출은 모델이 실제 환경에서 성능을 평가할 때, 과도하게 좋은 성능을 보이게 하고, 실전에서는 성능이 크게 떨어지게 만듭니다.

```
### 코드에서 데이터 누출이 발생하는 이유
```

주어진 코드에서 데이터 누출이 발생하는 이유는 `df\_prob4\_train` 데이터프레임의 피처(`cont{}\_q`)들이
타겟 변수(`target`)를 사용해 생성되었기 때문입니다.

이 피처들은 이미 타겟 변수의 정보를 반영하고 있기 때문에,

모델이 학습 과정에서 이 피처를 사용할 경우, 타겟 변수에 대한 사전 정보를 얻게 됩니다.

이로 인해 모델은 실제로 일반화되지 않은, 단순히 학습 데이터에서 잘 작동하는 성능을 보이게 됩니다.

데이터 누출이 발생하는 부분

1. 파생변수 생성:

- 코드에서는 ``cont{}`_q`` 피처들이 ``df_prob4_train['target']``을 사용하여 만들어졌습니다.
이는 모델이 학습할 때 타겟 값에 대한 정보를 이미 포함한 피처를 사용한다는 의미입니다.
- 이러한 피처는 테스트 데이터에서 모델을 평가할 때 사용할 수 없으며, 실제 성능을 과대평가하게 만듭니다.

2. 교차 검증 과정에서의 문제:

- 교차 검증(``cross_val_score``)을 수행할 때, 데이터셋을 여러 폴드로 나누고, 각 폴드마다 훈련과 테스트를 반복합니다.
그러나 피처(``cont{}`_q`` 피처들)가 이미 타겟 값을 반영하고 있기 때문에, 훈련 폴드에서 만들어진 피처들은 테스트 폴드의 타겟 정보를 이미 가지고 있습니다.
- 이는 마치 모델이 정답을 알고 있는 상태에서 문제를 푸는 것과 같습니다.
교차 검증 결과가 왜곡되어 실제 성능을 정확히 평가할 수 없게 됩니다.

데이터 누출을 방지하기 위한 방법

1. 타겟 변수를 사용한 피처 생성 시 주의:

- 타겟 변수를 사용해 파생변수를 만들 때는, 반드시 훈련 데이터와 테스트 데이터를 완전히 분리한 후, 훈련 데이터만을 사용해 파생변수를 생성해야 합니다.
- 교차 검증을 사용할 경우, 각 폴드 내에서만 파생변수를 생성하고, 해당 폴드의 훈련 데이터만을 사용해 파생변수를 만들어야 합니다.

2. 피처 엔지니어링 시 시점 고려:

- 피처를 생성할 때, 모델이 실전에서 어떤 데이터를 사용할 수 있을지 고려해야 합니다.
특히, 미래 데이터를 사용해 현재의 피처를 생성하는 경우가 없도록 해야 합니다.

개선 방법

파생변수를 생성하는 코드를 교차 검증 안으로 이동시켜, 각 폴드의 훈련 데이터에서만 파생변수를 만들고, 테스트 폴드에는 그 파생변수를 사용하지 않도록 코드를 수정해야 합니다.

예를 들어:

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.base import clone

cv = KFold(n_splits=5, random_state=123, shuffle=True)
scores = []

for train_idx, test_idx in cv.split(df_prob4_train):
    X_train, X_test = df_prob4_train.iloc[train_idx], df_prob4_train.iloc[test_idx]
    y_train, y_test = df_prob4_train['target'].iloc[train_idx], df_prob4_train['target'].iloc[test_idx]
```

```

# 각 폴드의 훈련 데이터에서만 파생변수 생성
X_train_q = generate_features_based_on_target(X_train, y_train)
X_test_q = generate_features_based_on_target(X_test, y_test) # 주의: 타겟 사용 금지

# 모델 복제 및 훈련
model = clone(reg_rd)
model.fit(X_train_q, y_train)

# 성능 평가
score = model.score(X_test_q, y_test)
scores.append(score)

print("Cross-validation scores:", scores)
print("Mean score:", np.mean(scores))

이렇게 하면, 각 폴드 내에서만 파생변수가 생성되므로, 데이터 누출 문제를 방지할 수 있습니다.
'''
print('')

```

In [179...

```

'''
위에 나온 누출이 발생한 코드의 수정 버전(GPT 답변 - 검증 필요!!!)

데이터 누출 문제를 해결하기 위해서는 파생변수(`cont{}_q`)를 교차 검증의 각 폴드 내에서 생성하도록 코드를 수정해야 합니다.
아래는 그에 따른 수정된 코드입니다.

### 수정된 코드

from sklearn.model_selection import cross_val_score, KFold
from sklearn.base import clone
import numpy as np

# 교차 검증을 위한 KFold 설정
cv = KFold(n_splits=5, random_state=123, shuffle=True)

# 점수를 저장할 리스트 초기화
scores = []

# 각 폴드에 대해 반복
for train_idx, test_idx in cv.split(df_prob4_train):
    # 훈련 폴드와 테스트 폴드로 데이터 분할

```

```

train_data, test_data = df_prob4_train.iloc[train_idx], df_prob4_train.iloc[test_idx]

# 파생변수 생성 (주의: 훈련 데이터에서만 생성)
for i in cont_cols:
    # 훈련 데이터에서 파생변수 생성
    train_data[f'{i}_q'] = pd.qcut(train_data[i], q=4, labels=False)

    # 테스트 데이터에서는 훈련 데이터의 정보를 기반으로 파생변수 생성
    # 예: 동일한 qcut 기준을 사용하여 테스트 데이터에 적용
    bins = pd.qcut(train_data[i], q=4, retbins=True)[1] # 훈련 데이터에서의 bins 추출
    test_data[f'{i}_q'] = pd.cut(test_data[i], bins=bins, labels=False, include_lowest=True)

# 사용할 피쳐 정의
X_train = train_data[[f'{i}_q' for i in cont_cols] + dum_cols]
X_test = test_data[[f'{i}_q' for i in cont_cols] + dum_cols]
y_train = train_data['target']
y_test = test_data['target']

# 모델 복제 및 학습
model = clone(reg_rd)
model.fit(X_train, y_train)

# 테스트 폴드에 대해 성능 평가
score = model.score(X_test, y_test) # 기본 스코어 메서드 사용
scores.append(score)

# 결과 출력
print("Cross-validation scores:", scores)
print("Mean score:", np.mean(scores))

### 설명

1. 파생변수 생성:
    - 훈련 데이터에서만 `qcut`을 사용하여 파생변수를 생성합니다.
    - 테스트 데이터는 훈련 데이터의 분포에 맞춘 `bins`을 사용하여 동일한 기준으로 파생변수를 생성합니다.
      이렇게 하면 데이터 누출을 방지할 수 있습니다.

2. 각 폴드 내에서 파생변수 생성:
    - `for` 루프를 사용해 교차 검증의 각 폴드에서 훈련 데이터와 테스트 데이터를 분리한 후, 훈련 데이터에서만 파생변수를 생성합니다.
    - 그런 다음, 훈련 데이터에서 생성된 `bins`을 사용해 테스트 데이터의 파생변수를 생성합니다.

```

3. 모델 학습 및 평가:

- 훈련 폴드에서 모델을 학습시키고, 해당 모델을 테스트 폴드에서 평가하여 점수를 기록합니다.

이렇게 하면 각 폴드에서 데이터 누출 없이 모델의 성능을 평가할 수 있으며, 실제 테스트 상황과 유사한 환경에서 성능을 평가할 수 있습니다.

```
print('')
```

In [180...

```
'''
`cv = KFold(n_splits=5, random_state=123, shuffle=True)`와 같이 `KFold` 객체를 생성한 경우,
`cv.split()` 함수는 교차 검증을 위해 데이터셋을 여러 개의 훈련/테스트 세트로 분할합니다.

### `cv.split()` 함수
#### 인자 (Parameters)
- `X`: 배열 형태의 입력 데이터. 일반적으로 특성(feature) 배열을 의미하며, 크기는 (n_samples, n_features)입니다.
- `y`: (선택 사항) 타깃 레이블 배열로, 크기는 (n_samples,)입니다. 회귀 문제에서 사용할 수 있습니다.
  `X`와 같은 길이어야 합니다.
- `groups`: (선택 사항) 그룹 레이블 배열로, 크기는 (n_samples,)입니다. 그룹별로 데이터를 나누기 위해 사용할 수 있습니다.
  `GroupKFold`와 같은 그룹별 분할 방법에서 사용됩니다.

cv.split(X, y=None, groups=None)

#### 리턴값 (Returns)
`cv.split()` 함수는 각 분할에 대한 인덱스의 제너레이터를 반환합니다.
각 분할은 (train_index, test_index)의 튜플로 구성되어 있습니다. 여기서:
- `train_index`: 훈련 세트의 샘플 인덱스 배열
- `test_index`: 테스트 세트의 샘플 인덱스 배열

예를 들어, 5-폴드 교차 검증에서 `cv.split(X)`를 호출하면, 5개의 (train_index, test_index) 튜플이 생성됩니다.
splits = cv.split(X)
for train_index, test_index in splits:
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

### 속성 (Attributes)
- `n_splits`: 교차 검증 분할 수. 여기서는 5입니다.
- `random_state`: 난수 시드 값. 주어진 난수 시드로 결과의 재현성을 보장합니다.
- `shuffle`: 데이터를 분할하기 전에 섞을지 여부. 여기서는 `True`로 설정되어 있습니다.

### 함수 (Methods)
- `get_n_splits()`: 분할 수를 반환합니다.
    n_splits = cv.get_n_splits() # n_splits는 5
'''
```

- ``split(X, y=None, groups=None)``: 교차 검증을 위해 데이터셋을 훈련/테스트 세트로 분할합니다.
위에서 설명한 인자 및 리턴값을 가집니다.

아래는 ``KFold``와 ``cv.split()``를 사용하는 예제 코드입니다.

아래 예제에서는 ``KFold``를 사용해 ``X``와 ``y``를 5-폴드 교차 검증으로 분할하고, 각 분할에 대해 훈련 세트와 테스트 세트를 출력합니다.
...

```
from sklearn.model_selection import KFold
import numpy as np

# 데이터 생성
X = np.arange(10).reshape((5, 2))
y = np.array([0, 1, 0, 1, 0])

# KFold 객체 생성
cv = KFold(n_splits=5, random_state=123, shuffle=True)

# 데이터 분할
for train_index, test_index in cv.split(X, y):
    print("Train Index:", train_index, "Test Index:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    print("X_train:", X_train, "X_test:", X_test)
    print("y_train:", y_train, "y_test:", y_test)
    print('-----')
```

```

Train Index: [0 2 3 4] Test Index: [1]
X_train: [[0 1]
[4 5]
[6 7]
[8 9]] X_test: [[2 3]]
y_train: [0 0 1 0] y_test: [1]
-----

```

```

Train Index: [0 1 2 4] Test Index: [3]
X_train: [[0 1]
[2 3]
[4 5]
[8 9]] X_test: [[6 7]]
y_train: [0 1 0 0] y_test: [1]
-----

```

```

Train Index: [0 1 2 3] Test Index: [4]
X_train: [[0 1]
[2 3]
[4 5]
[6 7]] X_test: [[8 9]]
y_train: [0 1 0 1] y_test: [0]
-----

```

```

Train Index: [1 2 3 4] Test Index: [0]
X_train: [[2 3]
[4 5]
[6 7]
[8 9]] X_test: [[0 1]]
y_train: [1 0 1 0] y_test: [0]
-----

```

```

Train Index: [0 1 3 4] Test Index: [2]
X_train: [[0 1]
[2 3]
[6 7]
[8 9]] X_test: [[4 5]]
y_train: [0 1 1 0] y_test: [0]
-----

```

In [181...

'''

아래 코드는 교차 검증을 사용하여 여러 열에 대해 분위수를 계산하고,
각 분위구간에 대해 평균 `target` 값을 구하여 새로운 특성 열을 생성한 후, 모델을 훈련하고 검증하는 과정을 다룹니다.

```

# 0과 1 사이를 0.01 간격으로 나눈 리스트를 생성합니다.
q = [i for i in np.arange(0, 1.01, 0.01)]

```

```

이 라인은 0과 1 사이를 0.01 간격으로 나눈 값들을 리스트로 생성합니다. `q`는 [0.00, 0.01, 0.02, ..., 1.00]의 값을 가지는 리스트가 됩니다.

# 빈 리스트를 생성하여 점수를 저장합니다.
score_ = []
이 라인은 빈 리스트 `score_`를 생성하여 교차 검증에서 얻은 점수를 저장할 준비를 합니다.

# 연속형 열 이름에 `_q`를 추가한 리스트와 더미 변수 열 이름을 합친 리스트를 생성합니다.
X_rd = [i + `_q` for i in cont_cols] + dum_cols
이 라인은 연속형 열(`cont_cols`)의 이름에 `_q`를 추가한 열 이름과 더미 변수 열(`dum_cols`) 이름을 합쳐서 새로운 리스트 `X_rd`를 생성합니다. 이 리스트는 모델 훈련에 사용할 특성 열 이름들을 포함합니다.

# 교차 검증을 위해 데이터셋을 나눕니다.
for train_idx, valid_idx in cv.split(df_prob4_train):
이 라인은 `cv` 객체를 사용하여 `df_prob4_train` 데이터프레임을 훈련 세트와 검증 세트로 나누는 작업을 시작합니다. `cv.split(df_prob4_train)`은 훈련 세트의 인덱스(`train_idx`)와 검증 세트의 인덱스(`valid_idx`)를 반환합니다.

    # 훈련 세트와 검증 세트를 생성합니다.
    df_cv_train, df_valid = df_prob4_train.iloc[train_idx].copy(), df_prob4_train.iloc[valid_idx].copy()
    이 라인은 훈련 세트와 검증 세트를 `df_prob4_train` 데이터프레임에서 인덱스를 사용하여 분리하고, 각각 `df_cv_train`과 `df_valid`에 저장합니다. `copy()`를 사용하여 원본 데이터의 변경을 방지합니다.

# 각 열에 대해 반복 작업을 수행합니다.
for i in range(0, 14):
이 라인은 연속형 열(`cont0`부터 `cont13`까지)에 대해 반복 작업을 시작합니다.

    # 현재 열 이름을 생성합니다.
    col = 'cont{}'.format(i)
    이 라인은 현재 반복 중인 열 이름을 생성하여 `col`에 저장합니다.

    # 현재 열에 대한 분위수를 계산합니다.
    qt = df_cv_train[col].quantile(q)
    이 라인은 현재 열 `col`에 대해 위에서 생성한 분위점 `q`를 사용하여 분위수를 계산합니다. `qt`는 현재 열 `col`의 분위수 값을 포함하는 시리즈가 됩니다.

    # 첫 번째 분위수보다 작은 값은 첫 번째 구간, 마지막 분위수보다 큰 값은 마지막 구간에 속하게 합니다.
    qt.iloc[[0, -1]] = [-np.inf, np.inf]
    이 라인은 `qt`의 첫 번째 값과 마지막 값을 각각 `-np.inf`와 `np.inf`로 설정하여 첫 번째 분위수보다 작은 값은 첫 번째 구간에, 마지막 분위수보다 큰 값은 마지막 구간에 속하도록 합니다.

    # 현재 열에 대해 분위구간을 생성합니다.
    q_cut = pd.cut(df_cv_train[col], bins=qt)
    이 라인은 현재 열 `col`의 값을 `qt`에 따라 구간으로 나눕니다.

```

`pd.cut` 함수는 현재 열 `col`의 값을 `qt`의 분위구간에 맞춰 구간을 나누고, 그 결과를 `q\_cut`에 저장합니다.

각 분위구간에 대해 target 값의 평균을 계산합니다.

```
q_mean = df_cv_train.groupby(q_cut)['target'].mean()
```

이 라인은 `q\_cut`에 따라 `df\_cv\_train` 데이터를 그룹화하고, 각 구간에 속하는 `target` 열의 평균을 계산합니다. 결과는 `q\_mean`에 저장됩니다. `q\_mean`은 각 구간의 `target` 평균값을 포함하는 시리즈가 됩니다.

훈련 세트에 새로운 분위구간 평균값 열을 생성합니다.

```
df_cv_train[col + '_q'] = q_cut.map(q_mean).astype('float')
```

이 라인은 `df\_cv\_train` 데이터프레임에 새로운 열을 생성합니다. 새로운 열의 이름은 현재 열 이름에 `\_q`를 붙인 형태이며, `q\_cut` 값을 `q\_mean` 값으로 매핑한 결과를 저장합니다. `astype('float')`은 결과를 float 형식으로 변환합니다.

검증 세트에 대해서도 동일한 작업을 수행하여 새로운 분위구간 평균값 열을 생성합니다.

```
df_valid[col + '_q'] = pd.cut(df_valid[col], bins=qt).map(q_mean).astype('float')
```

이 라인은 `df\_valid` 데이터프레임에 대해 동일한 작업을 수행합니다. `df\_valid`의 현재 열 `col` 값을 `qt`에 따라 구간으로 나. 각 구간에 대해 `q\_mean`의 값을 매핑하여 새로운 열을 생성합니다.

새로운 열의 이름은 `df\_cv\_train`과 동일하게 현재 열 이름에 `\_q`를 붙인 형태입니다.

모델을 훈련 세트로 훈련합니다.

```
reg_rd.fit(df_cv_train[X_rd], df_cv_train['target'])
```

이 라인은 `reg\_rd` 모델을 훈련 세트 `df\_cv\_train`의 특성 열 `X\_rd`와 목표 변수 `target`을 사용하여 훈련합니다.

검증 세트로 모델을 평가하고 점수를 저장합니다.

```
score_.append(mean_squared_error(df_valid['target'], reg_rd.predict(df_valid[X_rd])))
```

이 라인은 `reg\_rd` 모델을 사용하여 검증 세트 `df\_valid`의 특성 열 `X\_rd`에 대한 예측값을 생성하고, `target`과의 평균 제곱 오차(mean squared error)를 계산하여 `score\_` 리스트에 저장합니다.

전체 과정 요약

1. 0과 1 사이를 0.01 간격으로 나눈 값을 생성합니다.
2. 빈 리스트를 생성하여 점수를 저장합니다.
3. 연속형 열 이름에 `\_q`를 추가한 리스트와 더미 변수 열 이름을 합친 리스트를 생성합니다.
4. 교차 검증을 위해 데이터를 훈련 세트와 검증 세트로 나눕니다.
5. 각 연속형 열에 대해 반복 작업을 수행합니다.
6. 현재 열에 대한 분위수를 계산합니다.
7. 첫 번째 분위수보다 작은 값은 첫 번째 구간, 마지막 분위수보다 큰 값은 마지막 구간에 속하도록 설정합니다.
8. 현재 열을 분위수에 따라 구간으로 나눕니다.
9. 각 구간에 속하는 `target` 값의 평균을 계산합니다.
10. 훈련 세트와 검증 세트에 새로운 분위구간 평균값 열을 생성합니다.
11. 모델을 훈련 세트로 훈련합니다.
12. 모델을 검증 세트로 평가하고 점수를 저장합니다.

이 과정을 통해 데이터의 분위구간 기반 특성을 생성하고, 모델의 성능을 교차 검증을 통해 평가할 수 있습니다.

...

...

이 코드는 교차 검증을 통해 Ridge 회귀 모델의 성능을 평가하는 과정에서 데이터 누출을 방지합니다. 이를 위해 각 검증 분할에 대해 별도로 분위수를 계산하고 매핑하는 과정을 수행합니다.

코드 설명

1. 준비 작업

- ``q``는 0부터 1까지 0.01 간격의 분위수를 나타내는 리스트입니다.
- ``score_``는 각 교차 검증 분할에서 계산된 MSE 점수를 저장할 리스트입니다.
- ``X_rd``는 회귀 모델에 사용할 특징 변수 목록으로, 각 연속형 변수의 분위 구간별 평균(``_q``)과 범주형 변수의 원-핫 인코딩된 열 이름을 포함합니다.

2. 교차 검증 루프

- ``cv.split(df_prob4_train)``는 데이터를 ``train_idx``와 ``valid_idx``로 나누어, 교차 검증을 위한 훈련 데이터와 검증 데이터를 만듭니다.
- ``df_cv_train``과 ``df_valid``는 각각 훈련 분할과 검증 분할의 데이터프레임입니다.

3. 분위수 계산 및 매핑

- 이 루프는 각 연속형 변수(``cont0``부터 ``cont13``까지)에 대해 분위수를 계산하고, 분위 구간별로 목표 변수(``target``)의 평균 값을 계산합니다.
- ``qt``는 훈련 데이터(``df_cv_train[col]``)에서 분위수를 계산한 값입니다.
- ``q_cut``은 훈련 데이터를 분위수 구간으로 나눈 결과입니다.
- ``q_mean``은 각 분위수 구간에 대해 목표 변수의 평균 값을 계산한 결과입니다.
- ``df_cv_train[col + '_q']``는 훈련 데이터의 분위 구간별 평균 값을 매핑한 결과입니다.
- ``df_valid[col + '_q']``는 검증 데이터의 각 값이 훈련 데이터에서 계산된 분위수 구간에 속하는지에 따라 평균 값을 매핑한 결과입니다.

4. 모델 학습 및 평가

- ``reg_rd.fit(df_cv_train[X_rd], df_cv_train['target'])``는 훈련 데이터로 Ridge 회귀 모델을 학습합니다.
- ``score_.append(mean_squared_error(df_valid['target'], reg_rd.predict(df_valid[X_rd])))``는 검증 데이터로 예측한 값과 실제 값 사이의 평균 제곱 오차(MSE)를 계산하여 ``score_`` 리스트에 추가합니다.

5. 결과 계산

- ``np.mean(score_)``는 교차 검증의 각 분할에서 계산된 MSE의 평균 값을 계산합니다.
- ``score_``는 각 분할의 MSE 값을 저장합니다.

데이터 누출 방지

이 코드에서는 훈련 데이터(``df_cv_train``)와 검증 데이터(``df_valid``)를 분리한 후, 훈련 데이터에서만 분위수를 계산하고, 그 분위수를 사용하여 검증 데이터의 값을 매핑합니다. 이는 교차 검증 과정에서 검증 데이터의 정보가 훈련 데이터에 누출되는 것을 방지하여,

모델이 실제 테스트 데이터에서의 성능을 더 정확하게 반영할 수 있도록 합니다.

이러한 접근 방식은 특히 데이터 변환이 데이터셋 전체에 대해 사전 수행될 때 발생할 수 있는 누출을 방지하는 데 매우 중요합니다.
...

위에서 발생한 *Leak*을 바로 잡아 교차검증을 합니다.

```
q = [i for i in np.arange(0, 1.01, 0.01)]
print('q = ', q)
print('-----')
```

```
score_ = []
X_rd = [i + '_q' for i in cont_cols] + dum_cols
print('X_rd = ', X_rd)
print('-----')
```

```
for train_idx, valid_idx in cv.split(df_prob4_train):
    df_cv_train, df_valid = df_prob4_train.iloc[train_idx].copy(), df_prob4_train.iloc[valid_idx].copy()
    # 검증셋에서 train으로 파생변수를 만들고
    # 검증셋의 test(검외셋)에 검증셋의 train으로 만든 통계값(mean)을 반영합니다.
    for i in range(0, 14):
        col = 'cont{}'.format(i)
        qt = df_cv_train[col].quantile(q)
        qt.iloc[[0, -1]] = [-np.inf, np.inf]
        q_cut = pd.cut(df_cv_train[col], bins=qt)
        q_mean = df_cv_train.groupby(q_cut)['target'].mean()
        df_cv_train[col + '_q'] = q_cut.map(q_mean).astype('float')
        df_valid[col + '_q'] = pd.cut(df_valid[col], bins=qt).map(q_mean).astype('float')

    reg_rd.fit(df_cv_train[X_rd], df_cv_train['target'])
    score_.append(mean_squared_error(df_valid['target'], reg_rd.predict(df_valid[X_rd])))
```

Leak을 제거하니 위 df_prob4_test의 성능에 가깝게 나옵니다.
Leak을 없애 것이 이 검증이 Test에서 상황을 좀 더 잘 반영하게 된것입니다.

```
np.mean(score_), score_
```

```
q = [0.0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16, 0.17, 0.18, 0.19, 0.2,
0.21, 0.22, 0.23, 0.24, 0.25, 0.26, 0.27, 0.28, 0.29, 0.3, 0.31, 0.32, 0.33, 0.34, 0.35000000000000003, 0.36, 0.37, 0.38, 0.39,
0.4, 0.41000000000000003, 0.42, 0.43, 0.44, 0.45, 0.46, 0.47000000000000003, 0.48, 0.49, 0.5, 0.51, 0.52, 0.53, 0.54, 0.55, 0.5
6, 0.57000000000000001, 0.58, 0.59, 0.6, 0.61, 0.62, 0.63, 0.64, 0.65, 0.66, 0.67, 0.68, 0.69000000000000001, 0.70000000000000001,
0.71, 0.72, 0.73, 0.74, 0.75, 0.76, 0.77, 0.78, 0.79, 0.8, 0.81, 0.82000000000000001, 0.83000000000000001, 0.84, 0.85, 0.86, 0.87,
0.88, 0.89, 0.9, 0.91, 0.92, 0.93, 0.94000000000000001, 0.95000000000000001, 0.96, 0.97, 0.98, 0.99, 1.0]
```

```
X_rd = ['cont0_q', 'cont1_q', 'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q', 'cont8_q', 'cont9_q', 'cont10_
q', 'cont11_q', 'cont12_q', 'cont13_q', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C', 'cat5_B', 'cat5_C', 'cat5_
D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F', 'cat9_G', 'cat
9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

```
Out[181]: (0.7213562985785659,
[0.7117010644566664,
0.7236117939458264,
0.7246524557901769,
0.727192271050518,
0.7196239076496416])
```

Tip! 사용자 정의 지표를 만들어봅시다

시험장 버전 sklearn에는 mape가 구현되어 있지 않습니다.

$$MAPE = \frac{100}{N} \sum_{i=1}^n \left| \frac{y - \hat{y}}{y} \right|$$

```
In [182... from sklearn.metrics import make_scorer

# MAPE 지표 평가 함수입니다.
def mape(y_true, y_prd):
    return np.mean(np.abs((y_true - y_prd) / y_true)) * 100
```

```
In [183... '''
`make_scorer()` 함수는 `scikit-learn`에서 모델 평가를 위한 사용자 정의 평가 지표(scorer)를 생성할 때 사용됩니다.
이 함수는 평가 지표 함수를 받아 이를 스코어러 객체로 변환하여 `cross_val_score`, `GridSearchCV`와 같은 함수에서 사용할 수 있도록 함

### 1. 인자 (Parameters)
- `score_func`:
    - 평가 지표 함수입니다. 이 함수는 `(y_true, y_pred, kwargs)` 형식의 인자를 받아야 하며, 점수(스코어)를 반환합니다.
    - 대표적인 평가 지표로 `accuracy_score`, `f1_score`, `mean_squared_error` 등이 있습니다.
- `greater_is_better`:
```

- ``True``이면 높은 점수가 더 좋은 모델을 의미합니다. 기본값은 ``True``입니다.
- ``False``로 설정하면 점수의 부호가 반전됩니다.
예를 들어, 손실 함수에서 낮은 값이 더 좋을 경우 ``greater_is_better=False``로 설정할 수 있습니다.
- ``needs_proba``:
 - ``True``이면 모델의 ``predict_proba`` 메서드가 예측에 사용됩니다. 기본값은 ``False``입니다.
 - 분류 모델에서 확률 예측이 필요한 경우 사용됩니다. 예를 들어, AUC와 같은 지표를 사용할 때 필요합니다.
- ``needs_threshold``:
 - ``True``이면 모델의 ``decision_function`` 메서드가 예측에 사용됩니다. 기본값은 ``False``입니다.
 - ROC AUC와 같은 지표를 사용할 때 사용됩니다.
- ``kwargs``:
 - ``score_func``에 추가로 전달할 인자입니다. 예를 들어, ``average='macro'``와 같은 옵션을 설정할 수 있습니다.

2. 리턴값 (Return Value)

- ``scorer``:
 - ``make_scorer()`` 함수는 ``BaseScorer`` 또는 ``Callable`` 객체를 반환합니다.
 - 이 객체는 ``score_func``와 ``make_scorer``에서 설정한 옵션을 기반으로 모델의 성능을 평가합니다.

3. 속성 (Attributes)

``make_scorer`` 자체는 속성을 가지지 않지만, 반환된 스코어러 객체는 내부적으로 다음과 같은 속성을 가집니다:

- ``_score_func``:
 - 스코어러가 사용하는 평가 지표 함수입니다.
- ``_sign``:
 - ``greater_is_better`` 인자에 따라 스코어에 적용되는 부호입니다. (``1`` 또는 ``-1``)
- ``_needs_proba``:
 - ``True``이면 확률 예측이 필요합니다.
- ``_needs_threshold``:
 - ``True``이면 결정 함수 값이 필요합니다.

4. 함수 (Methods)

``make_scorer()``에서 반환된 스코어러 객체는 다음과 같은 주요 메서드를 제공합니다:

- ``__call__(estimator, X, y_true, kwargs)``:
 - 스코어러 객체가 호출될 때 실행됩니다.
 - ``estimator``는 평가할 모델 객체이고, ``X``는 입력 데이터, ``y_true``는 실제 타겟 값입니다.
 - 모델의 예측을 바탕으로 ``score_func`` 함수를 호출하여 점수를 반환합니다.

요약

- ``make_scorer()`` 함수는 사용자 정의 평가 지표를 ``scikit-learn``의 스코어러 객체로 변환합니다.
- 다양한 인자를 통해 평가 지표의 동작 방식을 세밀하게 조정할 수 있습니다.
- 반환된 스코어러는 모델 평가와 하이퍼파라미터 튜닝에서 유용하게 사용됩니다.

예제 설명

아래 예제에서는 사용자 정의 평가 지표를 생성하여 ``GridSearchCV``에서 사용하는 방법을 보여줍니다:

```
- `f1_scorer`: `f1_score` 함수를 기반으로 한 스코어러 객체를 생성합니다.
- `mse_scorer`: `mean_squared_error` 함수를 기반으로 한 스코어러 객체를 생성하며,
  `greater_is_better=False`로 설정하여 낮은 값이 더 좋은 경우로 만듭니다.
- `GridSearchCV`에서 `scoring` 인자로 사용자 정의 스코어러를 전달하여 모델을 평가합니다.
...
```

```
from sklearn.metrics import make_scorer, f1_score, mean_squared_error
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
```

```
# 데이터 생성
```

```
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
```

```
# 모델 생성
```

```
model = RandomForestClassifier(random_state=42)
```

```
# 사용자 정의 평가 지표: F1 스코어
```

```
f1_scorer = make_scorer(f1_score, average='macro')
```

```
# 사용자 정의 평가 지표: MSE (낮을수록 좋음)
```

```
mse_scorer = make_scorer(mean_squared_error, greater_is_better=False)
```

```
# 그리드 서치
```

```
param_grid = {'n_estimators': [50, 100, 200]}
```

```
grid_search = GridSearchCV(model, param_grid, scoring=f1_scorer, cv=5)
```

```
grid_search.fit(X, y)
```

```
print(f"최적의 하이퍼파라미터: {grid_search.best_params_}")
```

```
print(f"최고의 F1 스코어: {grid_search.best_score_}")
```

```
최적의 하이퍼파라미터: {'n_estimators': 50}
```

```
최고의 F1 스코어: 0.8999039792228576
```

In [184...

```
...
```

이 코드는 Ridge 회귀 모델의 성능을 평가하기 위해 MAPE(Mean Absolute Percentage Error)를 사용자 정의 평가지표로 사용하고, 교차 검증을 수행하여 평균 MAPE 점수를 계산합니다.

```
### 코드 설명
```

```
1. MAPE 함수 정의
```

```
- `mape` 함수는 실제 값(`y_true`)과 예측 값(`y_prd`) 사이의 절대 퍼센트 오차의 평균을 계산합니다.
```

- `(y_true - y_prd) / y_true`는 상대 오차를 계산합니다.
- `np.abs()`를 사용하여 절대 값을 취하고, `np.mean()`을 사용하여 평균을 계산한 후, **100**을 곱하여 퍼센트로 변환합니다.

2. MAPE 스코어러 생성

- `make_scorer` 함수를 사용하여 `mape` 함수를 사용자 정의 평가지표로 변환합니다.
- `greater_is_better=False`는 MAPE가 작을수록 모델 성능이 더 좋다는 것을 의미합니다.

3. 교차 검증 설정

- `KFold`를 사용하여 5겹 교차 검증을 설정합니다.
- `n_splits=5`는 데이터를 5개의 폴드로 나눕니다.
- `random_state=123`는 무작위 분할을 재현할 수 있도록 시드 값을 설정합니다.
- `shuffle=True`는 데이터를 무작위로 섞어서 각 폴드에 고르게 분포시키도록 합니다.

4. 특징 변수 설정

`X_rd = [i + '_q' for i in cont_cols] + dum_cols`

- `X_rd`는 회귀 모델에 사용할 특징 변수 목록입니다.
- 연속형 변수(`cont_cols`)의 분위 구간별 평균(`_q`)과 범주형 변수의 원-핫 인코딩된 열 이름(`dum_cols`)을 포함합니다.

5. 교차 검증 수행

```
score_ = cross_val_score(
    reg_rd,
    df_prob4_train[X_rd],
    df_prob4_train['target'],
    cv=cv, scoring=mape_scorer
)
```

- `cross_val_score`를 사용하여 5겹 교차 검증을 수행합니다.
- `reg_rd`는 Ridge 회귀 모델입니다.
- `df_prob4_train[X_rd]`는 훈련 데이터의 특징 변수입니다.
- `df_prob4_train['target']`는 훈련 데이터의 목표 변수입니다.
- `cv=cv`는 5겹 교차 검증 설정을 지정합니다.
- `scoring=mape_scorer`는 사용자 정의 MAPE 스코어러를 사용하여 모델 성능을 평가합니다.
- `score_`는 각 교차 검증 폴드에서 계산된 MAPE 점수를 저장합니다.

6. 평균 MAPE 계산

- `np.mean(score_)`는 5겹 교차 검증의 각 폴드에서 계산된 MAPE 점수의 평균을 계산합니다.
- `-`` 기호를 붙여서 MAPE가 작을수록 성능이 더 좋다는 것을 반영합니다.

요약

- `mape` 함수는 MAPE를 계산합니다.
- `make_scorer`로 `mape`를 사용자 정의 평가지표로 설정합니다.
- `KFold`로 5겹 교차 검증을 설정합니다.
- `cross_val_score`를 사용하여 교차 검증을 수행하고, 각 폴드에서 MAPE를 계산합니다.

- 교차 검증의 평균 **MAPE**를 계산하여, 모델 성능을 평가합니다.

이 코드에는 데이터 누출(**leakage**)이 없으며, 교차 검증 과정에서 훈련 데이터와 검증 데이터를 철저히 분리하여 평가하고 있습니다.

make_scorer에 함수를 전달합니다. 클수록 안 좋은 성격의 지표입니다.

```
mape_scorer = make_scorer(mape, greater_is_better=False)
```

```
from sklearn.model_selection import cross_val_score, KFold
```

```
cv = KFold(n_splits=5, random_state=123, shuffle=True)
```

```
X_rd = [i + '_q' for i in cont_cols] + dum_cols
```

```
print('X_rd = ', X_rd)
```

```
print('-----')
```

```
score_ = cross_val_score(
    reg_rd,
    df_prob4_train[X_rd],
    df_prob4_train['target'],
    cv=cv, scoring=mape_scorer
)
```

```
print(score_)
```

```
print('-----')
```

```
-np.mean(score_)
```

```
X_rd = ['cont0_q', 'cont1_q', 'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q', 'cont8_q', 'cont9_q', 'cont10_q', 'cont11_q', 'cont12_q', 'cont13_q', 'cat0_B', 'cat1_B', 'cat2_B', 'cat3_C', 'cat3_D', 'cat4_C', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_B', 'cat6_C', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

```
-----
[-9.70891848 -9.77411863 -9.78520922 -9.82802403 -9.7602062 ]
-----
```

Out[184]: 9.771295313422385

문제 5

XGBoost 분류 모델의 최적의 하이퍼 파라미터(Hyper-Parameter, 초매개변수)를 탐색한다.

In []:

단계 5-1

basetable에 cat0 ~ 9 제외하는 범주 없이 원핫 인코딩(One-Hot Encoding) 하여 prob5를 만든다.

In [185...

```
'''
df_basetable[cat_cols].apply(lambda x: print(x))

id
267387    A
410037    A
139373    A
113765    A
179915    A
..
320975    A
29591     A
46717     A
463191    A
415856    A
Name: cat0, Length: 240000, dtype: object
id
267387    B
410037    A
139373    A
113765    A
179915    B
..
320975    A
29591     B
46717     B
463191    A
415856    A
Name: cat1, Length: 240000, dtype: object
id
267387    B
410037    A
139373    B
```

```
113765    A
179915    A
..
320975    A
29591     A
46717     B
463191    A
415856    A
Name: cat2, Length: 240000, dtype: object
id
267387    C
410037    A
139373    A
113765    C
179915    C
..
320975    C
29591     C
46717     C
463191    C
415856    C
Name: cat3, Length: 240000, dtype: object
id
267387    B
410037    B
139373    B
113765    B
179915    B
..
320975    B
29591     B
46717     B
463191    B
415856    B
Name: cat4, Length: 240000, dtype: object
id
267387    B
410037    B
139373    B
113765    B
179915    B
..
```

```
320975    B
29591     B
46717     D
463191    D
415856    D
Name: cat5, Length: 240000, dtype: object
id
267387    A
410037    A
139373    A
113765    A
179915    A
..
320975    A
29591     A
46717     A
463191    A
415856    A
Name: cat6, Length: 240000, dtype: object
id
267387    E
410037    E
139373    E
113765    E
179915    E
..
320975    E
29591     E
46717     E
463191    E
415856    E
Name: cat7, Length: 240000, dtype: object
id
267387    G
410037    C
139373    C
113765    C
179915    E
..
320975    C
29591     C
46717     G
```

```

463191      E
415856      G
Name: cat8, Length: 240000, dtype: object
id
267387      G
410037      F
139373      J
113765      G
179915      L
..
320975      I
29591       F
46717       F
463191      F
415856      I
Name: cat9, Length: 240000, dtype: object
cat0      None
cat1      None
cat2      None
cat3      None
cat4      None
cat5      None
cat6      None
cat7      None
cat8      None
cat9      None
dtype: object
...
print('')

```

In [186...

```

'''
코드는 데이터 프레임 `df_basetable`의 범주형 변수를 원-핫 인코딩(one-hot encoding)하여
새로운 데이터 프레임 `df_prob5`를 생성하고, 원-핫 인코딩된 열 이름을 `dum_cols` 리스트에 저장합니다.

### 코드 설명

1. 원-핫 인코딩 수행
- `pd.get_dummies` 함수는 데이터 프레임 `df_basetable`에서 `cat_cols`에 포함된 범주형 변수들을 원-핫 인코딩합니다.
- 원-핫 인코딩은 각 범주형 변수의 각 고유 값에 대해 새로운 이진(dummy) 변수를 생성합니다.
- 예를 들어, 범주형 변수 `cat1`이 'A', 'B', 'C'의 세 가지 값을 가지면,
  `cat1_A`, `cat1_B`, `cat1_C`의 세 개의 새로운 열이 생성됩니다.

```

- 원-핫 인코딩된 데이터 프레임은 `df\_prob5`에 저장됩니다.

2. 원-핫 인코딩된 열 이름 생성

- `df\_basetable[cat\_cols]`는 `cat\_cols`에 해당하는 열들로 이루어진 데이터 프레임을 반환합니다.
- `apply` 메서드는 각 열에 대해 지정된 람다 함수를 적용합니다.
- 람다 함수 `lambda x: ['{}\_{}'.format(x.name, i) for i in np.sort(x.unique())]`는 다음과 같은 작업을 수행합니다:
 - `np.sort(x.unique())`는 해당 열의 고유 값을 오름차순으로 정렬하여 리스트로 반환합니다.
 - 리스트 내포(list comprehension) `[ '{}\_{}'.format(x.name, i) for i in np.sort(x.unique()) ]`는 각 고유 값에 대해 '열 이름_고유 값' 형식의 문자열을 생성합니다.
 - 예를 들어, 열 `cat1`이 'A', 'B', 'C'의 값을 가지면, `[ 'cat1\_A', 'cat1\_B', 'cat1\_C' ]`가 생성됩니다.
- `explode` 메서드는 각 리스트를 행으로 분리하여, 하나의 긴 시리즈로 만듭니다.
- `tolist` 메서드는 이 시리즈를 리스트로 변환합니다.
- 최종적으로, `dum\_cols`는 모든 원-핫 인코딩된 열 이름을 포함하는 리스트가 됩니다.

요약

- `pd.get\_dummies` 함수는 `df\_basetable`의 범주형 변수를 원-핫 인코딩하여 새로운 데이터 프레임 `df\_prob5`를 생성합니다.
- `cat\_cols`의 각 열에 대해, 고유 값들을 이용해 원-핫 인코딩된 열 이름을 생성하고, 이 이름들을 `dum\_cols` 리스트에 저장합니다.
- 결과적으로, `df\_prob5`는 원-핫 인코딩된 데이터 프레임이고, `dum\_cols`는 해당 열 이름들의 리스트입니다.

```
df_prob5 = pd.get_dummies(df_basetable, columns=cat_cols)
dum_cols = df_basetable[cat_cols].apply(lambda x: ['{}_{}'.format(x.name, i) for i in np.sort(x.unique())]).explode().tolist()
print('dum_cols = ', dum_cols)
```

```
df_prob5[dum_cols]
```

```
dum_cols = ['cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']
```

Out[186]:

	cat0_A	cat0_B	cat1_A	cat1_B	cat2_A	cat2_B	cat3_A	cat3_C	cat3_D	cat4_B	...	cat9_F	cat9_G	cat9_H	cat9_I	cat9_J	cat9_K	cat9_L	cat9_
id																			
267387	1	0	0	1	0	1	0	1	0	1	...	0	1	0	0	0	0	0	
410037	1	0	1	0	1	0	1	0	0	1	...	1	0	0	0	0	0	0	
139373	1	0	1	0	0	1	1	0	0	1	...	0	0	0	0	1	0	0	
113765	1	0	1	0	1	0	0	1	0	1	...	0	1	0	0	0	0	0	
179915	1	0	0	1	1	0	0	1	0	1	...	0	0	0	0	0	0	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
320975	1	0	1	0	1	0	0	1	0	1	...	0	0	0	1	0	0	0	
29591	1	0	0	1	1	0	0	1	0	1	...	1	0	0	0	0	0	0	
46717	1	0	0	1	0	1	0	1	0	1	...	1	0	0	0	0	0	0	
463191	1	0	1	0	1	0	0	1	0	1	...	1	0	0	0	0	0	0	
415856	1	0	1	0	1	0	0	1	0	1	...	0	0	0	1	0	0	0	

240000 rows x 20 columns

Tip! 열방향 groupby

groupby axis=1로 주면 열을 기준으로 groupby를 하게 됩니다.

입력 변수가 굉장히 많고, 변수명에 규칙성이 있다면 열방향 GroupBy를 활용하면

전처리 시간을 줄일 수 있습니다.

Ex) 가변수명을 활용해서 범주의 범주값들이 행마다 등장한 횟수를 추출합니다.

그리고, target과 pearson 상관도 분석을 해봅니다.

In [205...

```
'''
각 코드의 동작을 자세히 설명하겠습니다.
```

```
### 1. `np.array([i.split('_')[1] for i in dum_cols])`
```

코드는 `dum\_cols` 리스트에서 각 원-핫 인코딩된 열 이름을 '_' 기준으로 분할하고, 두 번째 부분(고유 값)을 추출하여 배열로 변환합니다.

```
### 2. `df_prob5[dum_cols].T.groupby(np.array([i.split('_')[1]
                                             for i in dum_cols])).sum().T.sort_index()`
```

이 코드는 다음과 같은 작업을 수행합니다:

- `df\_prob5[dum\_cols]`: `df\_prob5` 데이터 프레임에서 `dum\_cols`에 해당하는 열들만 선택합니다.
- `.T`: 선택된 데이터 프레임을 전치(transpose)합니다.
- `.groupby(np.array([i.split('\_')[1] for i in dum\_cols]))`: 각 열을 고유 값 기준으로 그룹화합니다.
- `.sum()`: 각 그룹의 열을 합칩니다.
- `.T`: 다시 전치하여 원래 형식으로 복원합니다.
- `.sort\_index()`: 인덱스를 정렬합니다.

```
### 3. `df_prob5[dum_cols].groupby([i.split('_')[1] for i in dum_cols],
                                   axis=1).sum().apply(lambda x: pd.Series(pearsonr(x,
                                   df_prob5['target']), index=['corr', 'pvalue']))`
```

이 코드는 다음과 같은 작업을 수행합니다:

- `df\_prob5[dum\_cols]`: `df\_prob5` 데이터 프레임에서 `dum\_cols`에 해당하는 열들만 선택합니다.
- `.groupby([i.split('\_')[1] for i in dum\_cols], axis=1).sum()`: 고유 값 기준으로 열을 그룹화하고 합칩니다.
- `.apply(lambda x: pd.Series(pearsonr(x, df\_prob5['target']), index=['corr', 'pvalue']))`: 각 그룹의 합과 `target` 열 간의 피어슨 상관 계수와 p-value를 계산합니다.

```
### 4. `df_basetable[cat_cols].reset_index().melt(id_vars='id')
      .pivot_table(index='id', columns='value', aggfunc='size')
      .fillna(0).astype('int')`
```

이 코드는 다음과 같은 작업을 수행합니다:

- `df\_basetable[cat\_cols]`: `df\_basetable` 데이터 프레임에서 `cat\_cols`에 해당하는 열들만 선택합니다.
- `.reset\_index()`: 인덱스를 초기화합니다.
- `.melt(id\_vars='id')`: 데이터를 길게(unpivot) 변환하여 'id' 열을 유지합니다.
- `.pivot\_table(index='id', columns='value', aggfunc='size')`: 'id'를 인덱스로, 'value'를 열로 하는 피벗 테이블을 만듭니다.
- 각 교차점의 값은 빈도수가 됩니다.
- `.fillna(0).astype('int')`: 결측값을 0으로 채우고, 정수형으로 변환합니다.

```
### 5. `df_prob5[cont_cols].groupby([int(i[4:]) % 2
                                     for i in cont_cols], axis=1).mean()`
```

이 코드는 다음과 같은 작업을 수행합니다:

```
- `df_prob5[cont_cols]`:
    `df_prob5` 데이터 프레임에서 `cont_cols`에 해당하는 열들만 선택합니다.
- `.groupby([int(i[4:]) % 2 for i in cont_cols], axis=1).mean()`:
    열 이름에서 숫자 부분을 추출하여 이를 기준으로 짝수와 홀수 열로 그룹화한 후, 각 그룹의 평균을 계산합니다.
```

```
### 6. `df_prob5[cont_cols].groupby([int(i[4:]) // 3
                                     for i in cont_cols], axis=1).mean()`
```

이 코드는 다음과 같은 작업을 수행합니다:

```
- `df_prob5[cont_cols]`:
    `df_prob5` 데이터 프레임에서 `cont_cols`에 해당하는 열들만 선택합니다.
- `.groupby([int(i[4:]) // 3 for i in cont_cols], axis=1).mean()`:
    열 이름에서 숫자 부분을 추출하여 이를 기준으로 3개씩 묶어 그룹화한 후, 각 그룹의 평균을 계산합니다.
```

요약

1. 열 이름에서 고유 값 추출
 - `dum\_cols` 리스트에서 각 열 이름의 고유 값을 추출합니다.
 2. 그룹별 합 계산
 - 원-핫 인코딩된 열을 고유 값 기준으로 그룹화하고 합을 계산하여 정렬합니다.
 3. 피어슨 상관 계수 계산
 - 원-핫 인코딩된 열을 고유 값 기준으로 그룹화하고 `target`과의 피어슨 상관 계수와 p-value를 계산합니다.
 4. 피벗 테이블 생성
 - 범주형 변수를 길게 변환한 후 피벗 테이블을 생성하여 각 범주의 빈도수를 계산합니다.
 5. 짝수/홀수 열 그룹화 및 평균 계산
 - 연속형 변수 열을 짝수와 홀수 그룹으로 나누어 각 그룹의 평균을 계산합니다.
 6. 3개씩 묶어 그룹화 및 평균 계산
 - 연속형 변수 열을 3개씩 묶어 그룹화한 후 각 그룹의 평균을 계산합니다.
- ...
- ```
print('')
```

```
In [188... # 가변수의 컬럼명의 뒷자리만 떼어내어 이를 기준으로 열방향 GroupBy를 해봅니다.
np.array([i.split('_')[1] for i in dum_cols])
```

```
Out[188]: array(['A', 'B', 'A', 'B', 'A', 'B', 'A', 'C', 'D', 'B', 'C', 'A', 'B',
 'C', 'D', 'A', 'B', 'C', 'B', 'D', 'E', 'G', 'A', 'C', 'D', 'E',
 'G', 'A', 'B', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O'],
 dtype='<U1')
```

```
In [189... df_prob5[dum_cols].T
```

Out[189]:

|        | id | 267387 | 410037 | 139373 | 113765 | 179915 | 166456 | 29503 | 476724 | 310468 | 113326 | ... | 367501 | 76711 | 215151 | 244056 | 324013 | 320975 | 2 |
|--------|----|--------|--------|--------|--------|--------|--------|-------|--------|--------|--------|-----|--------|-------|--------|--------|--------|--------|---|
| cat0_A |    | 1      | 1      | 1      | 1      | 1      | 1      | 1     | 1      | 0      | 1      | ... | 1      | 1     | 1      | 1      | 1      | 1      |   |
| cat0_B |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 1      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      |   |
| cat1_A |    | 0      | 1      | 1      | 1      | 0      | 1      | 0     | 1      | 1      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 1 |
| cat1_B |    | 1      | 0      | 0      | 0      | 1      | 0      | 1     | 0      | 0      | 1      | ... | 1      | 1     | 1      | 1      | 1      | 1      | 0 |
| cat2_A |    | 0      | 1      | 0      | 1      | 1      | 1      | 1     | 1      | 1      | 1      | ... | 1      | 1     | 0      | 1      | 0      | 0      | 1 |
| cat2_B |    | 1      | 0      | 1      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 1      | 0      | 1      | 1      | 0 |
| cat3_A |    | 0      | 1      | 1      | 0      | 0      | 0      | 0     | 0      | 0      | 1      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat3_C |    | 1      | 0      | 0      | 1      | 1      | 1      | 1     | 0      | 1      | 0      | ... | 1      | 1     | 1      | 1      | 1      | 1      | 1 |
| cat3_D |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 1      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat4_B |    | 1      | 1      | 1      | 1      | 1      | 1      | 1     | 1      | 1      | 1      | ... | 1      | 1     | 1      | 1      | 1      | 1      | 1 |
| cat4_C |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat5_A |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat5_B |    | 1      | 1      | 1      | 1      | 1      | 1      | 1     | 1      | 1      | 0      | ... | 1      | 0     | 1      | 1      | 1      | 0      | 1 |
| cat5_C |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat5_D |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 1      | ... | 0      | 1     | 0      | 0      | 0      | 1      | 0 |
| cat6_A |    | 1      | 1      | 1      | 1      | 1      | 1      | 1     | 1      | 1      | 1      | ... | 1      | 1     | 1      | 1      | 1      | 1      | 1 |
| cat6_B |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat6_C |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat7_B |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat7_D |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 1     | 0      | 0      | 0      | 0      | 0 |
| cat7_E |    | 1      | 1      | 1      | 1      | 1      | 1      | 1     | 1      | 1      | 1      | ... | 1      | 0     | 1      | 1      | 1      | 1      | 1 |
| cat7_G |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat8_A |    | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat8_C |    | 0      | 1      | 1      | 1      | 0      | 0      | 0     | 1      | 0      | 1      | ... | 1      | 0     | 0      | 0      | 0      | 0      | 1 |

| id     | 267387 | 410037 | 139373 | 113765 | 179915 | 166456 | 29503 | 476724 | 310468 | 113326 | ... | 367501 | 76711 | 215151 | 244056 | 324013 | 320975 | 2 |
|--------|--------|--------|--------|--------|--------|--------|-------|--------|--------|--------|-----|--------|-------|--------|--------|--------|--------|---|
| cat8_D | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat8_E | 0      | 0      | 0      | 0      | 1      | 1      | 1     | 0      | 1      | 0      | ... | 0      | 1     | 1      | 1      | 0      | 0      | 0 |
| cat8_G | 1      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 1      | 0      | 0 |
| cat9_A | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_B | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_F | 0      | 1      | 0      | 0      | 0      | 1      | 1     | 0      | 0      | 0      | ... | 0      | 1     | 1      | 1      | 1      | 1      | 0 |
| cat9_G | 1      | 0      | 0      | 1      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_H | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 1      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_I | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 1      | 0      | ... | 1      | 0     | 0      | 0      | 0      | 0      | 1 |
| cat9_J | 0      | 0      | 1      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_K | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 1      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_L | 0      | 0      | 0      | 0      | 1      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_M | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_N | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |
| cat9_O | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0 |

20 rows x 240000 columns

In [190...

```
df_prob5[dum_cols].T.groupby(
 np.array([i.split('_')[1] for i in dum_cols])
).sum()
```

Out[190]:

| id | 267387 | 410037 | 139373 | 113765 | 179915 | 166456 | 29503 | 476724 | 310468 | 113326 | ... | 367501 | 76711 | 215151 | 244056 | 324013 | 320975 | 29591 |
|----|--------|--------|--------|--------|--------|--------|-------|--------|--------|--------|-----|--------|-------|--------|--------|--------|--------|-------|
| A  | 2      | 5      | 4      | 4      | 3      | 4      | 3     | 4      | 3      | 4      | ... | 3      | 3     | 2      | 3      | 2      | 4      | 3     |
| B  | 4      | 2      | 3      | 2      | 3      | 2      | 3     | 2      | 3      | 2      | ... | 3      | 2     | 4      | 3      | 3      | 2      | 3     |
| C  | 1      | 1      | 1      | 2      | 1      | 1      | 1     | 1      | 1      | 1      | ... | 2      | 1     | 1      | 1      | 1      | 2      | 2     |
| D  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 1      | 0      | 1      | ... | 0      | 2     | 0      | 0      | 1      | 0      | 0     |
| E  | 1      | 1      | 1      | 1      | 2      | 2      | 2     | 1      | 2      | 1      | ... | 1      | 1     | 2      | 2      | 1      | 1      | 1     |
| F  | 0      | 1      | 0      | 0      | 0      | 1      | 1     | 0      | 0      | 0      | ... | 0      | 1     | 1      | 1      | 1      | 0      | 1     |
| G  | 2      | 0      | 0      | 1      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 1      | 0      | 0     |
| H  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 1      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |
| I  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 1      | 0      | ... | 1      | 0     | 0      | 0      | 0      | 1      | 0     |
| J  | 0      | 0      | 1      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |
| K  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 1      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |
| L  | 0      | 0      | 0      | 0      | 1      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |
| M  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |
| N  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |
| O  | 0      | 0      | 0      | 0      | 0      | 0      | 0     | 0      | 0      | 0      | ... | 0      | 0     | 0      | 0      | 0      | 0      | 0     |

15 rows × 240000 columns

```
In [191]: # 먼저 DataFrame을 Transpose 하여 행방향 groupby를 하고 다시 Transpose 합니다.
df_prob5[dum_cols].T.groupby(
 np.array([i.split('_')[1] for i in dum_cols])
).sum().T.sort_index()
```

Out[191]:

|        | A   | B   | C   | D   | E   | F   | G   | H   | I   | J   | K   | L   | M   | N   | O   |
|--------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| id     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
| 2      | 5   | 3   | 0   | 0   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 4      | 4   | 1   | 1   | 1   | 1   | 0   | 1   | 0   | 0   | 0   | 1   | 0   | 0   | 0   | 0   |
| 6      | 4   | 3   | 1   | 0   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 7      | 4   | 3   | 1   | 0   | 2   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 8      | 3   | 2   | 1   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| ...    | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 499992 | 5   | 1   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 499993 | 3   | 3   | 1   | 0   | 2   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   | 0   |
| 499997 | 3   | 3   | 2   | 0   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   |
| 499998 | 2   | 4   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 499999 | 4   | 2   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   | 0   | 0   |

240000 rows × 15 columns

```
In [192... df_prob5[dum_cols].groupby([i.split('_')[1] for i in dum_cols], axis=1).sum()
```

Out[192]:

|        | A   | B   | C   | D   | E   | F   | G   | H   | I   | J   | K   | L   | M   | N   | O   |
|--------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| id     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
| 267387 | 2   | 4   | 1   | 0   | 1   | 0   | 2   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 410037 | 5   | 2   | 1   | 0   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 139373 | 4   | 3   | 1   | 0   | 1   | 0   | 0   | 0   | 0   | 1   | 0   | 0   | 0   | 0   | 0   |
| 113765 | 4   | 2   | 2   | 0   | 1   | 0   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 179915 | 3   | 3   | 1   | 0   | 2   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   | 0   |
| ...    | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 320975 | 4   | 2   | 2   | 0   | 1   | 0   | 0   | 0   | 1   | 0   | 0   | 0   | 0   | 0   | 0   |
| 29591  | 3   | 3   | 2   | 0   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 46717  | 2   | 3   | 1   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 463191 | 4   | 1   | 1   | 1   | 2   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 415856 | 4   | 1   | 1   | 1   | 1   | 0   | 1   | 0   | 1   | 0   | 0   | 0   | 0   | 0   | 0   |

240000 rows × 15 columns

In [193...

```
'''
id
267387 2
410037 5
139373 4
113765 4
179915 3
..
320975 4
29591 3
46717 2
463191 4
415856 4
Name: A, Length: 240000, dtype: uint8
id
267387 4
410037 2
```

```
139373 3
113765 2
179915 3
..
320975 2
29591 3
46717 3
463191 1
415856 1
Name: B, Length: 240000, dtype: uint8
id
267387 1
410037 1
139373 1
113765 2
179915 1
..
320975 2
29591 2
46717 1
463191 1
415856 1
Name: C, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 0
..
320975 0
29591 0
46717 1
463191 1
415856 1
Name: D, Length: 240000, dtype: uint8
id
267387 1
410037 1
139373 1
113765 1
179915 2
```

```
..
320975 1
29591 1
46717 1
463191 2
415856 1
Name: E, Length: 240000, dtype: uint8
id
267387 0
410037 1
139373 0
113765 0
179915 0
..
320975 0
29591 1
46717 1
463191 1
415856 0
Name: F, Length: 240000, dtype: uint8
id
267387 2
410037 0
139373 0
113765 1
179915 0
..
320975 0
29591 0
46717 1
463191 0
415856 1
Name: G, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 0
..
320975 0
29591 0
```

```
46717 0
463191 0
415856 0
Name: H, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 0
..
320975 1
29591 0
46717 0
463191 0
415856 1
Name: I, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 1
113765 0
179915 0
..
320975 0
29591 0
46717 0
463191 0
415856 0
Name: J, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 0
..
320975 0
29591 0
46717 0
463191 0
415856 0
```

```
Name: K, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 1
..
320975 0
29591 0
46717 0
463191 0
415856 0
Name: L, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 0
..
320975 0
29591 0
46717 0
463191 0
415856 0
Name: M, Length: 240000, dtype: uint8
id
267387 0
410037 0
139373 0
113765 0
179915 0
..
320975 0
29591 0
46717 0
463191 0
415856 0
Name: N, Length: 240000, dtype: uint8
id
267387 0
```

```

410037 0
139373 0
113765 0
179915 0
..
320975 0
29591 0
46717 0
463191 0
415856 0
Name: 0, Length: 240000, dtype: uint8
A None
B None
C None
D None
E None
F None
G None
H None
I None
J None
K None
L None
M None
N None
O None
dtype: object
...
...
df_prob5[dum_cols].groupby([i.split('_')[1] for i in dum_cols], axis=1).sum().apply(
 lambda x: print(x)
)
...
print('')

```

In [194...

```

'''
아래 코드는 특정 열 이름 패턴을 가진 더미 변수 열(`dum_cols`)을 그룹화하고,
각 그룹의 합계를 계산하여 해당 그룹 합계와 목표 변수(`target`) 간의
피어슨 상관 계수(Pearson correlation coefficient)와 p-값(p-value)을 계산하는 과정을 다룹니다.

특정 열 이름 패턴을 가진 더미 변수 열을 그룹화하고, 그룹별 합계를 계산하여 상관 분석을 수행합니다.

```

```
df_prob5[dum_cols].groupby([i.split('_')[1] for i in dum_cols], axis=1).sum().apply(
이 부분은 여러 작업을 포함합니다.
```

1. ``df_prob5[dum_cols]``:
    - ``df_prob5`` 데이터프레임에서 ``dum_cols`` 리스트에 포함된 열만 선택합니다.
  2. ``.groupby([i.split('_')[1] for i in dum_cols], axis=1)``:
    - ``dum_cols`` 리스트에 포함된 각 열 이름 ``i``를 ``_``로 분리하고, 두 번째 부분(``i.split('_')[1]``)을 그룹화 기준으로 사용합니다. 예를 들어, ``dum_var1``과 ``dum_var2``는 ``var``라는 공통의 두 번째 부분을 가지므로 같은 그룹으로 묶입니다.
    - ``axis=1``은 열 기준으로 그룹화를 수행한다는 의미입니다.
  3. ``.sum()``:
    - 그룹화된 각 열의 합계를 계산합니다. 예를 들어, 같은 그룹에 속하는 여러 열의 값을 합칩니다.
  4. ``.apply(``:
    - 그룹별 합계에 대해 함수를 적용할 준비를 합니다.
 

```
lambda x: pd.Series(pearsonr(x, df_prob5['target']), index=['corr', 'pvalue'])
```
- 이 부분은 각 그룹별 합계에 대해 피어슨 상관 분석을 수행합니다.
1. ``lambda x: ...``:
    - 익명 함수를 정의합니다. 여기서 ``x``는 그룹별 합계를 의미합니다.
  2. ``pearsonr(x, df_prob5['target'])``:
    - ``x``와 ``df_prob5['target']`` 간의 피어슨 상관 계수와 p-값을 계산합니다.
    - ``pearsonr`` 함수는 상관 계수와 p-값을 반환합니다.
  3. ``pd.Series(..., index=['corr', 'pvalue'])``:
    - ``pearsonr`` 함수의 반환값을 ``pd.Series``로 변환하고, 인덱스를 ``['corr', 'pvalue']``로 설정합니다.
    - 이는 상관 계수와 p-값을 각각 ``corr``와 ``pvalue``로 나타내는 시리즈를 생성합니다.

전체 코드를 요약하면, ``df_prob5`` 데이터프레임의 ``dum_cols`` 리스트에 포함된 열을 특정 패턴에 따라 그룹화하고, 각 그룹의 합계를 계산한 후, 해당 그룹 합계와 목표 변수(``target``) 간의 피어슨 상관 계수와 p-값을 계산하여 결과를 시리즈로 반환합니다.

### 전체 과정 요약

1. ``dum_cols`` 리스트에 포함된 열을 선택합니다.
2. 열 이름의 두 번째 부분을 기준으로 그룹화합니다.
3. 그룹화된 열의 값을 합칩니다.
4. 각 그룹의 합계와 ``target`` 간의 피어슨 상관 계수와 p-값을 계산합니다.
5. 상관 계수와 p-값을 시리즈로 반환합니다.

이 과정을 통해 각 그룹별 합계와 목표 변수 간의 상관 관계를 파악할 수 있습니다.

```
...
```

```
from scipy.stats import pearsonr
구분할 항목을 groupby로 전달하고 axis=1로 열방향으로 지정하여
```

```
전달한 그룹이 동일한 것끼리 묶어 통계량을 구합니다.
이어, 피어슨 상관도를 구해봅니다.
df_prob5[dum_cols].groupby([i.split('_')[1] for i in dum_cols], axis=1).sum().apply(
 lambda x: pd.Series(pearsonr(x, df_prob5['target']), index=['corr', 'pvalue'])
)
```

Out[194]:

|        | A         | B        | C            | D            | E            | F            | G             | H         | I             | J             | K             | L        |         |
|--------|-----------|----------|--------------|--------------|--------------|--------------|---------------|-----------|---------------|---------------|---------------|----------|---------|
| corr   | -0.123781 | 0.101553 | 2.234329e-02 | 2.105080e-02 | 2.978301e-02 | 2.017009e-02 | -4.309416e-02 | -0.002058 | -6.669296e-02 | -2.005445e-02 | -1.977614e-02 | 0.081757 | 0.00394 |
| pvalue | 0.000000  | 0.000000 | 6.850167e-28 | 6.101729e-25 | 3.075829e-48 | 4.970866e-23 | 5.052612e-99  | 0.313303  | 1.163493e-234 | 8.737671e-23  | 3.351849e-22  | 0.000000 | 0.0534  |

```
In [195... df_basetable[cat_cols].reset_index().melt(id_vars='id').pivot_table(index='id', columns='value', aggfunc='size')\
.fillna(0).astype('int')
```

Out[195]:

| value  | A   | B   | C   | D   | E   | F   | G   | H   | I   | J   | K   | L   | M   | N   | O   |
|--------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| id     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
| 2      | 5   | 3   | 0   | 0   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 4      | 4   | 1   | 1   | 1   | 1   | 0   | 1   | 0   | 0   | 0   | 1   | 0   | 0   | 0   | 0   |
| 6      | 4   | 3   | 1   | 0   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 7      | 4   | 3   | 1   | 0   | 2   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 8      | 3   | 2   | 1   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| ...    | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 499992 | 5   | 1   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 499993 | 3   | 3   | 1   | 0   | 2   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   | 0   |
| 499997 | 3   | 3   | 2   | 0   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   |
| 499998 | 2   | 4   | 1   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 499999 | 4   | 2   | 1   | 1   | 1   | 0   | 0   | 0   | 0   | 0   | 1   | 0   | 0   | 0   | 0   |

240000 rows × 15 columns

In [196...

cont\_cols

Out[196]:

['cont0',  
'cont1',  
'cont2',  
'cont3',  
'cont4',  
'cont5',  
'cont6',  
'cont7',  
'cont8',  
'cont9',  
'cont10',  
'cont11',  
'cont12',  
'cont13']

In [197...

```
'''
아래 코드는 `df_prob5` 데이터프레임에서 연속형 열(`cont_cols`)을 인덱스의 홀수/짝수에 따라 그룹화하고,
각 그룹의 평균값을 계산하는 과정을 다룹니다.

df_prob5[cont_cols].groupby([int(i[4:]) % 2 for i in cont_cols], axis=1).mean()

1. `df_prob5[cont_cols]`:
- `df_prob5` 데이터프레임에서 `cont_cols` 리스트에 포함된 열만 선택합니다.
- 예를 들어, `cont_cols`가 `['cont0', 'cont1', 'cont2', 'cont3']`라면, `df_prob5[cont_cols]`는
 `df_prob5`에서 이 열들만 선택한 서브 데이터프레임을 반환합니다.
2. `[int(i[4:]) % 2 for i in cont_cols]`:
- 리스트 컴프리헨션을 사용하여 `cont_cols` 리스트에 포함된 각 열 이름 `i`에 대해 다음 작업을 수행합니다.
 - `i[4:]`: 열 이름 `i`의 네 번째 문자부터 끝까지의 문자열을 추출합니다. 예를 들어, 'cont0'이면 '0'이 추출됩니다.
 - `int(i[4:])`: 추출된 문자열을 정수로 변환합니다. 예를 들어, '0'은 0으로 변환됩니다.
 - `int(i[4:]) % 2`: 변환된 정수를 2로 나눈 나머지를 계산하여 홀수는 1, 짝수는 0으로 구분합니다.
- 예를 들어, `cont_cols`가 `['cont0', 'cont1', 'cont2', 'cont3']`인 경우,
 리스트 컴프리헨션은 `[0, 1, 0, 1]`을 반환합니다. 이는 인덱스 0과 2가 짝수, 인덱스 1과 3이 홀수임을 나타냅니다.
3. `.groupby(..., axis=1)`:
- 열(axis=1) 기준으로 그룹화를 수행합니다.
- `[0, 1, 0, 1]`을 그룹화 기준으로 사용하여, 열 이름의 인덱스가 짝수인 열들은 그룹 0에, 홀수인 열들은 그룹 1에 속하게 됩니다.
4. `.mean()`:
- 그룹화된 각 그룹의 평균값을 계산합니다.
- 예를 들어, 그룹 0에 속하는 열들(`cont0`과 `cont2`)의 평균값을 계산하고,
 그룹 1에 속하는 열들(`cont1`과 `cont3`)의 평균값을 계산합니다.

전체 과정 요약
1. `df_prob5` 데이터프레임에서 `cont_cols` 리스트에 포함된 열만 선택합니다.
2. `cont_cols`의 각 열 이름에서 숫자를 추출하여 2로 나눈 나머지에 따라 그룹화 기준 리스트를 생성합니다.
 (짝수 인덱스는 0, 홀수 인덱스는 1)
3. 생성된 그룹화 기준 리스트를 사용하여 열(axis=1) 기준으로 그룹화합니다.
4. 그룹화된 각 그룹의 평균값을 계산합니다.

이 과정을 통해 연속형 열들을 인덱스의 홀수/짝수에 따라 그룹화하고,
각 그룹의 평균값을 계산하여 데이터의 분포나 패턴을 분석할 수 있습니다.
'''

다른 예를 보여드립니다.
cont의 뒷자리 숫자가 짝수/홀수 별 평균을 구해봅니다.
df_prob5[cont_cols].groupby([int(i[4:]) % 2 for i in cont_cols], axis=1).mean()
```

Out[197]:

|        | 0        | 1        |
|--------|----------|----------|
| id     |          |          |
| 267387 | 0.604819 | 0.418539 |
| 410037 | 0.490341 | 0.506139 |
| 139373 | 0.379206 | 0.457589 |
| 113765 | 0.411349 | 0.487164 |
| 179915 | 0.432804 | 0.383754 |
| ...    | ...      | ...      |
| 320975 | 0.350579 | 0.568836 |
| 29591  | 0.522896 | 0.625043 |
| 46717  | 0.628611 | 0.763600 |
| 463191 | 0.335617 | 0.353714 |
| 415856 | 0.673560 | 0.560446 |

240000 rows × 2 columns

In [198...

```
다른 예를 보여드립니다.
cont의 뒷자리 숫자가 3개씩 묶어 평균을 구합니다.

df_prob5[cont_cols].groupby([int(i[4:]) // 3 for i in cont_cols] , axis=1).mean()
```

Out[198]:

|        | 0        | 1        | 2        | 3        | 4        |
|--------|----------|----------|----------|----------|----------|
| id     |          |          |          |          |          |
| 267387 | 0.487983 | 0.443463 | 0.490887 | 0.699477 | 0.399035 |
| 410037 | 0.450557 | 0.365157 | 0.666627 | 0.483487 | 0.538940 |
| 139373 | 0.506270 | 0.291730 | 0.361440 | 0.406760 | 0.579480 |
| 113765 | 0.586333 | 0.275163 | 0.390190 | 0.483943 | 0.541350 |
| 179915 | 0.498157 | 0.274377 | 0.531200 | 0.433803 | 0.251650 |
| ...    | ...      | ...      | ...      | ...      | ...      |
| 320975 | 0.487783 | 0.353270 | 0.451003 | 0.513100 | 0.510215 |
| 29591  | 0.473930 | 0.571670 | 0.679463 | 0.748567 | 0.307340 |
| 46717  | 0.589080 | 0.738133 | 0.715883 | 0.668877 | 0.804780 |
| 463191 | 0.481487 | 0.255250 | 0.167043 | 0.499010 | 0.308475 |
| 415856 | 0.600373 | 0.527903 | 0.740890 | 0.618180 | 0.588000 |

240000 rows × 5 columns

## 단계 5-2

prob5를 80%는 학습용 prob5\_train으로, 20%는 테스트용 prob5\_test로 나눈다.

### 함수가이드

sklearn.model\_selection.train\_test\_split, random\_state=123

문제 지시사항 외 Default 값 사용

```
In [199... from sklearn.model_selection import train_test_split
```

```
df_prob5_train, df_prob5_test = train_test_split(df_prob5, train_size=0.8, random_state=123)
```

```
In [200]: Image('3-5.png')
```

Out[200]: **단계 5-3**

다음 회귀 분석 모델을 아래 3개의 하이퍼 파라미터(Hyper-Parameter, 초매개변수)를 사용하여 prob5\_train으로 target를 학습시킨다.

prob5\_test으로 테스트했을 때 , 최적의 성능을 낸 하이퍼 파라미터 셋의 번호를 A, 성능지표를 B라고 한다.

입력 변수] cont0~13, cat0 ~ 9을 원핫 인코딩하여 나온 파생 변수(변수명, 범주명 알파벳순)

대상 변수] target

성능 지표]  $MSE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

No 하이퍼 파라미터(Hyper-parameter, 초매개변수)

|   |                                         |
|---|-----------------------------------------|
| 1 | colsample_bytree=0.25, n_estimators=300 |
| 2 | colsample_bytree=0.5, n_estimators=300  |
| 3 | colsample_bytree=0.25, n_estimators=500 |

공통적으로 트리의 최대 깊이는 2, 랜덤 시드는 123 이다.

함수가이드

xgboost 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [201]:
```

```
'''
```

코드는 주어진 `params` 목록의 파라미터 조합을 사용하여 XGBoost 모델을 학습하고 테스트 및 훈련 데이터 세트에 대한 평균 제곱 오차(MSE)를 계산하여 비교하는 과정을 설명합니다. 마지막에 테스트 데이터 세트에서 가장 낮은 MSE를 제공하는 파라미터 조합을 찾습니다.

1. 파라미터 설정
  - 세 가지 파라미터 조합을 설정합니다. 각각의 조합은 ``colsample_bytree``와 ``n_estimators`` 값으로 구성됩니다.
2. 특징 선택
  - ``cont_cols``와 ``dum_cols``를 합쳐서 모델에 사용할 특징 열을 결정합니다.
3. 오류 리스트 초기화
  - 테스트 및 훈련 데이터에 대한 MSE 값을 저장할 리스트를 초기화합니다.
4. 모델 학습 및 평가
  - 각 파라미터 조합에 대해 다음을 수행합니다:
    1. ``XGBRegressor`` 모델을 해당 파라미터로 초기화합니다.
    2. 훈련 데이터(``df_prob5_train``)로 모델을 학습시킵니다.
    3. 테스트 데이터(``df_prob5_test``)에 대한 예측을 수행하고, MSE를 계산하여 ``errors_test`` 리스트에 추가합니다.
    4. 훈련 데이터에 대한 예측을 수행하고, MSE를 계산하여 ``errors_train`` 리스트에 추가합니다.
    5. 현재 파라미터와 테스트 및 훈련 데이터의 MSE를 출력합니다.
5. 최적의 파라미터 찾기
  - 테스트 데이터에서 가장 낮은 MSE를 제공하는 파라미터의 인덱스를 찾습니다.  
(``np.argmax``은 0부터 시작하므로, 인덱스에 1을 더합니다)
  - 가장 낮은 MSE 값을 찾습니다.

#### ### 요약

코드는 XGBoost 회귀 모델을 여러 파라미터 조합으로 학습하고, 각각에 대해 테스트 및 훈련 데이터 세트의 MSE를 계산하여 최적의 파라미터를 찾는 과정입니다.

``errors_test``와 ``errors_train`` 리스트에 각 파라미터 조합에 대한 MSE를 저장하고, 테스트 데이터 세트에서 가장 낮은 MSE를 제공하는 파라미터 조합을 출력합니다.

- 파라미터 조합: 모델에 사용할 파라미터를 설정합니다.
- 데이터 분할: 훈련 데이터와 테스트 데이터로 분할합니다.
- 모델 학습 및 평가: 각 파라미터 조합에 대해 모델을 학습시키고 MSE를 계산합니다.
- 최적의 파라미터 찾기: 테스트 데이터에서 가장 낮은 MSE를 제공하는 파라미터를 찾습니다.
- ...

```
import xgboost as xgb
from sklearn.metrics import mean_squared_error

params = [
 {'colsample_bytree': 0.25, 'n_estimators': 300},
 {'colsample_bytree': 0.5, 'n_estimators': 300},
 {'colsample_bytree': 0.25, 'n_estimators': 500}
]
```

```

X_xgb = cont_cols + dum_cols
print('X_xgb = ', X_xgb)
print('-----')

errors_test = list()
errors_train = list()

for param in params:
 reg_xgb = xgb.XGBRegressor(max_depth=2, random_state=123, **param)

 reg_xgb.fit(df_prob5_train[X_xgb], df_prob5_train['target'])

 errors_test.append(
 mean_squared_error(df_prob5_test['target'], reg_xgb.predict(df_prob5_test[X_xgb]))
)

 errors_train.append(
 mean_squared_error(df_prob5_train['target'], reg_xgb.predict(df_prob5_train[X_xgb]))
)

 print(param, errors_test[-1], errors_train[-1])

X_xgb = ['cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'cat0_A', 'cat0_B', 'cat1_A', 'cat1_B', 'cat2_A', 'cat2_B', 'cat3_A', 'cat3_C', 'cat3_D', 'cat4_B', 'cat4_C', 'cat5_A', 'cat5_B', 'cat5_C', 'cat5_D', 'cat6_A', 'cat6_B', 'cat6_C', 'cat7_B', 'cat7_D', 'cat7_E', 'cat7_G', 'cat8_A', 'cat8_C', 'cat8_D', 'cat8_E', 'cat8_G', 'cat9_A', 'cat9_B', 'cat9_F', 'cat9_G', 'cat9_H', 'cat9_I', 'cat9_J', 'cat9_K', 'cat9_L', 'cat9_M', 'cat9_N', 'cat9_O']

{'colsample_bytree': 0.25, 'n_estimators': 300} 0.7215793793731834 0.7172290699688967
{'colsample_bytree': 0.5, 'n_estimators': 300} 0.721219679405562 0.7166355943476501
{'colsample_bytree': 0.25, 'n_estimators': 500} 0.7164353991627278 0.710087034418028

```

In [202...] errors\_test

Out[202]: [0.7215793793731834, 0.721219679405562, 0.7164353991627278]

```

In [203...] A = np.argmin(errors_test) + 1
 B = np.min(errors_test)

```

In [204...] A, B, A + B, round(A + B, 2)

Out[204]: (3, 0.7164353991627278, 3.716435399162728, 3.72)

A + B 를 소수점 셋째 자리에서 반올림하여 둘째 자리까지 구하라.

**3.72**

In [ ]: